

Руководство пользователя панели 3X-UI

العربية · English · Español · فارسی · Bahasa Indonesia · 日本語 · Português · Русский ·
Türkçe · Українська · Tiếng Việt · 简体中文 · 繁體中文

Версия 3X-UI: 3.5.0. Руководство составлено по этой версии и актуально для неё. Сводка изменений 3.5.0 относительно 3.4.2 — в разделе [«Что нового в 3.5.0»](#).

Подробное русскоязычное руководство по веб-панели **3X-UI** (управление Xray-core): функции, настройка и эксплуатация, с разбором каждого поля и переключателя в интерфейсе.

Названия и подписи соответствуют интерфейсу панели; русские термины приведены так, как они отображаются в интерфейсе. Слова *inbound* / *outbound* не переводятся.

Содержание

- Что нового в 3.5.0
- 1. Введение, требования и установка
 - 1.1. Что такое 3X-UI
 - 1.2. Поддерживаемые операционные системы и архитектуры
 - 1.3. Способы установки
 - 1.4. Первый запуск и учётные данные по умолчанию
 - 1.5. Расположение файлов
 - 1.6. Команда управления `x-ui` (меню скрипта)
 - 1.7. Подкоманды `x-ui` (без интерактивного меню)
 - 1.8. Миграция SQLite → PostgreSQL
- 2. Вход в панель и безопасность доступа
 - 2.1. Форма входа
 - 2.2. Двухфакторная аутентификация (2FA / TOTP)
 - 2.3. Ограничение попыток входа (login limiter / защита от перебора)
 - 2.4. Смена логина и пароля администратора
 - 2.5. Секретный путь (URI-путь / webBasePath) и порт панели
 - 2.6. Время жизни сессии (таймаут)
 - 2.7. LDAP (синхронизация и аутентификация)
- 3. Обзор / Дашборд
 - 3.1. Общие принципы сбора данных
 - 3.2. ЦП (CPU)
 - 3.3. Память (RAM)
 - 3.4. Подкачка (Swap)
 - 3.5. Диск (Storage)
 - 3.6. Время работы системы (Uptime)
 - 3.7. Загрузка системы (Load average)
 - 3.8. Сеть: скорость и общий объём трафика
 - 3.9. IP-адреса сервера
 - 3.10. TCP/UDP соединения
 - 3.11. Статус Xray и управление процессом
 - 3.12. Обновление панели (3X-UI)
 - 3.13. Обновление гео-файлов (GeoIP / GeoSite)
 - 3.14. Резервное копирование и восстановление базы данных
 - 3.15. Дополнительные элементы интерфейса
- 4. Inbounds: создание и общие параметры
 - 4.1. Общие поля формы
 - 4.2. Sniffing (Сниффинг)
 - 4.3. Allocate (стратегия распределения портов)
 - 4.4. External Proxy (Внешний прокси)
 - 4.5. Fallbacks (Fallback'и)
 - 4.6. Периодический сброс трафика
 - 4.7. JSON входящего (advanced)
 - 4.8. Действия с inbound: QR / Edit / Reset / Delete и статистика

- 5. Протоколы
 - 5.1. Список поддерживаемых протоколов
 - 5.2. Какие протоколы поддерживают TLS / REALITY / транспорт
 - 5.3. VLESS
 - 5.4. VMess
 - 5.5. Trojan
 - 5.6. Shadowsocks
 - 5.7. Dokodemo-door / Tunnel (прозрачный форвардер)
 - 5.8. SOCKS / HTTP (протокол `mixed`)
 - 5.9. WireGuard (inbound)
 - 5.10. Hysteria (по умолчанию v2)
 - 5.11. MTPROTO (прокси для Telegram)
 - 5.12. Краткая шпаргалка по выбору протокола
- 6. Транспорт (Stream Settings)
 - 6.1. Выбор сети передачи
 - 6.2. RAW / TCP (`tcpSettings`)
 - 6.3. mKCP (`kcpSettings`)
 - 6.4. WebSocket (`wsSettings`)
 - 6.5. gRPC (`grpcSettings`)
 - 6.6. HTTPUpgrade (`httpupgradeSettings`)
 - 6.7. XHTTP / SplitHTTP (`xhttpSettings`)
 - 6.8. Транспорт Hysteria (`hysteriaSettings`)
 - 6.9. Сопутствующие параметры
- 7. Безопасность соединения: TLS,XTLS и REALITY
 - 7.1. В чём разница: TLS vs XTLS vs REALITY
 - 7.2. Режим «Нет» (`none`)
 - 7.3. Режим TLS
 - 7.4. Режим REALITY
 - 7.5. Практические рекомендации по настройке
- 8. Клиенты
 - 8.1. Поля клиента
 - 8.2. Привязка к inbound
 - 8.3. Операции над клиентом
 - 8.4. Массовые операции
 - 8.5. Поиск, фильтры и сортировка
 - 8.6. Значки и статусы
- 9. Группы клиентов
 - 9.1. Что такое группа клиентов и зачем она нужна
 - 9.2. Связь группы с клиентами, inbound, нодами и протоколами
 - 9.3. Справочник групп и «пустые» группы
 - 9.4. Поля и столбцы группы
 - 9.5. Создание группы
 - 9.6. Переименование группы
 - 9.7. Добавление клиентов в группу
 - 9.8. Удаление клиентов из группы (без удаления самих клиентов)
 - 9.9. Сброс трафика группы

- 9.10. Удаление группы и удаление клиентов группы
- 9.11. Связь со страницей «Клиенты»
- 9.12. Сводка эндпоинтов API
- 9.13. Трафик по группе
- 10. Подписки (Subscription)
 - 10.1. Что такое subId и как формируется ссылка
 - 10.2. Настройки сервера подписок
 - 10.3. Форматы вывода
 - 10.4. Страница информации о подписке и QR-коды
 - 10.5. Кастомные шаблоны страницы подписки
- 11. Xray: маршрутизация, outbounds, DNS и расширения
 - 11.1. Структура редактора: вкладки/режимы
 - 11.2. Основные настройки (General)
 - 11.3. Правила маршрутизации (routing)
 - 11.4. Outbounds (исходящие подключения)
 - 11.5. Балансировщики (Balancers)
 - 11.6. DNS
 - 11.7. Fake DNS
 - 11.8. WireGuard / WARP / NordVPN
 - 11.9. Reverse-прокси и TUN
 - 11.10. Логи и статистика (Stats, metrics)
 - 11.11. Сохранение, перезапуск и автоматические преобразования
 - 11.12. Outbound из подписки (с авто-обновлением)
 - 11.13. Ротация IP в WARP
- 12. Узлы (мультипанель, master/slave)
 - 12.1. Сводка вверху списка
 - 12.2. Добавление и редактирование узла
 - 12.3. Проверка TLS (для https-узлов)
 - 12.4. Что показывается по каждому узлу
 - 12.5. Действия над узлом
 - 12.6. История метрик
 - 12.7. Как синхронизируются inbounds и клиенты
 - 12.8. Цепочки узлов (подузлы / транзитивные узлы)
 - 12.9. Исправления 3.5.0 (стабильность узлов)
 - 12.10. Узлы: новое в 3.3.0
- 13. Настройки панели
 - 13.1. Сохранение и перезапуск панели
 - 13.2. Общие настройки (вкладка «Панель» / *General*)
 - 13.3. Доступ к панели: IP, порт, путь, домен, сертификат
 - 13.4. Сессия, прокси панели и доверенные прокси (вкладка «Прокси и сервер» / *Proxy and Server*)
 - 13.5. Telegram-бот (вкладка «Telegram-бот» / *Telegram Bot*)
 - 13.6. Дата и время (вкладка «Дата и время» / *Date and Time*)
 - 13.7. Внешний трафик и поведение Xray (вкладка «Внешний трафик» / *External Traffic*)
 - 13.8. Прочее: шаблон конфигурации Xray и проверочный URL
 - 13.9. Учётная запись администратора и API-токены
 - 13.10. Изменения API в 3.3.0 (важно для интеграций)

- 14. Telegram-бот
 - 14.1. Включение и настройка бота
 - 14.2. Главное меню и кнопки
 - 14.3. Команды бота
 - 14.4. Управление клиентом (только администратор)
 - 14.5. Уведомления и отчёты
 - 14.6. Бэкап и логи
 - 14.7. Особенности работы
- 15. Гео-базы (geoip / geosite и кастомные)
 - 15.1. Что такое geoip.dat и geosite.dat
 - 15.2. Стандартные гео-файлы и их обновление
 - 15.3. Авто-обновление гео-данных средствами Xray (Geodata Auto-Update)
 - 15.4. Валидация и ограничения
 - 15.5. Автопроверка при старте панели
 - 15.6. Использование гео-баз в правилах маршрутизации
- 16. Эксплуатация: бэкапы, логи, обновление, CLI
 - 16.1. Резервное копирование и восстановление базы данных
 - 16.2. Просмотр логов
 - 16.3. Уровень и настройка логирования Xray
 - 16.4. Управление Xray: остановка и перезапуск
 - 16.5. Перезапуск и обновление панели
 - 16.6. Периодические задачи (cron)
 - 16.7. Консольное меню и CLI (`x-ui`)
 - 16.8. Удаление панели
 - 16.9. Команда `x-ui migrateDB`

Что нового в 3.5.0

Версия 3.5.0 — крупный релиз: MTProto переведён на мультиклиентную модель (движок mtg-multi, персональные секреты, квоты и ad-tag), управляемые хосты стали групповыми (несколько inbound и адресов в одной записи), восстановление на PostgreSQL-панели принимает SQLite-бэкапы, у outbound появились «Target Strategy», тест «Real delay» и колонки Egress/Country, балансировщик может использовать другой балансировщик как fallback. В комплекте — ядро Xray 26.7.11. Ниже — изменения относительно 3.4.2 по разделам руководства.

Изменения в разделе 1 — Введение, требования и установка

- Ядро Xray обновлено до **26.7.11**. Попутные авто-миграции: шифры Shadowsocks `none / plain` и VMess `none / zero` удалены из ядра (сохранённые конфиги переписываются автоматически), незашифрованный VLESS/Trojan-outbound на публичный адрес отклоняется при сохранении.
- Новая команда `x-ui pgclient [версия]` и пункт **10. Install/Upgrade client tools (pg_dump/pg_restore)** в меню PostgreSQL — установка/обновление клиентских инструментов PostgreSQL.
- Исправления скриптов: установка PostgreSQL и fail2ban на RHEL-семействе (EPEL), Arch без полного `распан -Syu`, корректное имя бинаря Xray на 32-битном ARM (`xray-linux-arm32`), подтверждение авто-определённого IPv4 перед выпуском IP-сертификата, исправлен ложный «Your input is invalid» при выборе ACME-порта по умолчанию.

Изменения в разделе 2 — Вход в панель и безопасность доступа

- Лимит IP: «мёртвое» соединение теперь банится **один раз**, а не при каждом 10-секундном сканировании — счётчики fail2ban больше не накручиваются, завышать `maxretry` не нужно.

Изменения в разделе 4 — Inbounds: создание и общие параметры

- В списке inbound появился **поиск** (по примечанию, порту и протоколу), а выпадающие списки узлов («Развернуть на», фильтр «Узлы») стали поисковыми.

Изменения в разделе 5 — Протоколы

- **MTProto переведён на мультиклиентную модель** (движок mtg-multi): пользователи MTProto — теперь обычные клиенты со своим секретом, квотой, сроком, ad-tag и персональной ссылкой `tg://proxy`. Поле «Secret» на уровне inbound убрано (существующие inbound автоматически конвертируются), «FakeTLS domain» стал доменом по умолчанию для новых секретов. Новые поля inbound: **Max connections** (ограничение соединений), **Public IPv4/IPv6** (для ad-tag middle proxy). Изменения клиентов применяются «на лету», не роняя чужие Telegram-сессии.
- WireGuard: меню inbound получило полный набор клиентских действий (Export All URLs, привязка/отвязка, группы), экспорт разделён на вкладки **Config** и **Links**, поле «Разрешённые IP WireGuard» стало **редактируемым** (несколько записей через запятую), в конфиге клиента узлового inbound `Endpoint` теперь указывает на адрес узла.

Изменения в разделе 7 — Безопасность соединения: TLS, XTLS и REALITY

- Комбинация **Finalmask + REALITY отклоняется** при сохранении (она приводила к падению Xray-core при первом подключении); плейсхолдер minClientVer обновлён до 26.3.27.
- Новый тип TCP-маски Finalmask — **XMC (Minecraft)**: маскировка потока под трафик Minecraft (Hostname, Usernames, обязательный Password с автогенерацией).

Изменения в разделе 8 — Клиенты

- Новая колонка **«Скорость»** — живая скорость каждого клиента (↑/↓, скользящее ~5-секундное среднее).
- Поиск клиентов снова находит по **Telegram ID**; в форме клиента выключенные inbound скрыты из списка привязки; исправлено накопление дубликатов в окне «Отвязать».
- У MTPROTO-клиентов — свои поля: **«MTPROTO secret»** (с регенерацией) и **«Ad-tag (спонсорский канал)»** (32 hex-символа); квота и срок теперь реально применяются к MTPROTO.

Изменения в разделе 9 — Группы клиентов

- В окне информации о клиенте теперь показывается его **группа**.

Изменения в разделе 10 — Подписки (Subscription)

- **Управляемые хосты стали групповыми**: одна запись охватывает **несколько inbound** (мульти-выбор) и **несколько адресов** (теги, у каждой записи возможен свой :порт ; автодополнение адресов; пусто — наследование адреса inbound). Колонки списка показывают чипы адресов и inbound (с «+N»), действия и API работают с группами (groupId), появился массовый POST /panel/api/hosts/bulk/add . Сортировка хостов теперь глобальная (по порядку, затем по примечанию).
- Текст **анонса** (subAnnounce) теперь показывается баннером на инфо-странице подписки; в кастомных шаблонах доступна переменная announce .
- Инфо-страница в браузере открывается теперь и по **JSON/Clash-ссылкам** (не только по основной).
- Host-настройки **Final Mask** и **Allow insecure** теперь действуют и в raw-ссылках (fm=) и для **Hysteria2** (insecure=1 / skip-cert-verify: true) соответственно.
- Диапазон «Интервалы обновления» (subUpdates) исправлен на **0–525600** (прежний UI-предел 168 блокировал сохранение настроек после апгрейда с 2.x).
- Нативные **WireGuard-клиенты теперь попадают в Clash- и JSON-подписки** (раньше — только в raw).

Изменения в разделе 11 — Xray: маршрутизация, outbounds, DNS и расширения

- Outbound-редактор: новое поле **«Target Strategy»** (11 значений от AsIs до ForceIPv4), режим теста **«Real delay»** (полное время с установлением туннеля; HTTP-режим теперь меряется по «тёплому» соединению), колонки **Egress** (IP выхода за «глазом») и **Country** (флаг + страна, метка WARP) после HTTP/Real-теста.
- **Fallback балансировщика может указывать на другой балансировщик**: панель сама строит скрытый loopback-объект (_bl_...), защищает от циклов и удаления используемого балансировщика; префикс _bl_ зарезервирован.

- Вкладка «Базовая маршрутизация» получила селектор **«Default Outbound»** — какой outbound обрабатывает трафик, не попавший ни под одно правило (выбранный перемещается на первую позицию).
- DNS-серверы на частных IP больше не блокируются правилом `geoip:private` — панель сама поддерживает управляемое allow-правило.
- Happy Eyeballs в настройках dial (sockopt) теперь действительно включается; «Try delay» по умолчанию 250 мс, явный 0 сохраняется.
- Импорт подписок outbound: `y ss:// -ссылка с ?plugin=/хвостовым /` корректно разбирается порт.

Изменения в разделе 12 — Узлы (мультипанель, master/slave)

- Пакет исправлений: сохранение клиента без изменений больше не рушит живой трафик узловых inbound; Host-переопределения узла принимаются в мастер при первом принятии; авто-продление открывает свежее окно квоты; удаление клиента на мастере полностью удаляет его на узлах; inbound узла не выметаются до первого принятия; один некорректный inbound не останавливает синхронизацию трафика узла; проверка конфликта портов ограничена своим узлом.

Изменения в разделе 14 — Telegram-бот

- В меню команд бота добавлены `/usage`, `/inbound`, `/restart` и новая админ-команда `/clearall` (сброс трафика всех клиентов, с подтверждением).
- Список онлайн-клиентов подписывается как `email` — примечание `inbound`; сообщения бэкапа и бан-лога содержат имя хоста; поиск по Telegram ID работает независимо от форматирования настроек.

Изменения в разделе 16 — Эксплуатация: бэкапы, логи, обновление, CLI

- **Восстановление на PostgreSQL-панели принимает SQLite-файлы:** обычный бэкап `.db` или миграционный `.dump` импортируются прямо в PostgreSQL (одной транзакцией, с проверками до остановки Xray). Диалог выбора файла принимает `.dump`, `.db` на обеих СУБД; «Скачать миграцию» остался только на PostgreSQL-панелях.
- Перед восстановлением `pg_dump` -архива панель проверяет читаемость дампа и при несовпадении версий подсказывает точную команду `x-ui pgclient <версия>`.
- Автопочинки при старте: переполненные счётчики трафика зажимаются и восстанавливаются; снимается устаревшее ограничение UNIQUE на порт inbound (мешало multi-node).
- Логи Xray: новый джоб каждые 10 минут обрезает access- и error-лог при превышении **64 МБ**; ежедневная очистка теперь чистит оба.
- Docker: автопродление сертификатов починено (crond запускается, состояние `acme.sh` сохраняется в томе).

1. Введение, требования и установка

1.1. Что такое 3X-UI

3X-UI — это открытая веб-панель управления для серверов [Xray-core](#). Панель предоставляет единый многоязычный веб-интерфейс для развёртывания, настройки и мониторинга широкого набора прокси- и VPN-протоколов: от одиночного VPS до распределённых конфигураций из нескольких узлов (нод).

3X-UI — это расширенный форк оригинального проекта X-UI. По сравнению с ним добавлены поддержка большего числа протоколов, повышенная стабильность, поучётный (по каждому клиенту) учёт трафика и множество удобных функций.

Основные возможности:

- **Inbound разных протоколов** — VLESS, VMess, Trojan, Shadowsocks, WireGuard, Hysteria2, HTTP, SOCKS (Mixed), Dokodemo-door / Tunnel, TUN и **MTPProto** (прокси Telegram, добавлен в 3.3.0).
- **Современные транспорты и шифрование** — TCP (Raw), mKCP, WebSocket, gRPC, HTTPUpgrade и XHTTP, защищаемые TLS, XTLS и REALITY.
- **Fallback** — обслуживание нескольких протоколов на одном порту (например, VLESS и Trojan на 443) средствами fallback в Xray.
- **Управление по каждому клиенту** — квоты трафика, даты истечения, лимиты IP, отображение статуса «онлайн», ссылки-приглашения в один клик, QR-коды и подписки.
- **Статистика трафика** — по каждому inbound, клиенту и outbound, с возможностью сброса.
- **Поддержка нескольких узлов (нод)** — управление и масштабирование на несколько серверов из одной панели.
- **Outbound и маршрутизация** — WARP, NordVPN, пользовательские правила маршрутизации, балансировщики нагрузки, цепочки прокси.
- **Встроенный сервер подписок** с несколькими форматами вывода.
- **Telegram-бот** для удалённого мониторинга и управления.
- **REST API** со встроенной документацией Swagger.
- **Гибкое хранилище** — SQLite (по умолчанию) или PostgreSQL.
- **13 языков интерфейса**, тёмная и светлая темы.
- **Интеграция с Fail2ban** для применения лимитов IP по каждому клиенту.

Важно: проект предназначен только для личного использования. Не рекомендуется применять его в незаконных целях или в продакшен-окружении.

1.2. Поддерживаемые операционные системы и архитектуры

Операционные системы

Установочный скрипт определяет дистрибутив по полю `ID` из `/etc/os-release` (или `/usr/lib/os-release`). Официально поддерживаются:

Ubuntu, Debian, Armbian, Fedora, CentOS, RHEL, AlmaLinux, Rocky Linux, Oracle Linux, Amazon Linux, Virtuozzo, Arch, Manjaro, Parch, openSUSE (Tumbleweed / Leap), Alpine, а также Windows.

На системах семейства Alpine используется служба OpenRC (`rc-service` / `rc-update`), на остальных – `systemd`. Для CentOS 7 пакеты устанавливаются через `yum`, для более новых выпусков – через `dnf`. Если дистрибутив не распознан, скрипт по умолчанию пытается использовать менеджер пакетов `apt-get`.

Архитектуры процессора

Архитектура определяется по выводу `uname -m` и приводится к одному из поддерживаемых значений:

Значение <code>uname -m</code>	Архитектура 3X-UI
<code>x86_64</code> , <code>x64</code> , <code>amd64</code>	<code>amd64</code>
<code>i*86</code> , <code>x86</code>	<code>386</code>
<code>armv8*</code> , <code>arm64</code> , <code>aarch64</code>	<code>arm64</code>
<code>armv7*</code> , <code>arm</code>	<code>armv7</code>
<code>armv6*</code>	<code>armv6</code>
<code>armv5*</code>	<code>armv5</code>
<code>s390x</code>	<code>s390x</code>

Если архитектура не входит в этот список, скрипт выводит сообщение «Unsupported CPU architecture!» и прекращает установку.

Базовые зависимости

Перед установкой панели скрипт автоматически устанавливает базовый набор пакетов (имена различаются по дистрибутивам): `cron` / `cronie` / `dcron`, `curl`, `tar`, `tzdata` / `timezone`, `socat`, `ca-certificates`, `openssl`.

1.3. Способы установки

Способ 1. Установочный скрипт (рекомендуется)

Установка выполняется одной командой от имени `root`:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/master/install.sh)
```

Скрипт обязательно требует прав `root`: при запуске не от `root` выводится «Please run this script with root privilege» и завершается с ошибкой.

Что делает установщик пошагово:

1. Определяет ОС и архитектуру.
2. Устанавливает базовые зависимости.

3. Скачивает архив релиза `x-ui-linux-<arch>.tar.gz` и распаковывает его в каталог `/usr/local/x-ui`.
4. Скачивает управляющий скрипт `x-ui.sh` и устанавливает его как команду `/usr/bin/x-ui`.
5. Создаёт каталог логов `/var/log/x-ui`.
6. Запускает первичную настройку: выбор базы данных, генерация учётных данных, выбор порта, опциональная настройка SSL.
7. Устанавливает и запускает службу автозапуска (systemd-юнит `x-ui.service` или init-скрипт OpenRC для Alpine).

Выбор базы данных при установке. Установщик предлагает:

- 1) SQLite (по умолчанию, рекомендуется при числе клиентов < 500) — один файл `/etc/x-ui/x-ui.db`, не требует настройки.
- 2) PostgreSQL (рекомендуется при большом числе клиентов или нескольких нодах). PostgreSQL можно установить локально (создаётся выделенный пользователь и БД с именем `xui`) либо указать DSN к уже существующему серверу. Параметры подключения записываются в файл окружения службы (`/etc/default/x-ui`, `/etc/conf.d/x-ui` или `/etc/sysconfig/x-ui` в зависимости от дистрибутива) в виде переменных `XUI_DB_TYPE` и `XUI_DB_DSN`.

Пример: запись параметров PostgreSQL в файл окружения службы. После выбора PostgreSQL и указания DSN установщик добавит в файл окружения примерно такие строки:

```
XUI_DB_TYPE=postgres
XUI_DB_DSN=postgres://xui:S3cretPass@127.0.0.1:5432/xui?sslmode=disable
```

Здесь `xui` — имя пользователя и базы, `127.0.0.1:5432` — адрес и порт сервера, `sslmode=disable` подходит для локального подключения (для удалённого сервера обычно используют `require`).

Установка конкретной (старой) версии. Можно явно указать тег версии — установщик скачает соответствующий релиз:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/v2.4.0/install.sh)
v2.4.0
```

Минимально допустимая версия для такой установки — `v2.3.5`; при указании более старой выводится «Please use a newer version (at least v2.3.5)».

Установка dev-сборки. Помимо тега версии установщик принимает аргумент `dev-latest` (псевдоним `dev`) — это устанавливает rolling dev-сборку по последнему коммиту ветки `main`:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/master/install.sh) dev-
latest
```

Dev-сборка — это per-commit предрелиз (тег `dev-latest`), а не стабильная версия, поэтому проверка минимально допустимой версии для неё не выполняется. При запуске выводится предупреждение «Installing the rolling dev build (tag: dev-latest). This is a per-commit pre-release, not a stable version.». Без аргумента установщик ставит последний стабильный релиз. Использовать dev-сборку имеет смысл только

для проверки ещё не вышедших исправлений; в обычной эксплуатации устанавливайте стабильные версии.

Способ 2. Docker

Запуск с базой SQLite по умолчанию:

```
docker compose up -d
```

Для запуска со встроенным сервисом PostgreSQL нужно раскомментировать строки `XUI_DB_*` в `docker-compose.yml` и запустить с профилем:

```
docker compose --profile postgres up -d
```

Образ включает Fail2ban (по умолчанию активен) для применения лимитов IP по клиентам. Fail2ban блокирует нарушителей через `iptables`, что требует возможности `NET_ADMIN`. В `docker-compose.yml` она уже предоставлена через `cap_add`. С 3.5.0 в контейнере починено **автопродление сертификатов**, выпущенных через SSL-меню: `entrypoint` запускает `crond` и заново регистрирует cron-задание `acme.sh`, а `docker-compose.yml` получил отдельный том для состояния `acme.sh` — продление переживает пересоздание контейнера (раньше сертификаты молча истекали через ~90 дней). При ручном запуске через `docker run` возможности нужно добавить самостоятельно, иначе блокировки будут только логироваться, но не применяться:

Пример: полная команда `docker run`. Минимальный вариант с прокидыванием порта панели, сетевыми возможностями и постоянным томом для базы:

```
docker run -d \
  --name 3x-ui \
  --restart unless-stopped \
  --cap-add=NET_ADMIN --cap-add=NET_RAW \
  -v $PWD/db:/etc/x-ui \
  -v $PWD/cert:/root/cert \
  -p 2053:2053 \
  ghcr.io/mhsanaei/3x-ui:latest
```

Том `/etc/x-ui` сохраняет файл `x-ui.db` между перезапусками контейнера, иначе настройки и аккаунты будут теряться.

```
docker run -d --cap-add=NET_ADMIN --cap-add=NET_RAW ... ghcr.io/mhsanaei/3x-ui
```

В Docker панель является основным процессом контейнера: автозапуск регулируется политикой перезапуска контейнера (например, `restart: unless-stopped`), а не службой внутри контейнера.

1.4. Первый запуск и учётные данные по умолчанию

При первой установке (когда ещё используются учётные данные по умолчанию) установщик **генерирует случайные значения** имени пользователя, пароля и веб-пути, а также порта:

Параметр	Как формируется при установке	Примечание
Имя пользователя (Username)	случайная строка из 10 символов	генерируется автоматически
Пароль (Password)	случайная строка из 10 символов	генерируется автоматически
Веб-путь панели (WebBasePath)	случайная строка из 18 символов	защищает панель от обнаружения по корневому URL
Порт панели (Port)	по умолчанию случайный порт в диапазоне 1024–62000; при желании можно задать вручную	«заводское» значение <code>webPort</code> равно <code>2053</code> , но установщик его перезаписывает

В конце установки скрипт выводит итоговую сводку: имя пользователя, пароль, порт, веб-путь, API-токен и готовую ссылку для входа (Access URL) вида:

```
https://<домен-или-IP>:<порт>/<веб-путь>
```

Если SSL-сертификат не настроен, ссылка будет по `http://`, и скрипт выведет предупреждение о необходимости настроить SSL (пункт меню 19).

Обязательная смена учётных данных. Поскольку логин и пароль генерируются случайно, их следует **сохранить сразу после установки**. Сменить их в любой момент можно пунктом меню «Reset Username & Password» (см. ниже) либо из веб-интерфейса в настройках панели. После сброса скрипт напоминает: «Please use the new login username and password to access the X-UI panel. Also remember them!».

После установки для открытия меню управления используется команда `x-ui` (см. раздел 1.6).

1.5. Расположение файлов

Путь	Назначение
<code>/usr/local/x-ui/</code>	каталог установки панели (бинарь <code>x-ui</code> , скрипт <code>x-ui.sh</code>)
<code>/usr/local/x-ui/bin/xray-linux-<arch></code>	бинарь Xray-core (на armv5/armv6/armv7 переименовывается в <code>xray-linux-arm</code>)
<code>/usr/bin/x-ui</code>	управляющий скрипт (команда <code>x-ui</code>)
<code>/etc/x-ui/x-ui.db</code>	файл базы данных SQLite (по умолчанию)
<code>/var/log/x-ui/</code>	каталог логов панели
<code>/etc/systemd/system/x-ui.service</code>	systemd-юнит службы (не для Alpine)
<code>/etc/init.d/x-ui</code>	init-скрипт OpenRC (только Alpine)
<code>/etc/default/x-ui</code> · <code>/etc/conf.d/x-ui</code> · <code>/etc/sysconfig/x-ui</code>	файл переменных окружения службы (путь зависит от дистрибутива); сюда пишутся <code>XUI_DB_TYPE</code> / <code>XUI_DB_DSN</code>

Каталог базы данных можно переопределить переменной окружения `XUI_DB_FOLDER` (по умолчанию `/etc/x-ui`), а каталог бинарей Xray — переменной `XUI_BIN_FOLDER` (по умолчанию `bin` относительно каталога панели). Имя файла БД — `x-ui.db`.

Пример: вынос базы на отдельный диск. Чтобы хранить `x-ui.db` не в `/etc/x-ui`, а, например, на смонтированном диске `/data`, задайте переменную в файле окружения службы и перезапустите панель:

```
echo 'XUI_DB_FOLDER=/data/x-ui' >> /etc/default/x-ui
mkdir -p /data/x-ui
systemctl restart x-ui
```

Полный путь к базе станет `/data/x-ui/x-ui.db`.

Основные переменные окружения

Переменная	Назначение	По умолчанию
<code>XUI_DB_TYPE</code>	бэкенд БД: <code>sqlite</code> или <code>postgres</code>	<code>sqlite</code>
<code>XUI_DB_DSN</code>	строка подключения PostgreSQL (при <code>XUI_DB_TYPE=postgres</code>)	—
<code>XUI_DB_FOLDER</code>	каталог файла БД SQLite	<code>/etc/x-ui</code>
<code>XUI_INIT_WEB_BASE_PATH</code>	начальный URI-путь веб-панели (только при первой инициализации)	<code>/</code>
<code>XUI_DB_MAX_OPEN_CONNS</code>	максимум открытых соединений (пул PostgreSQL)	—
<code>XUI_DB_MAX_IDLE_CONNS</code>	максимум простаивающих соединений (пул PostgreSQL)	—
<code>XUI_ENABLE_FAIL2BAN</code>	включить применение лимитов IP через Fail2ban	<code>true</code>
<code>XUI_LOG_LEVEL</code>	уровень логирования (<code>debug</code> , <code>info</code> , <code>warning</code> , <code>error</code>)	<code>info</code>
<code>XUI_DEBUG</code>	режим отладки	<code>false</code>

Пример: временно включить подробное логирование. Для диагностики проблемы поднимите уровень логов до `debug` и перезапустите службу:

```
echo 'XUI_LOG_LEVEL=debug' >> /etc/default/x-ui
systemctl restart x-ui
x-ui log # просмотр отладочного лога
```

После диагностики верните значение `info`, чтобы лог не разрастался.

Начальный путь веб-панели через окружение. Переменная `XUI_INIT_WEB_BASE_PATH` задаёт URI-путь веб-панели (`webBasePath`) при первичной инициализации настроек. Это удобно при развёртывании в Docker или через systemd, чтобы сразу зафиксировать путь входа в панель. Значение нормализуется автоматически — ведущий и замыкающий слэши добавляются при необходимости, а пустое или состоящее из пробелов значение игнорируется (тогда применяется путь по умолчанию `/`). Переменная влияет **только на первичную инициализацию**: если настройки уже созданы, путь меняется в веб-интерфейсе или пунктом меню «Reset Web Base Path».

1.6. Команда управления `x-ui` (меню скрипта)

После установки команда `x-ui` (запускается от `root`) открывает интерактивное меню «3X-UI Panel Management Script». Выбор пункта производится вводом его номера (диапазон 0–27). Многие пункты доступны и в виде подкоманд для скриптов (см. раздел 1.7).

Меню разбито на тематические блоки.

Установка и обновление

- **1. Install** — установка панели (запускает `install.sh`). Перед установкой проверяется, что панель ещё не установлена.
- **2. Update** — обновление всех компонентов `x-ui` до последней версии. Данные при этом не теряются; после обновления панель перезапускается автоматически. Требуется подтверждения.
- **3. Update Menu** — обновление только управляющего скрипта (`x-ui.sh` / команды `x-ui`) до актуальной версии без переустановки панели.
- **4. Legacy Version** — установка указанной (старой) версии панели. Скрипт запрашивает номер версии (например, `2.4.0`) и скачивает соответствующий релиз.
- **5. Uninstall** — полное удаление панели **вместе с Xray**. Останавливается и отключается служба, удаляются каталоги `/etc/x-ui/` и `/usr/local/x-ui/`, файл окружения службы и сам управляющий скрипт. Требуется подтверждения (по умолчанию «нет»).

Учётные данные и настройки

- **6. Reset Username & Password** — сброс имени пользователя и пароля панели. Можно ввести свои значения или оставить пустыми для случайной генерации (случайное имя — 10 символов, случайный пароль — 18 символов). Дополнительно предлагается отключить двухфакторную аутентификацию (2FA), если она настроена. После сброса панель перезапускается.
- **7. Reset Web Base Path** — сброс веб-пути панели: генерируется новый случайный путь (18 символов), панель перезапускается. Используется, если прежний путь скомпрометирован или забыт.
- **8. Reset Settings** — сброс всех настроек панели к значениям по умолчанию. **Учётные данные (имя пользователя и пароль) и данные аккаунтов при этом не теряются.** Требуется подтверждения; после сброса панель перезапускается.
- **9. Change Port** — смена порта веб-панели. Запрашивается номер порта (1–65535); после установки требуется перезапуск, чтобы порт вступил в силу.
- **10. View Current Settings** — просмотр текущих настроек (`x-ui setting -show`). Показывает в том числе используемый бэкенд БД (SQLite или PostgreSQL с замаскированным паролем в DSN) и готовую ссылку доступа (Access URL). Если SSL не настроен, предлагает выпустить сертификат Let's Encrypt для IP-адреса.

Управление службой

- **11. Start** — запуск службы панели. Если панель уже работает, выводится сообщение, что повторный запуск не нужен.
- **12. Stop** — остановка службы панели.
- **13. Restart** — перезапуск службы панели.

- **14. Restart Xray** — перезапуск только ядра Xray-core без перезапуска самой панели (через `systemctl reload x-ui`, в Docker — сигналом `USR1` процессу панели).
- **15. Check Status** — проверка состояния службы (`systemctl status x-ui` или `rc-service x-ui status`).
- **16. Logs Management** — управление логами: просмотр отладочного лога (Debug Log, через `journalctl`) и, кроме Alpine, очистка всех логов (Clear All logs).

Автозапуск

- **17. Enable Autostart** — включить автозапуск панели при загрузке ОС (`systemctl enable x-ui` или `rc-update add`).
- **18. Disable Autostart** — отключить автозапуск при загрузке ОС.

В Docker автозапуск регулируется политикой перезапуска контейнера, поэтому эти пункты лишь выводят соответствующую подсказку.

Безопасность и сеть

- **19. SSL Certificate Management** — управление SSL-сертификатами через `acme.sh`: выпуск сертификата для домена, отзыв, принудительное обновление, просмотр существующих доменов, указание путей к сертификату для панели, а также выпуск короткоживущего (~6 дней, с автообновлением) сертификата для IP-адреса.
- **20. Cloudflare SSL Certificate** — выпуск SSL-сертификата через DNS-валидацию Cloudflare.
- **21. IP Limit Management** — управление лимитами числа IP по клиентам (на базе Fail2ban): просмотр и снятие блокировок и т. п.
- **22. Firewall Management** — управление межсетевым экраном (открытие/закрытие портов и просмотр правил).
- **23. SSH Port Forwarding Management** — настройка проброса SSH-портов, чтобы открывать панель с локальной машины через SSH-туннель.

Производительность и обслуживание

- **24. Enable BBR** — включение/отключение алгоритма управления перегрузкой TCP BBR (подменю с пунктами Enable BBR / Disable BBR).
- **25. Update Geo Files** — обновление geo-баз (файлы `.dat`) с выбором источника: Loyalsoldier (`geoip.dat` , `geosite.dat`), chocolate4u (`geoip_IR.dat` , `geosite_IR.dat`), runetfreedom (`geoip_RU.dat` , `geosite_RU.dat`) или All (все сразу). После обновления панель перезапускается.
- **26. Speedtest by Ookla** — запуск теста скорости сети через Speedtest by Ookla.
- **27. PostgreSQL Management** — управление встроенным/связанным экземпляром PostgreSQL (включение и сопутствующие операции).
- **0. Exit Script** — выход из меню.

1.7. Подкоманды `x-ui` (без интерактивного меню)

Для использования в скриптах команда `x-ui` поддерживает прямые подкоманды (вывод `x-ui` без аргументов открывает меню):

Команда	Действие
<code>x-ui</code>	открыть меню управления
<code>x-ui start</code>	запустить панель
<code>x-ui stop</code>	остановить панель
<code>x-ui restart</code>	перезапустить панель
<code>x-ui restart-xray</code>	перезапустить Xray
<code>x-ui status</code>	текущее состояние службы
<code>x-ui settings</code>	текущие настройки
<code>x-ui enable</code>	включить автозапуск при загрузке ОС
<code>x-ui disable</code>	отключить автозапуск
<code>x-ui log</code>	просмотр логов
<code>x-ui banlog</code>	просмотр логов блокировок Fail2ban
<code>x-ui update</code>	обновить панель
<code>x-ui update-all-geofiles</code>	обновить все гео-файлы
<code>x-ui migrateDB [file]</code>	конвертация <code>.db</code>  <code>.dump</code> (SQLite)
<code>x-ui legacy</code>	установить старую версию
<code>x-ui install</code>	установить панель
<code>x-ui uninstall</code>	удалить панель

1.8. Миграция SQLite → PostgreSQL

Существующую установку на SQLite можно перенести на PostgreSQL:

```
x-ui migrate-db --dsn "postgres://xui:password@127.0.0.1:5432/xui?sslmode=disable"
# затем задать XUI_DB_TYPE и XUI_DB_DSN в /etc/default/x-ui и перезапустить:
systemctl restart x-ui
```

Исходный файл SQLite остаётся нетронутым — удалите его вручную только после проверки работы нового бэкенда.

Пример: проверка переключения на PostgreSQL. После миграции убедитесь, что панель действительно работает на новом бэкенде, командой просмотра настроек — в выводе должен быть указан PostgreSQL (пароль в DSN маскируется):

```
x-ui settings | grep -i -E 'db|dsn'
```

Если панель открывается, а аккаунты на месте, исходный `x-ui.db` можно удалить.

2. Вход в панель и безопасность доступа

Этот раздел описывает всё, что относится к аутентификации администратора панели 3X-UI: форму входа, двухфакторную аутентификацию (TOTP), защиту от перебора пароля, смену учётных данных, изменение секретного пути и порта панели, время жизни сессии, а также синхронизацию/аутентификацию через LDAP.

2.1. Форма входа

Страница входа отдаётся по корню секретного пути панели (`webBasePath`). Если пользователь уже авторизован, он автоматически перенаправляется на `.../panel/`. На странице есть переключатель темы, выбор языка интерфейса и собственно форма.

Поля формы:

Поле	Подсказка/заголовок (RU)	Обязательно	Описание
Имя пользователя	«Имя пользователя»	Да	Логин администратора. Пустое значение отклоняется ещё на клиенте, а на сервере — сообщением «Введите имя пользователя».
Пароль	«Пароль»	Да	Пароль администратора. Пустое значение отклоняется сообщением «Введите пароль».
Код 2FA	«Код 2FA»	Только при включённой 2FA	Поле появляется только если на панели включена двухфакторная аутентификация. 6-значный код из приложения-аутентификатора.

Кнопка «**Войти**» отправляет форму на `POST /login`.

Поведение и сообщения:

- При успешном входе показывается «Вход выполнен успешно» и происходит переход на `.../panel/`.
- При любой ошибке учётных данных или неверном коде 2FA сервер возвращает **единое** сообщение: «Неверные данные учетной записи.» (англ.: *Invalid username or password or two-factor code.*). Это сделано намеренно — панель не подсказывает, что именно неверно (логин, пароль или код), чтобы не облегчать перебор.
- Поле «Код 2FA» панель показывает или скрывает на основании запроса `POST /getTwoFactorEnable`, который возвращает текущий статус 2FA ещё до авторизации.
- Если истекла серверная сессия, при следующем запросе показывается «Сессия истекла. Войдите в систему снова», и пользователь перенаправляется на страницу входа.

Примечание о CSRF: перед отправкой формы клиент получает CSRF-токен (`GET /csrf-token`); запросы `/login` и `/logout` защищены CSRF-проверкой.

Пример: вход через API. Когда 2FA выключена, достаточно логина и пароля; при включённой 2FA добавляется поле `twoFactorCode` :

```
# Без 2FA
curl -i -X POST https://panel.example.com:2053/мой-секрет/login \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'username=admin&password=ВашПароль'

# С включённой 2FA — добавляется 6-значный код
curl -i -X POST https://panel.example.com:2053/мой-секрет/login \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'username=admin&password=ВашПароль&twoFactorCode=123456'
```

При успехе сервер вернёт `Set-Cookie` с сессионной cookie — её и нужно передавать в последующих запросах к `/panel/api/...`.

2.2. Двухфакторная аутентификация (2FA / TOTP)

2FA в 3X-UI реализована по стандарту **TOTP** и совместима с любым приложением-аутентификатором (Google Authenticator, Aegis, FreeOTP и т.п.). Параметры жёстко заданы: алгоритм **SHA1**, **6** цифр, период **30** секунд, эмитент (issuer) `3x-ui`, метка `Administrator`.

Пример: otpauth-URI, который кодирует QR-код. Если приложение-аутентификатор не сканирует камеру, токен можно добавить вручную по такой ссылке (подставьте свой Base32-секрет вместо `JBSWY3DPEHPK3ZPXP`):

```
otpauth://totp/3x-ui:Administrator?secret=JBSWY3DPEHPK3ZPXP&issuer=3x-
ui&algorithm=SHA1&digits=6&period=30
```

Параметры `algorithm=SHA1`, `digits=6`, `period=30` соответствуют жёстко заданным значениям панели — менять их не нужно.

Настройки находятся в разделе **Настройки** → **Учетная запись**, вкладка **«Двухфакторная аутентификация»**.

Элемент	Текст (RU)	Описание
Переключатель	«Включить 2FA»	Включает/отключает двухфакторную аутентификацию.
Описание	«Добавляет дополнительный уровень аутентификации для повышения безопасности.»	Подсказка под переключателем.

Как включить 2FA

При включении переключателя панель **локально генерирует новый секрет** — случайную строку в кодировке Base32 (алфавит `A–Z` и `2–7`). Открывается окно «Включить двухфакторную аутентификацию» с пошаговой инструкцией:

1. **«Отсканируйте этот QR-код в приложении для аутентификации или скопируйте токен рядом с QR-кодом и вставьте его в приложение».** Под QR-кодом отображается сам секрет в текстовом виде — по клику на QR-код секрет копируется в буфер обмена (всплывает «Скопировано»).
2. **«Введите код из приложения»** — нужно ввести 6-значный код, который сгенерировало приложение. Код проверяется **на стороне браузера**: панель сама вычисляет текущий TOTP по только что

сгенерированному секрету и сравнивает с введённым. Если код неверный — «Неверный код»; поле принимает только ровно 6 цифр.

Только после успешного подтверждения секрет и флаг включения сохраняются. При сохранении показывается «Двухфакторная аутентификация была успешно установлена».

Важно: изменения в разделе настроек применяются общей кнопкой **«Сохранить»**, после чего, как правило, требуется перезапуск панели («Сохраните изменения и перезапустите панель для их применения»). При первом включении 2FA сервер дополнительно **инвалидирует все активные сессии** (увеличивает «login erosch»), поэтому после применения настройки потребуются повторный вход — уже с кодом 2FA.

Как отключить 2FA

Повторное нажатие переключателя открывает окно «Отключить двухфакторную аутентификацию» с подсказкой «Введите код из приложения, чтобы отключить двухфакторную аутентификацию». После ввода верного кода (с 3.4.2 он проверяется **на сервере**) флаг и секрет очищаются, показывается «Двухфакторная аутентификация была успешно удалена».

Проверка кода при входе

При входе сервер берёт сохранённый секрет и сравнивает текущий TOTP с присланным кодом 2FA. Несовпадение трактуется как неуспешный вход, но пользователю показывается то же объединённое сообщение «Неверные данные учетной записи.».

Восстановление доступа (recovery)

Отдельного механизма «кодов восстановления» в 3X-UI **нет**. Если доступ к приложению-аутентификатору потерян, восстановить вход через интерфейс панели нельзя. Единственный путь — отключить 2FA напрямую в базе данных на сервере: сбросить ключ `twoFactorEnable` в `false` (и при необходимости очистить `twoFactorToken`) в таблице настроек, после чего перезапустить панель. Поэтому секрет (Base32-токен) при включении 2FA рекомендуется сохранить в надёжном месте.

Пример: аварийное отключение 2FA на сервере. Получив доступ к серверу по SSH, остановите панель, сбросьте ключи в таблице настроек и запустите панель снова:

```
x-ui stop
sqlite3 /etc/x-ui/x-ui.db "UPDATE settings SET value='false' WHERE
key='twoFactorEnable';"
sqlite3 /etc/x-ui/x-ui.db "UPDATE settings SET value='' WHERE key='twoFactorToken';"
x-ui start
```

После этого вход выполняется только по логину и паролю, а 2FA при желании настраивается заново.

Связь со сменой учётных данных: при смене логина/пароля (см. 2.4) 2FA **автоматически отключается** на сервере, чтобы старый секрет не блокировал доступ под новой учётной записью.

2.3. Ограничение попыток входа (login limiter / защита от перебора)

Панель содержит встроенный лимитер неудачных входов (аналог fail2ban на уровне приложения).
 Параметры заданы в коде и **не настраиваются** через интерфейс:

Параметр	Значение	Назначение
Максимум неудач	5	Сколько неудачных попыток допускается в пределах окна.
Окно подсчёта	5 минут	Скользящее окно, в котором накапливаются неудачи (более старые отбрасываются).
Блокировка (cooldown)	15 минут	На сколько блокируется ключ после превышения порога.

Как это работает:

- Ключ блокировки строится из **связки «IP + логин»** (логин приводится к нижнему регистру, пробелы обрезаются). То есть блокировка применяется к конкретной паре «адрес + имя пользователя», а не ко всей панели.
- При каждой неудачной попытке (неверные логин/пароль или неверный код 2FA) счётчик растёт. Достигнув **5** неудач за **5 минут**, ключ блокируется на **15 минут**. Во время блокировки любые попытки этой пары немедленно отклоняются тем же сообщением «Неверные данные учетной записи.», даже если данные верны.
- **Успешный вход немедленно сбрасывает** счётчик и снимает блокировку для этой пары.
- IP-адрес клиента определяется с учётом доверенных прокси (см. `trustedProxyCIDRs`): заголовки `X-Real-IP` и `X-Forwarded-For` принимаются только если запрос пришёл с доверенного адреса. Иначе используется реальный адрес соединения, а при невозможности его извлечь — строка `unknown`.

Все попытки логируются. Для неудачных пишется предупреждение в лог сервера с именем пользователя, IP, причиной и, при блокировке, временем `blocked_until`. Если включено уведомление о входе через Telegram-бота (`tgNotifyLogin` — «Уведомление о входе»), администратор дополнительно получает имя пользователя, IP и время как успешных, так и неуспешных и заблокированных попыток.

Пример: уведомление о входе в Telegram. При включённом `tgNotifyLogin` после каждой попытки администратор получает сообщение примерно такого вида:

```
Уведомление о входе
Пользователь: admin
IP: 203.0.113.45
Время: 2026-06-10 14:32:07
Статус: успешно
```

Для заблокированной пары «IP + логин» в статусе будет указано, что попытка отклонена лимитером.

2.4. Смена логина и пароля администратора

Раздел **Настройки** → **Учетная запись**, вкладка «**Учетные данные администратора**». Поля:

Поле	Текст (RU)	Описание
Текущий логин	«Текущий логин»	Действующее имя пользователя. Должно совпасть с текущим логином, иначе изменение отклоняется.
Текущий пароль	«Текущий пароль»	Действующий пароль для подтверждения личности.
Новый логин	«Новый логин»	Новое имя пользователя. Не может быть пустым.
Новый пароль	«Новый пароль»	Новый пароль. Не может быть пустым.

Изменение применяется кнопкой **«Подтвердить»** и отправляется на `POST /panel/setting/updateUser`.

Логика и сообщения сервера:

- Если «Текущий логин» не совпадает с реальным или «Текущий пароль» неверен — «Произошла ошибка при изменении учетных данных администратора.» с пояснением «Неверное имя пользователя или пароль».
- Если новый логин или новый пароль пуст — пояснение «Новое имя пользователя и новый пароль должны быть заполнены».
- При успехе — «Вы успешно изменили учетные данные администратора.». Пароль хранится в виде bcrypt-хеша.

Подтверждение кодом 2FA (с 3.4.2). Если на панели включена двухфакторная аутентификация, смена логина/пароля дополнительно требует **ввести текущий код** из приложения-аутентификатора. Открывается окно **«Смена учётных данных»** с подсказкой «Введите код из приложения, чтобы изменить учётные данные администратора.»; код (`twoFactorCode`) проверяется на сервере, при неверном или пустом коде изменение не применяется. То же подтверждение теперь требуется и при отключении 2FA (см. 2.2).

Пример: смена учётных данных через API. Запрос требует действующей сессионной cookie (получается при входе) и подтверждения текущих логина/пароля:

```
curl -X POST https://panel.example.com:2053/мой-секрет/panel/setting/updateUser \
  -b 'session=ВАША_СЕССИОННАЯ_COOKIE' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'oldUsername=admin&oldPassword=СтарыйПароль&newUsername=root&newPassword=НовыйСложныйПароль'
```

После успеха текущая сессия аннулируется — потребуются войти заново уже с новыми данными.

Важные эффекты смены учётных данных:

- **Все существующие сессии аннулируются** (увеличивается счётчик `login_epoch` у пользователя), поэтому после смены панель автоматически выполняет выход и перенаправляет на страницу входа — нужно войти заново.
- Если на момент смены была включена **2FA, она автоматически отключается** (флаг и секрет сбрасываются). Двухфакторную аутентификацию после смены логина/пароля придётся настроить заново.

Если 2FA включена, перед отправкой формы открывается окно «Изменить учетные данные» с подсказкой «Введите код из приложения, чтобы изменить учетные данные администратора.» – менять учётные данные можно только подтвердив текущий код 2FA.

2.5. Секретный путь (URI-путь / webBasePath) и порт панели

Эти параметры находятся в разделе **Настройки** → **Панель** и напрямую влияют на «скрытность» и доступность панели. Применяются после сохранения и **перезапуска панели**.

Поле	Текст (RU)	Значение по умолчанию	Описание
Порт панели	«Порт панели» (<code>panelPort</code>), подсказка «Порт, на котором работает панель»	2053	TCP-порт веб-интерфейса.
URI-путь	«URI-путь» (<code>panelUrlPath</code>), подсказка «Должен начинаться с '/' и заканчиваться '/'»	/	Секретный базовый путь (<code>webBasePath</code>). Панель доступна только по нему (например, <code>/мой-секрет/</code>).
IP-адрес для управления панелью	«IP-адрес для управления панелью» (<code>panelListeningIP</code>), подсказка «Оставьте пустым для подключения с любого IP»	пусто	Адрес, на котором слушает панель. Пусто = все интерфейсы.
Домен панели	«Домен панели» (<code>panelListeningDomain</code>), подсказка «Оставьте пустым для подключения с любых доменов и IP»	пусто	Ограничение доступа по домену (Host).
Путь к публичному ключу сертификата панели	<code>publicKeyPath</code> , подсказка «Введите полный путь, начинающийся с '/'»	пусто	Сертификат TLS для HTTPS-доступа к панели.
Путь к приватному ключу сертификата панели	<code>privateKeyPath</code> , та же подсказка	пусто	Приватный ключ TLS.

Поведение базового пути (`webBasePath`):

- Значение нормализуется автоматически: если не начинается с `/` , символ добавляется в начало; если не заканчивается на `/` , добавляется в конец. То есть фактически путь всегда вида `/.../` .
- Базовый путь применяется к самой панели, к ассетам и к cookie сессии (cookie выдаётся только для этого пути).

Рекомендации безопасности (раздел «Предупреждения безопасности»): панель сама показывает предупреждения, если конфигурация «слишком публична»:

- «Панель работает по обычному HTTP – настройте TLS для продакшна.»
- «Стандартный порт 2053 широко известен – измените его на случайный.»

- «Базовый путь по умолчанию "/" широко известен — измените его на случайный.»

Иными словами, для боевого сервера следует задать **нестандартный порт, нетривиальный URI-путь и TLS-сертификат**.

Пример: «скрытная» конфигурация панели для продакшна. В разделе **Настройки** → **Панель** задайте примерно такие значения:

Поле	Значение
Порт панели	34571 (случайный, вместо 2053)
URI-путь	/aXf9Qm2/ (нетривиальный, начинается и заканчивается на /)
Путь к публичному ключу сертификата панели	/etc/letsencrypt/live/panel.example.com/fullchain.pem
Путь к приватному ключу сертификата панели	/etc/letsencrypt/live/panel.example.com/privkey.pem

После сохранения и перезапуска панель будет доступна только по `https://panel.example.com:34571/aXf9Qm2/`, а предупреждения безопасности исчезнут.

2.6. Время жизни сессии (таймаут)

Поле **«Продолжительность сессии»** (`sessionMaxAge`) находится среди настроек панели/интервалов.

Поле	Текст (RU)	Значение по умолчанию	Единица	Описание
Продолжительность сессии	«Продолжительность сессии», подсказка «Продолжительность сессии в системе (значение: минута)»	360	минуты	Срок жизни cookie-сессии администратора.

Поведение:

- Значение задаётся в **минутах** (по умолчанию 360 минут = 6 часов) и при настройке cookie переводится в секунды.
- Если значение **больше 0**, у cookie сессии выставляется соответствующий `MaxAge`. По истечении этого срока cookie перестает действовать и при следующем запросе пользователь получает «Сессия истекла. Войдите в систему снова».
- Сессия также становится недействительной досрочно при смене учётных данных или первом включении 2FA (за счёт механизма `login_epoch`, см. 2.4 и 2.2) и при явном выходе (`POST /logout`).
- Cookie сессии помечается `HttpOnly`, с политикой `SameSite=Lax`; флаг `Secure` выставляется при прямом HTTPS-доступе к панели.

Помимо самого таймаута есть связанное уведомление: **«Задержка уведомления об истечении сессии»** (`expireTimeDiff`, подсказка «Получение уведомления об истечении срока действия сессии до достижения порогового значения (значение: день)», по умолчанию `0`) — позволяет получать предупреждение заранее.

2.7. LDAP (синхронизация и аутентификация)

Раздел LDAP даёт две возможности: (1) аутентифицировать вход администратора по LDAP, если локальный пароль не подошёл, и (2) периодически синхронизировать состояние клиентов (включён/выключен VLESS-флаг) из каталога.

Как используется при входе: сервер сначала проверяет локальный bcrypt-хеш пароля. Если он **не совпал** и LDAP включён, панель пытается аутентифицировать пользователя в каталоге: при заданном **Bind DN** выполняется служебный bind, затем по фильтру и атрибуту ищется запись пользователя, и делается попытка bind под найденным DN с введённым паролем. Успех означает вход. (После успешной LDAP-аутентификации, если включена 2FA, всё равно проверяется код TOTP.)

Поля раздела:

Поле	Текст (RU)	Значение по умолчанию	Описание
Включить LDAP-синхронизацию	«Включить LDAP-синхронизацию» (<code>enable</code>)	false	Главный выключатель LDAP-интеграции.
LDAP-хост	«LDAP-хост» (<code>host</code>)	пусто	Адрес LDAP-сервера.
Порт LDAP	«Порт LDAP» (<code>port</code>)	389	Порт. Для LDAPS обычно 636.
Использовать TLS (LDAPS)	«Использовать TLS (LDAPS)» (<code>useTls</code>)	false	При включении используется схема <code>ldaps://</code> (по умолчанию — с проверкой сертификата сервера).
Пропускать проверку TLS-сертификата	«Пропускать проверку TLS-сертификата» (<code>ldapInsecureSkipVerify</code>)	false	Добавлено в 3.4.2. Отключает проверку сертификата сервера при LDAPS — для внутренних/самоподписанных CA. Подсказка: «Небезопасно — отключает проверку сертификата сервера. Используйте только с внутренними/недоверенными CA». Переключатель доступен только при включённом «Использовать TLS (LDAPS)».
Bind DN	«Bind DN» (<code>bindDn</code>)	пусто	DN служебной учётной записи для первичного bind/поиска. Если пусто — bind не выполняется (анонимный поиск).
Пароль bind	подсказки: «Настроено; оставьте пустым, чтобы сохранить текущий пароль.» / «Не настроено.» / «Настроено — введите новое значение для замены»	пусто	Пароль для Bind DN . Хранится отдельно; чтобы оставить прежний, поле оставляют пустым.

Поле	Текст (RU)	Значение по умолчанию	Описание
Base DN	«Base DN» (<code>baseDn</code>)	пусто	Корень поддерева, в котором ведётся поиск (поиск рекурсивный, по всему поддереву).
Фильтр пользователя	«Фильтр пользователя» (<code>userFilter</code>)	(<code>objectClass=person</code>)	LDAP-фильтр выбора учётных записей. При аутентификации логин подставляется в фильтр с экранированием.
Атрибут пользователя (username/ email)	«Атрибут пользователя (username/ email)» (<code>userAttr</code>)	<code>mail</code>	Атрибут, сопоставляемый с логином/идентификатором клиента (например, <code>mail</code> или <code>uid</code>).
Атрибут VLESS-флага	«Атрибут VLESS-флага» (<code>vlessField</code>)	<code>vless_enabled</code>	Атрибут, определяющий, должен ли VLESS-доступ клиента быть включён.
Общий атрибут флага (опц.)	«Общий атрибут флага (опц.)» (<code>flagField</code>), подсказка «Если задано, переопределяет флаг VLESS – напр. <code>shadowInactive</code> .»	пусто	Если задан, используется вместо <code>vless_enabled</code> .
Truthy-значения	«Truthy-значения» (<code>truthyValues</code>), подсказка «Через запятую; по умолчанию: <code>true,1,yes,on</code> »	<code>true,1,yes,on</code>	Список значений атрибута флага, трактуемых как «включено».
Инвертировать флаг	«Инвертировать флаг» (<code>invertFlag</code>), подсказка «Включите, когда атрибут означает «отключено» (напр. <code>shadowInactive</code>).»	<code>false</code>	Инвертирует смысл флага.
Расписание синхронизации	«Расписание синхронизации» (<code>syncSchedule</code>), подсказка «Строка типа <code>cron</code> , напр. <code>@every 1m</code> »	<code>@every 1m</code>	Периодичность синхронизации в cron-подобном формате.
Теги входящих	«Теги входящих» (<code>inboundTags</code>), подсказка «Входящие, на которых LDAP-синхронизация может авто-создавать или авто-удалять клиентов.»	пусто	Ограничивает, на каких <code>inbound</code> разрешены авто-операции. Если <code>inbound</code> нет: «Входящие не найдены. Сначала создайте входящий.»
Авто-создание клиентов	«Авто-создание клиентов» (<code>autoCreate</code>)	<code>false</code>	Создавать клиента в указанных <code>inbound</code> , если он появился в каталоге.
Авто-удаление клиентов	«Авто-удаление клиентов» (<code>autoDelete</code>)	<code>false</code>	Удалять клиента, если он исчез из каталога.
Объём по умолчанию (ГБ)	«Объём по умолчанию (ГБ)» (<code>defaultTotalGb</code>)	<code>0</code>	Лимит трафика для авто-создаваемых клиентов (0 = без лимита).

Поле	Текст (RU)	Значение по умолчанию	Описание
Срок по умолчанию (дни)	«Срок по умолчанию (дни)» (defaultExpiryDays)	0	Срок действия для авто-создаваемых клиентов (0 = бессрочно).
Лимит IP по умолчанию	«Лимит IP по умолчанию» (defaultIpLimit)	0	Ограничение числа одновременных IP (0 = без ограничения).

Особенности логики флага синхронизации: при чтении атрибута флага (`flagField` , по умолчанию `vless_enabled`) значение считается «включено», если оно входит в список `truthy`-значений; при включённой инверсии результат меняется на противоположный. Атрибут пользователя (`userAttr`) используется как ключ сопоставления (`email/имя`) — записи без значения этого атрибута пропускаются.

Безопасность: рекомендуется включать **TLS (LDAPS)**, чтобы пароли bind и проверяемые пароли не передавались открытым текстом, а для Bind DN использовать учётную запись с минимально необходимыми правами на чтение.

Пример: типовая конфигурация LDAP-синхронизации (Active Directory). Заполнение полей раздела для каталога, где статус доступа хранится в атрибуте `userAccountControl` -подобного флага, а сопоставление идёт по почте:

Поле	Значение
LDAP-хост	<code>ldap.example.com</code>
Порт LDAP	<code>636</code>
Использовать TLS (LDAPS)	включено
Bind DN	<code>CN=svc-3xui,OU=Service,DC=example,DC=com</code>
Base DN	<code>OU=Users,DC=example,DC=com</code>
Фильтр пользователя	<code>(objectClass=person)</code>
Атрибут пользователя (username/email)	<code>mail</code>
Атрибут VLESS-флага	<code>vless_enabled</code>
Truthy-значения	<code>true,1,yes,on</code>
Расписание синхронизации	<code>@every 5m</code>

При такой настройке каждые 5 минут панель пройдёт по поддереву `OU=Users` , сопоставит клиентов по `mail` и включит/выключит VLESS-доступ по значению `vless_enabled` .

3. Обзор / Дашборд

Дашборд («Дашборд», в английском интерфейсе — *Overview*) — это стартовая страница панели. Она в режиме реального времени показывает состояние сервера и процесса Xray. Все показатели приходят со стороны сервера. Фоновый планировщик пересобирает снимок **каждые 2 секунды** и рассылает его всем открытым вкладкам через WebSocket; раз в минуту накопленные ряды метрик сбрасываются на диск. HTTP-эндпоинт `GET /status` отдаёт последний кэшированный снимок.

Ниже разобран каждый показатель и каждый элемент управления на странице.

В боковом меню панели (и в мобильной шторке) с версии 3.4.2 есть кнопка **«Документация»** (иконка книги) — открывает официальную документацию <https://docs.sanaei.dev/>. Также в 3.4.2 проведено сквозное улучшение **доступности**: значки-кнопки и кликабельные элементы получили aria-подписи и активацию по Enter/Space (для скринридеров и навигации с клавиатуры).

3.1. Общие принципы сбора данных

- Снимок собирается библиотекой `gopsutil`. Если конкретный замер не удался, поле остаётся нулевым, а в лог пишется предупреждение (`get cpu percent failed`, `get uptime failed` и т.п.) — это не валит весь дашборд, просто соответствующая плитка покажет 0/N-A.
- «Мгновенные» скорости (CPU %, сеть, диск I/O) вычисляются как разница между текущим и предыдущим снимком, делённая на интервал в секундах. Поэтому при первой загрузке страницы значения скоростей могут быть нулевыми, пока не накопится второй замер.
- Историю можно посмотреть в разделе «История системы» (*System History*) — графики строятся по тем же рядам данных, что описаны ниже (см. п. 3.12).

3.2. ЦП (CPU)

Плитка «ЦП» (*CPU*) показывает текущую загрузку процессора в процентах, а также параметры самого процессора.

Показатель	Описание
Загрузка ЦП, %	Доля занятого процессорного времени за последний интервал. Сглаживается экспоненциальным средним (ЕМА, коэффициент <code>alpha = 0.3</code>), чтобы скачки не дёргали индикатор. Значение всегда зажато в диапазоне 0–100 %. При самом первом замере возвращается 0 (инициализация базовой точки).
Логические процессоры	Количество логических ядер — то есть с учётом Hyper-Threading.
Физические ядра	Число физических ядер.
Частота	Базовая частота процессора в МГц. Запрашивается лениво и кэшируется: первый успешный замер сохраняется, повторная попытка делается не чаще чем раз в 5 минут, а сам запрос ограничен таймаутом 1,5 с (на некоторых системах запрос частоты отвечает медленно).

CPU-загрузка алгоритмически считается так: при наличии нативной платформенной реализации используется она, иначе — расчёт по дельтам счётчиков процессорного времени (busy / total). Guest- и GuestNice-время исключаются, чтобы не учитывать его дважды.

3.3. Память (RAM)

Плитка «Память» (RAM) показывает использовано и всего. Отображается в виде «использовано / всего» и/или процента заполнения. В историю записывается процент.

3.4. Подкачка (Swap)

Плитка «Подкачка» (Swap) показывает использовано и всего. Если файл/раздел подкачки не настроен (всего = 0), показатель равен нулю; в исторический ряд при отсутствии swap пишется 0.

3.5. Диск (Storage)

Плитка «Диск» (Storage) показывает использовано и всего, при этом учитывается **только корневой раздел /** . В историю «Использование диска» (Disk Usage) пишется процент заполнения. Отдельно собирается дисковый ввод-вывод (чтение / запись, байт/с) как дельта счётчиков за интервал — он отображается на вкладке «Диск I/O» истории.

3.6. Время работы системы (Uptime)

Показатель «Время работы системы» (Uptime). Это время с момента загрузки **всего сервера** (в секундах), а не время работы панели или Xray. Отдельно хранится аптайм процесса Xray (см. п. 3.9), а также число потоков панели (в переводе — «Потоки» / Threads).

Память, занимаемая панелью

Рядом с показателями процесса панели выводится объём оперативной памяти, который занимает сам процесс 3X-UI. Это значение берётся из реального RSS процесса (как его видит операционная система) и совпадает с тем, что показывают системные утилиты. Число снижается по мере освобождения памяти. Раньше панель выводила внутренний счётчик Go, который завывал расход памяти (например, ~300 МБ на простаивающем сервере с одним клиентом) и никогда не уменьшался — теперь этого артефакта нет. Дополнительно периодический фоновый процесс возвращает неиспользуемую память операционной системе, чтобы показатель отражал фактическое потребление.

3.7. Загрузка системы (Load average)

Блок «Нагрузка на систему» (System Load) — массив из трёх чисел [Load1, Load5, Load15] . Подпись-подсказка: «Средняя загрузка системы за последние 1, 5 и 15 минут» (System load average for the past 1, 5, and 15 minutes). График истории называется «Средняя нагрузка системы (1 / 5 / 15 мин)». В исторические ряды значения пишутся отдельно: load1 , load5 , load15 .

Это стандартный Unix-показатель: среднее число процессов, находящихся в очереди на выполнение. Ориентир — сравнивать с числом ядер: загрузка, устойчиво превышающая количество физических ядер, говорит о перегрузке.

3.8. Сеть: скорость и общий объём трафика

Учитываются **только физические интерфейсы**. Виртуальные и туннельные интерфейсы исключаются: это `lo / lo0`, а также всё, что начинается с `loopback`, `docker`, `br-`, `veth`, `virbr`, `tun`, `tap`, `wg`, `tailscale`, `zt`. Значения суммируются по всем оставшимся интерфейсам.

Общая скорость (*Overall Speed*) — мгновенная скорость, дельта счётчиков за интервал:

Показатель	Описание
Загрузка / отдача (подпись «Загрузка» / <i>Upload</i>)	Исходящая скорость, байт/с.
Скачивание / приём (подпись «Скачать» / <i>Download</i>)	Входящая скорость, байт/с.

Общий объём трафика (*Total Data*) — накопленные счётчики с момента старта системы:

Показатель	Описание
Отправлено (подпись «Отправлено» / <i>Sent</i>)	Всего отправлено байт.
Получено (подпись «Получено» / <i>Received</i>)	Всего получено байт.

Дополнительно собираются скорости пакетов (пакетов/с) и суммарные счётчики пакетов — они показываются на вкладке «Сетевые пакеты» (*Network Packets*) истории. Ряды истории по сети: `netUp`, `netDown`, `pktUp`, `pktDown`.

3.9. IP-адреса сервера

Блок «IP-адреса сервера» (*IP Addresses*) показывает IPv4 и IPv6. Внешние адреса определяются через сторонние сервисы (`api4.ipify.org`, `ipv4.icanhazip.com`, `v4.api.ipinfo.io/ip`, `ipv4.myexternalip.com/raw`, `4.ident.me`, `check-host.net/ip` для IPv4 и аналогичные для IPv6). Список перебирается по очереди до первого успешного ответа; таймаут на каждый запрос — 3 с.

Особенности:

- Результат **кэшируется** на время жизни процесса: успешно определённый адрес повторно не запрашивается.
- Если ни один сервис не ответил, в поле остаётся N/A. Для IPv6 при первом N/A запросы IPv6 вообще отключаются, чтобы не тратить время на сетях без IPv6.
- Рядом есть кнопка-«глаз» для скрытия/показа адресов — подсказка «Скрыть или показать IP-адреса сервера» (*Toggle visibility of the IP*). Это только визуальное скрытие в интерфейсе (например, для скриншотов), на сами адреса оно не влияет.

3.10. TCP/UDP соединения

Блок «Количество соединений» (*Connection Stats*) показывает общее число активных TCP- и UDP-соединений на сервере (по всей системе, не только Xray). График истории — «Активные соединения (TCP / UDP)» (*Active Connections*), ряды `tcpCount`, `udpCount`.

3.11. Статус Xray и управление процессом

Карточка «Xray» показывает состояние процесса Xray-core и даёт им управлять.

Состояния

Значение	Подпись	Перевод	Когда выставляется
running	«Запущен»	Running	Процесс Xray запущен.
stop	«Остановлен»	Stopped	Процесс не запущен, и при этом нет зафиксированной ошибки запуска.
error	«Ошибка»	Error	Процесс не запущен, но зафиксирована ошибка запуска. Текст ошибки показывается во всплывающем окне с заголовком «Ошибка при запуске Xray» (<i>An error occurred while running Xray</i>).
—	«Неизвестно»	Unknown	Отображается, пока статус ещё не получен.

Рядом со статусом выводится **версия Xray**.

Кнопки управления

- **Стоп (Stop)**. Вызывает `POST /stopXrayService`. При успехе панель рассылает по WebSocket новое состояние `stop` и уведомление «Xray успешно остановлен» (*Xray service has been stopped*), при ошибке — состояние `error` с текстом. Важно: если панель доступна *через* сам Xray, остановка Xray может оборвать связь с панелью — при прямом подключении к панели проблемы нет.
- **Перезапуск (Restart)**. Вызывает `POST /restartXrayService`. Перед действием показывается подтверждение «Перезапустить xray?» с пояснением «Перезагружает сервис xray с сохранённой конфигурацией». При успехе — состояние `running` и уведомление «Xray успешно перезапущен» (*Xray service has been restarted successfully*). Перезапуск применяет текущую сохранённую конфигурацию — используйте его после изменения настроек.

Примечание. В этом форке для дашборда добавлено полноценное управление Start / Stop / Restart для всех типов авторизации; в исходном UI 3x-ui отдельной кнопки «старт» нет — запуск выполняется перезапуском.

Кнопка просмотра логов Xray

На карточке Xray есть кнопка просмотра логов Xray (*Logs*). Она появляется только тогда, когда в конфигурации Xray настроен access-лог: встроенный просмотрщик читает именно этот файл, поэтому без access-лога кнопка скрыта. Видимость кнопки привязана к отдельному признаку `accessLogEnable` и больше не зависит от лимита IP — онлайн-список и лимит IP-адресов продолжают работать даже без access-лога (см. п. 8).

Выбор версии Xray

Раздел «Выбор версии» (*Version*) позволяет переключить Xray-core на другой релиз. Список версий загружается через `GET /getXrayVersion`:

- Источник — GitHub API репозитория `XTLS/Xray-core` (`/releases`). Запросы кэшируются на **15 минут**; при сбое GitHub отдаётся последний удачно полученный список, чтобы пикер не пустел.
- В список попадают только релизы вида `X.Y.Z` и **не старше 26.4.25**.

Подсказки: «Выберите нужную версию» (*Choose the version you want to switch to.*) и предупреждение «Важно: старые версии могут не поддерживать текущие настройки» (*Choose carefully, as older versions may not be compatible with current configurations.*).

Переключение: `POST /installXray/:version`. Сценарий:

Пример. Переключиться на конкретную версию Xray-core (cookie сессии должен быть уже получен авторизацией):

```
curl -X POST 'https://panel.example.com:2053/xpanel/installXray/v25.6.8' \
  -b cookie.txt
```

Здесь `v25.6.8` — тег из списка, который отдаёт `GET /getXrayVersion`. Версия обязана присутствовать в этом списке, иначе панель ответит отказом. С версии 3.4.2 минимально допустимая для установки версия Xray поднята до **26.6.27**, а в комплекте 3.5.0 идёт ядро **Xray 26.7.11**. Попутные изменения ядра: шифры Shadowsocks `none / plain` и VMess `none / zero` удалены (сохранённые конфиги переписываются автоматически: SS — на поддерживаемый шифр, VMess — на `auto`), а незашифрованный VLESS/Trojan-outbound на публичный адрес отклоняется при сохранении — ядро с таким конфигом не запустилось бы.

1. Выбранная версия проверяется на наличие в актуальном списке релизов (иначе — отказ).
2. Xray останавливается.
3. Под текущую ОС и архитектуру скачивается архив `Xray-<os>-<arch>.zip` с GitHub (поддерживаются amd64/64, arm64-v8a, arm32-v7a/v6/v5, 386/32, s390x; для Windows — `xray.exe`). Размер архива и бинарника ограничены 200 МБ.
4. Бинарник заменяется атомарно (через временный файл + переименование) и помечается исполняемым.
5. Xray запускается заново.

Перед переключением показывается диалог «Переключить версию Xray» (*Do you really want to change the Xray version?*) с описанием «Это изменит версию Xray на `#version#`». При успехе — уведомление «Xray успешно обновлён» (*Xray updated successfully*).

3.12. Обновление панели (3X-UI)

Блок проверки обновлений панели. Данные приходят через `GET /getPanelUpdateInfo`:

Поле	Описание
Текущая версия панели	Версия установленной панели.

Поле	Описание
Последняя версия панели	Последний релиз 3x-ui, полученный с GitHub.
Доступно обновление	Признак того, что последняя версия новее текущей. Если обновление не нужно – показывается «Панель обновлена» / «Обновлено».

Кнопка **«Обновить панель»** (*Update Panel*) запускает `POST /updatePanel`. Подсказка: «Это обновит 3X-UI до последнего релиза и перезапустит сервис панели». Перед запуском – подтверждение «Вы действительно хотите обновить панель?» с текстом «Это обновит 3X-UI до версии #version# и перезапустит сервис панели».

Особенности и ограничения:

- Самообновление поддерживается **только на Linux** (на других ОС возвращается ошибка).
- Скрипт-обновлятор скачивается из официального репозитория (`raw.githubusercontent.com/MHSanaei/3x-ui/main/update.sh`, лимит 2 МБ) и запускается через `bash`, по возможности изолированно через `systemd-run`.
- При успешном запуске показывается «Обновление панели началось» (*Panel update started*); если проверка обновления не удалась – «Проверка обновления панели не удалась». Во время установки выводится предупреждение «Установка в процессе. Не обновляйте страницу».

3.13. Обновление гео-файлов (GeoIP / GeoSite)

Кнопка/диалог обновления гео-баз вызывает `POST /updateGeofile` (все файлы) или `POST /updateGeofile/:fileName` (один файл). Обновление работает по строгому белому списку имён и источников:

Файл	Источник
<code>geoip.dat</code> , <code>geosite.dat</code>	<code>Loyalsoldier/v2ray-rules-dat</code> (latest)
<code>geoip_IR.dat</code> , <code>geosite_IR.dat</code>	<code>chocolate4u/Iran-v2ray-rules</code> (latest)
<code>geoip_RU.dat</code> , <code>geosite_RU.dat</code>	<code>runetfreedom/russia-v2ray-rules-dat</code> (latest)

Поведение:

- Имя файла валидируется: запрещены `..`, слэши, абсолютные пути; допустимы только `[a-zA-Z0-9._-]+.dat`. Файлы вне белого списка не скачиваются.
- Используется условный запрос `If-Modified-Since`: если на сервере-источнике файл не менялся (HTTP 304), повторно он не качается, лишь обновляется отметка времени.
- После загрузки Xray **перезапускается** (чтобы подхватить новые базы).

Пример. Обновить только российские гео-базы, не трогая остальные файлы:

```
curl -X POST 'https://panel.example.com:2053/xpanel/updateGeofile/geoup_RU.dat' -b
cookie.txt
curl -X POST 'https://panel.example.com:2053/xpanel/updateGeofile/geosite_RU.dat' -b
cookie.txt
```

Чтобы обновить сразу все файлы из белого списка — вызовите `POST /updateGeofile` без имени файла.

- Диалоги: «Вы действительно хотите обновить геофайл?» с «Это обновит файл #filename#» для одного файла и «Это обновит все геофайлы» для кнопки «Обновить все». Успех — «Геофайлы успешно обновлены».

3.14. Резервное копирование и восстановление базы данных

Блок «Бэкап и восстановление» (*Backup & Restore*). Поведение зависит от используемой СУБД (SQLite по умолчанию или PostgreSQL).

Экспорт базы (Резервная копия)

Кнопка «Экспорт базы данных» / «Резервная копия» (*Back Up*) вызывает `GET /getDb`. Файл отдаётся как вложение:

- **SQLite**: сначала выполняется checkpoint (сброс WAL), затем скачивается файл `x-ui.db`. Подсказка: «Нажмите, чтобы скачать файл .db, содержащий резервную копию вашей текущей базы данных...».
- **PostgreSQL**: скачивается дамп `x-ui.dump` в кастомном формате (`pg_dump --format=custom --no-owner --no-privileges`). На сервере должны быть установлены клиентские инструменты PostgreSQL; иначе — ошибка про отсутствие `pg_dump`.

Импорт базы (Восстановление)

Кнопка «Импорт базы данных» / «Восстановление» (*Restore*) загружает файл через `POST /importDB` (поле формы `db`). Подсказка: «Нажмите, чтобы выбрать и загрузить файл .db... для восстановления базы данных из резервной копии».

Сценарий для **SQLite** безопасный, с откатом:

1. Файл проверяется на формат SQLite и сохраняется во временный файл, затем проверяется целостность.
2. Храу останавливается, текущая БД закрывается и переименовывается в `*.backup` (фолбэк).
3. Новый файл встаёт на место рабочей БД, выполняется инициализация и миграция. Если что-то пошло не так — восстанавливается фолбэк.
4. Храу запускается заново.

Для **PostgreSQL** с 3.5.0 «Восстановление» принимает **три вида файлов** (тип определяется по содержимому, не по расширению): архив `pg_dump` (`PGDMP`) — применяется через `pg_restore --clean --if-exists --single-transaction ...`; **SQLite-базу** `.db` (обычный бэкап) и **SQLite-миграционный** `.dump` — они проверяются, при необходимости пересобираются и импортируются в PostgreSQL **одной транзакцией** тем же движком, что и `x-ui migrate-db --dsn` (при ошибке PostgreSQL остаётся нетронутым). Диалог выбора

файла принимает `.dump`, `.db` на обеих СУБД; загрузка `pg_dump` -архива на SQLite-панель даёт понятную ошибку. Подсказка прямо предупреждает: «Это заменит все текущие данные».

Сообщения: «База данных успешно импортирована», «Произошла ошибка при импорте базы данных», «...при чтении базы данных», «...при получении базы данных».

Файл миграции (между SQLite и PostgreSQL)

Кнопка «Скачать файл миграции» (*Download Migration*) вызывает `GET /getMigration`. С 3.5.0 она показывается **только на PostgreSQL-панелях** и скачивает `x-ui.db` — готовую SQLite-базу, собранную из данных PostgreSQL (путь «обратно на SQLite»). На SQLite-панелях кнопка убрана как избыточная: обычный бэкап `.db` теперь и так восстанавливается прямо в PostgreSQL-панель.

3.15. Дополнительные элементы интерфейса

- **Индикатор онлайн-клиентов.** Дашборд ведёт ряд `online` (*Online Clients* / «Клиенты онлайн») — число клиентов с активным соединением. Считается при работающем Xray (иначе 0) и записывается в историю на том же 2-секундном такте. График — вкладка «Онлайн».
- **История системы (графики).** Кнопка/раздел «Графики» → «История системы» с вкладками: «Пропускная способность», «Пакеты», «Диск I/O», «Онлайн», «Нагрузка», «Соединения», «Использование диска». Данные тянутся по `GET /history/:metric/:bucket`; допустимые интервалы агрегации (bucket, сек): **2, 30, 60, 180, 360, 720, 1440, 2880, 10080**, на вкладку приходит до 60 точек. В самом селекторе диапазона на странице доступны кнопки **2m, 1h, 3h, 6h, 12h, 24h, 2d, 7d** (bucket'ы **2, 60, 180, 360, 720, 1440, 2880, 10080** соответственно). На длинных диапазонах **2d** и **7d** подписи времени по оси дополнены датой в формате `MM-DD HH:MM`. Хранение организовано трёхуровневым прореживанием (rollup): свежие данные держатся с шагом 2 с за последний **час**, далее усредняются до шага 1 мин за **48 часов** и до шага 10 мин за **7 суток**. Поэтому графики (CPU, RAM, трафик, пакеты, соединения, диск, онлайн, нагрузка) можно просматривать за период **до 7 суток** (раньше — до 48 часов), причём чем дальше в прошлое, тем грубее детализация. Допустимые метрики: `cpu`, `mem`, `swap`, `netUp`, `netDown`, `pktUp`, `pktDown`, `diskRead`, `diskWrite`, `diskUsage`, `tcpCount`, `udpCount`, `online`, `load1`, `load5`, `load15`. Подпись «Последние 2 минуты» соответствует `bucket = 2` (режим реального времени).

Пример. Получить ряд загрузки ЦП за последние ~2 минуты (`bucket = 2` с, до 60 точек) и тот же ряд, агрегированный по 5 минут (`bucket = 300` с):

```
curl 'https://panel.example.com:2053/xpanel/history/cpu/2' -b cookie.txt
curl 'https://panel.example.com:2053/xpanel/history/cpu/300' -b cookie.txt
```

Метрику можно заменить на любую допустимую (`mem`, `netUp`, `tcpCount`, `load1` и т.д.). Bucket вне белого списка **2, 30, 60, 180, 360, 720, 1440, 2880, 10080** будет отвергнут.

- **Метрики Xray** — отдельный блок с потреблением памяти и сборкой мусора Xray (ряды `xrAlloc`, `xrSys`, `xrHeapObjects`, `xrNumGC`, `xrPauseNs`) и «Обсерваторией» (состояние исходящих соединений). Работают, только если в конфигурации Xray задан блок `metrics` (`listen 127.0.0.1:11111`, `tag metrics_out`); иначе показывается «Конечная точка метрик Xray не настроена».

В окне метрик Xray свой селектор диапазона с кнопками **2m, 1h, 3h, 6h, 12h** (bucket'ы 2, 60, 180, 360, 720).

Пример блока, который включает плитку метрик Xray. В разделе настроек Xray должны присутствовать одновременно `metrics` (с тегом) и `inbound`, который этот тег слушает:

```
{
  "metrics": {
    "tag": "metrics_out"
  },
  "inbounds": [
    {
      "listen": "127.0.0.1",
      "port": 11111,
      "protocol": "dokodemo-door",
      "settings": { "address": "127.0.0.1" },
      "tag": "metrics_out"
    }
  ]
}
```

Адрес `127.0.0.1:11111` намеренно не выставляется наружу — панель опрашивает его локально.

- **Переключатель тёмной темы.** Находится в общем меню/шапке, а не в самом дашборде. Варианты: «Тема» (*Theme*) с вариантами «Темная» и «Очень темная» (*Ultra Dark*). Это чисто визуальная настройка оформления, на работу панели не влияет.
- **Прочие ссылки** в окружении дашборда (из меню/нижней панели): «Логи», «Конфигурация» — просмотр итогового JSON Xray (GET /getConfigJson), «Документация».

4. Inbounds: создание и общие параметры

Раздел **«Входящие»** (англ. *Inbounds*) — это список всех точек входа Xray, через которые подключаются клиенты. Каждый inbound хранит как «панельные» поля (примечание, лимит трафика, расписание сброса), так и сырые JSON-блоки конфигурации Xray (`settings`, `streamSettings`, `sniffing`).

Создание выполняется кнопкой **«Создать подключение»** (*Add Inbound*), редактирование — **«Изменить подключение»** (*Modify Inbound*). Обе операции отправляются на API-эндпоинты `POST /add` и `POST /update/:id`.

Ниже разобраны все поля формы, **не** относящиеся к настройкам конкретного протокола (клиенты, шифрование, REALITY/TLS) и **не** относящиеся к транспорту/потoku (вкладки **«Поток»**, **«Безопасность»**) — это темы отдельных разделов.

4.1. Общие поля формы

Remark (Примечание)

Параметр	Значение
Поле	<code>remark</code>
Тип	строка
По умолчанию	пусто

Человекочитаемое имя inbound, отображаемое в списке и в заголовках диалогов («Удалить подключение "{remark}"?» и т. п.). Подпись поля — **«Примечание»**. На работу Xray не влияет, нужно только для удобства администрирования; рекомендуется задавать уникальные осмысленные имена, так как они подставляются в названия экспортируемых файлов и в подтверждения массовых операций.

Protocol (Протокол)

Параметр	Значение
Поле	<code>protocol</code>
Подпись	«Протокол»
Валидация	<code>required,oneof=vmess vless trojan shadowsocks wireguard hysteria http mixed tunnel tun</code>

Выпадающий список протокола inbound. Допустимые значения:

Значение	Примечание
<code>vmess</code>	
<code>vless</code>	
<code>trojan</code>	

Значение	Примечание
shadowsocks	
wireguard	
hysteria	Hysteria v2 — это hysteria с streamSettings.version = 2, отдельного протокола нет
http	
mixed	socks/http на одном порту
tunnel	
tun	принимается валидатором, отдельной константы протокола нет

Поле обязательное (required). Выбор протокола определяет, какие поля настроек клиентов и какой транспорт будут доступны (см. протокол-специфичные разделы).

Важно: при сохранении сервис нормализует streamSettings. Транспортные настройки оставляются только для протоколов vmess, vless, trojan, shadowsocks, hysteria; для остальных (http, mixed, tunnel, wireguard, tun) поле streamSettings **принудительно очищается**.

Для inbound типа tunnel /Trojan, у которых блок streamSettings не содержит ключа security (бестранспортный вариант), форма открывается и сохраняется без ошибки валидации streamSettings.security Invalid input.

Listen IP (IP прослушивания)

Параметр	Значение
Поле	listen
Тип	строка
По умолчанию	пусто → Xray слушает на 0.0.0.0 (все IP)

IP-адрес, на котором inbound принимает соединения. Подсказка к полю:

«Оставьте пустым для прослушивания всех IP-адресов».

При генерации конфигурации Xray пустое значение заменяется на 0.0.0.0. Помимо IP, поле принимает **путь Unix-сокета** — подсказка:

«Можно также указать путь Unix-сокета (например, /run/xray/in.sock) или имя абстрактного сокета с префиксом @ (например, @xray/in.sock), чтобы слушать сокет вместо TCP-порта — в этом случае задайте порт 0».

Таким образом, поле принимает две формы Unix-сокета: путь в файловой системе (`/run/xray/in.sock`) и абстрактное имя сокета с префиксом `@` (`@xray/in.sock`). В обоих случаях задайте `Port` равным `0` .

Меняют это поле, когда нужно ограничить inbound одним интерфейсом (например, `127.0.0.1` для inbound, работающего только как fallback-цель за Nginx) либо когда inbound слушает Unix-сокеты.

Пример. Inbound, который слушает только локальный интерфейс (типичная fallback-цель за Nginx) и Unix-сокеты:

```
listen = 127.0.0.1   порт = 8443
listen = /run/xray/in.sock   порт = 0
```

Port (Порт)

Параметр	Значение
Поле	<code>port</code>
Подпись	«Порт»
Валидация	<code>gte=0,lte=65535</code>
По умолчанию	— (задаётся пользователем)

TCP/UDP-порт прослушивания. Допустимы значения от `0` до `65535` . Значение `0` используется только в паре с прослушиванием на Unix-сокете (см. выше).

При сохранении сервис проверяет конфликт порта: два inbound не могут одновременно занимать пересекающиеся `listen:port` для одного и того же транспорта (TCP/UDP). Транспорт вычисляется из протокола и `streamSettings/settings` : например, `hysteria` и `wireguard` всегда занимают UDP, `kcp` / `quic` — UDP, а большинство остальных — TCP. При конфликте сохранение отклоняется с ошибкой.

Отдельно панель не даёт занять **зарезервированный порт внутреннего Xray API** (тег `api` , по умолчанию `62789` на `127.0.0.1`): локальный TCP-inbound, чей адрес прослушивания пересекается с этим портом на loopback, отклоняется с той же ошибкой конфликта порта. Реальный порт API читается из шаблона конфигурации Xray (с запасным значением `62789`). На узлах (nodes) это ограничение не действует — у них собственный Xray.

Тег Xray (`Tag` , уникальный) генерируется автоматически из порта и транспорта в формате `in-<порт>-<tcp|udp|tcpudp|any>` ; для inbound, развёрнутого на узле, добавляется префикс `n<nodeId>-` . При коллизии к тегу дописывается `-2` , `-3` и т. д. Пользователь обычно тег не редактирует.

Total traffic (Всего трафика, GB)

Параметр	Значение
Поле	<code>total</code> (в байтах)
Подпись	«Общий расход»
По умолчанию	<code>0</code>

Суммарный лимит трафика inbound. В форме значение вводится в гигабайтах, в БД хранится в байтах. Подсказка к полю:

«= Безлимит. (единица: ГБ)».

То есть `0` означает **безлимит**. Это лимит на уровень всего inbound (а не отдельных клиентов); фактический израсходованный трафик хранится в полях `up` (отправлено) и `down` (получено) и сравнивается с `total`.

Expiry date / Duration (Дата окончания / срок)

Параметр	Значение
Поле	<code>expiryTime</code> (Unix-метка времени)
Подпись	«Дата окончания» (англ. <i>Duration</i>)
По умолчанию	пусто / <code>0</code>

Срок действия inbound. Подсказка:

«Оставьте пустым, чтобы было бесконечным».

Пустое значение (`0`) означает бессрочный inbound. Значение хранится как Unix-метка; форма позволяет задавать как конкретную дату, так и срок в днях (относительный отсчёт от текущего момента — англ. подпись поля *Duration*).

Enabled (Включить)

Параметр	Значение
Поле	<code>enable</code>
Подпись	«Включить» (англ. <i>Enabled</i>)
По умолчанию	задаётся при создании

Признак активности inbound. Переключение этого флага в списке обрабатывается отдельным «лёгким» эндпоинтом `POST /setEnabled/:id`, а не полным обновлением — это сделано специально, чтобы не пересериализовывать весь блок `settings` (всех клиентов) при каждом щелчке по переключателю на

inbound с тысячами клиентов. При выключении inbound удаляется из работающего Xray, при включении — добавляется обратно.

Node / Deploy to (Узел / Развернуть на)

Параметр	Значение
Поле	<code>nodeId</code>
Подпись	«Развернуть на», «Локальная панель»
По умолчанию	пусто (локальная панель)

Выбор, где физически работает inbound: на локальной панели или на одном из зарегистрированных узлов. С 3.5.0 список «Развернуть на» (как и фильтр «Узлы» над списком inbound) поддерживает **поиск по вводу**, а над самим списком inbound появилось **поле поиска** (по примечанию, порту и протоколу; работает вместе с фильтром узлов). Особенность реализации: `nodeId = 0` нормализуется в `nil`, так как `0` — это не валидный id узла, а артефакт привязки формы; `nil / 0` означает локальную панель. При сохранении inbound на офлайн-узле возможен тост «изменение синхронизируется, когда узел переподключится». С версии 3.4.2 этот тост показывается только когда узел действительно офлайн или выключен (раньше он мог появляться и при сохранении на онлайн-узел).

Стратегия адреса для ссылок (Share address strategy)

Параметр	Значение
Поле	стратегия + (опционально) пользовательский адрес
Подпись	«Стратегия адреса для ссылок» (англ. <i>Share address strategy</i>)
По умолчанию	«Адрес прослушивания inbound» (<i>Inbound listen</i>)

Выпадающий список определяет, какой адрес подставляется в **экспортируемые ссылки-шеринга и QR-коды** этого inbound. Значения:

Значение	Подпись	Что подставляется
<code>node</code>	«Адрес узла» (<i>Node address</i>)	адрес узла, на котором работает inbound
<code>listen</code>	«Адрес прослушивания inbound» (<i>Inbound listen</i>)	адрес прослушивания самого inbound
<code>custom</code>	«Пользовательская» (<i>Custom</i>)	свой адрес из поля «Пользовательский адрес для ссылок» (<i>Custom share address</i>)

При выборе «Пользовательская» появляется поле «Пользовательский адрес для ссылок»; в него вводят хост или IP **без схемы и порта** (значение валидируется). Вариант «Адрес узла» показывается в списке, только если существует включённый узел, на котором этот inbound может работать; иначе он скрыт, а значение приводится к «Адрес прослушивания inbound».

Эта стратегия влияет **только** на прямые ссылки-шеринга и QR-коды. На выдачу подписки она **не** влияет — там адрес по-прежнему определяется обычной логикой панели.

4.2. Sniffing (Сниффинг)

Вкладка «**Сниффинг**» редактирует блок `sniffing` конфигурации Xray, который хранится как сырой JSON. Sniffing позволяет Xray «подсматривать» реальное доменное имя/протокол внутри соединения для целей маршрутизации.

Подполе	Подпись	Назначение
<code>enabled</code>	(переключатель вкладки)	Включает/выключает сниффинг для inbound
<code>destOverride</code>	—	Список протоколов, для которых перехватывается адрес назначения: <code>http</code> , <code>tls</code> , <code>quic</code> , <code>fakedns</code>
<code>metadataOnly</code>	«Только метаданные»	Использовать только метаданные соединения, без чтения полезной нагрузки
<code>routeOnly</code>	«Только маршрутизация»	Применять результат сниффинга только для маршрутизации, не переписывая адрес назначения
<code>domainsExcluded</code>	«Исключённые домены»	Домены, исключаемые из сниффинга
(исключённые IP)	«Исключённые IP»	IP-адреса, исключаемые из сниффинга

- **`destOverride`** — набор снифферов: `http` (определяет домен из HTTP-заголовка Host), `tls` (из SNI), `quic` (из QUIC ClientHello), `fakedns` (сопоставление с FakeDNS-пулом). Обычно для определения домена включают `http` и `tls`.

Пример блока `sniffing` (определять домен по HTTP и TLS, использовать результат только для маршрутизации, не трогая локальную сеть):

```
{
  "enabled": true,
  "destOverride": ["http", "tls"],
  "routeOnly": true,
  "domainsExcluded": ["courier.push.apple.com"]
}
```

- **`metadataOnly`** — когда включён, Xray не считывает содержимое первого пакета и опирается только на метаданные; полезно, чтобы не нарушать протоколы, у которых нельзя «подсматривать» данные.
- **`routeOnly`** — результат сниффинга используется только правилами маршрутизации; адрес соединения в outbound при этом не переписывается на распознанный домен.

Примечание: панель хранит

`sniffing` как непрозрачный JSON-блок и при сохранении ничего к нему не добавляет — все значения по умолчанию для этих галочек формируются на стороне приложения-клиента. В сыром виде блок можно отредактировать через раздел «JSON входящего» (см. ниже).

4.3. Allocate (стратегия распределения портов)

Блок `allocate` в `streamSettings` управляет тем, как Xray распределяет порты прослушивания. Это часть конфигурации Xray; панель хранит и передаёт его как часть `streamSettings` /JSON inbound. Параметры (по терминологии Xray-core):

Подполе	Назначение	Значения / по умолчанию
<code>strategy</code>	Стратегия выделения портов	<code>always</code> — всегда слушать заданный порт (по умолчанию); <code>random</code> — периодически менять прослушиваемые порты внутри диапазона
<code>refresh</code>	Интервал смены портов (минуты) при <code>random</code>	целое число минут (рекомендуется 5; минимум — 2)
<code>concurrency</code>	Сколько портов держать открытыми одновременно при <code>random</code>	целое (по умолчанию 3; не более трети ширины диапазона портов)

`strategy: always` оставляет inbound на одном порту (стандартный режим). `strategy: random` нужен для сценариев anti-blocking, когда inbound периодически «прыгает» по диапазону портов; в этом случае имеют смысл `refresh` и `concurrency`. Менять эти значения нужно только при сознательном использовании режима случайных портов.

Пример блока `allocate` в `streamSettings` (режим случайных портов: держать 3 порта открытыми, менять каждые 5 минут):

```
{
  "allocate": {
    "strategy": "random",
    "refresh": 5,
    "concurrency": 3
  }
}
```

Чтобы это работало, `port inbound` задают диапазоном (например, `20000–20100`).

4.4. External Proxy (Внешний прокси)

Поле **«External Proxy»** относится к настройкам формирования ссылок-приглашений и хранится в `streamSettings inbound`. Оно задаёт список альтернативных внешних адресов (хост/порт, при необходимости с принудительным TLS — **«Принудительный TLS»**), которые подставляются в клиентские ссылки вместо реального `listen:port inbound`.

Используется, когда клиенты должны подключаться не напрямую к серверу, а через внешний прокси/реверс/CDN: тогда в общих ссылках указывается публичный адрес такого фронтенда. На сам процесс приёма соединений Xray это не влияет — это «косметика» генерируемых ссылок. Связанные поля формы: **«Принудительный TLS»**, **«Fingerprint»**, подписи каждой записи.

4.5. Fallbacks (Fallback'и)

Раздел **«Fallback'и»** задаёт правила перенаправления соединений, не совпавших ни с одним клиентом inbound. Доступен для master-inbound на TLS-транспорте (VLESS/Trojan TCP-TLS). Управляется через эндпоинты `GET /:id/fallbacks / POST /:id/fallbacks`.

Подсказка к разделу:

«Когда соединение на этом инбаунде не совпадает ни с одним клиентом, оно перенаправляется в другое место. Выберите дочерний инбаунд ниже, чтобы поля маршрутизации (SNI / ALPN / Path / xver) заполнились автоматически из его транспорта, либо оставьте выбор пустым и задайте Dest напрямую (например, 8080 или 127.0.0.1:8080), чтобы перенаправить на внешний сервер, такой как Nginx. Каждый дочерний инбаунд должен слушать на 127.0.0.1 с security=none».

Раздел fallback'ов показывается только для inbound VLESS/Trojan поверх RAW (TCP) с безопасностью TLS или REALITY. Новый inbound стартует с `security=none`, поэтому раздел сначала может выглядеть отсутствующим. В этом состоянии (VLESS/Trojan, RAW/TCP, безопасность ещё не настроена) вместо раздела выводится встроенная подсказка: fallback'и станут доступны после выбора TLS или Reality на вкладке **«Безопасность»**.

Поля строки fallback

Поле	По умолчанию	Описание
(дочерний inbound)	—	Выбор дочернего inbound (подпись «Выберите инбаунд»). Если выбран, поля Name/Alpn/Path/Dest могут авто-заполниться из его транспорта
Name	пусто (= любой)	Условие совпадения по имени (SNI/имени). Подпись «любой» — «любой»
Alpn	пусто	Условие совпадения по ALPN
Path	пусто	Условие совпадения по пути (для WS/HTTP-транспортов дочернего inbound)
Dest	авто	Куда перенаправлять. Плейсхолдер «авто (listen:порт дочернего)» . Можно указать порт (8080) или host:port (127.0.0.1:8080)
Xver	0	Версия PROXY-протокола («Xver»): 0 — выключен, 1 или 2 — соответствующая версия PROXY protocol
(порядок)	по позиции	Порядок применения правил; задаётся кнопками «Вверх»/«Вниз»

Логика сохранения: весь список фолбэков мастера заменяется атомарно. Строка, у которой нет ни выбранного дочернего inbound (`childId <= 0`), ни заданного Dest, **пропускается**. Если выбранный дочерний inbound равен id самого мастера, он обнуляется. При генерации итогового JSON: если Dest пуст, он вычисляется из дочернего inbound как listen:port, причём 0.0.0.0 / :: / :::0 заменяются на 127.0.0.1; пустые поля name / alpn / path в выходной JSON не попадают; xver добавляется только если он больше 0.

Пример итогового `settings.fallbacks` (трафик с `alpn=h2` уходит на WS-цель по пути `/ws`, всё остальное — на локальный Nginx на порту 8080):

```
{
  "fallbacks": [
    { "alpn": "h2", "path": "/ws", "dest": "127.0.0.1:2001", "xver": 1 },
    { "dest": 8080 }
  ]
}
```

Последняя строка без `name / alpn / path` — это правило «по умолчанию», ловящее всё остальное.

Кнопки и подсказки раздела `fallbacks`

- **«Добавить фолбэк»** — добавить строку; **«Фолбэков пока нет»** — пустое состояние.
- **«Быстро добавить все подходящие»** / **«Добавить все»** — добавляет строку fallback для каждого подходящего inbound, ещё не подключённого. Результат: «Добавлено {n} фолбэк(ов)» или «Нет новых подходящих инбаундов».
- **«Заполнить из дочернего»** — переподтянуть поля маршрутизации (SNI/ALPN/Path/xver) из транспорта выбранного дочернего inbound; после выполнения — «Заполнено из дочернего».
- **«Изменить поля маршрутизации»** / **«Скрыть расширенные»** — показать/скрыть тонкие поля строки.
- Подписи **«Маршрутизирует, когда»** и **«По умолчанию — ловит всё остальное»** поясняют условие срабатывания каждой строки.

После сохранения фолбэков сервер вызывает рестарт Xray, чтобы новые `settings.fallbacks` вступили в силу.

4.6. Периодический сброс трафика

Блок **«Сброс трафика»** настраивает автоматический сброс счётчиков трафика inbound по расписанию. Описание:

«Автоматический сброс счетчика трафика через указанные интервалы».

Параметр	Значение
Поле	<code>trafficReset</code>
Валидация	<code>omitempty,oneof=never hourly daily weekly monthly</code>
По умолчанию	<code>never</code>
Сопутствующее поле	<code>lastTrafficResetTime</code> — метка последнего сброса (подпись «Последний сброс»)

Выпадающий список:

Значение	Подпись
<code>never</code>	«Никогда»

Значение	Подпись
hourly	«Ежечасно»
daily	«Ежедневно»
weekly	«Еженедельно»
monthly	«Ежемесячно»

Для каждого периода зарегистрирован cron-джоб, запускающийся по соответствующему расписанию (@hourly, @daily, @weekly, @monthly). Джоб выбирает все inbound с заданным trafficReset и для каждого сбрасывает счётчики самого inbound (up=0, down=0) и трафик всех его клиентов. То есть периодический сброс затрагивает и inbound, и его клиентов.

Пример значения поля. Чтобы счётчики обнулялись первого числа каждого месяца, в форме выбирают «Ежемесячно», что сохраняется как:

```
{ "trafficReset": "monthly" }
```

Значение never (по умолчанию) полностью отключает автосброс.

4.7. JSON входящего (advanced)

Раздел **«Разделы JSON входящего»** даёт прямой доступ к сырым JSON-блокам inbound. Описание:

«Полный JSON входящего и отдельные редакторы для settings, sniffing и streamSettings».

Доступны редакторы:

Вкладка	Подпись	Что редактирует
Всё	«Полный объект входящего со всеми полями в одном редакторе»	весь объект Inbound
Настройки	«Обёртка блока settings Xray»	поле settings
Sniffing	«Обёртка блока sniffing Xray»	поле sniffing
Stream	«Обёртка блока stream Xray»	поле streamSettings

Эти поля сериализуются как вложенные JSON-объекты: пустые блоки отдаются как null, а текст, не являющийся валидным JSON, оборачивается в строку, чтобы данные не терялись. Ошибки парсинга при сохранении выводятся с префиксом **«Расширенный JSON»**.

Окно просмотра «JSON входящего», как и окно импорта inbound, использует полноценный редактор кода с подсветкой синтаксиса JSON (вместо обычного текстового поля): просмотр конфигурации — в подсвеченном режиме только для чтения, а импорт — в редактируемом, что упрощает чтение и правку.

4.8. Действия с inbound: QR / Edit / Reset / Delete и статистика

В списке и в карточке inbound доступны следующие действия (меню «**Меню**»):

Статистика трафика

Отображается агрегированный трафик inbound: «**Отправлено/получено**» (поля `up / down`), «**Всего трафика**», «**Всего подключений**». В карточке также — «**Создано**», «**Обновлено**», «**Дата окончания**».

В списке inbounds есть колонка **Speed** с текущей скоростью трафика по каждому inbound (отдача/загрузка), вычисляемой из приращений счётчиков между опросами; та же живая скорость показывается в окне статистики inbound. Когда очередной опрос не даёт приращения, значение скорости сбрасывается.

В сводке клиентов на странице inbounds статус определяется по приоритету «исчерпан/завершён»: клиенты, у которых истёк срок или исчерпан трафик (и которым авто-задание сняло `enable`), относятся к статусу «**Исчерпан/завершён**» (*Depleted/Ended*), а не к серому «**Отключён**» (*Disabled*), и не считаются дважды. Классификация совпадает с той, что показана в карточке самого клиента, и корректно учитывает клиентов, привязанных к нескольким inbound.

QR-код и копирование ссылок

- «**Подробнее**» — раскрывает ссылки подключения и подписки.
- QR-код клиента: подсказка «**Нажмите на QR-код, чтобы скопировать**».
- «**Копировать ссылку**» (англ. *Copy URL*), «**Экспорт ссылок**». С версии 3.4.2 страничное действие «**Экспортировать все ссылки**» (`GET /panel/api/inbounds/allLinks` ; окно «Экспорт всех ссылок inbound», файл «All-Inbounds») формирует ссылки через движок подписок — применяет шаблон ремарки к каждому клиенту и предпочитает управляемые Host-эндпоинты (раньше использовалась фиксированная ремарка `inbound-email`).

Edit (Изменить)

«**Изменить подключение**» — открывает форму редактирования (`POST /update/:id`). При обновлении сервис перечитывает существующую строку, переносит изменённые поля, при необходимости регенерирует тег (если старый тег был авто-сгенерирован) и синхронизирует рантайм Xray. Успех — тост «**Подключение успешно обновлено**».

Reset Traffic (Сбросить трафик)

«**Сбросить трафик**» — обнуляет счётчики `up / down` именно этого inbound (`POST /:id/resetTraffic` , ставит `up=0` , `down=0`). Подтверждение:

«Сбросить трафик "{remark}"?» / «Сбрасывает счётчики отправки/получения этого подключения до 0».

Сброс трафика inbound **не** трогает счётчики его клиентов (для них есть отдельные действия «Сброс трафика клиентов»). После сброса инициируется рестарт Xray. Успех — тост «**Входящий трафик сброшен**». Есть и массовый вариант — «**Сброс трафика всех подключений**» (`POST /resetAllTraffics`).

Delete (Удалить)

«Удалить подключение» (`POST /del/:id`). Подтверждение:

«Удалить подключение "{remark}"?» / «Подключение и все его клиенты будут удалены. Это действие нельзя отменить».

Удаление снимает inbound с работающего Xray (при необходимости с рестартом). Успех — тост **«Подключение успешно удалено»**. Массовое удаление — `POST /bulkDel`, с пер-элементной отчётностью и не более чем одним рестартом Xray.

Прочие действия с клиентами inbound

В меню также доступны: **«Клонировать»** (копия inbound с новым портом и пустым списком клиентов), **«Удалить всех клиентов»** (`POST /:id/delAllClients` — удаляет всех клиентов, сам inbound сохраняется), **«Удалить отключенных клиентов»**, **«Привязать/Отвязать клиентов»**, **«Импортировать»/«Экспорт подключений»** (`POST /import`). Детали клиентских операций относятся к разделу о клиентах.

5. Протоколы

При создании входящего соединения (inbound) первым выбирается **Протокол** («Protocol»). Протокол определяет, какой способ аутентификации и шифрования трафика Xray-core будет применять к этому inbound, какой набор полей в `settings` потребуется заполнить, а также какие транспорты (`network`) и виды безопасности (TLS / REALITY) для него доступны.

Поле протокола задаётся один раз при создании inbound и **не меняется при редактировании** (в форме редактирования выпадающий список заблокирован). Чтобы сменить протокол, нужно создать новый inbound.

5.1. Список поддерживаемых протоколов

Сервер принимает следующий набор значений поля `Protocol` :

```
oneof=vless vless trojan shadowsocks wireguard hysteria http mixed tunnel tun mtproto
```

Начиная с версии **3.3.0** в перечень добавлено значение `mtproto` (прокси Telegram).

Значение в конфиге	Назначение	Клиентская модель
<code>vless</code>	Основной прокси-протокол (по умолчанию при создании inbound)	Клиенты с UUID, поддержка flow и пост-квантового шифрования
<code>vmess</code>	Классический прокси-протокол Xray	Клиенты с UUID и параметром <code>security</code>
<code>trojan</code>	Прокси, маскирующийся под обычный HTTPS	Клиенты с паролем
<code>shadowsocks</code>	Прокси Shadowsocks (включая SIP022 / 2022-blake3)	Один пользователь или несколько (2022)
<code>wireguard</code>	Inbound WireGuard	Peer'ы (а не клиенты)
<code>hysteria</code>	Inbound Hysteria (по умолчанию версия 2)	Клиенты с токеном <code>auth</code>
<code>http</code>	Классический HTTP-прокси (forward proxy)	Аккаунты user/pass, без учёта трафика
<code>mixed</code>	Совмещённый SOCKS + HTTP-прокси	Аккаунты user/pass
<code>tunnel</code>	Прозрачный форвардер (xray <code>dokodemo-door</code>)	Без клиентов
<code>tun</code>	TUN-интерфейс (только рендеринг существующих)	Без клиентов
<code>mtproto</code>	Прокси Telegram (MTProto), добавлен в 3.3.0; обслуживается отдельным процессом <code>mtg</code> , а не Xray	Без клиентов (доступ по секрету)

Замечание о `tun` : значение оставлено в списке для совместимости и **отображения** ранее сохранённых inbound, но в актуальной версии backend его создание не рекомендуется — поддержка признана устаревшей. Создание новых inbound этого типа смысла не имеет.

Замечание о Hysteria 2: отдельного протокола «hysteria2» нет. Это протокол `hysteria` с полем `streamSettings.version = 2`. Схема ссылки `hysteria2://` при генерации share-ссылок выбирается автоматически, когда версия потока равна 2.

Не все протоколы поддерживают распределение по узлам (nodes). На узлы можно разворачивать только: `vless`, `vmess`, `trojan`, `shadowsocks`, `hysteria`, `wireguard`. Протоколы `http`, `mixed`, `tunnel`, `tun`, `mtproto` работают только на локальной панели.

5.2. Какие протоколы поддерживают TLS / REALITY / транспорт

Возможность включить тот или иной слой безопасности и транспорт зависит от протокола и выбранной сети (`streamSettings.network`):

Возможность	Доступна для протоколов	Допустимые сети (<code>network</code>)
TLS	<code>vmess</code> , <code>vless</code> , <code>trojan</code> , <code>shadowsocks</code> (а также всегда для <code>hysteria</code>)	<code>tcp</code> , <code>ws</code> , <code>http</code> , <code>grpc</code> , <code>httpupgrade</code> , <code>xhttp</code>
REALITY	<code>vless</code> , <code>trojan</code>	<code>tcp</code> , <code>http</code> , <code>grpc</code> , <code>xhttp</code>
flow (<code>xtls-rprx-vision</code>)	только <code>vless</code>	только <code>tcp</code> , при <code>security = tls</code> или <code>reality</code>
Stream / транспорт (вкладка «Поток»)	<code>vmess</code> , <code>vless</code> , <code>trojan</code> , <code>shadowsocks</code> , <code>hysteria</code>	—

Для протоколов `http`, `mixed`, `tunnel`, `tun`, `wireguard` вкладка транспорта недоступна — у них нет stream-настроек Xray.

5.3. VLESS

Назначение: основной современный прокси-протокол. Поддерживает XTLS-Vision (`flow`), REALITY, а также пост-квантовое шифрование на уровне самого VLESS (поля `decryption` / `encryption`). Используется по умолчанию для новых inbound.

Поля блока `settings` :

Поле	Значение по умолчанию	Описание
<code>clients</code>	<code>[]</code>	Список клиентов. У каждого: <code>id</code> (UUID), <code>email</code> (обязателен), <code>flow</code> , лимиты (<code>limitIp</code> , <code>totalGB</code> , <code>expiryTime</code>), <code>enable</code> , <code>tgId</code> , <code>subId</code> , <code>comment</code> , <code>reset</code>

Поле	Значение по умолчанию	Описание
decryption	none	Параметр расшифрования на стороне сервера. Подпись в UI: «Расшифрование» (англ. «Decryption»)
encryption	none	Парный параметр шифрования (попадает в клиентскую ссылку). Подпись: «Шифрование» (англ. «Encryption»)
fallbacks	[]	Список fallback'ов (см. раздел про fallback'и); доступен, когда network = tcp и security = TLS или REALITY
testseed	(4 числа: 900, 500, 900, 256)	«Vision testseed» — 4 положительных целых для XTLS-Vision padding. Применяется только к клиентам с flow xtls-rprx-vision, иначе игнорируется

flow (xtls-rprx-vision)

flow задаётся **на клиенте**, а не на inbound, и принимает одно из трёх значений:

Значение	Смысл
`` (пусто)	Без XTLS-flow (по умолчанию)
xtls-rprx-vision	XTLS-Vision — рекомендуемый режим для VLESS поверх TCP+TLS/REALITY
xtls-rprx-vision-udp443	Тот же Vision, но с обработкой UDP/443 (QUIC)

Поле flow доступно для выбора только когда выполнены все условия: протокол vless, network = tcp и security = tls либо reality. Поле **Vision testseed** в форме показывается только при тех же условиях.

Исключение для XHTTP: при VLESS поверх network = xhttp с включённой пост-квантовой аутентификацией VLESS (encryption / decryption, vlessenc) flow xtls-rprx-vision также допустим — независимо от слоя безопасности, в том числе с REALITY. В этом случае панель корректно передаёт xtls-rprx-vision в share-ссылки и в подписки (включая формат Clash/Mihomo), так что клиент получает конфигурацию именно с Vision.

Расшифрование / Шифрование (пост-квантовая аутентификация VLESS)

Поля decryption и encryption — это аутентификация на уровне самого VLESS (отдельно от транспортного TLS/REALITY). По умолчанию оба равны none. В форме под этими полями расположен блок «Генерация ключей» — выпадающий список режима и кнопка «Сгенерировать» (рядом — кнопка «Очистить»). Выпадающий список содержит шесть вариантов: **X25519 (native)**, **X25519 (xorpub)**, **X25519 (random)**, **ML-KEM-768 (native)**, **ML-KEM-768 (xorpub)**, **ML-KEM-768 (random)** — то есть два типа ключа (классический X25519 и пост-квантовый ML-KEM-768), каждый в трёх режимах:

- **native** — базовая пара ключей выбранного типа;
- **xorpub** — производный режим с дополнительной обработкой публичной части;
- **random** — производный режим со случайной составляющей.

Выберите нужный режим в списке и нажмите **«Сгенерировать»**: панель заполнит **оба** поля (`decryption` и `encryption`) готовой парой значений под этот режим. Кнопка **«Очистить»** сбрасывает оба поля обратно в `none` .

Под блоком выводится строка состояния **«Выбрано: ...»**, которая распознаёт по сгенерированной строке как тип ключа (X25519 или ML-KEM-768), так и режим (native / xorpub / random) и показывает их. Пустые поля или `none` отображаются как «None».

Технически кнопки обращаются к `GET /panel/api/server/getNewVlessEnc` (генерация ключей через `xray vlessenc`) и заполняют **оба** поля парными значениями вида `mlkem768x25519plus.native.<rtt>.<role>` (например, `decryption = mlkem768x25519plus.native.600s.server-x25519` , `encryption = mlkem768x25519plus.native.0rtt.client-x25519`). Параметр `decryption` остаётся на сервере, `encryption` уходит в клиентскую ссылку.

Важно: при генерации конфигурации inbound для Xray панель удаляет лишнее: если в `settings` остаётся `encryption` (которое относится к клиентской стороне), оно **вырезается** из серверного конфига. На самом сервере остаётся только `decryption` .

Когда выбирать VLESS: это рекомендуемый вариант по умолчанию для нового inbound, особенно в связке с REALITY (без собственного сертификата) или с TLS + XTLS-Vision.

Пример: блок `settings` VLESS-inbound с одним клиентом и XTLS-Vision. Поле `flow` стоит на клиенте, `decryption` остаётся на сервере:

```
{
  "clients": [
    {
      "id": "d342d11e-d424-4583-b36e-524ab1f0afa4",
      "email": "user1",
      "flow": "xtls-rprx-vision",
      "limitIp": 2,
      "totalGB": 0,
      "expiryTime": 0,
      "enable": true
    }
  ],
  "decryption": "none"
}
```

Для связки REALITY соответствующий блок `streamSettings` (вкладка «Transport» → Security: REALITY) выглядит так:

```
{
  "network": "tcp",
  "security": "reality",
  "realitySettings": {
    "dest": "www.microsoft.com:443",
    "serverNames": ["www.microsoft.com"],
    "privateKey": "<приватный ключ X25519>",
    "shortIds": ["", "6ba85179e30d4fc2"]
  }
}
```

5.4. VMess

Назначение: классический прокси-протокол Xray. Аутентификация по UUID, на клиенте дополнительно настраивается метод шифрования полезной нагрузки (security).

Поля блока settings :

Поле	Значение по умолчанию	Описание
clients	[]	Список клиентов

У каждого клиента VMess (помимо общих полей email , лимитов, enable , tgId , subId , comment , reset):

Поле клиента	Значение по умолчанию	Описание
id	—	UUID клиента
security	auto	Метод шифрования полезной нагрузки VMess. Допустимые значения: aes-128-gcm , chacha20-poly1305 , auto , none , zero

Значение security :

- auto — Xray сам выбирает шифр в зависимости от платформы (рекомендуется);
- aes-128-gcm , chacha20-poly1305 — фиксированные AEAD-шифры;
- none — без шифрования полезной нагрузки (имеет смысл только поверх TLS);
- zero — без шифрования и без аутентификации полезной нагрузки.

Историческая совместимость: старые записи могли хранить security: "" — при чтении пустая строка приводится к auto . При генерации серверного конфига поле security у клиентов VMess **удаляется** из settings , так как для inbound оно не требуется.

Когда выбирать VMess: для совместимости со старыми клиентами или существующими конфигурациями. Для новых развёртываний обычно предпочтительнее VLESS.

5.5. Trojan

Назначение: прокси, имитирующий обычный HTTPS-трафик. Аутентификация по паролю. Как и VLESS, поддерживает fallback'и и (при `network = tcp`) REALITY/TLS.

Поля блока `settings` :

Поле	Значение по умолчанию	Описание
<code>clients</code>	<code>[]</code>	Список клиентов
<code>fallbacks</code>	<code>[]</code>	Список fallback'ов (доступен при <code>network = tcp</code> и TLS/REALITY)

У каждого клиента Trojan ключевое поле:

Поле клиента	Значение по умолчанию	Описание
<code>password</code>	—	Пароль клиента (обязателен, минимум 1 символ)
<code>email</code>	—	Уникальный идентификатор клиента

Остальные поля клиента — общие (`limitIp`, `totalGB`, `expiryTime`, `enable`, `tgId`, `subId`, `comment`, `reset`).

Когда выбирать Trojan: когда нужна маскировка под HTTPS на 443 порту, в том числе с fallback'ами на веб-сервер (Nginx) для непрошенных подключений.

Пример: блок `settings` Trojan с fallback на локальный веб-сервер. Непрошенные подключения (без валидного пароля) уходят на Nginx, который слушает `127.0.0.1:8080` :

```
{
  "clients": [
    { "password": "S3cret-Pass-1", "email": "user1" }
  ],
  "fallbacks": [
    { "dest": "127.0.0.1:8080" }
  ]
}
```

Для fallback нужны `network = tcp` и Security = TLS или REALITY; иначе поле `fallbacks` недоступно.

5.6. Shadowsocks

Назначение: лёгкий прокси Shadowsocks. Поддерживает как устаревшие AEAD-шифры, так и новые методы SIP022 (2022-blake3-*). Может работать в однопользовательском или многопользовательском режиме.

Поля блока `settings` :

Поле	Значение по умолчанию	Описание
method	2022-blake3-aes-256-gcm	Метод шифрования inbound. Подпись в UI: «Метод шифрования» (англ. «Encryption method»)
password	``	Пароль inbound (для 2022-методов генерируется автоматически под выбранный метод)
network	tcp,udp	Транспорт. Подпись: «Сеть» (англ. «Network»). Варианты: tcp,udp (TCP, UDP), tcp , udp
clients	[]	Список клиентов
ivCheck	false (выкл.)	Переключатель «ivCheck» – защита от повторного использования IV

Методы шифрования (method)

Допустимый набор:

Метод	Категория
aes-256-gcm	Устаревший AEAD
chacha20-poly1305	Устаревший AEAD
chacha20-ietf-poly1305	Устаревший AEAD
xchacha20-ietf-poly1305	Устаревший AEAD
2022-blake3-aes-128-gcm	SS 2022 (рекомендуется)
2022-blake3-aes-256-gcm	SS 2022 (по умолчанию)
2022-blake3-chacha20-poly1305	SS 2022, однопользовательский

Логика панели по методам:

- **2022-методы** (2022-blake3-*) считаются «SS 2022». Метод 2022-blake3-chacha20-poly1305 – **однопользовательский** (multi-user не поддерживается); прочие 2022-методы допускают несколько клиентов. Поле пароля (с кнопкой генерации, подгоняющей длину ключа под метод) показывается в форме именно для 2022-методов.
- **Устаревшие шифры** (aes-* , chacha20-*) работают по классической схеме «один метод + один пароль».

Нормализация перед запуском Xray: для устаревших шифров каждый клиент должен нести method , совпадающий с методом inbound (иначе Xray падает с «unsupported cipher method:»). Для 2022-методов наоборот – поле method у клиента должно быть **пустым** (иначе Xray отвергает inbound с «users must have empty method»). Панель сама приводит данные в порядок при переключении метода.

Перегенерация клиентских ключей при смене размера ключа: для Shadowsocks-2022 при смене метода шифрования на метод с другим размером ключа (например между 2022-blake3-aes-256-

gcm и 2022-blake3-aes-128-gcm) панель автоматически регенерирует клиентские PSK под новую длину при сохранении inbound. Иначе старые ключи остались бы прежней длины, и Xray отверг бы их. Следствие: затронутым клиентам нужно заново получить подписку — прежние ссылки перестанут подключаться.

Когда выбирать Shadowsocks: для простых развёртываний без TLS-маскировки; современный выбор — 2022-blake3-методы.

Пример: блок settings Shadowsocks для метода 2022-blake3 (многопользовательский режим). У inbound — свой пароль (ключ base64 нужной длины), у каждого клиента — свой пароль, поле method клиента пустое:

```
{
  "method": "2022-blake3-aes-256-gcm",
  "password": "d2hhdGV2ZXItMzItYnl0ZS1iYXNlNjQta2V5LWhlcmlU=",
  "network": "tcp,udp",
  "clients": [
    {
      "email": "user1",
      "password": "Y2xpZW50LWtleS0zMl1ieXRlcy1iYXNlNjQtaGVyZQ==",
      "method": ""
    }
  ]
}
```

Для legacy-шифров (aes-256-gcm и т. п.) — наоборот: пароль один на inbound, а method клиента должен совпадать с методом inbound.

5.7. Dokodemo-door / Tunnel (прозрачный форвардер)

Назначение: прозрачный форвардер (в панели — протокол tunnel, реализующий поведение dokodemo-door). Принимает трафик и переадресует его на заданный адрес/порт, без аутентификации и клиентов.

Поля блока settings:

Поле	Значение по умолчанию	Описание
rewriteAddress	(нет)	«Переписать адрес» (англ. «Rewrite address») — целевой адрес, на который перенаправляется трафик
rewritePort	(нет)	«Переписать порт» (англ. «Rewrite port») — целевой порт (0–65535)
allowedNetwork	tcp,udp	«Разрешённая сеть» (англ. «Allowed network»). Варианты: tcp,udp, tcp, udp
portMap	{}	«Сопоставление портов» — карта порт→порт (Record<string,string>)
followRedirect	false (выкл.)	«Следовать redirect» (англ. «Follow redirect») — использовать исходный адрес назначения из перехваченного соединения

Вкладка «Transport» для Tunnel: у inbound этого типа доступна вкладка **«Transport»**, ограниченная настройкой `sockopt` — этого достаточно для режима **TProxy** (прозрачное проксирование/redirect через `sockopt.tproxy`). Выпадающий список выбора транспорта (`network`) и вкладка «Security» для Tunnel скрыты, так как TLS/REALITY этим типом не поддерживаются.

Когда выбирать: для прозрачного проксирования/перенаправления портов на внутренние сервисы.

Поле «Переписать порт» (`rewritePort`) можно оставлять пустым: при очистке значение просто исключается из настроек inbound, а не вызывает ошибку сохранения. (Раньше очистка этого поля приводила к ошибке валидации `settings.rewritePort` и блокировала сохранение, в том числе через вкладку JSON.)

5.8. SOCKS / HTTP (протокол `mixed`)

В этой сборке отдельного протокола `socks` нет — SOCKS и HTTP-прокси объединены в протокол `mixed` (совмещённый SOCKS + HTTP). Кроме того, есть отдельный чистый `http` -прокси.

5.8.1. Mixed (SOCKS + HTTP)

Поля блока `settings` :

Поле	Значение по умолчанию	Описание
<code>auth</code>	<code>password</code>	«Auth» — режим аутентификации. Варианты: <code>password</code> (по логину/паролю) или <code>noauth</code> (без авторизации)
<code>accounts</code>	(опционально)	«Аккаунты» — список пар user/pass. При <code>auth = noauth</code> поле не пишется в конфиг
<code>udp</code>	<code>false</code> (выкл.)	Переключатель «UDP» — поддержка UDP через SOCKS
<code>ip</code>	<code>127.0.0.1</code>	«UDP IP» — локальный адрес для UDP-ассоциаций. Поле показывается только при включённом <code>udp</code>

Аккаунты добавляются кнопкой «Добавить»; при добавлении генерируются случайные логин (8 символов) и пароль (12 символов), которые можно отредактировать.

5.8.2. HTTP (чистый прокси)

Назначение: классический форвард-прокси HTTP. На уровне Xray не отслеживает клиентов как «биллинговых» (нет email/лимитов) — есть только список аккаунтов.

Поля блока `settings` :

Поле	Значение по умолчанию	Описание
accounts	[]	«Аккаунты» — список пар user/pass (оба поля обязательны)
allowTransparent	false (выкл.)	«Разрешить прозрачный» (англ. «Allow transparent») — пересылать запросы с оригинальным заголовком Host

Когда выбирать SOCKS/HTTP: для локального или служебного прокси-доступа без сложной маскировки. mixed удобен тем, что один порт обслуживает и SOCKS, и HTTP-клиентов.

5.9. WireGuard (inbound)

Назначение: inbound WireGuard — VPN-туннель WireGuard, а не маскируемый прокси. Транспорт и TLS/REALITY к нему неприменимы.

С версии 3.4.2 WireGuard переведён на мультиклиентную модель (как VLESS/VMess/Trojan/Shadowsocks/Hysteria). Сам inbound хранит только серверную часть; устройства-пиры теперь добавляются как обычные **клиенты** (см. раздел 8) — с автоматической выдачей адреса в туннеле, поддержкой подписок, лимитов трафика/срока и групп. Прежний инлайн-список «Peer'ы / Добавить реер» из формы inbound убран: новый inbound создаётся без пиров, а каждому клиенту адрес выдаётся автоматически.

Поля inbound (блок settings):

Поле	Значение по умолчанию	Описание
secretKey	—	Приватный ключ сервера (обязателен). Рядом кнопка генерации; публичный ключ (Public Key) выводится из него автоматически (только для чтения; отдельно хранившийся pubKey убран)
mtu	1420	MTU интерфейса
dns	1.1.1.1, 1.0.0.1	Новое в 3.4.2. DNS, записываемый в строку DNS = клиентского .conf
noKernelTun	false (выкл.)	«TUN без kernel» — userspace-TUN вместо ядерного
domainStrategy	(опционально)	«Domain Strategy»: ForceIP, ForceIPv4, ForceIPv4v6, ForceIPv6, ForceIPv6v4

Параметры WireGuard у клиента — вкладка «Учётные данные» в окне добавления/редактирования клиента (показываются, когда привязанный inbound — WireGuard):

Поле	Описание
«Приватный ключ WireGuard»	Приватный ключ клиента (редактируемое, есть кнопка «Перегенерировать»); при вводе из него автоматически выводится публичный ключ
«Публичный ключ WireGuard»	Только для чтения; вычисляется из приватного

Поле	Описание
«Общий ключ WireGuard» (PSK)	Необязательный дополнительный общий ключ
«Разрешённые IP WireGuard»	С 3.5.0 — редактируемое (плейсхолдер <code>10.0.0.2/32</code> , подсказка «Оставьте пустым для авто-выдачи; записи разделяйте запятыми»). Пусто — адрес выдаётся автоматически; сервер проверяет каждую запись (IP или CIDR), одиночные адреса нормализует до <code>/32</code> и отклоняет адрес, занятый другим клиентом этого inbound

Адрес в туннеле выдаётся сервером автоматически из подсети inbound (по умолчанию `10.0.0.0/24` : сервер занимает `.1`, клиенты — начиная с `.2`); если существующие клиенты используют другую `/24`, новые адреса выдаются в той же подсети.

Domain Strategy и вкладка Transport

Помимо перечисленных, у WireGuard-inbound есть поле **Domain Strategy** (стратегия резолвинга доменов: `ForceIP`, `ForceIPv4`, `ForceIPv4v6`, `ForceIPv6`, `ForceIPv6v4`). Поле необязательно и пишется в конфиг, только если задано.

Поле **Workers** (`workers`, число рабочих потоков) убрано из форм WireGuard (как inbound, так и outbound): начиная с `hysteria-core v26.6.22` движок его больше не использует и полагается на внутренний механизм `wireguard-go`. Ранее сохранённые конфиги работают без изменений — при разборе поле просто отбрасывается, миграция не нужна.

Для WireGuard также доступна вкладка «**Transport**» — но в урезанном виде: на ней настраиваются только `sockopt` и обфускация **Finalmask**. Выпадающий список выбора транспорта (`network`) скрыт, поскольку WireGuard всегда слушает по UDP. В шумовых записях (noise) Finalmask отдельным полем задаётся **Rand Range** (диапазон байт 0–255, с валидацией), а в качестве метода обфускации для WireGuard и Hysteria доступен **Salamander**.

Действия inbound и экспорт (с 3.5.0)

Меню строки WireGuard-inbound теперь содержит полный «мультиклиентный» набор действий — «**Экспортировать все ссылки**», экспорт по подписке, **привязка/отвязка клиентов**, добавление в группу и удаление всех клиентов (раньше были только экспорт/сброс/клонирование/удаление), а в списке отображается счётчик клиентов. Для WireGuard окно «Экспорт всех ссылок» разделено на две вкладки: **Config** (склеенные `.conf`-блоки, по одному на клиента) и **Links** (персональные ссылки `wireguard://`); «Копировать» и «Скачать» действуют на открытую вкладку. Для inbound, размещённого на узле, Endpoint в клиентском `.conf` /QR теперь указывает **адрес узла**, а не главной панели.

Конфигурация и шеринг клиента WireGuard

У WireGuard-клиента в окнах «Информация»/QR доступна **сворачиваемая карточка конфигурации**: полный `.conf` (`[Interface]` с `PrivateKey` / `Address` / `DNS` / `MTU` и `[Peer]` с `PublicKey` / `PresharedKey` / `AllowedIPs = 0.0.0.0/0, ::/0` / `Endpoint` / `PersistentKeepalive`) с кнопками **Копировать**, **Скачать** и

QR. Поддерживается также ссылка вида `wireguard:// / wg://`, которую клиент-приложение разбирает в готовый `.conf`.

Когда выбирать WireGuard: когда нужен именно VPN-туннель WireGuard, а не маскируемый прокси.

5.10. Hysteria (по умолчанию v2)

Назначение: inbound Hysteria поверх QUIC. Панель по умолчанию работает с версией 2. Каждый клиент аутентифицируется токеном `auth` вместо UUID/пароля. TLS для Hysteria доступен всегда (см. таблицу возможностей в 5.2).

Поля блока `settings`:

Поле	Значение по умолчанию	Описание
<code>version</code>	2	Версия протокола (минимум 1; панель по умолчанию 2)
<code>clients</code>	[]	Список клиентов

У каждого клиента ключевое поле — `auth` (токен, обязателен) плюс общие поля (`email`, лимиты, `enable`, `tgId`, `subId`, `comment`, `reset`).

Дополнительные параметры задаются в `streamSettings.hysteriaSettings`:

Поле	Значение / варианты	Описание
<code>version</code>	зафиксировано 2 (поле заблокировано)	«Версия» (англ. «Version»)
<code>udpIdleTimeout</code>	(целое $\geq$ 1, сек.)	«UDP idle timeout (с)» — тайм-аут простоя UDP
<code>masquerade</code>	выключен по умолчанию	«Masquerade» — маскировка под обычный веб-сервер при «непрошенных» запросах

При включении `masquerade` доступен выбор типа (`type`):

- ```` — default (страница 404);
- `proxy` — обратный прокси (поля «Upstream URL», «Переписать Host», «Пропустить TLS verify»);
- `file` — раздача каталога (поле «Директория», например `/var/www/html`);
- `string` — фиксированный ответ (поля «Код статуса», «Body», «Заголовки»).

Когда выбирать Hysteria: при потребности в QUIC-транспорте и устойчивости на нестабильных/мобильных каналах; маскировка повышает скрытность точки входа.

5.11. MTProto (прокси для Telegram)

Добавлен в версии **3.3.0**. Значение протокола — `mtproto`.

MTProto — протокол собственного прокси Telegram. В 3X-UI такой inbound **обслуживается не Xray, а отдельным процессом mtg**, которым управляет сама панель. Панель периодически сверяет включённые MTProto-inbound с запущенными процессами `mtg`: поднимает недостающие, останавливает лишние и снимает счётчики трафика с метрик `mtg`. Поэтому **учёт трафика** по такому inbound работает как у обычных протоколов.

С версии 3.5.0 MTProto — мультиклиентный протокол (движок сменён на форк `mtg-multi`): один inbound обслуживает много пользователей, каждый из которых — обычный **клиент** (см. раздел 8) со своим FakeTLS-секретом, квотой трафика, сроком, переключателем включения, необязательным ad-tag и персональной ссылкой `tg://проху`. Прежняя модель «один inbound = один секрет» упразднена: поле «Secret» на уровне inbound из формы убрано, а при обновлении панели существующий одиночный секрет автоматически превращается в первого клиента этого inbound (ручных действий не требуется).

Следствия:

- Вкладка **«Транспорт» (Stream Settings)** к этому inbound **не применяется**; клиенты же теперь управляются как у остальных протоколов — привязка/отвязка, лимиты, группы, подписки.
- Изменения клиентов (добавление, удаление, смена секрета, включение/выключение, ad-tag) применяются **«на лету»** через management-API `mtg`, не разрывая Telegram-сессии остальных пользователей; лишь структурные изменения inbound (адрес, domain fronting, лимит соединений, публичные IP) перезапускают процесс.
- MTProto-inbound запускается **только на основной панели**; на дочерние узлы (nodes) он не разворачивается (inbound с заданным `NodeID` пропускаются).
- Вкладка **«Sniffing»** для MTProto скрыта — этот протокол обслуживается процессом `mtg`, а не Xray, поэтому сниффинг к нему неприменим.

Поля. Хранятся в `settings inbound`:

Поле в UI	Ключ	Описание
Remark	<code>remark</code>	Метка inbound.
Listen IP	<code>listen</code>	IP для прослушивания (пусто = все интерфейсы).
Port	<code>port</code>	Порт прокси.
Домен прикрытия (FakeTLS)	<code>settings.fakeTlsDomain</code>	По умолчанию <code>www.cloudflare.com</code> . С 3.5.0 — домен по умолчанию для генерации секретов новых клиентов (подсказка: «Default FakeTLS domain used to generate a new client's secret. Each client can front its own domain»); каждый клиент может прикрываться собственным доменом через свой секрет.
Max connections	<code>settings.throttleMaxConnections</code>	Новое в 3.5.0. Ограничение одновременных соединений на всех пользователей с честным делением; <code>0</code> — без ограничения.
Public IPv4 / Public IPv6	<code>settings.publicIpv4</code> / <code>settings.publicIpv6</code>	Новое в 3.5.0. Публичный адрес сервера для ad-tag middle proxy (плейсхолдеры <code>1.2.3.4</code> / <code>2001:db8::1</code>); пусто — <code>mtg</code> определит сам.

Формат секрета (FakeTLS). Секрет теперь живёт у **каждого клиента** (поле «**MTProto secret**» в форме клиента, с кнопкой регенерации; рядом — необязательный «**Ad-tag (спонсорский канал)**», ровно 32 hex-символа). Вид секрета прежний: `ee` + 32 hex-символа + hex-код домена прикрытия (`ee<hex32><hex(домен)>`); при привязке нового клиента к MTProto-inbound секрет генерируется автоматически от домена inbound по умолчанию. Квота трафика и срок действия клиента с 3.5.0 **реально применяются** (через лимиты `mtg-multi`): исчерпанный или истёкший клиент отключается, а сброс трафика возвращает доступ сразу.

Domain-fronting и расширенные опции mtg

У MTProto-inbound есть дополнительные параметры процесса `mtg`. Поля **Domain fronting IP**, **Domain fronting port** и **Domain fronting PROXY protocol** задают, куда `mtg` отправляет не-Telegram трафик (например, на фейковый сайт NGINX): если IP оставить пустым, используется FakeTLS-домен через DNS, порт по умолчанию — `443`. Дополнительно доступны **Accept PROXY protocol** (для слушателя), **IP preference** (`prefer-ipv6` / `prefer-ipv4` / `only-ipv6` / `only-ipv4`) и **Debug logging**. Каждое значение записывается в файл `mtg-<id>.toml`, только если оно задано.

Маршрутизация Telegram-трафика через Xray

Переключатель «**Route through Xray**» (по умолчанию выключен) и необязательное поле **Outbound** позволяют подчинить egress Telegram маршрутизатору Xray. При включении панель встраивает в конфиг Xray локальный SOCKS-мост с тегом самого inbound, и `mtg` отправляет Telegram-трафик через него. После этого трафик можно сопоставлять правилами на вкладке «**Routing**» либо принудительно направить в выбранный outbound или балансировщик через поле **Outbound** (если поле пусто, решают правила маршрутизации).

Как раздать пользователю. Для MTProto-inbound панель генерирует ссылку-приглашение:

Пример: секрет FakeTLS и готовая ссылка. Если домен прикрытия — `www.cloudflare.com`, секрет собирается как `ee` + 32 hex-символа + hex-код домена, например:

```
secret = ee1a2b3c4d5e6f70819293a4b5c6d7e8f7777772e636c6f7564666c6172652e636f6d
```

Готовая ссылка-приглашение (её и QR-код отправляют пользователю в Telegram):

```
tg://proxy?
server=203.0.113.10&port=443&secret=ee1a2b3c4d5e6f70819293a4b5c6d7e8f7777772e636c6f7564666c6172652e636f6d
```

```
tg://proxy?server=<адрес>&port=<порт>&secret=<секрет>
```

(эквивалент — `https://t.me/proxy?server=...&port=...&secret=...`). С 3.5.0 ссылка **персональная** — строится из секрета конкретного клиента, а фрагмент `#remark` из неё убран (нестрогие парсеры Telegram склеивали его с секретом и ломали импорт); примечание показывается отдельной меткой в окне информации. Эту ссылку и QR-код нужно отправить пользователю Telegram — при открытии прокси сразу добавляется в приложение. Ссылка также отдаётся через сервер подписок.

Когда использовать. Штатный способ обхода блокировок Telegram; FakeTLS-маскировка (домен прикрытия) делает трафик похожим на обычный визит на указанный сайт.

5.12. Краткая шпаргалка по выбору протокола

- **VLESS** — выбор по умолчанию; лучший вариант с REALITY или TLS + XTLS-Vision, поддерживает пост-квантовую аутентификацию.
 - **Trojan** — маскировка под HTTPS с fallback'ами на веб-сервер.
 - **VMess** — совместимость со старыми клиентами.
 - **Shadowsocks** — простой прокси без TLS; современный выбор — методы `2022-blake3-*`.
 - **Hysteria** — QUIC, устойчивость на плохих каналах.
 - **mixed / http** — служебные SOCKS/HTTP-прокси.
 - **WireGuard** — полноценный VPN-туннель.
 - **tunnel** — прозрачное перенаправление портов.
 - **MTPROTO** — прокси для обхода блокировок Telegram (FakeTLS); отдельный процесс `mtg`.
-

6. Транспорт (Stream Settings)

Транспорт (в интерфейсе панели — поле «**Транспорт**», англ. *Transmission*) определяет способ, которым Xray-core передаёт данные внутри inbound: какой сетевой протокол используется поверх TLS/Reality и как именно кадрируется трафик. Эти параметры сохраняются в объект `streamSettings` конфигурации Xray и задаются на вкладке транспорта в редакторе inbound. Шифрование (TLS / Reality) рассматривается в отдельном разделе — здесь описывается только выбор сети и её параметры.

6.1. Выбор сети передачи

Сеть выбирается в выпадающем списке «**Транспорт**» (`streamSettings.network`). Значение по умолчанию — `tcp` (в списке отображается как **RAW**). Доступны следующие варианты:

Значение в списке	Поле <code>network</code>	Транспорт
RAW	<code>tcp</code>	Обычный TCP (в новых версиях Xray переименован в RAW), опционально с HTTP-обфускацией
mKCP	<code>kcp</code>	Надёжный UDP-транспорт mKCP
WebSocket	<code>ws</code>	WebSocket поверх HTTP(S)
gRPC	<code>grpc</code>	gRPC-туннель (HTTP/2)
HTTPUpgrade	<code>httpupgrade</code>	HTTP Upgrade
XHTTP	<code>xhttp</code>	XHTTP / SplitHTTP — современный мультиплексируемый транспорт

При смене значения панель очищает блок настроек предыдущей сети и заполняет блок новой сети значениями по умолчанию из её схемы, поэтому каждое поле под-формы всегда имеет осмысленное начальное значение.

Важно. В данной сборке панели **транспорт HTTP/2 (h2) в списке отсутствует** — он был исключён из набора сетей; для двунаправленного HTTP/2-подобного туннеля используется gRPC, а для современного HTTP-маскированного транспорта — XHTTP. Транспорт **Hysteria (hysteria)** не выбирается через этот список: он жёстко привязан к протоколу Hysteria и появляется автоматически, когда сам inbound создаётся с протоколом Hysteria (см. п. 6.8).

Ниже каждая сеть и каждое её поле разобраны отдельно.

6.2. RAW / TCP (`tcpSettings`)

Базовый TCP-транспорт. По умолчанию трафик передаётся «как есть»; опционально его можно замаскировать под обычный HTTP/1.1-обмен.

Поле	Значение по умолчанию	Описание
Proxy Protocol (<code>acceptProxyProtocol</code>)	<code>false</code> (выкл.)	Принимать заголовок PROXY protocol от вышестоящего балансировщика/прокси
HTTP-обфускация (<code>header.type</code>)	<code>none</code> (выкл.)	Включает маскировку трафика под HTTP/1.1

Proxy Protocol

Переключатель **«Proxy Protocol»** (`acceptProxyProtocol`). Когда включён, Xray ожидает на входящем соединении заголовок PROXY protocol и извлекает из него реальный IP клиента. Включают только если перед панелью стоит обратный прокси/балансировщик (например, HAProxy или nginx со `send-proxy`), который этот заголовок добавляет. По умолчанию выключен.

HTTP-обфускация (camouflage)

Переключатель **«HTTP Обфускация»**. Управляет полем `header` :

- **Выключен** → `header.type = "none"` (на проводе поле `header` просто отсутствует). Чистый TCP.
- **Включён** → `header.type = "http"` . Трафик кадрируется под вид HTTP/1.1-запроса и ответа. При включении панель сразу заполняет под-объекты `request` и `response` значениями по умолчанию.

При включённой HTTP-обфускации появляются поля настройки имитируемых запроса и ответа.

Заголовок запроса (`header.request`):

Поле	Ключ	Значение по умолчанию	Описание
Версия запроса	<code>request.version</code>	<code>1.1</code>	Версия HTTP в стартовой строке запроса
Метод запроса	<code>request.method</code>	<code>GET</code>	HTTP-метод имитируемого запроса
Путь запроса	<code>request.path</code>	<code>/</code>	Путь(и). Вводится списком значений через запятую; на проводе это массив строк. Если оставить пустым, подставляется <code>/</code>
Заголовки запроса	<code>request.headers</code>	<code>{}</code> (пусто)	Таблица «Имя/Значение» HTTP-заголовков. Хранится как карта <code>имя → [значения]</code> (одному имени может соответствовать несколько значений)

Заголовок ответа (`header.response`):

Поле	Ключ	Значение по умолчанию	Описание
Версия ответа	<code>response.version</code>	<code>1.1</code>	Версия HTTP в стартовой строке ответа
Статус ответа	<code>response.status</code>	<code>200</code>	HTTP-код статуса имитируемого ответа

Поле	Ключ	Значение по умолчанию	Описание
Причина ответа	<code>response.reason</code>	OK	Текстовое описание статуса (reason-phrase)
Заголовки ответа	<code>response.headers</code>	<code>{}</code> (пусто)	Таблица «Имя/Значение» заголовков ответа (карта <code>имя → [значения]</code>)

Поля заголовков редактируются построчно — каждая строка задаёт имя заголовка (`Имя`) и его значение (`Значение`). Эти параметры используют только для маскировки внешнего вида трафика; на криптографию они не влияют. Значения по умолчанию (`GET / HTTP/1.1` , ответ `200 OK`) подходят для большинства сценариев — менять их стоит, только если требуется имитировать конкретный сайт/сервис.

Пример `streamSettings` для RAW с HTTP-обфускацией:

```
{
  "network": "tcp",
  "tcpSettings": {
    "acceptProxyProtocol": false,
    "header": {
      "type": "http",
      "request": {
        "version": "1.1",
        "method": "GET",
        "path": ["/"],
        "headers": {
          "Host": ["www.example.com"]
        }
      },
      "response": {
        "version": "1.1",
        "status": "200",
        "reason": "OK"
      }
    }
  }
}
```

Обратите внимание: `path` на проводе — массив строк, а каждый заголовок — массив значений (`Host → ["www.example.com"]`).

6.3. mKCP (`kcpSettings`)

mKCP — надёжный транспорт поверх UDP. Полезен на каналах с потерями пакетов и высокой задержкой, но создаёт повышенный служебный трафик. Все значения по умолчанию совпадают с рекомендованными в xray-core.

Поле	Ключ	По умолчанию	Допустимо	Описание
MTU	<code>mtu</code>	<code>1350</code>	576–1460	Максимальный размер пакета (байт). Уменьшают при проблемах фрагментации
TTI (мс)	<code>tti</code>	<code>20</code>	10–100	Интервал передачи (ms). Меньше — ниже задержка, но выше накладные расходы
Uplink (МБ/с)	<code>uplinkCapacity</code>	<code>5</code>	≥ 0	Расчётная пропускная способность на отдачу (МБ/с)
Downlink (МБ/с)	<code>downlinkCapacity</code>	<code>20</code>	≥ 0	Расчётная пропускная способность на приём (МБ/с)
Множитель CWND	<code>cwndMultiplier</code>	<code>1</code>	≥ 1	Множитель окна перегрузки (congestion window)
Макс. окно отправки	<code>maxSendingWindow</code>	<code>2097152</code>	≥ 0	Максимальный размер окна отправки

Замечания по полям:

- **Uplink / Downlink capacity** задают, насколько агрессивно mKCP занимает канал. Их выставляют в соответствии с реальной шириной канала: завышенные значения ведут к лишнему трафику, заниженные — к недоиспользованию канала.
- **TTI** напрямую влияет на компромисс «задержка [?] накладные расходы»: меньшие значения снижают задержку, но увеличивают объём служебных пакетов.
- **MTU** ограничивает размер одного пакета mKCP; снижение помогает на каналах, где крупные UDP-пакеты режутся или теряются.

В данной сборке поле «seed» (пароль обфускации mKCP) и выпадающий список **типа заголовка/обфускации** (`none` , `srtп` , `utp` , `wechat-video` , `dtls` , `wireguard`) в под-форме mKCP **отсутствуют как отдельные поля** — обфускация транспортного уровня вынесена в общий механизм «FinalMask» (включая режим `mkcp-legacy`), описываемый в соответствующем разделе. Параметр «congestion» как отдельная галочка также не выведен; управление перегрузкой задаётся через `cwndMultiplier` и `maxSendingWindow` .

6.4. WebSocket (`wsSettings`)

WebSocket-транспорт поверх HTTP(S). Хорошо проходит через CDN и обратные прокси, маскируется под обычный веб-трафик.

Поле	Ключ	По умолчанию	Описание
Proxy Protocol	<code>acceptProxyProtocol</code>	<code>false</code>	Принимать заголовок PROXY protocol от вышестоящего прокси (см. п. 6.2)

Поле	Ключ	По умолчанию	Описание
Хост	<code>host</code>	<code>""</code> (пусто)	Значение HTTP-заголовка <code>Host</code> . Указывают при работе через CDN/домен-фронтинг
Путь	<code>path</code>	<code>/</code>	Путь в строке запроса WebSocket-рукопожатия
Период heartbeat	<code>heartbeatPeriod</code>	<code>0</code>	Интервал отправки heartbeat-кадров (в секундах). <code>0</code> отключает heartbeat
Заголовки	<code>headers</code>	<code>{}</code> (пусто)	Дополнительные HTTP-заголовки рукопожатия. Карта «Имя → Значение» (только строковые значения, без массивов)

Замечания:

- **Путь** должен совпадать на сервере (inbound) и у клиента. Часто за этим путём на стороне веб-сервера маскируют точку входа.
- **Хост** имеет смысл задавать, если inbound стоит за CDN или используется фронтинг по домену; иначе можно оставить пустым.
- **Период heartbeat** держит соединение «живым» при прохождении через прокси/CDN, агрессивно разрывающие неактивные сессии. По умолчанию (`0`) heartbeat выключен.
- В отличие от RAW, таблица заголовков WebSocket использует «плоский» формат `имя → значение` (одна строка значения на заголовок).

Пример `streamSettings` для WebSocket за CDN:

```
{
  "network": "ws",
  "wsSettings": {
    "acceptProxyProtocol": false,
    "host": "cdn.example.com",
    "path": "/ray",
    "heartbeatPeriod": 0,
    "headers": {
      "User-Agent": "Mozilla/5.0"
    }
  }
}
```

Значения `host` и `path` должны совпадать у клиента; в отличие от RAW значение заголовка здесь — обычная строка, а не массив.

6.5. gRPC (`grpcSettings`)

Самый «лёгкий» по числу параметров транспорт. Туннелирует трафик внутри gRPC-вызовов (поверх HTTP/2); хорошо совместим с CDN, поддерживающими gRPC. Обфускации заголовков нет.

Поле	Ключ	По умолчанию	Описание
Имя сервиса (Service Name)	serviceName	"" (пусто)	Имя gRPC-сервиса (фактически — «путь» туннеля). Должно совпадать у сервера и клиента
Authority	authority	"" (пусто)	Значение псевдо-заголовка :authority (аналог Host для HTTP/2). Указывают при работе через CDN/домен
Multi Mode	multiMode	false (выкл.)	Включает мультиплексирование нескольких параллельных gRPC-потоків внутри одного соединения

Замечания:

- **Service Name** — основной идентификатор gRPC-канала; он должен быть одинаковым на обеих сторонах. Пустое значение допустимо, но обычно задают неочевидную строку для маскировки.
- **Authority** влияет на то, какой :authority отправляется в HTTP/2-кадрах; нужен в первую очередь при проксировании через CDN.
- **Multi Mode** позволяет нескольким логическим потокам идти через одно физическое соединение; включают для повышения производительности, когда и сервер, и клиент это поддерживают.

Пример streamSettings для gRPC:

```
{
  "network": "grpc",
  "grpcSettings": {
    "serviceName": "GunService",
    "authority": "grpc.example.com",
    "multiMode": false
  }
}
```

serviceName (здесь GunService) играет роль «пути» туннеля и должен совпадать у сервера и клиента.

6.6. HTTPUpgrade (httpupgradeSettings)

Транспорт на основе механизма HTTP Upgrade (как у WebSocket, но без самого WebSocket-протокола). Также хорошо проходит через прокси и CDN. Набор полей повторяет WebSocket, но **без** периода heartbeat.

Поле	Ключ	По умолчанию	Описание
Proxy Protocol	acceptProxyProtocol	false	Принимать заголовок PROXY protocol от вышестоящего прокси
Хост	host	"" (пусто)	Значение HTTP-заголовка Host
Путь	path	/	Путь HTTP-запроса с заголовком Upgrade
Заголовки	headers	{ } (пусто)	Дополнительные HTTP-заголовки. «Плоская» карта имя → значение (как у WebSocket)

Назначение полей **Хост**, **Путь** и **Заголовки** совпадает с WebSocket (п. 6.4). Heartbeat для HTTPUpgrade не предусмотрен — это специфика WebSocket.

6.7. ХНТТP / SplitHTTр (xhttpSettings)

ХНТТP (он же SplitHTTр) — современный мультиплексируемый HTTP-транспорт xray-core. Разделяет восходящий и нисходящий потоки на отдельные HTTP-запросы, что хорошо подходит для CDN и сред с ограничениями по длительности соединений. Поля видимы в редакторе не все сразу: часть из них появляется в зависимости от выбранного режима (mode) и включённых переключателей.

Базовые поля (видны всегда)

Поле	Ключ	По умолчанию	Описание
Хост	host	"" (пусто)	Значение HTTP-заголовка Host
Путь	path	/	Базовый путь HTTP-запросов
Режим (Mode)	mode	auto	Режим передачи (см. ниже)
Server Max Header Bytes	serverMaxHeaderBytes	0	Лимит размера заголовков запроса на сервере (байт). 0 — значение по умолчанию xray-core
Padding Bytes	xPaddingBytes	100–1000	Диапазон случайного «набивочного» паддинга (в байтах, формат мин–макс) для затруднения анализа размеров
Заголовки	headers	{ } (пусто)	Дополнительные HTTP-заголовки. «Плоская» карта имя → значение
HTTP-метод Uplink	uplinkHTTPMethod	"" (Default = POST)	HTTP-метод восходящих запросов. Варианты: пусто (по умолчанию POST), POST , PUT , GET (последний доступен только в режиме packet-up)
Padding Obfs Mode	xPaddingObfsMode	false (выкл.)	Включает расширенную обфускацию паддинга и открывает дополнительные поля (см. ниже)
No SSE Header	noSSEHeader	false (выкл.)	Не отправлять заголовок Content-Type: text/event-stream (SSE). Включают, если он мешает прохождению через промежуточные узлы

Поле «Режим» (mode)

Выпадающий список со значениями:

Значение	Описание
auto	Авто-выбор режима (по умолчанию)
packet-up	Восходящий поток разбивается на отдельные HTTP-запросы (по пакету на запрос)
stream-up	Восходящий поток передаётся одним длительным потоковым запросом

Значение	Описание
stream-one	Один общий двунаправленный потоковый запрос

Выбор режима определяет, какие дополнительные поля становятся видимыми.

Поля, видимые только при `mode = packet-up` :

Поле	Ключ	По умолчанию	Описание
Макс. буферизованная загрузка	scMaxBufferedPosts	30	Максимум одновременно буферизуемых POST-запросов восходящего потока
Макс. размер загрузки (байт)	scMaxEachPostBytes	1000000	Максимальный размер одного восходящего POST-запроса (байт)
Uplink Data Placement	uplinkDataPlacement	"" (Default = body)	Где размещать данные восходящего потока: <code>body</code> , <code>header</code> , <code>cookie</code> , <code>query</code>
Uplink Data Key	uplinkDataKey	""	Имя ключа/заголовка для данных uplink. Появляется, только если <code>uplinkDataPlacement</code> задан и не равен <code>body</code>

Поле, видимое только при `mode = stream-up` :

Поле	Ключ	По умолчанию	Описание
Stream-Up Server	scStreamUpServerSecs	20-80	Диапазон времени удержания серверного потокового соединения (в секундах, формат <code>мин-макс</code>)

Поля обфускации паддинга (видны при `xPaddingObfsMode = вкл.`)

Поле	Ключ	По умолчанию	Описание
Padding Key	xPaddingKey	"" (placeholder <code>x_padding</code>)	Имя ключа для паддинга
Padding Header	xPaddingHeader	"" (placeholder <code>X-Padding</code>)	Имя HTTP-заголовка, в котором передаётся паддинг
Padding Placement	xPaddingPlacement	"" (Default = <code>queryInHeader</code>)	Где размещать паддинг: <code>queryInHeader</code> , <code>header</code> , <code>cookie</code> , <code>query</code>
Padding Method	xPaddingMethod	"" (Default = <code>repeat-x</code>)	Метод генерации паддинга: <code>repeat-x</code> или <code>tokenish</code>

Размещение сессии и последовательности (видны всегда)

Поле	Ключ	По умолчанию	Описание
Session ID Placement	<code>sessionIDPlacement</code>	<code>""</code> (Default = path)	Где передавать идентификатор сессии: <code>path</code> , <code>header</code> , <code>cookie</code> , <code>query</code>
Session ID Key	<code>sessionIDKey</code>	<code>""</code> (placeholder <code>x_session</code>)	Имя ключа сессии. Появляется, только если <code>sessionIDPlacement</code> задан и не равен <code>path</code>
Session ID Table	<code>sessionIDTable</code>	<code>""</code> (placeholder <code>Base62</code>)	Набор символов для генерации идентификаторов сессии. Можно выбрать predetermined из выпадающего списка-автодополнения (<code>ALPHABET</code> , <code>Alphabet</code> , <code>BASE36</code> , <code>Base62</code> , <code>HEX</code> , <code>alphabet</code> , <code>base36</code> , <code>hex</code> , <code>number</code>) или ввести произвольную ASCII-строку. Пусто – значение по умолчанию <code>xray-core</code>
Session ID Length	<code>sessionIDLength</code>	<code>""</code> (пусто)	Длина или диапазон (например <code>8–16</code>) генерируемых идентификаторов. Показывается, только когда задан <code>Session ID Table</code> ; минимум должен быть больше 0
Sequence Placement	<code>seqPlacement</code>	<code>""</code> (Default = path)	Где передавать порядковый номер пакета: <code>path</code> , <code>header</code> , <code>cookie</code> , <code>query</code>
Sequence Key	<code>seqKey</code>	<code>""</code> (placeholder <code>x_seq</code>)	Имя ключа последовательности. Появляется, только если <code>seqPlacement</code> задан и не равен <code>path</code>

Поля сессии переименованы под `xray-core v26.6.22`: ранее они назывались **Session Placement / Session Key** (`sessionPlacement` / `sessionKey`) – теперь это **Session ID Placement / Session ID Key** (`sessionIDPlacement` / `sessionIDKey`); старого имени ядро больше не понимает. Inbound, сохранённые до обновления, мигрируются на новые ключи автоматически – пересохранять их не требуется.

Рекомендации:

- Для большинства установок достаточно оставить **Режим** = `auto`, задать **Путь/Хост** и (при работе через CDN) согласовать их с клиентом.
- Поля размещения (`*Placement` / `*Key`) и обфускации паддинга нужны только для тонкой подстройки под конкретный анти-DPI/CDN-сценарий; при пустых значениях используются значения по умолчанию `xray-core`, указанные в скобках.
- Параметры, относящиеся к стороне клиента/outbound (например, интервалы повторных POST, размеры чанков), в форме inbound не выводятся – сервер-слушатель их игнорирует. Мультиплексор XMUX, напротив, в форме inbound доступен (см. ниже).
- Служебные дефолты не проставляются.** Панель больше не записывает в конфиги XHTTP служебные значения по умолчанию `scMaxEachPostBytes` и `scMinPostsIntervalMs` – применяются внутренние значения `xray-core`. Это устраняет постоянную DPI-сигнатуру (литерал `scMinPostsIntervalMs=30`), по которой раньше мог блокироваться трафик. Для уже сохранённых inbound совпадающие с дефолтами `xray-core` значения не выводятся в ссылках и подписке (пересохранять inbound не требуется); заданные вручную значения сохраняются.

Пример `streamSettings` для ХНТТР (режим `auto`):

```
{
  "network": "xhttp",
  "xhttpSettings": {
    "host": "xhttp.example.com",
    "path": "/yourpath",
    "mode": "auto",
    "xPaddingBytes": "100-1000"
  }
}
```

Для большинства установок достаточно этих четырёх полей; поля размещения сессии/последовательности и обфускации паддинга оставляют пустыми — тогда используются значения по умолчанию `xgray-core`.

ХМУХ (мультиплексирование соединений)

Переключатель **ХМУХ** (`enableXmux`) включает слой мультиплексирования, который распределяет параллельные запросы по небольшому пулу физических соединений. При включении раскрываются шесть настраиваемых полей (хранятся в `xhttpSettings.xmux`):

Поле	Ключ	По умолчанию	Описание
Max Concurrency	<code>maxConcurrency</code>	16-32	Максимум одновременных запросов на одно соединение (диапазон мин-макс)
Max Connections	<code>maxConnections</code>	6	Максимум физических соединений (0 — без ограничения). С версии 3.4.2 новые inbound создаются со значением 6 ; ранее созданные сохраняют своё.
Max Reuse Times	<code>cMaxReuseTimes</code>	"" (пусто)	Сколько раз переиспользовать соединение
Max Request Times	<code>hMaxRequestTimes</code>	600-900	Максимум запросов на соединение (диапазон)
Max Reusable Secs	<code>hMaxReusableSecs</code>	1800-3000	Время, в течение которого соединение пригодно для повторного использования (секунды, диапазон)
Keep Alive Period	<code>hKeepAlivePeriod</code>	"" (пусто)	Период keep-alive для удержания соединения

Важно. Задавать одновременно **Max Connections** и **Max Concurrency** нельзя — `xgray-core` отклонит такой конфиг. По умолчанию при включении ХМУХ панель проставляет `Max Concurrency = 16-32` ; если вы зададите **Max Connections** (значение больше 0), панель уберёт значение `Max Concurrency` по умолчанию, чтобы избежать конфликта.

6.8. Транспорт Hysteria (`hysteriaSettings`)

Транспорт **Hysteria** не выбирается в списке «Транспорт»: он автоматически активируется, когда inbound создаётся с протоколом Hysteria, и для других протоколов скрыт (при уходе с протокола Hysteria сеть принудительно возвращается к `tcp`). Параметры:

Поле	Ключ	По умолчанию	Описание
Версия	<code>version</code>	2 (зафиксировано, поле заблокировано)	Версия Hysteria. Поддерживается только Hysteria 2
UDP idle timeout (с)	<code>udpIdleTimeout</code>	60	Таймаут простоя UDP-сессии (секунды). Допустимый диапазон — 2–600; xray-core отклоняет значения вне этого интервала при запуске
Masquerade	<code>masquerade</code>	выкл. (отсутствует)	Включает маскировку слушателя под HTTP/3-сервер при зондировании

При включённом **Masquerade** появляется выбор типа (`type`) и зависящие от него поля:

- **"" — default (404 page)**: отдаётся стандартная страница 404 (доп. поля не требуются).
- **proxy (reverse proxy)**: обратное проксирование на внешний сайт.
 - `url` (**Upstream URL**, placeholder `https://www.example.com`) — целевой адрес;
 - `rewriteHost` (**Переписать Host**, по умолчанию `false`) — подменять заголовок `Host` ;
 - `insecure` (**Пропустить TLS verify**, по умолчанию `false`) — не проверять TLS-сертификат апстрима.
- **file (serve directory)**: раздача файлов из каталога.
 - `dir` (**Директория**, placeholder `/var/www/html`).
- **string (fixed body)**: фиксированный HTTP-ответ.
 - `statusCode` (**Код статуса**, по умолчанию `0` , диапазон 0–599);
 - `content` (**Body**) — тело ответа;
 - `headers` (**Заголовки**) — карта `имя → значение` .

Masquerade позволяет inbound на основе Hysteria выглядеть как обычный HTTP/3-сервер для активных проб, что повышает скрытность. По умолчанию маскировка выключена.

Пример `hysteriaSettings` с обратным проксированием (`masquerade` → `proxy`):

```
{
  "version": 2,
  "udpIdleTimeout": 60,
  "masquerade": {
    "type": "proxy",
    "url": "https://www.example.com",
    "rewriteHost": true,
    "insecure": false
  }
}
```

Здесь при зондировании слушатель отдаёт ответ от `https://www.example.com` , маскируясь под обычный HTTP/3-сайт.

6.9. Сопутствующие параметры

Помимо выбора сети, на той же вкладке доступны два общих блока, не зависящих от конкретного транспорта (подробно — в соответствующих разделах):

- **External Proxy** (`externalProxy`) — список внешних адресов/портов, которые подставляются в ссылки-подписки вместо адреса самой панели.
- **Sockopt** (`sockopt`) — низкоуровневые опции сокета (TCP Fast Open, mark, домен-стратегия, прозрачное проксирование и т. п.).

Real client IP (определение реального IP за CDN/релеем)

Когда inbound стоит за посредником (CDN вроде Cloudflare, L4-туннель/релей или другая панель), Xray видит адрес посредника, а не реального посетителя. Этот адрес попадает в список онлайн-клиентов и по нему считается лимит IP на клиента, из-за чего оба становятся бесполезны за прокси. Чтобы восстановить реальный IP, в секции **Sockopt** формы inbound есть выбор пресета **Real client IP**, объединяющий настройки `acceptProxyProtocol` и `trustedXForwardedFor`:

Пресет	Что делает	Когда применять
Off / direct	Очищает оба поля.	Inbound доступен клиентам напрямую
Cloudflare CDN	Устанавливает <code>sockopt.trustedXForwardedFor = ["CF-Connecting-IP"]</code> .	WebSocket / HTTPUpgrade / XHTTP / gRPC за CDN Cloudflare (оранжевое облако)
L4 relay / Spectrum (PROXY)	Включает <code>acceptProxyProtocol = true</code> .	L4-туннель/релей перед inbound или Cloudflare Spectrum

Пресеты взаимоисключающие: выбор одного очищает поле другого, поэтому устаревший `trustedXForwardedFor` не перебивает восстановленный по PROXY-протоколу IP. Ниже пресета остаются видны «сырые» переключатель **Proxy Protocol** и список **Trusted X-Forwarded-For** — пресет лишь заполняет их за вас, а при необходимости их правят вручную. Если выбранный пресет не поддерживается текущим транспортом (например, PROXY-протокол на mKCP), форма показывает предупреждение. Эти поля относятся только к стороне сервера и **никогда не отправляются клиентам в подписках**.

Используйте что-то одно. `acceptProxyProtocol` читает реальный IP из L4-заголовка PROXY-протокола, а `trustedXForwardedFor` — из HTTP-заголовка запроса; смешивать их вручную стоит, только если этого требует ваша цепочка посредников.

- **FinalMask** (`finalmask`) — общий механизм обфускации транспортного уровня (включая legacy-обфускацию mKCP), который заменил собой отдельные поля «seed»/«header type» внутри под-форм сетей.

7. Безопасность соединения: TLS, XTLS и REALITY

Каждый inbound, поддерживающий передачу через transport-поток (VMess, VLESS, Trojan, Shadowsocks, Hysteria), имеет вкладку «Безопасность» в редакторе. На ней настраивается, как именно шифруется и маскируется транспортный канал. Доступны три режима, переключаемые кнопками-радио:

Режим	Подпись в UI	Когда доступен
none	Нет	Всегда (кроме Hysteria, где TLS обязателен)
tls	TLS	Для VMess/VLESS/Trojan/Shadowsocks на сетях tcp, ws, http, grpc, httpupgrade, xhttp; для Hysteria — всегда
reality	Reality	Только для VLESS/Trojan на сетях tcp, http, grpc, xhttp

Кнопка **Нет** не показывается, если протокол — Hysteria (для него TLS обязателен). Кнопка **Reality** появляется только при допустимой комбинации протокола и сети (см. таблицу выше).

При смене режима панель полностью пересобирает блок `streamSettings`: удаляются `tlsSettings` и `realitySettings` от предыдущего режима и подставляются значения по умолчанию для выбранного. В частности, при выборе **Reality** панель сразу автоматически: подставляет случайную пару `target` + `serverNames` (SNI) из встроенного списка популярных доменов, генерирует случайные `shortIds`, а также делает запрос к серверу за свежей парой ключей X25519 (`privateKey/publicKey`).

7.1. В чём разница: TLS vs XTLS vs REALITY

- **TLS** — классическое шифрование транспорта по протоколу TLS 1.2/1.3. На сервере должен лежать действительный сертификат (свой домен + цепочка). Трафик выглядит как обычный HTTPS, но для активного цензора характерен распознаваемый TLS-handshake к вашему домену; при «отстреле» по SNI или при отсутствии доверенного сертификата соединение блокируется/выдаёт ошибку.
- **XTLS (Vision)** — это не отдельный режим в списке «Безопасность», а *flow*-механизм поверх TLS или Reality. Включается на стороне клиента inbound через поле **Flow** = `xtls-rprx-vision` (или `xtls-rprx-vision-udp443`). Vision доступен для VLESS на сети `tcp` при `security = tls` или `security = reality`, а также для VLESS поверх транспорта `xhttp` при включённом VLESS-шифровании (`vlessenc` / ML-KEM) — в этом случае поле **Flow** также можно установить в `xtls-rprx-vision`, и значение корректно попадает в ссылку `vless://` (`flow=xtls-rprx-vision`). Vision снижает «двойное шифрование» (TLS-in-TLS), отдавая полезную нагрузку напрямую после рукопожатия, что ускоряет передачу и улучшает маскировку. Поэтому связка **VLESS + Reality + Flow xtls-rprx-vision** считается рекомендуемой современной конфигурацией.

Автоматическое восстановление flow Vision. Если у VLESS/XHTTP-inbound шифрование (ML-KEM, поля `decryption/encryption`) включается уже после того, как были добавлены клиенты, inbound становится flow-пригодным. В этой ситуации панель сама восстанавливает `flow = xtls-rprx-vision` у тех клиентов, которым он положен, но у кого поле **Flow** оказалось пустым. Раньше при таком сценарии Vision молча пропадал из конфигов, share-ссылок и подписок (особенно заметно на узловых inbound). Никаких ручных действий не требуется: исправление применяется автоматически

при сохранении inbound и однократно при обновлении панели. Поведение консервативное — панель не выдумывает flow и не перезаписывает явно заданное клиентом значение.

- **REALITY** — механизм маскировки без собственного сертификата. Сервер «заимствует» TLS-рукопожатие реального стороннего сайта (`target / serverNames`), поэтому для наблюдателя соединение неотличимо от обращения к этому сайту, а сертификат не нужен вовсе. Аутентификация строится на паре ключей X25519 и наборе `shortIds`. REALITY устойчив к активным пробам (`active probing`) и блокировке по SNI, поскольку SNI указывает на настоящий внешний домен. Цена — более строгие требования к настройке (корректный `target` с портом, синхронизация ключей с клиентом).

Короткое правило выбора:

- есть свой домен и валидный сертификат, нужен простой HTTPS-вид → **TLS** (по возможности с Vision);
- нет домена/сертификата либо нужна максимальная скрытность от DPI → **REALITY** (с Vision для VLESS/TCP).

7.2. Режим «Нет» (`none`)

Транспорт передаётся без TLS-обёртки: блоки `tlsSettings` и `realitySettings` из `streamSettings` исключаются. Дополнительных полей у режима нет. Подходит, когда:

- inbound слушает только на `127.0.0.1` и служит fallback-целью (по правилу панели дочерний inbound для fallback должен слушать на `127.0.0.1` с `security=none`);
- шифрование/маскировку обеспечивает внешний слой (например, обратный прокси Nginx перед панелью);
- используется протокол с собственным шифрованием (Shadowsocks) во внутренней сети.

Для inbound, доступных извне, режим «Нет» не рекомендуется.

Пример: блок `streamSettings` для TLS на сети `tcp` (VLESS/Trojan/VMess). Так выглядит результат после выбора режима **TLS** и заполнения SNI и путей к сертификату:

```

"streamSettings": {
  "network": "tcp",
  "security": "tls",
  "tlsSettings": {
    "serverName": "vpn.example.com",
    "minVersion": "1.2",
    "maxVersion": "1.3",
    "alpn": ["h2", "http/1.1"],
    "settings": { "fingerprint": "chrome" },
    "certificates": [
      {
        "certificateFile": "/root/cert/vpn.example.com.crt",
        "keyFile": "/root/cert/vpn.example.com.key",
        "ocspStapling": 3600,
        "usage": "encipherment"
      }
    ]
  }
}

```

7.3. Режим TLS

Поля блока `tlsSettings`. Значения по умолчанию взяты из схемы панели.

Основные параметры

Поле (подпись)	Значение по умолчанию	Описание
SNI (<code>serverName</code>)	<code>""</code> (пусто)	Server Name Indication — имя домена, предъявляемое в TLS-рукопожатии. Должно совпадать с доменом сертификата. Англ. подсказка-плейсхолдер: «Server Name Indication».
Cipher Suites (<code>cipherSuites</code>)	<code>""</code> → Авто	Список разрешённых наборов шифров. По умолчанию пусто — выбор отдаётся на усмотрение Xray/Go (вариант Авто). Менять только при необходимости явно ограничить шифры.
Мин/Макс версия (<code>minMaxVersion</code>)	min = <code>1.2</code> , max = <code>1.3</code>	Минимальная и максимальная версии TLS. Доступные значения: <code>1.0</code> , <code>1.1</code> , <code>1.2</code> , <code>1.3</code> . Рекомендуется оставить <code>1.2</code> – <code>1.3</code> ; понижать минимум до 1.0/1.1 нежелательно (устаревшие, небезопасные версии).
uTLS (<code>settings.fingerprint</code>)	<code>chrome</code> (в форме — пункт None = <code>""</code> доступен)	Имитируемый TLS-отпечаток клиентского хелло (uTLS fingerprint), чтобы рукопожатие выглядело как у популярного браузера. См. список ниже. В TLS первый пункт списка — None (<code>""</code>), отключающий имитацию.
ALPN (<code>alpn</code>)	<code>["h2", "http/1.1"]</code>	Список протоколов прикладного уровня, согласуемых в TLS (множественный выбор). Допустимые значения: <code>h3</code> , <code>h2</code> , <code>http/1.1</code> . По умолчанию предлагаются <code>h2</code> и <code>http/1.1</code> .

Возможные значения **uTLS fingerprint** (одинаковы для TLS и REALITY): `chrome` , `firefox` , `safari` , `ios` , `android` , `edge` , `360` , `qq` , `random` , `randomized` , `randomizednoalpn` , `unsafe` . В TLS-форме дополнительно доступен пустой вариант **None** (имитация отпечатка не применяется).

Доступные значения **Cipher Suites** (TLS 1.3 и ECDHE-наборы): TLS_AES_128_GCM_SHA256 , TLS_AES_256_GCM_SHA384 , TLS_CHACHA20_POLY1305_SHA256 , TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA , TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA , TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA , TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA , TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 , TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 , TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 , TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 , TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256 , TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256 .

Переключатели TLS

Переключатель	По умолчанию	Описание
Отклонить неизвестный SNI (<code>rejectUnknownSni</code>)	выкл. (<code>false</code>)	Если включено, сервер разрывает соединение, когда предъявленный клиентом SNI не совпадает с именем в сертификате. Повышает скрытность (сервер не отвечает на «чужие» запросы), но требует точного совпадения SNI у клиента.
Отключить System Root (<code>disableSystemRoot</code>)	выкл. (<code>false</code>)	Отключает использование системного хранилища доверенных корневых сертификатов.
Возобновление сессии (<code>enableSessionResumption</code>)	выкл. (<code>false</code>)	Включает возобновление TLS-сессии (session resumption / session tickets).

Дополнительные параметры TLS (vcp, кривые, лог ключей, ECH Socket)

Под основными настройками TLS доступны дополнительные поля.

Поле (подпись)	По умолчанию	Описание
Verify Peer Cert By Name (<code>settings.verifyPeerCertByName</code>)	""	Имена (через запятую), по которым клиент проверяет сертификат сервера вместо SNI. Это современная замена удалённого из Xray после 2026-06-01 поля <code>allowInsecure</code> . Значение только для панели: на сервер в конфиг xray не пишется, но передаётся в ссылки-приглашения и подписки (<code>vcp=...</code>), чтобы клиент применил его сам. Плейсхолдер: <code>example.com</code> .
Curve Preferences (<code>curvePreferences</code>)	""	Ограничение и порядок кривых обмена ключами TLS, в порядке предпочтения (например <code>X25519MLKEM768</code> , <code>X25519</code>). Пусто — берутся значения по умолчанию Xray-core.
Master Key Log (<code>masterKeyLog</code>)	""	Путь для записи TLS master keys в формате <code>SSLKEYLOGFILE</code> (для расшифровки трафика в Wireshark при отладке). Плейсхолдер: <code>/path/to/sslkeylog.txt</code> . В продакшене оставлять пустым — файл позволяет расшифровать весь трафик.

Поле (подпись)	По умолчанию	Описание
ECH Sockopt (echSockopt)	выкл.	Переключатель с параметрами сокета для соединения, через которое Xray запрашивает ECH config list. При включении доступны: Dialer Proxy (dialerProxy — пропустить запрос через указанный outbound по тегу), Domain Strategy (domainStrategy), TCP Fast Open (tcpFastOpen), Multipath TCP (tcpMptcp). Без необходимости оставлять выключенным.

Поля `verifyPeerCertByName`, `curvePreferences`, `masterKeyLog` и `echSockopt` располагаются на верхнем уровне `tlsSettings` и переживают «обрезку» панельных полей при сохранении конфигурации.

Сертификаты

Раздел **SSL-сертификат** (в UI заголовок «SSL-сертификат») задаётся списком: кнопкой **+** добавляется новая запись сертификата, кнопкой **– Удалить** — убирается (кнопка удаления доступна, только если записей больше одной). По умолчанию при включении TLS создаётся одна пустая запись.

Для каждой записи переключатель режима ввода (`useFile`):

- **Путь к сертификату** (значение `useFile = true`, по умолчанию) — указываются пути к файлам на сервере:
 - **Публичный ключ** (`certificateFile`) — путь к файлу сертификата (`.crt` / `.pem`);
 - **Приватный ключ** (`keyFile`) — путь к файлу приватного ключа (`.key`).
- **Содержимое сертификата** (значение `useFile = false`) — содержимое вставляется прямо в поля (многострочные текстовые области):
 - **Публичный ключ** (`certificate`) — PEM-содержимое сертификата;
 - **Приватный ключ** (`key`) — PEM-содержимое ключа.

Под полями режима «Путь к сертификату» доступны две кнопки:

- **Установить сертификат панели** — подставляет в поля пути к собственному SSL-сертификату панели. Для inbound на центральной панели берётся её сертификат (`POST /panel/setting/all → webCertFile / webKeyFile`); для inbound, назначенного на ноду, — сертификат самой ноды (`GET /panel/api/nodes/webCert/{nodeId}`), потому что пути центральной панели на ноду не существуют. Если сертификат не настроен, выводится предупреждение: «Для панели не настроен сертификат. Сначала установите его в Настройках.» (сам сертификат панели задаётся в разделе «Настройки → Безопасность»).
- **Очистить** — стирает оба пути.

Дополнительные поля каждой записи сертификата:

Поле	По умолчанию	Описание
OCSP Stapling (ocsStapling)	0 (выкл.)	Интервал обновления OCSP-степлинга в секундах (минимум 0). Для новых inbound по умолчанию выключен (0): это убирает ошибки в логах xray для сертификатов без OCSP-респондера (например, Let's Encrypt, отказавшегося от OCSP). Включайте только для сертификатов, которые поддерживают stapling.

Поле	По умолчанию	Описание
Однократная загрузка (<code>oneTimeLoading</code>)	выкл. (<code>false</code>)	Если включено, сертификат читается с диска один раз при старте и не перечитывается при изменении файла.
Опция использования (<code>usage</code>)	<code>encipherment</code>	Назначение сертификата. Допустимо: <code>encipherment</code> (шифрование – обычный серверный сертификат), <code>verify</code> (проверка), <code>issue</code> (выпуск – сервер сам подписывает/выпускает сертификаты).
Build Chain (<code>buildChain</code>)	выкл. (<code>false</code>)	Показывается только при <code>usage = issue</code> . Достаивать цепочку сертификатов.

Самоподписанный сертификат как отдельной кнопки в редакторе inbound нет: панель не генерирует самоподписанный сертификат на лету для inbound. Сертификат либо указывается путём/содержимым, либо подтягивается из настроек панели кнопкой «Установить сертификат панели». Выпуск/получение SSL-сертификата самой панели (включая загрузку файлов и привязку к домену) выполняется в разделе **Настройки** → **Безопасность**; ACME/Let's Encrypt-эндпоинтов для inbound здесь нет.

ECH и закрепление сертификата (расширенные поля TLS)

Поле	По умолчанию	Описание
ECH key (<code>echServerKeys</code>)	<code>""</code>	Серверные ключи Encrypted Client Hello.
ECH config (<code>settings.echConfigList</code>)	<code>""</code>	ECH config list (клиентская часть, попадает в ссылку).
SHA-256 сертификата пира (<code>settings.pinnedPeerCertSha256</code>)	<code>[]</code>	SHA-256-хеши сертификата пира (hex-строки, через запятую). Дословная подсказка: «SHA-256-хеши сертификата пира в виде шестнадцатеричной строки (напр. <code>e8e2d3...</code>), через запятую. Только для панели – не записывается в конфиг <code>хгау</code> сервера, но включается в ссылки-приглашения, чтобы клиенты могли закрепить сертификат.»

Кнопки: Рядом с полем **SHA-256 сертификата пира** есть две кнопки автозаполнения:

- **Fill from this inbound's certificate** (иконка щита) – подставляет SHA-256-хеш сертификата самого этого inbound (берётся локально через эндпоинт `getCertHash`).
- **Fetch the hash by pinging the SNI (xray tls ping)** (иконка загрузки) – получает хеш живого сертификата сервера, выполнив TLS-подключение по указанному SNI (на сервере вызывается `getRemoteCertHash`). Поле **SNI** (`serverName`) должно быть заполнено – иначе выводится подсказка «Set the SNI (serverName) first to ping the remote certificate.»

Полученные хеши добавляются в поле (через запятую) и попадают в ссылки-приглашения, чтобы клиент мог закрепить сертификат.

- **Получить новый ECH-сертификат** – запрашивает у сервера новую ECH-пару под текущий SNI (`POST /panel/api/server/getNewEchCert`, на сервере выполняется `xray tls ech --serverName <SNI>`); заполняет поля **ECH key** и **ECH config**.

- **Очистить** — обнуляет оба ECH-поля.

7.4. Режим REALITY

Поля блока `realitySettings`. REALITY не использует SSL-сертификат: вместо него — заимствованное TLS-рукопожатие внешнего домена и пара ключей X25519.

Параметры маскировки

Поле (подпись)	Значение по умолчанию	Описание
Показать (<code>show</code>)	выкл. (<code>false</code>)	Отладочный вывод REALITY в логи Xray. Обычно оставляют выключенным.
Xver (<code>xver</code>)	0	Версия PROXY-протокола, передаваемая на бэкэнд (0 — выключено). Минимум 0 .
uTLS (<code>settings.fingerprint</code>)	chrome	Имитируемый TLS-отпечаток (тот же список, что в TLS, но без пустого варианта None).
Цель (<code>target</code>)	"" (с 3.4.2 при включении REALITY остаётся пустым)	Обязательное поле. Реальный домен, чьё TLS-рукопожатие заимствует REALITY. Дословная подсказка: «Обязательно. Должно содержать порт (например, <code>example.com:443</code>). Без порта Xray-core не запускается.» Валидация в панели проверяет наличие и корректность порта; иначе выдаются ошибки «Цель REALITY обязательна» / «Цель REALITY должна содержать порт...» / «У цели REALITY указан недопустимый порт». Рядом с полем — кнопки «Сканировать» (проверить текущую цель «вживую») и «Найти цели» (открыть сканер целей REALITY); см. ниже.
SNI (<code>serverNames</code>)	[] (подставляется вместе с целью)	Список допустимых SNI (множественный ввод тегами). Должен соответствовать домену из Цель . При успешном сканировании цели SNI заполняется по её сертификату.
Макс. разница во времени (мс) (<code>maxTimediff</code>)	0	Максимально допустимое расхождение часов клиента и сервера в миллисекундах (0 — без ограничения). Минимум 0 .
Мин. версия клиента (<code>minClientVer</code>)	""	Минимальная версия Xray-клиента (плейсхолдер с 3.5.0 — 26.3.27). Пусто — без ограничения.
Макс. версия клиента (<code>maxClientVer</code>)	""	Максимальная версия Xray-клиента. Пусто — без ограничения.
Short IDs (<code>shortIds</code>)	[] (генерируются при включении)	Список коротких идентификаторов (hex), различающих клиентов. Множественный ввод тегами; кнопка-обновление генерирует случайный набор.
SpiderX (<code>settings.spiderX</code>)	/	Путь «паука» (клиентская часть REALITY), используемый при имитации обращения к внешнему сайту. Попадает в ссылку-приглашение.

С версии 3.4.2 **Цель** (`target`) и **SNI** (`serverNames`) при включении REALITY больше **не** заполняются автоматически — оба поля остаются пустыми, а цель подбирается живым сканером (см. ниже). Выбирайте

«жирный», стабильный сторонний HTTPS-сайт с поддержкой TLS 1.3 и HTTP/2, не находящийся за вашим же сервером.

Поиск цели REALITY (живой сканер)

С версии 3.4.2 прежняя кнопка «случайная цель» (со статичным встроенным списком) заменена двумя действиями рядом с полем **Цель**:

- **«Сканировать»** — «вживую» проверяет текущую цель: устанавливает TLS-соединение и, если цель пригодна, подставляет SNI из её сертификата. Тосты: «Цель пригодна — заполнены цель и SNI.» / «Цель доступна, но не пригодна для REALITY.» / «Не удалось просканировать цель REALITY.».
- **«Найти цели»** — открывает окно **«Сканер целей REALITY»**: можно указать домен, **IP или диапазон CIDR** (тогда панель находит цели по сертификатам живых хостов) либо оставить поле пустым (проверяются встроенные кандидаты). Результаты ранжируются; столбцы — «Статус»/«Пригодно», «Обмен ключами», «Сертификат», TLS, ALPN, «Задержка». Кнопкой **«Использовать»** выбранная строка подставляется в поля inbound.

Цель считается **пригодной**, если сервер согласует **TLS 1.3, HTTP/2 (ALPN h2)**, обмен ключами **X25519/X25519MLKEM768** и предъявляет **доверенный** (не wildcard) сертификат. Сканирование выполняется на стороне сервера панели (`POST /panel/api/server/scanRealityTarget` и `.../scanRealityTargets`).

Пример: блок `streamSettings` для REALITY на сети `tcp` (VLESS). Сертификат не нужен — вместо него заимствованный домен и пара ключей X25519:

```
"streamSettings": {
  "network": "tcp",
  "security": "reality",
  "realitySettings": {
    "show": false,
    "xver": 0,
    "dest": "www.nvidia.com:443",
    "serverNames": ["www.nvidia.com"],
    "privateKey": "YOUR_X25519_PRIVATE_KEY",
    "shortIds": ["", "6ba85179e30d4fc2"],
    "settings": {
      "publicKey": "YOUR_X25519_PUBLIC_KEY",
      "fingerprint": "chrome",
      "spiderX": "/"
    }
  }
}
```

Здесь поле **Цель** (`target`) панели соответствует `dest` в готовом конфиге Xray. Если REALITY-inbound был создан с destination в ключе `dest` (старыми версиями панели, через API или внешними инструментами), панель при разборе нормализует `dest` → `target`, когда `target` пуст, — поэтому такой inbound корректно загружается, поле **Цель** не оказывается пустым, и пересохранение не ломает рабочий REALITY.

Ключи REALITY (X25519)

Поле	По умолчанию	Описание
Публичный ключ (settings.publicKey)	""	Публичный ключ X25519 (его клиент кладёт в свою конфигурацию/ссылку).
Приватный ключ (privateKey)	""	Приватный ключ X25519 (хранится только на сервере).

Кнопки под ключами:

- **Получить новый сертификат** — запрашивает у сервера новую пару ключей (GET /panel/api/server/getNewX25519Cert ; на сервере выполняется `xray x25519`), заполняет **Приватный ключ** и **Публичный ключ**. Эта же пара автоматически генерируется при первом включении режима REALITY.

Пример: получить пару ключей X25519 через API (вне формы, например для скрипта). Запрос возвращает приватный и публичный ключи:

```
curl -s -b cookie.txt https://your-panel:2053/panel/api/server/getNewX25519Cert
# Ответ:
# {"success":true,"obj":{"privateKey":"...","publicKey":"..."}}
```

cookie.txt — файл cookie сессии, полученный после входа через POST /login .

- **Очистить** — обнуляет оба ключа.

Пост-квантовая подпись ML-DSA-65 (mldsa65)

Дополнительный (необязательный) слой пост-квантовой аутентификации REALITY:

Поле	По умолчанию	Описание
mldsa65 Seed (mldsa65Seed)	""	Серверный seed ключа ML-DSA-65.
mldsa65 Verify (settings.mldsa65Verify)	""	Проверочное значение (клиентская часть, попадает в ссылку).

Кнопки:

- **Получить новый Seed** — запрашивает новую пару (GET /panel/api/server/getNewmldsa65 ; на сервере выполняется `xray mldsa65`), заполняет **mldsa65 Seed** и **mldsa65 Verify**.
- **Очистить** — обнуляет оба поля.

Ограничение скорости fallback и лог ключей REALITY

В настройках REALITY доступно ограничение скорости fallback-трафика — оно не даёт активным зондам использовать сервер как бесплатный канал к заимствованному домену. Настройка задаётся отдельно для двух направлений — **Limit Fallback Upload** и **Limit Fallback Download** (limitFallbackUpload / limitFallbackDownload), каждое с одинаковым набором полей:

Поле (подпись)	По умолчанию	Описание
After Bytes (afterBytes)	0	Сколько байт пропустить на полной скорости перед началом ограничения. 0 — ограничивать с первого байта.
Bytes Per Sec (bytesPerSec)	0	Потолок скорости fallback-трафика в байтах в секунду после порога. 0 — без ограничения (отключает это направление).
Burst Bytes Per Sec (burstBytesPerSec)	0	Запас для кратковременных всплесков сверх постоянной скорости (размер token-bucket). Если меньше Bytes Per Sec , поднимается до его значения.

Там же добавлено поле **Master Key Log** (masterKeyLog) — путь для записи TLS master keys в формате SSLKEYLOGFILE для отладки в Wireshark; в продакшене оставлять пустым.

7.5. Практические рекомендации по настройке

1. **VLESS + Reality (рекомендуется)**: создайте VLESS-inbound на сети tcp , на вкладке «Безопасность» выберите **Reality** — панель сама подставит случайные target /SNI, shortIds и сгенерирует ключи X25519. При необходимости нажмите «Получить новый сертификат» для своей пары ключей. Для клиентов VLESS включите **Flow** = xtls-rprx-vision (XTLS Vision) — это даст максимальную производительность и скрытность.

Пример: итоговая клиентская ссылка VLESS + Reality + Vision. Так выглядит ссылка-приглашение, которую панель выдаёт для такого inbound (значения ключей/ID — иллюстративные):

```
vless://uuid-клиента@1.2.3.4:443?
type=tcp&security=reality&pbk=ПУБЛИЧНЫЙ_КЛЮЧ&fp=chrome&sni=www.nvidia.com&sid=6ba85179e3
0d4fc2&spx=%2F&flow=xtls-rprx-vision#my-reality
```

Здесь pbk — публичный ключ X25519, sni — заимствованный домен из **Цель**, sid — один из **Short IDs**, flow=xtls-rprx-vision — включённый XTLS Vision. 2. **TLS со своим доменом**: выберите **TLS**, заполните **SNI** именем домена, добавьте сертификат (путём к файлам или содержимым), либо нажмите «Установить сертификат панели», если домен и сертификат уже настроены в «Настройки → Безопасность». Оставьте **Мин/Макс версия** = 1.2 – 1.3 и **uTLS** = chrome для маскировки под обычный браузер. 3. Не оставляйте режим **Нет** для inbound, открытых наружу, — используйте его только для локальных fallback-целей (127.0.0.1) или когда TLS обеспечивает внешний прокси. 4. Совет из интерфейса: для большинства расширенных полей действует подсказка «Рекомендуется оставить настройки по умолчанию» — меняйте их только при понимании последствий.

8. Клиенты

Клиент — это учётная запись пользователя VPN: набор учётных данных (UUID или пароль), привязанный к одному или нескольким inbound, с собственной квотой трафика, сроком действия и лимитом одновременных подключений. В этом форке клиент является самостоятельной сущностью (таблица `clients`): один и тот же клиент может быть привязан к нескольким inbound сразу, сохраняя общий UUID/пароль и общий счётчик трафика. Раздел **Клиенты** показывает все учётные записи панели вне зависимости от inbound, с поиском, фильтрами, сортировкой и массовыми операциями.

8.1. Поля клиента

Ниже разобрано каждое поле редактора **Добавить клиента** / **Изменить клиента**.

Форма клиента разбита на две вкладки: **Основные** (email, привязка к inbound, лимиты, срок, группа, комментарий, обратный тег) и **Учётные данные** (UUID/пароль/auth, Flow, VMess Security). В подписях полей квота указана как **Лимит трафика (ГБ)**, а сроки — как **Длительность (дней)** и **Автопродление (дней)**; у полей **Лимит трафика (ГБ)** и **Лимит IP** есть подсказки, поясняющие, что `0` означает «без ограничений». При редактировании уже существующего клиента кнопка генерации случайного email скрыта, а кнопка журнала IP вынесена прямо к полю **Лимит IP** и показывает число зафиксированных адресов.

Поле	Ключ JSON	По умолчанию	Описание
Email	<code>email</code>	— (обязательно)	Уникальный идентификатор клиента
UUID	<code>id</code>	генерируется	Идентификатор для VMess/VLESS
Пароль	<code>password</code>	генерируется	Пароль для Trojan/Shadowsocks
Авторизация	<code>auth</code>	генерируется	Пароль для Hysteria
Flow	<code>flow</code>	пусто	Flow control (XTLS), только VLESS
VMess Security	<code>security</code>	<code>auto</code>	Метод шифрования VMess
Лимит IP	<code>limitIp</code>	<code>0</code> (без лимита)	Максимум одновременных IP
Всего отправлено/получено (ГБ)	<code>totalGB</code>	<code>0</code> (без лимита)	Квота трафика
Срок действия	<code>expiryTime</code>	<code>0</code> (бессрочно)	Дата истечения
Автопродление	<code>reset</code>	<code>0</code> (выкл.)	Период сброса трафика, дни
ID пользователя Telegram	<code>tgId</code>	<code>0</code> (нет)	Числовой Telegram ID
ID подписки	<code>subId</code>	генерируется	Идентификатор подписки
Группа	<code>group</code>	пусто	Логическая метка группировки
Комментарий	<code>comment</code>	пусто	Произвольная заметка
Включён	<code>enable</code>	<code>true</code>	Активна ли учётная запись

Email (идентификатор)

Поле **Email** — главный и обязательный идентификатор клиента. Несмотря на название, это не обязательно почтовый адрес: подойдёт любая текстовая метка (имя пользователя, номер). Значение должно быть **уникальным** в пределах панели; попытка создать второго клиента с занятым email отклоняется (`email already in use`), за исключением случая, когда совпадает и `subId` (это трактуется как привязка того же клиента).

Email **нельзя оставлять пустым** (`client email is required`) и он **не может содержать пробелы, символы /, \ и управляющие символы** («Email не может содержать пробелы, '/', ' или управляющие символы»).

Email участвует в учёте трафика, в журнале IP, в онлайн-списке и в именах операций — менять его задним числом не рекомендуется.

UUID / Пароль / Авторизация (учётные данные)

Конкретное поле учётных данных зависит от протокола inbound, к которому привязывается клиент.

Значения подставляются автоматически, если оставить поле пустым:

- **UUID** (поле `id`) — для протоколов **VMess** и **VLESS**. Если не задан, генерируется случайный UUID v4.
- **Пароль** (поле `password`) — для **Trojan** и **Shadowsocks**. Для Trojan по умолчанию генерируется UUID без дефисов. Для Shadowsocks генерируется ключ нужной длины в Base64 в зависимости от метода шифрования inbound: 16 байт для `2022-blake3-aes-128-gcm`, 32 байта для `2022-blake3-aes-256-gcm` и `2022-blake3-chacha20-poly1305`; для прочих методов — UUID без дефисов. Если введённый вручную ключ не подходит под метод `2022-blake3`, он будет заменён сгенерированным.
- **Авторизация** (поле `auth`) — пароль для **Hysteria**. По умолчанию UUID без дефисов.

Поскольку один клиент может быть привязан к inbound разных протоколов, у записи клиента могут одновременно присутствовать и UUID, и пароль, и auth — для каждого протокола используется своё поле.

Пример: как учётные данные клиента выглядят в `settings` разных inbound. Один и тот же клиент в VLESS-inbound идентифицируется по `id`, в Trojan — по `password`, в Shadowsocks — по `password` (ключ Base64):

```
// фрагмент settings.clients у VLESS-inbound
{ "id": "b831381d-6324-4d53-ad4f-8cda48b30811", "email": "user-a", "flow": "xtls-rprx-vision" }

// тот же клиент в Trojan-inbound
{ "password": "b831381d63244d53ad4f8cda48b30811", "email": "user-a" }

// тот же клиент в Shadowsocks-inbound (метод 2022-blake3-aes-256-gcm)
{ "password": "GPy0aA3f7C00az53eaQ8eqMfRDjmBl0h7v1u3+Z+pHk=", "email": "user-a" }
```

Flow

Flow (поле `flow`) — управление потоком XTLS. Применимо **только к VLESS** и только когда inbound сконфигурирован для XTLS Vision: VLESS поверх транспорта **TCP** с security **tls** или **reality**. Допустимое

значение — `xtls-rprx-vision` (а также исторический `xtls-rprx-vision-udp443`); пустое значение означает отсутствие flow.

Если inbound не поддерживает XTLS-flow, заданный flow **молча обнуляется** при сохранении клиента: для одного и того же клиента, привязанного к нескольким inbound, flow применяется только там, где это допустимо. Менять стоит только если вы целенаправленно используете VLESS-Vision.

VMess Security

VMess Security (поле `security`) — метод шифрования полезной нагрузки для VMess. Значение по умолчанию — `auto` (Xray сам выбирает шифр). Допустимые значения — стандартные для VMess: `auto`, `aes-128-gcm`, `chacha20-poly1305`, `none`, `zero`. Для остальных протоколов поле не используется.

Лимит IP (одновременные подключения)

Лимит IP (поле `limitIp`) — максимальное число **разных IP-адресов**, с которых клиент может быть подключён одновременно. Значение по умолчанию — `0`, что означает **отсутствие ограничения**. При положительном значении панель отслеживает активные IP клиента и, при превышении лимита, отключает учётную запись фоновым заданием. (Начиная с **3.3.1** подсчёт IP идёт через online-stats API ядра Xray и **не требует** журнала доступа; на старых версиях ядра панель возвращается к чтению журнала доступа, который тогда должен быть включён.) Используйте, чтобы запретить раздачу одной подписки на много устройств: например, `2` — разрешить два устройства.

Лимит IP применяется средствами **Fail2ban**, поэтому поле **Лимит IP** активно только при установленном и работоспособном Fail2ban (панель проверяет его статус через `GET /panel/api/server/fail2banStatus`). Если Fail2ban не установлен, поле редактора клиента (и формы массового добавления) блокируется, а при наведении показывается подсказка с предложением установить Fail2ban из bash-меню `x-ui` («Fail2ban is not installed, so the IP limit cannot be enforced. Install Fail2ban from the x-ui bash menu to enable this option.»); на Windows подсказка сообщает, что Fail2ban там недоступен («Fail2ban is not available on Windows, so the IP limit cannot be enforced.»), а если функция отключена на сервере — «The IP limit feature is disabled on this server.». При обновлении панели у клиентов на серверах без Fail2ban сохранённый лимит IP обнуляется разовой миграцией, так как он там всё равно не применялся.

Пример значений. `limitIp: 0` — без ограничения; `limitIp: 1` — строго одно устройство одновременно; `limitIp: 3` — до трёх разных IP. При четвёртом активном IP фоновое задание выключит клиента (`enable = false`), пока вы не выполните **Сбросить лимит IP**.

Связанные операции: **Журнал IP** показывает список зафиксированных IP клиента; каждая запись содержит, помимо самого IP, время последнего обращения и метку ноды (`@ имя_ноды`), через которую зафиксировано подключение, — в мультипанельной конфигурации видно, через какой узел подключался клиент. **Сбросить лимит IP** очищает накопленный журнал IP, чтобы клиент снова смог подключиться, не дожидаясь естественного истечения записей.

Всего отправлено/получено (ГБ) — квота трафика

Всего отправлено/получено (ГБ) (поле `totalGB`) — суммарная квота трафика (отправка + приём). Значение по умолчанию — `0` — означает **безлимит**. По достижении квоты (`up + down >= total`) клиент считается **исчерпанным** (depleted) и отключается. В UI обычно вводится в гигабайтах; в базе хранится в байтах.

В списке клиентов столбец **Трафик** показывает цветную полосу использования: объём израсходованного трафика, метку лимита (или знак ∞ при безлимите) и подсказку при наведении с разбивкой на отправлено/получено и остаток. Тот же компактный индикатор отображается в карточках клиентов на телефоне.

Срок действия (Expiry)

Срок действия (поле `expiryTime`) задаёт момент истечения учётной записи. Поле имеет три режима:

- **Бессрочно** — `0`. Клиент никогда не истекает по времени.
- **Конкретная дата** — положительный Unix-timestamp (в миллисекундах). По наступлении (`expiryTime <= сейчас`) клиент считается истёкшим (expired) и отключается. В UI обычно задаётся выбором даты или длительностью в днях (**Длительность**, единица — **Дни**).
- **Старт после первого использования** — **отрицательное** значение, кодирующее длительность. Пока клиент не передал ни одного байта, срок остаётся отрицательным («отложенный старт»). На первом же тике учёта трафика панель конвертирует его в абсолютную дату: `сейчас + |длительность|`. Это позволяет продать, например, «30 дней с момента первого подключения», не зная заранее, когда клиент активируется. Конвертация выполняется один раз на email, чтобы все привязанные inbound получили один и тот же срок.

Пример кодирования срока. Фиксированная дата 1 марта 2026, 00:00 UTC →

`expiryTime: 1772323200000` (положительный timestamp в миллисекундах). «30 дней с первого подключения» → `expiryTime: -2592000000` (отрицательное значение, $30 \times 24 \times 60 \times 60 \times 1000$); при первом байте трафика панель заменит его на `сейчас + 2592000000`. Бессрочно → `expiryTime: 0`.

Автопродление (период сброса трафика клиента)

Поле **Автопродление** (поле `reset`) — это период автоматического продления/сброса в днях. Подсказка: «Автоматическое продление после окончания. (0 = отключено) (единица: день)».

- `0` — автопродление **отключено** (значение по умолчанию). По истечении срока клиент просто становится исчерпанным.
- `> 0` — фоновое задание при истечении срока **сбрасывает счётчики трафика в ноль** (`up = down = 0`), **сдвигает срок действия вперёд** на `reset` дней (если нужно — на несколько периодов, пока новый срок не окажется в будущем) и при необходимости снова **включает** клиента. Это реализует периодическую подписку (например, ежемесячную). Автопродление **не применяется к inbound на узлах-нодах** (`node_id IS NOT NULL`).

Важное следствие: клиенты с `reset > 0` **исключаются** из понятия «исчерпанный» в операциях массового удаления — их трафик/срок ожидаемо обнуляются автопродлением, а не делают учётную запись кандидатом на удаление.

ID пользователя Telegram

ID пользователя Telegram (поле `tgId`) — числовой Telegram-идентификатор пользователя для привязки к встроенному Telegram-боту панели (уведомления, самостоятельный просмотр статистики). Подсказка: «Числовой ID пользователя Telegram (0 = нет)». Значение по умолчанию `0` — привязка отсутствует. По этому полю доступна фильтрация (**Есть** / **Нет**).

ID подписки (subId)

ID подписки (поле `subId`) — идентификатор, под которым клиент включается в **подписку** (subscription). Все клиенты с одинаковым `subId` отдаются по одной ссылке подписки. Если поле оставлено пустым при создании, панель **автоматически генерирует случайный** `subId` (UUID). Значение должно быть **уникальным** среди клиентов с другим email (`subId already in use`) и подчиняется тем же ограничениям символов, что и email («ID подписки не может содержать пробелы, '/' или управляющие символы»).

Без `subId` ссылка подписки для клиента недоступна («У этого клиента нет subId, ссылка для общего доступа недоступна.»).

Вкладка Links (внешние ссылки и подписки)

Кроме вкладок **Основные** и **Учётные данные**, в редакторе клиента есть третья вкладка **Links** (подсказка: «Add third-party share links and remote subscription URLs to include in this client's subscription.»). На ней кнопкой **Add External Link** добавляют сторонние share-ссылки (`vless://`, `vmess://`, `trojan://`, `ss://`, `hysteria2://`, `wireguard://`), а кнопкой **Add External Subscription** — URL удалённых подписок (например, `https://provider.example/sub/...`).

Всё перечисленное подмешивается в выдачу подписки данного клиента (форматы raw, JSON и Clash): ссылки добавляются как есть, а удалённые подписки панель периодически скачивает (с кэшем и коротким таймаутом) и объединяет их конфигурации с собственными. Так в одной ссылке подписки клиента можно отдавать вместе со своими серверами ещё и внешние конфигурации.

Группа

Группа (поле `group`) — логическая метка для объединения связанных клиентов. Подсказка: «Логическая метка для группировки связанных клиентов (например, команда, клиент, регион). Фильтруется из панели инструментов.», плейсхолдер — «например, customer-a». Поле необязательное (по умолчанию пустое). По группе можно фильтровать список и выполнять массовые операции; для очистки метки у клиента используется действие **Разгруппировать**.

Снять группу можно и прямо в редакторе одного клиента: если очистить поле **Группа** и сохранить, метка корректно снимается, и клиент перестаёт отображаться под прежней группой.

Комментарий

Комментарий (поле `comment`) — произвольная текстовая заметка для администратора (по умолчанию пусто). Содержимое попадает в поиск и доступно для фильтрации (**Есть** / **Нет** комментарий).

Включён

Включён (поле `enable`) — флаг активности учётной записи. По умолчанию **включён** (`true`); при создании, даже если флаг не передан, панель принудительно ставит `true`. Выключенный клиент (`enable = false`) не может подключиться и в сводке относится к категории **неактивных** (`deactive`). Панель сама выключает клиентов, исчерпавших квоту, истёкших или превысивших лимит IP.

Поля только для чтения

В карточке клиента также отображаются служебные поля: **Создан** (`created_at`) и **Обновлён** (`updated_at`) — временные метки создания и последнего изменения, заполняются автоматически и не редактируются. Поле **Обратный тег** (`reverse`) — необязательный Reverse tag для простого обратного прокси VLESS («Необязательный Reverse tag»).

8.2. Привязка к inbound

Каждый клиент должен быть привязан хотя бы к одному inbound — при создании требуется минимум один (`at least one inbound is required`). В редакторе это поле **Привязанные входящие** с подсказкой **Выберите один или несколько входящих**.

- **Привязать** — добавить клиента к выбранным inbound (тот же UUID/пароль и общий трафик). Существующие привязки сохраняются.
- **Отвязать** — убрать клиента из выбранных inbound. Сама запись клиента сохраняется (для полного удаления используйте **Удалить**). Пары, где клиент не был привязан, тихо пропускаются.

При сохранении клиента, привязанного к нескольким inbound, поля, несовместимые с конкретным протоколом/транспортом (например, Flow вне VLESS-Vision), автоматически приводятся к допустимым значениям для каждого inbound.

Над списком выбора inbound (в форме клиента, при массовом добавлении клиентов и в окнах массового прикрепления/открепления) есть кнопки **Выбрать все** и **Очистить**. В этих списках каждый inbound подписывается своим примечанием (`remark`), если оно задано, иначе — тегом inbound.

8.3. Операции над клиентом

Для отдельного клиента (через карточку **Информация о клиенте** или контекстное меню **Действия**) доступны:

Просмотр информации, QR-кода и ссылки

- **Информация о клиенте** — карточка со всеми полями, использованным/оставшимся трафиком (**Остаток**), сроком действия и привязанными inbound. С 3.5.0 карточка также показывает **группу** клиента (строка «Группа» с меткой, над «Комментарий»), когда группа задана.

Запрос клиента через API (GET /panel/api/clients/get/:email) рядом с полями client и inboundIds дополнительно возвращает usedTraffic — фактически израсходованный трафик (отправлено + получено, с учётом данных узлов), что упрощает сопоставление расхода с квотой totalGB.

- **QR-код и Ссылка** — конфигурационная ссылка клиента для импорта в клиентское приложение. Формируется по всем привязанным inbound с поддерживаемым протоколом (GET /links/:email). Если подходящих ссылок нет: «Нет ссылок для общего доступа — сначала привяжите клиента к входящему с поддерживаемым протоколом.».
- **Ссылка подписки** — URL подписки по subId (GET /subLinks/:subId). Доступна, только если у клиента есть subId и сервис подписки включён в **Настройки панели** → **Подписка** (иначе «Сервис подписки отключён.»). Дополнительно отдаётся **URL JSON-подписки**.

Сбросить трафик

Сбросить трафик (POST /resetTraffic/:email) обнуляет счётчики отправки/приёма (up , down) конкретного клиента. Квота (totalGB) и срок действия **не затрагиваются** — обнуляется только использованный объём. Тост: «Трафик сброшен». Если клиент не привязан ни к одному inbound: «Сначала привяжите этого клиента к входящему.».

Кнопка **Сбросить трафик** доступна и из формы редактирования клиента — в нижней панели, рядом с **Отмена / Сохранить** (перед сбросом запрашивается подтверждение). Если клиент был отключён из-за исчерпания трафика, сброс (как одиночный, так и массовый) автоматически снова **включает** его (enable = true) и сразу рассылает это изменение на узлы-ноды — повторно включать клиента вручную на мастере и нодах больше не нужно.

Сбросить лимит IP

Очищает накопленный журнал IP клиента (POST /clearIps/:email), чтобы снять временную блокировку по превышению лимита одновременных подключений. Тост: «Лог был очищен».

Удалить

Удалить (POST /del/:email) — полное удаление клиента. Диалог подтверждения: заголовок «Удалить клиента {email}?», текст «Клиент будет удалён из всех привязанных входящих, а его запись трафика будет уничтожена. Это действие нельзя отменить.». Удаление снимает клиента со **всех** inbound и уничтожает его запись трафика. Тост: «Клиент удалён».

8.4. Массовые операции

В списке клиентов можно отметить несколько записей (**Выбрать всё, Очистить всё**); счётчик — «{count} выбрано». Над выбранными доступны:

- **Удалить ({count})** (POST /bulkDel) — групповое удаление. Подтверждение: «Удалить {count} клиентов?», «Каждый выбранный клиент удаляется из всех привязанных входящих, его запись трафика уничтожается. Это действие нельзя отменить.». Тост: «Удалено клиентов: {count}», при частичной неудаче — «Удалено: {ok}, не удалось: {failed}».

- **Изменить ({count}) / Корректировка** (POST /bulkAdjust) — массовое изменение срока и/или квоты. Диалог «Изменить {count} клиентов» с подсказкой «Положительные значения добавляют, отрицательные — уменьшают. Клиенты с неограниченным сроком или трафиком пропускаются для соответствующего поля.». Поля: **Добавить дни**, **Добавить трафик (ГБ)** и **Set flow**. Логика:
 - **Срок:** клиенты с бессрочным сроком (`expiryTime == 0`) пропускаются («unlimited expiry»); для клиентов с датой срок сдвигается на указанное число дней; для клиентов в режиме «после первого использования» (отрицательный срок) корректируется длительность ожидания. Уменьшение, выходящее за пределы остатка, пропускается («reduction exceeds remaining time/delay window»).
 - **Трафик:** клиенты с безлимитом (`totalGB == 0`) пропускаются («unlimited traffic»); иначе квота меняется на указанный объём, не опускаясь ниже нуля.
 - **Flow:** выпадающий список **Set flow** позволяет установить или сбросить XTLS flow сразу у всех выбранных клиентов. По умолчанию выбрано **No change** (без изменений). Вариант **Disable (clear flow)** сбрасывает flow, а значения `xtls-rprx-vision` и `xtls-rprx-vision-udp443` задают соответствующий vision-flow. Установка vision-flow применяется только к inbound, поддерживающим flow; неподходящие inbound остаются без изменений и помечаются как пропущенные, тогда как сброс flow допускается всегда.
 - **Авто-включение (с 3.4.2):** клиент, отключённый **только из-за исчерпания** (истёк срок или превышена квота), при продлении автоматически снова включается — локально и на его узле, — если корректировка возвращает его в пределы лимитов. Отключённые вручную или всё ещё исчерпанные остаются выключенными (поле `enable` теперь записывается явно, поэтому состояние сохраняется и у клиентов без inbound).
 - Если не указаны ни дни, ни трафик, ни flow: «Укажите дни, трафик или flow перед применением.». Тост: «Изменено: {count}» / «Изменено: {ok}, пропущено: {skipped}».

Пример: продлить выбранных клиентов на 30 дней и добавить 50 ГБ. В диалоге **Изменить** укажите **Добавить дни** = 30 , **Добавить трафик (ГБ)** = 50 . Чтобы, наоборот, отнять неделю и урезать квоту на 10 ГБ, введите отрицательные значения: **Добавить дни** = -7 , **Добавить трафик (ГБ)** = -10 (клиенты с бессрочным сроком или безлимитом по соответствующему полю будут пропущены).

- **Привязать ({count}) / Отвязать ({count})** (POST /bulkAttach / bulkDetach) — массовая привязка/отвязка выбранных клиентов к выбранным inbound. Цели — только многопользовательские inbound. Результат отвязки: «Отвязано {detached}, пропущено {skipped}». С 3.5.0 селектор inbound в форме клиента **скрывает выключенные inbound** (кроме тех, к которым клиент уже привязан), а в окне «Отвязать» устранено накопление дублей записей одного клиента (данные чинятся автоматически при старте панели).
- **Sub-ссылки ({count})** — сводная таблица URL подписок и JSON-подписок выбранных клиентов с кнопкой **Копировать все**. Если ни у кого нет subId: «Ни у одного из выбранных клиентов нет ID подписки.».
- **Добавить в группу и Разгруппировать** — назначение и снятие метки группы.
- **Включить ({count}) / Отключить ({count})** (POST /bulkEnable / bulkDisable) — массовое включение и отключение выбранных клиентов. **Включить** активирует каждого выбранного клиента на всех привязанных inbound; клиенты с исчерпанной квотой трафика или истёкшим сроком будут автоматически отключены снова. **Отключить** сразу лишает клиентов доступа, но их записи и накопленный трафик сохраняются. Перед выполнением панель запрашивает подтверждение, а после

операции показывает уведомление с количеством обработанных клиентов и, при наличии, с количеством тех, для которых действие не удалось.

Сброс трафика и удаление по статусу

- **Сбросить трафик всех клиентов** (`POST /resetAllTraffics`) — обнуляет счётчики `up / down` у **всех** клиентов панели. Подтверждение: «Сбросить трафик всех клиентов?» и «Счётчики отправки/приёма всех клиентов сбрасываются в ноль. Квоты и срок действия не затрагиваются. Это действие нельзя отменить.». Тост: «Трафик всех клиентов сброшен».
- **Удалить исчерпанных** (`POST /delDepleted`) — удаляет всех клиентов, у которых **исчерпана квота** (`total > 0 and up + down >= total`) **или истёк срок** (`expiry_time > 0 and expiry_time <= сейчас`), при условии `reset = 0` (клиенты с автопродлением не трогаются). Подтверждение: «Удалить исчерпанных клиентов?», «Удаляются все клиенты, у которых исчерпана квота трафика или истёк срок. Это действие нельзя отменить.». Тост: «Удалено исчерпанных клиентов: {count}».

Экспорт, импорт и удаление непривязанных клиентов

Когда ничего не выделено, в меню **Ещё** на странице **Клиенты** доступны три операции.

Экспорт клиентов (`GET /clients/export`) открывает просмотрщик с JSON-списком всех клиентов в формате `{client, inboundIds}` с кнопками копирования и скачивания (файл `clients-export.json`).

Импорт клиентов (`POST /clients/import`) открывает редактор, в который вставляют такой же JSON и жмут **Import**: клиенты с `inboundIds` создаются и привязываются к `inbound`, клиенты без привязок восстанавливаются как отдельные «голые» записи, а уже существующие email **никогда не перезаписываются** — они попадают в список пропущенных. Тосты: «{count} clients imported», «{ok} imported, {failed} skipped».

Удалить непривязанных клиентов (`POST /clients/delOrphans`) — опасная операция: удаляет всех клиентов, не привязанных ни к одному `inbound`, вместе с их записью трафика, журналом IP и внешними ссылками. Подтверждение: «Delete clients without an inbound?», «Removes every client that is not attached to any inbound, along with its traffic record. This cannot be undone.». Тост: «{count} unattached clients deleted». Действие необратимо.

8.5. Поиск, фильтры и сортировка

Над списком есть строка поиска («Поиск email, комментария, sub ID, UUID, пароля, auth, Telegram ID...») — она ищет по email, комментарию, subId, UUID, паролю, auth и (снова с 3.5.0) по **Telegram ID**. Счётчик результатов: «Показано {shown} из {total}».

Список клиентов обновляется автоматически: панель раз в несколько секунд подтягивает актуальную текущую страницу, поэтому вновь подключившиеся клиенты и изменившийся порядок сортировки появляются без ручного обновления (индикатор загрузки при фоновом опросе не мигает).

С 3.5.0 в таблице есть колонка **«Скорость»** (между «Трафик» и «Остаток»): живая скорость клиента — синяя метка `↑` отдача / `↓` загрузка (скользящее ~5-секундное среднее, обновляется каждые 5 секунд); при отсутствии трафика — серый прочерк. На мобильных скорость показывается строкой в карточке клиента.

Панель **Фильтр клиентов** позволяет отбирать по статусу (категории), протоколу, привязанному inbound, диапазону срока действия, диапазону использованного трафика, наличию автопродления (**Есть/Нет**), наличию Telegram ID и комментария, а также по группе. На панелях с узлами появляется мультиселект **Узлы**: можно ограничить список клиентами выбранных узлов; отдельный пункт **Локальная панель** отбирает клиентов inbound без привязки к узлу (фильтр виден только при наличии узлов). Сортировка: **Сначала старые/новые, Недавно обновлены, Недавно в сети, Email A→Я / Я→A, Больше трафика, Больше остатка, Скорее истекают.**

8.6. Значки и статусы

Приоритет статусов: исчерпанный/истёкший → неактивный → скоро истекает → активный.

- **В сети / Не в сети** — клиент с активным подключением (есть в текущем онлайн-списке) и **включён**. Онлайн-список обновляется отдельными запросами (`/onlines` , `/onlinesByGuid`).
- **Исчерпан** (depleted) — квота выбрана (`up + down >= totalGB`) **или** срок истёк (`expiryTime <=` сейчас). Такой клиент автоматически отключается и попадает под действие **Удалить исчерпанных**.
- **Скоро истекает / заканчивается** (expiring) — включённый клиент, у которого до истечения срока осталось меньше порогового интервала **или** до исчерпания квоты осталось меньше порогового объёма (пороги задаются в настройках панели). Не считается, если клиент уже исчерпан/отключён.
- **Неактивный** (deactive) — клиент с `enable = false` (выключен вручную или фоновым заданием).
- **Активный** (active) — включён, не исчерпан, срок не истёк и до порогов ещё далеко.

9. Группы клиентов

Это возможность данного форка 3X-UI. В оригинальном проекте 3x-ui (MHSanaei) понятия «группа клиентов» нет — здесь добавлены отдельная таблица групп, страница **Группы** в меню панели и соответствующие методы API. Если вы переносите конфигурацию на оригинальный 3x-ui, метка группы просто не будет нигде учитываться.

9.1. Что такое группа клиентов и зачем она нужна

Группа — это именованная логическая метка (label), которую можно навесить на одного или нескольких клиентов. Она не создаёт нового способа подключения и не является ни inbound, ни нодой — это сугубо организационный ярлык, по которому клиентов удобно фильтровать и обрабатывать массово.

Ключевая идея модели клиентов в этом форке: **клиент — это сущность верхнего уровня, идентифицируемая по email** (поле `email` в таблице `clients` имеет уникальный индекс). Один и тот же клиент (один email с одними и теми же учётными данными) может одновременно числиться в нескольких inbound и даже на нескольких нодах, в том числе с разными протоколами. Метка группы хранится **один раз на клиента**, поэтому она автоматически распространяется на все его привязки к inbound сразу.

Метка группы — это логическая метка для группировки:

Слой	Где хранится	Поле
Запись клиента (БД)	таблица <code>clients</code>	<code>group_name</code> (по умолчанию пустая строка <code>' '</code>)
Справочник групп (БД)	таблица <code>client_groups</code>	<code>name</code> (уникальный индекс, не пустой)
Настройки inbound (Xray)	JSON <code>settings.clients[].group</code>	дублируется в каждый клиентский объект каждого inbound, где состоит клиент

Зачем это нужно на практике:

- **Один клиент на нескольких inbound/нодах.** Если клиент «продаётся» как доступ сразу к нескольким inbound (например, разные протоколы или разные ноды), группа позволяет управлять им как единым целым: сбросить трафик, удалить, переименовать метку — одной операцией по всем его inbound.
- **Массовые операции и фильтрация.** На странице **Клиенты** список можно фильтровать по группе; на странице **Группы** доступны массовые действия над всеми участниками группы.
- **Организация большого парка клиентов.** Метки вида `vip`, `trial`, `team-A` помогают раскладывать тысячи клиентов по логическим корзинам, не плодя при этом отдельные inbound.

9.2. Связь группы с клиентами, inbound, нодами и протоколами

Это важнейший для понимания подраздел, поскольку синхронизация метки нетривиальна.

Группа описывает клиента, а не inbound. Метка живёт в записи клиента (`clients.group_name`). Когда клиент присоединён к нескольким inbound, при любой смене группы панель проходит по **всем** inbound, в которых этот клиент состоит, и проставляет/убирает поле `group` внутри их Xray-настроек

(`settings.clients[]`). Технически это делается так: по email клиента находятся все inbound, в которых он состоит, затем в JSON-настройках каждого такого inbound правится клиентский объект с этим email.

Поэтому:

- Группа **не зависит от протокола**. Один email может быть VLESS-клиентом в одном inbound и Hysteria-клиентом в другом — метка группы у него всё равно одна и применится к обоим (учётные данные при этом у каждого протокола свои и сохраняются отдельно).
- Группа **охватывает ноды**. Inbound, принадлежащие нодам, отличаются от inbound главной панели полем `nodeId` (у inbound главной панели оно `null / 0`). Метка группы распространяется на клиентские объекты в inbound вне зависимости от того, главный это inbound или нодовый — лишь бы клиент с таким email там присутствовал.

Метка группы устойчива к синхронизации с нод и к перестроению настроек. Это поведение специально заложено:

- Когда нода присылает снапшот трафика, её данные **не затирают** локальные `group_name` и `comment` клиента в БД панели. Группа и комментарий считаются «панель-локальными» полями — нода ими не управляет.
- При перестроении настроек inbound пустое значение `group` в приходящих данных **не сбрасывает** уже сохранённую метку. Группой управляют именно через страницу **Группы** (а не через редактирование Xray-настроек инбаунда), поэтому «пустая группа» при обычной перестройке настроек трактуется как «не трогать», а не как «очистить».

Практический вывод: единственные операции, которые **намеренно очищают** метку, — это удаление группы и явное удаление клиента из группы (см. ниже). Обычное редактирование inbound или фоновая синхронизация с нодой группу не потеряют.

9.3. Справочник групп и «пустые» группы

Список групп на странице формируется слиянием двух источников:

1. **Производные группы (derived)** — все непустые значения `group_name`, реально встречающиеся у клиентов, с подсчётом количества клиентов.
2. **Сохранённые группы (stored)** — записи из таблицы `client_groups`.

Это объединение даёт важный эффект: группа может существовать **без единого клиента**. Такая группа создаётся явной кнопкой «Добавить группу» (запись в `client_groups`) и в списке отображается со счётчиком `0`. Эти записи и считаются **пустыми группами**. Список всегда отсортирован по имени без учёта регистра.

Сводные счётчики на странице:

Поле (RU)	Что показывает
Всего групп	Общее число групп (сохранённых и производных вместе).
Клиенты с группой	Сколько клиентов имеют непустую метку группы.
Пустые группы	Сколько групп существует без клиентов (счётчик <code>0</code>).
Клиентов в группе	Число клиентов в конкретной группе (столбец таблицы).

9.4. Поля и столбцы группы

Запись группы в таблице `client_groups` содержит:

Поле	Тип	По умолчанию	Описание
<code>Id</code>	int	автоинкремент	Первичный ключ записи группы.
<code>Name</code>	string	— (обязательно)	Имя группы. Уникальный индекс, не может быть пустым. В UI — столбец Имя .
<code>CreatedAt</code>	int64 (мс)	время создания	Момент создания записи группы.
<code>UpdatedAt</code>	int64 (мс)	время изменения	Момент последнего изменения.

В таблице на странице отображаются как минимум столбцы **Имя** и **Клиентов в группе**, а также кнопки действий (см. ниже).

9.5. Создание группы

Кнопка **Добавить группу**.

Шаги:

1. Нажмите **Добавить группу**.
2. Введите имя группы.
3. Подтвердите.

Поведение бэкенда (POST `/panel/api/clients/groups/create`, тело `{"name": "..."}):`

- Имя обрезается от пробелов по краям. Пустое имя отклоняется с ошибкой «group name is required».
- Если группа с таким именем уже есть — ошибка «group already exists».
- При успехе создаётся запись в `client_groups` (изначально без клиентов — это пустая группа).

Сообщение об успехе: «Группа «`{name}`» создана.»

Пример: создать пустую группу через API. Подготовьте набор меток заранее, ещё до наполнения клиентами:

```
curl -s 'https://panel.example.com:2053/panel/api/clients/groups/create' \
  -H 'Content-Type: application/json' \
  -b cookie.txt \
  -d '{"name": "vip"}'
```

Ответ при успехе:

```
{ "success": true, "msg": "Группа «vip» создана.", "obj": null }
```

Повторный вызов с тем же именем вернёт `"success": false` и сообщение `group already exists`.

Создание пустой группы заранее удобно, когда вы хотите подготовить набор меток и затем наполнять их клиентами через «Добавить клиентов...».

9.6. Переименование группы

Кнопка **Переименовать**, заголовок диалога — «Переименовать {name}».

Поведение (POST /panel/api/clients/groups/rename ,тело {"oldName": "...", "newName": "..."}):

- Оба имени обрезаются от пробелов. Пустое старое имя — ошибка «old group name is required», пустое новое — «new group name is required».
- Если новое имя совпадает со старым — ничего не делается (затронуто 0 клиентов).
- Иначе переименование выполняется атомарно:
 - запись в `client_groups` переименовывается;
 - у всех клиентов с `group_name = oldName` поле обновляется на `newName` ;
 - во **всех inbound**, где состоят затронутые клиенты (включая нодовые), в Xray-настройках значение `group` правится со старого на новое.
- После переименования панель помечает Xray как требующий перезапуска и рассылает уведомление об изменении клиентов.

Сообщения:

- Успех: «Группа переименована для {count} клиент(ов).»
- Конфликт имён в UI: «Группа с именем «{name}» уже существует.»

9.7. Добавление клиентов в группу

Кнопка **Добавить клиентов...**, заголовок — «Добавить клиентов в группу «{name}»».

Дословная подсказка в диалоге:

«Выберите клиентов для добавления в эту группу. Существующие привязки к входящим сохраняются; меняется только метка группы. Клиенты, уже входящие в эту группу, не показываются.»

Если добавлять некого, показывается «Нет других клиентов для добавления.»

Поведение (POST /panel/api/clients/groups/bulkAdd ,тело {"emails": [...], "group": "..."}):

- Имя группы обязательно (иначе ошибка «group name is required»); пустой список email — операция ничего не делает.
- Если такой группы ещё нет ни в `client_groups` , ни среди производных — она будет создана автоматически.
- Для выбранных email у клиентов проставляется `group_name = group` ; **привязки клиентов к inbound не меняются** — затрагивается только метка. Затем во всех inbound этих клиентов проставляется поле `group` .
- Возвращается число затронутых записей клиентов; Xray помечается на перезапуск.

Сообщение об успехе: «Добавлено {count} клиент(ов) в {name}.»

Пример: пометить нескольких клиентов группой одним запросом. Клиенты указываются по email, привязки к inbound при этом не меняются:

```
curl -s 'https://panel.example.com:2053/panel/api/clients/groups/bulkAdd' \
  -H 'Content-Type: application/json' \
  -b cookie.txt \
  -d '{"emails": ["alice@example.com", "bob@example.com"], "group": "vip"}'
```

Если группы `vip` ещё нет, она будет создана автоматически. После запроса у этих клиентов в записи проставится `group_name = "vip"`, а в Xray-настройках каждого их inbound клиентский объект получит поле `"group": "vip"`:

```
{ "id": "6f1b...", "email": "alice@example.com", "group": "vip", "enable": true }
```

9.8. Удаление клиентов из группы (без удаления самих клиентов)

Кнопка **Удалить клиентов...**, заголовок — «Удалить клиентов из группы «{name}»».

Дословная подсказка:

«Выберите участников для удаления из этой группы. Сами клиенты сохраняются (используйте «Удалить клиентов группы» для полного удаления).»

Поведение (`POST /panel/api/clients/groups/bulkRemove`, тело `{"emails": [...]}`): технически это то же, что «Добавить в группу» с пустой группой. У выбранных клиентов очищается `group_name`, а в их inbound поле `group` удаляется из Xray-настроек. Сами клиенты и их привязки к inbound остаются.

Сообщение об успехе: «Удалено {count} клиент(ов) из {name}.»

9.9. Сброс трафика группы

Кнопка **Сбросить трафик**.

Диалог подтверждения:

- Заголовок: «Сбросить трафик группы {name}?»
- Текст: «Сбрасывается только счётчик трафика группы. Счётчики отдельных клиентов не затрагиваются.»

Поведение: обнуляется **только отображаемый счётчик самой группы** — панель запоминает текущую сумму `up/down` группы как базовую отметку и далее в списке групп показывает `max(0, сумма - базовая отметка)`. Счётчики `up / down`, квоты и поле `enable` отдельных клиентов **не изменяются**, перезапуск Xray не требуется. Это изменение поведения относительно прежних версий, где сброс обнулял трафик всех клиентов группы.

Запрос: `POST /panel/api/clients/groups/resetTraffic` с телом `{"name": "<имя группы>"}`.

Сообщение об успехе: **«Трафик группы {name} сброшен.»**

9.10. Удаление группы и удаление клиентов группы

На странице есть **две принципиально разные операции удаления** — их легко перепутать, поэтому различие критично.

9.10.1. Удалить группу (сохранить клиентов)

Кнопка **«Удалить группу (сохранить клиентов)»**.

Диалог:

- Заголовок: **«Удалить группу {name}?»**
- Текст: **«Это удаляет группу и очищает её метку у {count} клиент(ов). Сами клиенты не удаляются.»**

Поведение (`POST /panel/api/clients/groups/delete` , тело `{"name": "..."}`): запись группы удаляется из `client_groups` , у всех её клиентов `group_name` очищается, и из их `inbound` убирается поле `group` . **Клиенты, их подключения и трафик сохраняются.** Xray помечается на перезапуск.

Сообщение об успехе: **«Группа очищена у {count} клиент(ов).»**

9.10.2. Удалить клиентов группы (полное удаление)

Кнопка **«Удалить клиентов группы»**.

Диалог:

- Заголовок: **«Удалить всех клиентов в {name}?»**
- Текст: **«Это безвозвратно удаляет {count} клиент(ов) вместе с их записями трафика. Метка группы также очищается. Это нельзя отменить.»**

Это деструктивная операция: она удаляет самих клиентов (через массовое удаление по email, эндпоинт `POST /panel/api/clients/bulkDel`), включая их записи трафика, и тем самым убирает их из всех `inbound`.

Сообщения:

- Успех: **«Удалено {count} клиент(ов).»**
- Частичный результат: **«{ok} удалено, {failed} пропущено»**

Если группа пуста, действия над её участниками недоступны — выводится **«В этой группе пока нет клиентов.»**

9.11. Связь со страницей «Клиенты»

Метка группы видна и используется и вне страницы **Группы**:

- В компактной записи клиента есть поле `group`, поэтому в списке клиентов отображается принадлежность к группе.
- Список клиентов (`/panel/api/clients/list/paged`) принимает параметр фильтра `group`: можно передать одно имя или несколько через запятую. Сопоставление выполняется по принципу «ИЛИ» внутри поля, без учёта регистра. Особый случай: пустой элемент в списке групп фильтра означает «клиенты без группы» (у которых `group` пустая).
- В ответе страницы клиентов отдаётся массив `groups` — полный перечень имён существующих групп, чтобы UI мог построить выпадающий список фильтра.

Пример: фильтрация клиентов по группам. Запрос отдаёт только клиентов с метками `vip` или `trial` (несколько имён — через запятую, «ИЛИ»):

```
GET /panel/api/clients/list/paged?group=vip,trial
```

Чтобы получить клиентов **без** группы, передайте пустой элемент в списке — например, значение фильтра `group=` (пустая строка) или `group=vip,` (метка `vip` плюс клиенты без группы).

9.12. Сводка эндпоинтов API

Все маршруты групп смонтированы под `/panel/api/clients`:

Метод и путь	Назначение	Тело запроса
GET <code>/panel/api/clients/groups</code>	Список групп со счётчиками клиентов	—
GET <code>/panel/api/clients/groups/:name/emails</code>	Email всех участников группы (сортировка по email)	—
POST <code>/panel/api/clients/groups/create</code>	Создать пустую группу	<code>{"name"}</code>
POST <code>/panel/api/clients/groups/rename</code>	Переименовать группу	<code>{"oldName", "newName"}</code>
POST <code>/panel/api/clients/groups/delete</code>	Удалить группу, сохранив клиентов (очистка метки)	<code>{"name"}</code>
POST <code>/panel/api/clients/groups/bulkAdd</code>	Добавить клиентов в группу (по email)	<code>{"emails":[...], "group"}</code>
POST <code>/panel/api/clients/groups/bulkRemove</code>	Убрать клиентов из группы (очистка метки)	<code>{"emails":[...]}</code>
POST <code>/panel/api/clients/bulkDel</code>	Полное удаление клиентов (используется «Удалить клиентов группы»)	<code>{"emails": [...], "keepTraffic"}</code>

Пример: типовой сценарий жизненного цикла группы через API.

```
# 1. Создать метку trial
curl -s .../panel/api/clients/groups/create -d '{"name":"trial"}'

# 2. Навесить её на двух клиентов
curl -s .../panel/api/clients/groups/bulkAdd -d '{"emails":
["u1@example.com","u2@example.com"],"group":"trial"}'

# 3. Обнулить трафик всех участников (по email из /groups/trial/emails)
curl -s .../panel/api/clients/groups/bulkRemove -d '{"emails":["u2@example.com"]}'

# 4. Удалить группу, но сохранить клиентов (только очистка метки)
curl -s .../panel/api/clients/groups/delete -d '{"name":"trial"}'
```

Шаг 4 удаляет запись группы и очищает `group_name` у её клиентов, но сами клиенты, их подключения и трафик остаются. Для безвозвратного удаления самих клиентов вместо этого используется `bulkDel`.

Операции, меняющие метку у клиентов (`rename`, `delete`, `bulkAdd`, `bulkRemove`), помечают Xray как требующий перезапуска и рассылают уведомление об изменении клиентов.

9.13. Трафик по группе

Новинка версии 3.3.0: в разделе **Группы** (страница «Клиенты», вкладка управления группами) таблица групп теперь показывает не только количество клиентов в каждой группе, но и суммарный израсходованный трафик группы. Колонка подписана **«Использованный трафик»**.

Что показывает колонка

Для каждой строки-группы выводится сумма трафика по всем клиентам, входящим в эту группу — то есть сложенные `up` + `down` (отданный + принятый трафик) всех её участников. Это даёт быстрый ответ на вопрос «сколько суммарно скачала/раздала вся группа», без необходимости открывать клиентов по одному и складывать вручную.

Рядом в таблице групп выводятся:

Колонка	Что означает
Имя	Имя группы
Клиенты	Сколько клиентов помечено этой группой (раньше столбец назывался «Клиентов в группе»)
Отправлено	Суммарный <code>up</code> (отданный трафик) по всем клиентам группы
Получено	Суммарный <code>down</code> (принятый трафик) по всем клиентам группы
Использованный трафик	Суммарный <code>up</code> + <code>down</code> по всем клиентам группы

Отданный и принятый трафик выводятся отдельными столбцами **Отправлено** и **Получено**, а столбец **Использованный трафик** показывает их сумму. Столбец числа клиентов называется просто **Клиенты**.

Сводка над таблицей дополнительно показывает агрегаты по всем группам — **«Всего групп»** и **«Клиенты с группой»**, а суммарный трафик разбит на две карточки: **«Всего отправлено / получено»** (со стрелками

вверх/вниз — отдельно отданный и принятый трафик всех групп) и **«Общий трафик»** (с иконкой диаграммы — их суммарный итог).

Как считается

Расчёт выполняется одним SQL-запросом к таблице клиентов с присоединением (`LEFT JOIN`) таблицы учёта трафика:

- по полю-метке группы (`group_name`) клиенты группируются, считается их количество — это и есть «Клиентов в группе»;
- трафик берётся как сумма `up + down` из присоединённой таблицы `client_traffics`. То есть суммируются и отданные (`up`), и принятые (`down`) байты по каждому клиенту;
- поскольку email уникален и в таблице клиентов, и в таблице трафика, присоединение не задваивает трафик одного клиента.

Особенности значений:

- **Клиенты без записи трафика** учитываются в счётчике участников, но добавляют к сумме 0, поэтому свежесозданная группа показывает трафик `0`.
- **Пустые группы** (созданы, но без клиентов) также присутствуют в списке с нулевым счётчиком и нулевым трафиком: помимо групп, «выведенных» из меток клиентов, в результат подмешиваются явно сохранённые группы, после чего список сортируется по имени без учёта регистра.
- Клиенты без метки группы (`group_name` пустой) в расчёт не попадают.

Связанные действия

Из таблицы групп по-прежнему доступны действия над группой целиком, в том числе **«Сбросить трафик»** — обнуляет **только счётчик самой группы** (базовую отметку), не трогая трафик отдельных клиентов (см. [9.9](#)). После такого сброса колонка «Использованный трафик» для этой группы показывает `0`.

10. Подписки (Subscription)

Подписка (subscription) — это механизм, который позволяет выдать клиенту одну постоянную ссылку (URL), по которой VPN-клиент сам скачивает и периодически обновляет полный набор конфигураций. Вместо того чтобы вручную пересылать пользователю отдельную ссылку на каждый inbound, ему передаётся единый адрес вида `https://домен:порт/sub/<subId>`. По этому адресу панель на лету собирает все конфигурации, привязанные к данному клиенту, и отдаёт их в нужном клиенту формате. При изменении настроек на сервере (новый адрес, ротация Reality-ключей, добавление inbound) клиент получает актуальную конфигурацию при следующем автоматическом обновлении, не требуя никаких действий от пользователя.

Подписку обслуживает отдельный HTTP/HTTPS-сервер внутри панели, который запускается независимо от веб-панели и слушает на собственном порту. Это сделано из соображений безопасности: порт подписки можно открыть наружу, не открывая порт самой панели.

10.1. Что такое subId и как формируется ссылка

Каждый клиент в inbound имеет поле `subId` (в интерфейсе — «ID подписки»). Именно это значение является ключом подписки: панель ищет во всех inbound клиентов, у которых `subId` совпадает с запрошенным, и объединяет их конфигурации в один ответ.

- Если у нескольких клиентов (в одном или в разных inbound) задан одинаковый `subId`, их конфигурации попадут в одну подписку. Это штатный способ выдать одному пользователю сразу несколько серверов/протоколов по одной ссылке.

Пример: один пользователь — два сервера по одной ссылке. Допустим, есть два inbound (VLESS на сервере А и Trojan на сервере В). Чтобы выдать пользователю обе конфигурации одной ссылкой, задайте обоим его клиентам одинаковый `subId`:

```
Inbound 1 (VLESS): email = ivan@vpn, subId = ivan2025
Inbound 2 (Trojan): email = ivan@vpn, subId = ivan2025
```

Тогда по адресу `https://sub.example.com:2096/sub/ivan2025` панель отдаст обе конфигурации сразу. Добавьте позже третий inbound с тем же `subId` — он появится у пользователя при следующем автообновлении подписки, без пересылки новой ссылки.

- Если у клиента поле `subId` пустое, ссылкой для общего доступа поделиться нельзя. В интерфейсе на это указывает подсказка: «У этого клиента нет subId, ссылка для общего доступа недоступна.»

Внешние ссылки и подписки клиента (вкладка «Links»)

В форме клиента есть вкладка «Links», где для отдельного клиента можно прикрепить дополнительные источники конфигураций, которые подмешиваются именно в его подписку (форматы RAW, JSON и Clash):

- **Add External Link** — сторонняя share-ссылка (`vless://`, `trojan://`, `ss://` и т.д.). Добавляется в выдачу как есть, а для JSON/Clash дополнительно разбирается в конфигурацию.

- **Add External Subscription** — адрес внешней подписки. Панель сама скачивает её (с кэшированием и коротким таймаутом) и вливает полученные конфигурации в общий список клиента.

Это удобно, чтобы выдать клиенту дополнительные серверы поверх ваших inbound по той же единой ссылке. Если ответ удалённой подписки слишком велик, он больше не обрезается молча: панель возвращает ошибку и продолжает использовать последнее успешно закэшированное значение.

- Значение `subId` нельзя задать произвольно: при сохранении проверяется, что в нём нет пробелов, символов `/`, `\` и управляющих символов. Соответствующая подсказка валидации: «ID подписки не может содержать пробелы, `'`, `"` или управляющие символы».

Итоговая ссылка строится как `<база>/<subPath>/<subId>` (см. раздел о настройках сервера подписок и поле «URI обратного прокси»). Если по `subId` не найдено ни одного клиента (клиент удалён, `subId` не существует), сервер возвращает HTTP 404 без тела. При внутренней ошибке — HTTP 500. VPN-клиенты ориентируются только на код ответа, поэтому тело ошибки намеренно пустое.

Порядок ссылок inbound в подписке

У каждого inbound есть поле **«Порядок в подписке»** (`subSortIndex`) — число от 1, задающее позицию ссылок этого inbound в выдаче подписки. Меньшие значения идут первыми; при равных значениях сохраняется исходный порядок создания (по id). Порядок применяется ко всем форматам выдачи — сырой текст, страница подписки, JSON и Clash. На порядок inbounds в самой панели это поле не влияет.

Поле редактируется в форме inbound рядом с настройками адреса в ссылке (share address) и синхронизируется на узлы по обычным правилам. Если хотя бы у одного inbound порядок отличается от 1, в списке Inbounds появляется компактная колонка **«Порядок»**.

10.2. Настройки сервера подписок

Все параметры подписки находятся в разделе настроек панели на вкладке **«Подписка»**. Ниже разобран каждый параметр; в скобках указан внутренний ключ настройки и значение по умолчанию.

Сам раздел разбит на вкладки: **«Настройки панели»**, **«Информация»**, **«Профиль»**, **«Сертификаты»**, **«Happ»** и **«Clash / Mihomo»**. Поля заголовка подписки, URL поддержки, страницы профиля, объявления и каталога темы находятся на вкладке «Профиль»; правила маршрутизации Happ и Clash/Mihomo — на одноимённых вкладках; интервал обновления подписки — на вкладке «Информация».

Основные параметры

Поле (UI)	Ключ	Значение по умолчанию	Описание
Включить подписку	<code>subEnable</code>	<code>true</code> (включено)	Запускает отдельный сервер подписок. Подсказка: «Функция подписки с отдельной конфигурацией». Если выключено — сервер подписок не стартует, и ни одна из ссылок не работает.
Прослушивание IP	<code>subListen</code>	пусто	IP-адрес, на котором сервер подписок принимает соединения. Подсказка: «Оставьте пустым по умолчанию, чтобы отслеживать все IP-адреса».

Поле (UI)	Ключ	Значение по умолчанию	Описание
Порт подписки	subPort	2096	ТСР-порт сервера подписок. Подсказка: «Номер порта для обслуживания службы подписки не должен использоваться на сервере» — порт должен быть свободен и не конфликтовать с панелью или Xray.
URI-путь	subPath	/sub/	Путь, по которому отдаются обычные подписки. Подсказка: «Должен начинаться с '/' и заканчиваться на '/'».
Домен прослушивания	subDomain	пусто	Домен, по которому разрешён доступ к подписке (валидация Host). Подсказка: «Оставьте пустым по умолчанию, чтобы слушать все домены и IP-адреса». Если задан — запросы с другим Host отклоняются.

Важно по безопасности: путь по умолчанию `/sub/` (и `/json/` для JSON) широко известен и легко подбирается. Панель показывает предупреждение: «Путь подписки по умолчанию `/sub/` широко известен — измените его.» и аналогичное для JSON. Рекомендуется задать собственный неочевидный путь.

TLS / сертификат

Поле (UI)	Ключ	По умолчанию	Описание
Путь к файлу публичного ключа сертификата подписки	subCertFile	пусто	Полный путь к файлу сертификата (<code>.crt</code> / <code>fullchain</code>). Подсказка: «Введите полный путь, начинающийся с '/'».
Путь к файлу приватного ключа сертификата подписки	subKeyFile	пусто	Полный путь к файлу приватного ключа. Подсказка: «Введите полный путь, начинающийся с '/'».

Если заданы оба пути и сертификат успешно загружается, сервер подписок работает по **HTTPS**. Если поля пустые либо сертификат не удалось прочитать — сервер падает обратно на **HTTP** (ошибка пишется в лог). Наличие валидного TLS также влияет на формирование базового URL: при порте 443 с TLS и порте 80 без TLS номер порта в ссылке опускается.

Интервал обновления

Поле (UI)	Ключ	По умолчанию	Описание
Интервалы обновления подписки	subUpdates	12	Как часто (в часах) клиентское приложение должно перезапрашивать подписку. Подсказка: «Интервал между обновлениями в клиентском приложении (в часах)». С 3.5.0 поле принимает 0–525600 и показывает диапазон (прежний предел интерфейса 168 не совпадал с сервером и после апгрейда с 2.x блокировал сохранение любых настроек).

Значение передаётся клиенту в HTTP-заголовке `Profile-Update-Interval`; современные клиенты используют его как период автообновления конфигурации.

Формат и информация в ответе

Поле (UI)	Ключ	По умолчанию	Описание
Кодировать	subEncrypt	true	Подсказка: «Шифровать возвращенные конфиги в подписке». Технически это не шифрование, а Base64-кодирование всего тела обычной подписки (формат, который ожидает большинство клиентов). При выключении ссылки отдаются открытым текстом, по одной на строку.
Показать информацию об использовании	subShowInfo	true	Подсказка: «Отображать остаток трафика и дату окончания после имени конфигурации». Когда включено, к названию (remark) каждой конфигурации добавляются маркеры остатка трафика () и срока действия (например, 5D, 3H); при истёкшем/недоступном клиенте показывается N/A .
Включать Email в название	subEmailInRemark	true	Подсказка: «Включать email клиента в название профиля подписки». Добавляет email клиента в remark профиля.

Шаблон ремарки (Remark Template)

Отображаемое название (remark) каждой конфигурации в подписке формируется по **шаблону ремарки** — поле **«Шаблон примечания»** (remarkTemplate) на вкладке **«Информация»** настроек подписки. Прежний конструктор модели примечания (отдельный выбор частей inbound/email/external proxy и символа-разделителя) из интерфейса убран; теперь вы пишете произвольный формат имени и вставляете в него переменные. Значение по умолчанию — `{{INBOUND}}-{{EMAIL}}|{{TRAFFIC_LEFT}}|{{DAYS_LEFT}}D` (то есть по умолчанию имя профиля содержит email клиента). Если поле оставить пустым, применяется прежняя (не настраиваемая через интерфейс) модель ремарки.

Переменные сгруппированы по разделам **Client**, **Traffic** и **Time & status** и показываются рядом с полем как кликабельные чипы `{{VAR}}` с подсказкой при наведении; клик вставляет токен в шаблон, доступен живой предпросмотр. Каждая переменная подставляется индивидуально для конкретного клиента в момент генерации подписки. Допускается и упрощённая запись в одинарных скобках (`{DATA_LEFT}` , `{EXPIRE_DATE}` , `{PROTOCOL}` , `{TRANSPORT}` и т.п.) — панель сама приводит её к внутреннему формату `{{...}}` .

Доступные переменные:

- **Идентификация клиента:** `{{EMAIL}}` (синоним — `{{USERNAME}}`), `{{INBOUND}}` (ремарка самого inbound), `{{HOST}}` (ремарка хоста), `{{ID}}` (UUID), `{{SHORT_ID}}` (первые 8 символов UUID), `{{SUB_ID}}` , `{{COMMENT}}` , `{{TELEGRAM_ID}}` , `{{PROTOCOL}}` , `{{TRANSPORT}}` .
- **Трафик:** `{{TRAFFIC_USED}}` , `{{TRAFFIC_LEFT}}` , `{{TRAFFIC_TOTAL}}` (и их *_BYTES -варианты в точных байтах), `{{UP}}` , `{{DOWN}}` , `{{USAGE_PERCENTAGE}}` .
- **Срок и статус:** `{{DAYS_LEFT}}` , `{{TIME_LEFT}}` , `{{EXPIRE_DATE}}` (ГГГГ-ММ-ДД), `{{JALALI_EXPIRE_DATE}}` (дата по календарю джалали), `{{EXPIRE_UNIX}}` , `{{CREATED_UNIX}}` , `{{RESET_DAYS}}` , `{{STATUS}}` (active / expired / disabled / depleted), `{{STATUS_EMOJI}}` .

- **Подключение (Connection):** `{{PROTOCOL}}` — протокол (VLESS, VMess, Trojan и т.д.), `{{TRANSPORT}}` — транспортная сеть (tcp, ws, grpc и т.д.), `{{SECURITY}}` — безопасность транспорта (TLS, REALITY, NONE; выводится заглавными). Как и переменные расхода и срока, эти три переменные действуют только в теле подписки и автоматически убираются из ремарки на отображаемых ссылках в панели (QR/«Информация») и на странице информации о подписке.

Шаблон можно разбить на сегменты вертикальной чертой `|`. Сегмент, в котором переменная даёт «безлимитное» значение (∞) — например `{{TRAFFIC_LEFT}}` или `{{DAYS_LEFT}}` у клиента без ограничений, — автоматически скрывается. Кроме того, блок с расходом трафика и сроком показывается один раз, на первой ссылке клиента, чтобы не дублироваться в каждой конфигурации.

Пример. Шаблон `{{EMAIL}}|{{TRAFFIC_LEFT}}|{{DAYS_LEFT}}D` даст у клиента с остатком 42 ГБ и 7 днями имя вида `ivan@vpn 42.00GB 7D`, а у безлимитного клиента — просто `ivan@vpn` (сегменты с ∞ опущены).

С версии 3.4.2 переменная `{{EMAIL}}` (и синоним `{{USERNAME}}`) выводится только на **первой** ссылке клиента в теле подписки — как и блок остатка трафика/срока; на остальных ссылках того же клиента она опускается.

На отображаемых в панели ссылках (QR-код и окна «Информация» на странице «Клиенты») и на странице информации о подписке email клиента присутствует в имени профиля: вид «inbound-host-email» при заданном хосте либо «inbound-email» без хоста. Сведения о трафике и сроке (а также переменные группы «Подключение») в эти отображаемые имена не подставляются — они работают только в теле подписки, которое получает VPN-клиент.

Если строка статистики трафика клиента «осиротела» после удаления и повторного создания inbound, переменная `{{TRAFFIC_USED}}` (и другие показатели расхода) больше не показывает `0.00B`: панель дополнительно ищет статистику по email клиента и подставляет корректный использованный трафик. | Шаблон ремарки `| remarkTemplate | {{INBOUND}} | {{TRAFFIC_LEFT}} | {{DAYS_LEFT}}D |` Свободный шаблон отображаемого названия (remark) каждой конфигурации с подстановкой переменных `{{VAR}}`. Подставляется индивидуально для каждого клиента при генерации подписки. Прежний конструктор «модели примечания» (выбор inbound/email/external proxy и разделителя) убран из интерфейса и используется только как запасной вариант, если поле оставить пустым. Подробнее — см. «Шаблон ремарки (Remark Template)» ниже. |

Метаданные профиля (заголовки ответа)

Эти строки передаются клиенту в HTTP-заголовках ответа и отображаются в VPN-клиенте как метаданные профиля. Все они по умолчанию пустые.

Поле (UI)	Ключ	Заголовок	Описание
Заголовок подписки	<code>subTitle</code>	<code>Profile-Title</code> (в Base64)	«Название подписки, которое видит клиент в VPN-клиенте». Для Clash также используется как имя импортируемого профиля через <code>Content-Disposition</code> .
URL поддержки	<code>subSupportUrl</code>	<code>Support-Url</code>	«Ссылка на техническую поддержку, отображаемая в VPN-клиенте».

Поле (UI)	Ключ	Заголовок	Описание
URL профиля	subProfileUrl	Profile-Web-Page-Url	«Ссылка на ваш сайт, отображаемая в VPN-клиенте». Если не задано, подставляется фактический URL запроса подписки.
Объявление	subAnnounce	Announce (в Base64)	«Текст объявления, отображаемый в VPN-клиенте». С 3.5.0 текст также показывается информационным баннером наверху инфо-страницы подписки (когда непуст), а кастомным шаблонам доступна переменная <code>announce</code> .

Кроме того, в каждом ответе передаётся заголовок `Subscription-Userinfo` с агрегированными данными трафика клиента: `upload`, `download`, `total` и `expire` (момент истечения в секундах). По нему клиент показывает остаток трафика и срок действия.

Маршрутизация (только для клиента Haprr)

Поле (UI)	Ключ	По умолчанию	Описание
Включить маршрутизацию	subEnableRouting	false	«Глобальная настройка для включения маршрутизации в VPN-клиенте. (Только для Haprr)». Передаётся в заголовке <code>Routing-Enable</code> .
Правила маршрутизации	subRoutingRules	пусто	«Глобальные правила маршрутизации для VPN-клиента. (Только для Haprr)». Передаётся в заголовке <code>Routing</code> .

| Скрыть настройки сервера | `subHideSettings` | false | «Скрывать настройки сервера в подписке (только для Haprr)». Когда включено, в клиенте Haprr скрывается возможность просматривать и изменять параметры сервера. Опция действует только для клиента Haprr. |

Маршрутизация Incy (только для клиента Incy)

Для VPN-клиента **Incy** в настройках подписки есть отдельная вкладка **«Incy»** с двумя полями: переключатель **«Включить маршрутизацию»** (`subIncyEnableRouting`, по умолчанию выключено) и текстовое поле **«Правила маршрутизации»** (`subIncyRoutingRules`) формата `incy://routing/onadd/<base64>`. Когда маршрутизация включена и поле заполнено, эта строка дописывается отдельной строкой в тело подписки (формат raw) – так профиль маршрутизации доставляется клиенту Incy, не конфликтуя с заголовком `Routing` клиента Haprr. Настройки действуют только для клиента Incy.

URI обратного прокси

Поле (UI)	Ключ	По умолчанию	Описание
URI обратного прокси	subURI	пусто	«Изменить базовый URI URL-адреса подписки для использования за прокси-серверами».

Если поле пустое, базовый адрес ссылки панель строит сама из домена и порта подписки (с учётом TLS). Если же подписка раздаётся через внешний реверс-прокси/CDN на другом домене или пути, в это поле

задают итоговый базовый URI, и все ссылки будут строиться от него. Аналогичные отдельные поля есть для JSON (`subJsonURI`) и Clash (`subClashURI`).

Если задан только общий `subURI` , а отдельные поля для JSON и Clash оставлены пустыми, ссылки этих форматов на странице подписки наследуют схему и хост из `subURI` (а не порт sub-сервера и `http`) — так они совпадают с адресом обратного прокси.

Пример: подписка за реверс-прокси. Сама подписка слушает на `2096` , но снаружи доступна через nginx/CDN на `https://cfg.example.com/u/` . Чтобы ссылки в ответе строились от внешнего адреса, а не от внутреннего `домен:2096` , в поле «Reverse proxy URI» задают итоговый базовый URI:

```
Reverse proxy URI: https://cfg.example.com/u
```

Тогда конечная ссылка примет вид `https://cfg.example.com/u/ivan2025` . Для JSON- и Clash-форматов при необходимости заполняют отдельные поля `subJsonURI` и `subClashURI` тем же способом.

10.3. Форматы вывода

Подписка может отдаваться в трёх независимых форматах, каждый со своим эндпоинтом, который можно включать/выключать отдельно.

Адрес сервера и узлы в выдаче

Адрес сервера в ссылках подписки подставляется по той же стратегии адреса в ссылке, что и обычные ссылки и QR-коды в панели: «listen» — маршрутизируемый адрес привязки, «custom» — заданный пользовательский адрес (`shareAddr`), «node» (по умолчанию) — адрес узла. Для inbound без явно заданной стратегии выдача подписки не меняется. Это позволяет узловому inbound, привязанному к конкретному публичному IP, отдавать клиентам достижимый адрес. Стратегия применяется к сырому, JSON и Clash форматам.

Имя узла (Node) не добавляется к названию (remark) профиля в подписке: в клиентском приложении показывается только ремарка inbound, заданная администратором, без внутреннего суффикса вида `@имя-узла` . Чтобы различать одноимённые записи в мультиузловой подписке, задавайте им разные ремарки вручную либо используйте управляемые хосты (Hosts) с собственными Remark.

Если из-за рассинхронизации между узлами один и тот же клиент попал в служебный JSON inbound дважды, выдача подписки автоматически устраняет такие дубликаты по email во всех трёх форматах, поэтому повторяющиеся профили в выдаче не появляются.

Управляемые хосты (Hosts)

Раздел **Hosts** (пункт бокового меню; сводная страница с количеством Total/Enabled/Disabled и списком) задаёт переопределения адреса для ссылок подписки. Для каждого inbound можно добавить один или несколько **хостов** — эндпоинтов, которые подставляются в выдаваемые клиенту subscription-ссылки **вместо адреса, порта и TLS-параметров самого inbound**. Это удобно, чтобы раздавать трафик через CDN или релей, не меняя сам inbound.

С версии 3.5.0 хост — это «группа»: одна запись охватывает **несколько inbound** и **несколько адресов** сразу. Действия списка (включение/выключение/удаление/перестановка) и API работают с группами (строковый `groupId`); появился массовый эндпоинт `POST /panel/api/hosts/bulk/add ( {"inboundIds": [...], "hosts": [...], ...} )`, а `list / get / update / del / setEnable` оперируют объектами группы.

У каждого хоста (группы) задаются:

- **Remark** и описание (Description), привязка к **Inbounds** (мульти-выбор с поиском; минимум один), переключатель **Enable** и назначение на узлы (**Nodes**).
- **Address** — с 3.5.0 это **список адресов** (ввод тегами; разделители — запятая, точка с запятой, пробел; плейсхолдер `cdn.example.com, cdn2.example.com:443`). У каждой записи может быть свой встроенный `:` порт (в том числе IPv6 в скобках `:::1]:443`); выпадающая подсказка предлагает адреса, уже использованные другими хостами. Пустой список — наследуется адрес самого inbound (в списке это оранжевая метка **Inherits**). **Port** (`0` — наследуется порт inbound) служит портом по умолчанию для записей без встроенного порта; **Tags** (учитываются только в RAW-подписке).
- Вкладка **Security** — `same / tls / none / reality` с SNI, отпечатком (fingerprint), ALPN, привязкой сертификата (`pinned-cert`), `allowInsecure` и ECH.
- Вкладка **Advanced** — Host header, Path, маршрут VLESS, Mux, Sockopt, Final Mask и исключение хоста из отдельных форматов подписки (`raw / json / clash`). С 3.5.0 **Final Mask хоста попадает и в raw-ссылки** (`fm=`; раньше — только в JSON/Clash): TCP/UDP-маски хоста добавляются к маскам inbound, QUIC-параметры хоста берутся, только если их нет у inbound. Переключатель **Allow insecure** теперь действует и для **Hysteria/Hysteria2** (`insecure=1` в ссылке, `skip-cert-verify: true` в Clash).
- Вкладка **Clash (mihomo)** — версия IP, Mihomo X25519, перемешивание хостов (Shuffle host).

В списке колонка **Endpoint** показывает адреса чипами (первый виден, остальные — в поповере «+N»), а колонка **Inbounds** — чипы inbound, раскрашенные по протоколу. С 3.5.0 хосты упорядочиваются **глобально** (по порядку сортировки, затем по примечанию), а не в пределах inbound; массовые включение/выключение/удаление сохраняются. Управляемые хосты приходят на смену прежнему массиву External Proxy.

Маршрут VLESS (VLESS route). С версии 3.4.2 это одно число `0-65535` (а не список портов; подсказка — «одно значение VLESS route (0-65535), зашиваемое в UUID, например 443; пусто — без него», плейсхолдер `443`). Заданное значение реально «зашивается» в UUID каждой генерируемой подписки (`raw / JSON / Clash`): Xray читает байты 6-7 UUID и маскирует их перед аутентификацией, поэтому клиент по-прежнему совпадает. Пустое или некорректное значение UUID не меняет.

Обычные ссылки (SUB) — Base64 / plain text

Базовый формат, эндпоинт `subPath` (по умолчанию `/sub/`). Включён всегда (при включённой подписке в целом). Возвращает список ссылок Xray (`vless://`, `vmess://`, `trojan://`, `ss://` и т.д.) — по одной на строку. При включённой опции «Кодировать» (`subEncrypt`) весь список кодируется в Base64; при выключенной — отдаётся открытым текстом. Этот формат понимают практически все клиенты (v2rayNG, V2RayTun, Sing-box, NekoBox, Streisand, Shadowrocket, Hap и др.).

Пример: тело ответа с выключенным «Кодировать». При `subEncrypt = false` эндпоинт `/sub/` отдаёт открытый текст — по одной ссылке на строку:


```
vless://3c8f...@a.example.com:443?security=reality&...#srvA-ivan
trojan://p4ss@b.example.com:443?security=tls&...#srvB-ivan
```

При `subEncrypt = true` (по умолчанию) тот же список целиком кодируется в Base64 и отдаётся одной строкой — именно этот вид ожидает большинство клиентов.

JSON-подписка (sing-box и совместимые)

Эндпоинт `subJsonPath` (по умолчанию `/json/`), включается отдельной галочкой.

Поле (UI)	Ключ	По умолчанию	Описание
JSON-подписка	<code>subJsonEnable</code>	<code>false</code>	«Включить/отключить JSON-эндпоинт подписки независимо.»

Возвращает полную JSON-конфигурацию (формат, понятный sing-box и производным клиентам — Podkop, OpenWRT sing-box, Karing, NekoBox). С 3.5.0 в JSON-подписку попадают и клиенты **нативных WireGuard-inbound** (`secretKey`, `address`, `peers[]` с `publicKey` / `endpoint` / `preSharedKey` / `keepAlive` / `allowedIPs`, `mtu`) — раньше они молча пропускались. Для этого формата доступны дополнительные параметры (вкладка `subFormats`):

- **Mux** (`subJsonMux`, по умолчанию пусто) — JSON-настройки мультиплексирования (Mux), которые внедряются в outbound каждого потока JSON-подписки. «Передача нескольких независимых потоков данных в одном соединении.»
- **Final Mask** (`subJsonFinalMask`, по умолчанию пусто) — «Маски `finalmask` xray (TCP/UDP) и настройки QUIC, добавляемые в каждый поток JSON-подписки. Требуется свежая версия xray на клиенте». Настраивается через подполя: «Пакеты» (`packets`), «Длина» (`length`), «Интервал» (`interval`), «Макс. разбиение» (`maxSplit`), «Шумы» (`noises`: «Тип»/ `type`, «Пакет»/ `packet`, «Задержка (мс)»/ `delayMs`, «Применить к»/ `applyTo`, кнопка «+ Шум»), а также «Параллелизм» (`concurrency`), «Параллелизм xudp» (`xudpConcurrency`) и «xudp UDP 443» (`xudpUdp443`).
- **Правила маршрутизации** (`subJsonRules`, по умолчанию пусто) — глобальные правила, добавляемые в JSON-конфигурацию.

Clash / Mihomo-подписка (YAML)

Эндпоинт `subClashPath` (по умолчанию `/clash/`), включается отдельной галочкой.

Поле (UI)	Ключ	По умолчанию	Описание
Подписка Clash / Mihomo	<code>subClashEnable</code>	<code>false</code>	Включает генерацию YAML-конфигурации для клиентов Clash и Mihomo.
Включить маршрутизацию	<code>subClashEnableRouting</code>	<code>false</code>	«Добавлять глобальные правила маршрутизации Clash/Mihomo в сгенерированные YAML-подписки.»
Глобальные правила маршрутизации	<code>subClashRules</code>	пусто	«Правила Clash/Mihomo, добавляемые в начало каждой YAML-подписки перед <code>MATCH,PROXY</code> .»

Ответ отдаётся с типом `application/yaml; charset=utf-8`. Если задан «Заголовок подписки» (`subTitle`), он передаётся также в заголовке `Content-Disposition (attachment; filename*=UTF-8' '<title>)`, чтобы Clash-клиент назвал импортируемый профиль этим именем.

Формат генерируемых ссылок и YAML поддерживается в актуальном состоянии для современных клиентов: Shadowsocks-2022 (SS2022) больше не кодирует userinfo в Base64; ссылки Shadowsocks с http-обфускацией выдаются в формате SIP002 с плагином `obfs-local`; для подписок Clash/Mihomo реализован полный набор полей XHTTP. С 3.5.0 в Clash/Mihomo-подписку попадают и клиенты **нативных WireGuard-inbound** (поля `private-key`, `public-key`, `pre-shared-key`, `persistent-keepalive`, `ip/ipv6`, `mtu`, `dns`) — раньше они молча пропускались. Отдельных настроек это не требует — ссылки просто корректнее распознаются клиентами.

Примечание: в этой сборке поддерживаются именно три формата — обычные ссылки (Base64/текст), JSON (sing-box-совместимый) и Clash/Mihomo (YAML). Отдельного формата Outline в сервере подписок нет.

10.4. Страница информации о подписке и QR-коды

Если открыть **любую из трёх ссылок подписки** — raw, JSON или Clash — в браузере (или явно добавить к URL параметр `?html=1` либо `?view=html`, либо отправить заголовок `Accept: text/html`), сервер вместо «сырого» ответа отдаёт визуальную **страницу информации о подписке** («Информация о подписке»). До 3.5.0 так работала только основная ссылка `/sub/`, а `/json/` и `/clash/` отдавали в браузер сырой JSON/YAML. VPN-клиенты по-прежнему получают машинный ответ, так как они не запрашивают HTML.

Страница (одностраничное приложение, собранное Vite) показывает:

- **Информацию о подписке** (блок Descriptions):
 - «ID подписки» — значение `subId`;
 - «Статус» — «Активна», «Неактивна» или «Неограниченно». Статус «неактивна» выставляется, если клиент выключен, исчерпал лимит трафика или истёк срок действия;
 - «Загружено» и «Отправлено» — объёмы трафика;
 - «Общий лимит» — лимит трафика или ∞ , если не ограничен;
 - «Срок действия» — дата окончания или «Бессрочно»;
 - остаток трафика и время последнего онлайна.
 - Даты отображаются по григорианскому или джалали-календарю в зависимости от настройки «Calendar Type» панели (`datepicker`, по умолчанию `gregorian`).
- **Ссылки на подписку**: для каждого включённого формата — отдельная строка с цветным тегом (зелёный **SUB**, фиолетовый **JSON**, золотой **CLASH**), кнопкой копирования и кнопкой **QR-кода** (всплывающее окно, размер 240 px). Строка с JSON и CLASH появляется только если соответствующий формат включён в настройках.
- **Индивидуальные ссылки** («Скопировать ссылку»): полный список отдельных конфигураций, входящих в подписку, каждая со своим тегом протокола, кнопкой копирования и QR-кодом (для post-quantum-ссылок QR не строится).

- **Кнопка «Копировать все конфигурации»** (над списком индивидуальных ссылок): одним нажатием копирует в буфер обмена сразу все ссылки конфигураций (каждую с новой строки), не требуя копировать их по одной; по завершении показывается уведомление «Все конфигурации скопированы».
- **Кнопки быстрого импорта в приложения** (выпадающие меню по платформам): для Android — v2box, v2rayNG (deep-link `v2rayng://install-config?url=...`), Sing-box, V2RayTun, NPV Tunnel, Happ (`happ://add/...`), Incy (`incy://add/...`); для iOS — Shadowrocket (через параметр `flag=shadowrocket`), v2box (`v2box://install-sub?url=...&name=...`), Streisand (`streisand://import/...`), V2RayTun, NPV Tunnel, Happ, Incy. Эти кнопки либо открывают deep-link нужного приложения с уже подставленным адресом подписки, либо копируют ссылку в буфер обмена.

Страница информации отдаётся с заголовками запрета кэширования (`Cache-Control: no-cache`), чтобы клиент всегда видел актуальные данные о трафике и сроке действия.

10.5. Кастомные шаблоны страницы подписки

Начиная с 3.3.0 можно заменить стандартную страницу-лендинг подписки собственным HTML-шаблоном. По умолчанию по адресу подписки отдаётся встроенная страница, но если указать каталог со своим шаблоном, панель будет рендерить его и подставлять в него актуальные данные клиента (трафик, срок действия, ссылки и т. д.).

Важно: панель **не предоставляет** готовых шаблонов — свою тему нужно создать самостоятельно. Инструкция по созданию и список доступных переменных — в [docs/custom-subscription-templates.md](https://docs.v2ray.com/custom-subscription-templates.md).

Где включается

Каталог темы задаётся в настройках панели:

Настройки → Подписка → раздел «Информация о подписке», поле «Каталог темы подписки» (`subThemeDir`).

Описание поля в интерфейсе: «Абсолютный путь к папке с пользовательским шаблоном (`index.html/sub.html`) для страницы подписки (например, `/etc/3x-ui/sub_templates/my-theme/`). Оставьте пустым, чтобы использовать страницу по умолчанию.»

Рядом в том же разделе расположены смежные настройки, значения которых доступны в шаблоне:

В описании поля «Каталог темы подписки» есть ссылка **«Руководство по шаблонам ↗»** на документацию по созданию собственных шаблонов оформления страницы подписки.

- **«Заголовок подписки»** (`subTitle`) — название, видимое клиенту;
- **«URL поддержки»** (`subSupportUrl`) — ссылка на техподдержку.

Параметр настройки

Параметр	Значение по умолчанию	Назначение
subThemeDir	"" (пусто)	Абсолютный путь к каталогу с вашим HTML-шаблоном. Пусто = встроенная страница по умолчанию.

Как подставить свой шаблон

1. Создайте на сервере папку для темы (где угодно), например `/etc/3x-ui/sub_templates/my-theme/`.
2. Положите внутрь HTML-файл с именем `index.html` или `sub.html`.

Пример: путь к теме. Финальная раскладка на сервере и значение поля в настройках:

```
/etc/3x-ui/sub_templates/my-theme/
└─ index.html      (или sub.html — он имеет приоритет)
```

Настройки → Подписка → «Каталог темы подписки»:
`/etc/3x-ui/sub_templates/my-theme/`

Путь должен быть **абсолютным** (начинаться с `/`). Если в папке нет ни `index.html`, ни `sub.html`, панель отдаст встроенную страницу. 3. В панели откройте **Настройки** → **Подписка** и впишите **абсолютный** путь к этой папке в поле «Каталог темы подписки». 4. Сохраните настройки.

Поведение выбора файла и рендеринга:

- Если в каталоге есть `sub.html`, используется именно он; иначе берётся `index.html`. То есть `sub.html` имеет приоритет над `index.html`.
- Шаблон рендерится стандартным движком Go `html/template`.
- Разобранный шаблон **кэшируется** и перечитывается с диска только при изменении времени модификации файла. Поэтому правки шаблона подхватываются без перезапуска панели, но без накладных расходов на чтение/парсинг при каждом запросе.
- Ответ формируется в буфер целиком и только потом отдаётся клиенту: если шаблон упадёт во время выполнения, частично сгенерированная (битая) страница не уйдёт пользователю.

Поведение по умолчанию и откат (fallback)

- Поле пустое → отдаётся встроенная SPA-страница (данные внедряются в `window.__SUB_PAGE_DATA__`).
- Путь не существует или это не каталог → используется страница по умолчанию.
- В каталоге нет ни `index.html`, ни `sub.html` → в лог пишется предупреждение «subThemeDir set but no usable template found», отдаётся страница по умолчанию.
- Файл шаблона есть, но не парсится → в лог пишется ошибка «custom template parse failed», отдаётся страница по умолчанию.
- Ошибка при выполнении шаблона → в лог пишется «custom template execution failed», отдаётся страница по умолчанию.

То есть любая проблема с кастомным шаблоном не «ломает» подписку — панель всегда деградирует к встроенной странице. Все страницы подписки (и кастомная, и стандартная) отдаются с заголовками запрета кэширования (Cache-Control: no-cache, no-store, must-revalidate), чтобы клиенты всегда получали свежие данные о трафике и сроке.

Доступные переменные шаблона

В контекст шаблона передаётся набор данных клиента подписки. Обращение — через `{{ .имя }}`:

Переменная	Тип	Описание
<code>{{ .sId }}</code>	string	ID подписки (UUID).
<code>{{ .enabled }}</code>	bool	Включён ли клиент/подписка.
<code>{{ .download }}</code>	string	Форматированный объём загрузки (напр. «2.5 GB»).
<code>{{ .upload }}</code>	string	Форматированный объём отправки.
<code>{{ .total }}</code>	string	Форматированный общий лимит трафика.
<code>{{ .used }}</code>	string	Форматированный использованный трафик (download + upload).
<code>{{ .remained }}</code>	string	Форматированный остаток трафика.
<code>{{ .expire }}</code>	int64	Срок действия — Unix-время в секундах (0 = бессрочно). Для JS <code>Date</code> умножьте на 1000.
<code>{{ .lastOnline }}</code>	int64	Время последнего онлайна — Unix-время в миллисекундах (0 = ни разу).
<code>{{ .downloadByte }}</code>	int64	Загрузка в точных байтах.
<code>{{ .uploadByte }}</code>	int64	Отправка в точных байтах.
<code>{{ .totalByte }}</code>	int64	Общий лимит в точных байтах.
<code>{{ .subUrl }}</code>	string	URL страницы подписки.
<code>{{ .subJsonUrl }}</code>	string	URL JSON-конфигурации подписки.
<code>{{ .subClashUrl }}</code>	string	URL конфигурации Clash/Mihomo.
<code>{{ .subTitle }}</code>	string	Заголовок подписки из настроек (может быть пустым).
<code>{{ .subSupportUrl }}</code>	string	URL поддержки из настроек (может быть пустым).
<code>{{ .links }}</code>	[]string	Список строк-конфигов (VMess, VLESS, и т.д.). Перебор: <code>{{ range .links }} ... {{ end }}</code> .
<code>{{ .emails }}</code>	[]string	Список email, относящихся к подписке.
<code>{{ .datepicker }}</code>	string	Текущий формат календаря панели: <code>gregorian</code> или <code>jalali</code> (берётся из настройки «Тип календаря»; если пусто — <code>gregorian</code>).

Минимальный пример тела шаблона, использующего часть переменных:

```
<h1>{{ .subTitle }}</h1>
<p>Использовано: {{ .used }} из {{ .total }} (осталось {{ .remained }})</p>
{{ range .links }}<div>{{ . }}</div>{{ end }}
```

Пример: дата окончания из `expire`. Поле `{{ .expire }}` — это Unix-время в **секундах**, поэтому для JavaScript его умножают на 1000; значение `0` означает «без срока»:

```
<script>
  var exp = {{ .expire }};
  document.write(exp === 0
    ? 'Без срока'
    : 'До ' + new Date(exp * 1000).toLocaleDateString());
</script>
```

Обратите внимание: `{{ .lastOnline }}` задаётся уже в **миллисекундах** — его умножать на 1000 не нужно.

11. Xray: маршрутизация, outbounds, DNS и расширения

Раздел «**Настройки Xray**» — это редактор шаблона конфигурации Xray-core, на основе которого панель генерирует итоговый `config.json` для запуска ядра. Подсказка к разделу шаблона: «*На основе шаблона создаётся конфигурационный файл Xray.*» В отличие от inbounds (которые хранятся отдельно в БД и подставляются в шаблон при сборке конфигурации), всё остальное — логи, маршрутизация, outbounds, DNS, политика, статистика — задаётся именно здесь.

Важно: значение шаблона хранится в БД под ключом `xrayTemplateConfig`. При сохранении панель прогоняет его через ряд автоматических преобразований (см. 11.11). Любой синтаксически некорректный JSON будет отклонён с ошибкой «*xray template config invalid*».

Расположение в меню: «Исходящие» и «Маршрутизация»

«**Исходящие**» (Outbounds) и «**Маршрутизация**» (Routing) — это отдельные пункты бокового меню (сразу под «Хосты», над «Настройки панели»), у каждого свой адрес: `/outbound` и `/routing`. Прямые ссылки на эти страницы и перезагрузка страницы работают как ожидается. В подменю «**Конфигурации Xray**» при этом остаются только: Основное, Балансировщик, DNS и Расширенный шаблон. В описании ниже разделы 11.3 и 11.4 соответствуют страницам «Маршрутизация» и «Исходящие».

11.1. Структура редактора: вкладки/режимы

Редактор предлагает несколько режимов отображения шаблона (фильтры по разделам JSON):

Режим	Что показывает
Основное	Базовые секции (Лог, базовая маршрутизация, основные настройки)
Расширенный шаблон	Полный JSON-шаблон Xray
Все	Все секции одновременно

Логические группы настроек внутри редактора:

- **Основные настройки** (подсказка: «*Эти параметры описывают общие настройки*»).
- **Лог** (см. 11.10).
- **Базовые соединения**: блокировки и прямые маршруты.
- **Входящие** (подсказка: «*Изменение шаблона конфигурации для подключения определенных клиентов*»).
- **Исходящие** (см. 11.4).
- **Балансировщик** (см. 11.5).
- **Маршрутизация** (подсказка: «*Важен приоритет каждого правила!*», см. 11.3).
- **DNS / Fake DNS** (см. 11.6).

11.2. Основные настройки (General)

Freedom Protocol Strategy

Поле	Подпись	Описание	По умолчанию
FreedomStrategy	Настройка стратегии протокола Freedom	Стратегия вывода сети для прямого (freedom) outbound. Подсказка: «Установка стратегии вывода сети в протоколе Freedom». Управляет полем domainStrategy внутри settings outbound-a с протоколом freedom.	В эталонном шаблоне domainStrategy для freedom-outbound direct равен AsIs (адрес не резолвится, передаётся в исходном виде).

domainStrategy для freedom (значения Xray-core): AsIs (не резолвить домен на стороне сервера), а также семейство UseIP / UseIPv4 / UseIPv6 и их «принудительные» варианты ForceIP*, заставляющие выходной сервер резолвить домен и подключаться по полученному IP. Меняйте на UseIPv4, если на выходном сервере нет IPv6 или нужно принудительно ходить только по IPv4.

Freedom Happy Eyeballs (IPv4/IPv6)

Поле	Подпись	Описание
FreedomHappyEyeballs	Freedom Happy Eyeballs (IPv4/IPv6)	Подсказка: «Двухстековый набор для прямого (freedom) исходящего – полезно на выходных серверах с IPv4 и IPv6.» Включает алгоритм Happy Eyeballs (одновременная попытка по обоим семействам адресов) для freedom-outbound.

Отдельный Happy Eyeballs есть и в **настройках dial (sockopt) конкретного outbound**: с 3.5.0 он действительно включается (раньше конфиг сериализовался как выключенный и переключатель «слетал»). «Try delay (ms)» по умолчанию теперь **250**; явно заданный **0** (= выключено) сохраняется; также доступны Prioritize IPv6, interleave (1) и maxConcurrentTry (4). | try delay | (подсказка) | «Миллисекунды перед попыткой другого семейства адресов. 150–250 мс – хорошая отправная точка.» Задержка перед переключением на альтернативное семейство адресов. Рекомендуемый диапазон – 150–250 мс. |

Overall Routing Strategy

Поле	Подпись	Описание	По умолчанию
RoutingStrategy	Настройка маршрутизации доменов	Общая стратегия разрешения DNS для маршрутизации. Подсказка: «Установка общей стратегии маршрутизации разрешения DNS». Управляет полем routing.domainStrategy.	В эталонном шаблоне routing.domainStrategy = AsIs.

routing.domainStrategy определяет, как сопоставляются IP-правила маршрутизации с доменными запросами: AsIs (только доменные правила, без резолва), IPIfNonMatch (если домен не совпал с правилами – резолвить и проверить IP-правила), IPOnDemand (резолвить сразу при встрече IP-правила). Для работы IP-правил (например, geoip:*) при доменном запросе обычно требуется IPIfNonMatch.

Outbound Test URL

Поле	Подпись	Описание	По умолчанию
<code>outboundTestUrl</code>	URL для теста исходящего	URL для проверки связности при тестировании outbound. Подсказка: «URL для проверки подключения исходящего». Хранится отдельно от шаблона, под ключом <code>xrayOutboundTestUrl</code> .	<code>https://www.google.com/generate_204</code>

Значение проходит санитизацию. При самом тесте outbound оно дополнительно проверяется как публичный URL — это защита от SSRF: пользователь не может подsunуть произвольный (в т. ч. внутренний) URL через клиента, тестовый URL всегда берётся из серверной настройки. Пустое значение при сохранении/тесте заменяется на дефолтный `generate_204`.

Default Outbound (с 3.5.0)

Первый пункт вкладки «**Базовая маршрутизация**» — селектор **Default Outbound**: «Трафик, не совпавший ни с одним правилом маршрутизации, использует этот outbound (Xray берёт первый outbound в списке)». В списке — `direct`, `blocked` и все существующие теги; по умолчанию `direct`. Выбор **перемещает указанный outbound на первую позицию** шаблона; отсутствующие `direct` / `blocked` создаются на лету, а при переключении не удаляются.

Block BitTorrent

Поле	Подпись	Описание
<code>Torrent</code>	Заблокировать BitTorrent	Добавляет в <code>routing.rules</code> правило, отправляющее трафик с <code>protocol: ["bittorrent"]</code> на outbound <code>blocked</code> . В эталонном шаблоне это правило присутствует по умолчанию.

Ограничения соединения (Connection Limits)

Подсказка: «*Политики уровня соединения для пользователей уровня 0. Оставьте поле пустым, чтобы использовать значение Xray по умолчанию.*» Эти параметры пишутся в `policy.levels.0`.

Поле	Подпись	Описание	По умолчанию
<code>connIdle</code>	Тайм-аут простоя (секунд)	«Закрывает соединение после простоя в течение указанного числа секунд. Уменьшение значения быстрее освобождает память и файловые дескрипторы на нагруженных серверах (по умолчанию в Xray: 300).»	пусто → дефолт Xray 300
<code>bufferSize</code>	Размер буфера (КБ)	«Размер внутреннего буфера на соединение в КБ. Установите 0, чтобы минимизировать использование памяти на серверах с малым объёмом ОЗУ (значение Xray по умолчанию зависит от платформы).» Плейсхолдер: «авто».	пусто → зависит от платформы; 0 — минимизировать

Пример (`policy.levels.0`). Поля из этой группы пишутся в политику уровня 0. На нагруженном сервере с малым ОЗУ можно ускорить освобождение ресурсов так:

```

"policy": {
  "levels": {
    "0": {
      "connIdle": 120,
      "bufferSize": 0
    }
  }
}

```

Здесь соединение закрывается уже после 120 с простоя (вместо дефолтных 300), а `bufferSize: 0` минимизирует расход памяти на буферы. Оставленное пустым в форме поле просто не попадёт в JSON — и Xray применит своё значение по умолчанию.

11.3. Правила маршрутизации (routing)

Список правил `routing.rules`. **Порядок критичен** («Важен приоритет каждого правила!»): правила оцениваются сверху вниз, срабатывает первое совпавшее. Подсказка: «Перетащите для изменения порядка». Кнопки управления порядком: **Первый**, **Последний**, **Поднять вверх**, **Опустить вниз**.

Каждое правило имеет `type: "field"`. Кнопки: **Создать правило**, **Редактировать правило**. Подсказка к полям-спискам: «Элементы, разделённые запятыми».

На странице «Маршрутизация» кнопки «**Импорт правил**» и «**Экспорт правил**» собраны в выпадающее меню «ещё» (more) — как и на странице «Исходящие». Кнопка «**Экспорт правил**» не скачивает файл сразу, а открывает модальное окно с предпросмотром JSON и кнопками «**Копировать**» и «**Скачать**»: содержимое можно просмотреть перед сохранением. Аналогично работает экспорт исходящих на странице «Исходящие».

Route Tester (тестировщик маршрута)

На вкладке Routing есть подвкладка **Route Tester** — она спрашивает у работающего Xray, какой outbound обработал бы конкретное соединение, не отправляя реального трафика. Укажите домен или IP, порт, сеть (TCP/UDP) и, при необходимости, inbound и перехваченный протокол (`http / tls / quic / bittorrent`), затем нажмите **Test Route**. Решение берётся напрямую из живого движка маршрутизации.

В ответе показывается подобранный outbound, а при использовании балансировщика — ещё и тег балансировщика. Если ни одно правило не совпало, тестировщик сообщает, что трафик идёт в outbound по умолчанию (первый в списке `outbounds`). Это удобно для проверки порядка правил перед тем, как полагаться на них.

Включение и отключение отдельного правила

Отдельное правило маршрутизации можно временно **отключить** переключателем, не удаляя его. В таблице правил есть столбец «**Включить**» с переключателем (Switch), а в форме правила поле «**Включить**» — тоже переключатель. Отключённое правило не попадает в итоговую конфигурацию Xray, но сохраняется в шаблоне и его можно снова включить в любой момент.

Служебное правило статистики (`inboundTag: ["api"]` → `outboundTag: "api"`) отключить нельзя — его переключатель заблокирован, чтобы не сломать учёт трафика панели (см. [11.11](#)).

Поля формы правила

Поле формы	Подпись (RU)	JSON-поле	Описание
Источник	Источник	<code>source</code>	IP-адреса/подсети источника. Список через запятую.
Порт источника	Порт источника	<code>sourcePort</code>	Порт(ы) источника.
Пункт назначения	Пункт назначения	<code>domain + ip + port</code>	Целевые домены, IP и порты. Домены поддерживают префиксы <code>domain:</code> , <code>full:</code> , <code>regex:</code> , <code>keyword:</code> , а также <code>geosite:*</code> ; IP — <code>geoip:*</code> и CIDR.
Сеть	—	<code>network</code>	<code>tcp</code> , <code>udp</code> или <code>tcp,udp</code> . С 3.4.2 в конфиг пишется строчными (<code>tcp / udp</code>), а в таблице маршрутов отображается прописными (<code>TCP / UDP</code>); пусто — любая сеть.
Протокол	—	<code>protocol</code>	<code>http</code> , <code>tls</code> , <code>bittorrent</code> (определяется по sniffing).
Пользователь	Пользователь	<code>user</code>	Фильтр по e-mail/идентификатору пользователя.
Атрибуты / Значение	Атрибуты / Значение	<code>attrs</code>	Атрибуты HTTP-заголовков для сопоставления.
VLESS route	VLESS route	—	Маршрутизация по полю route для VLESS.
Теги входящих	Теги входящих	<code>inboundTag</code>	Один или несколько тегов inbound, к которым применяется правило (включая встроенные <code>api</code> , и DNS-тег из настроек DNS). В списках inbound отображается как «tag (remark)», если у inbound задано отдельное примечание, иначе — просто тег; в сохранённых правилах по-прежнему хранятся только теги.
Тег исходящего	Тег исходящего / Исходящее подключение	<code>outboundTag</code>	Куда направить совпавший трафик.
Тег балансировщика	Тег балансировщика / Балансировщик	<code>balancerTag</code>	Подсказка: « <i>Направляет трафик через один из настроенных балансировщиков нагрузки.</i> »

Взаимоисключение `outboundTag` и `balancerTag` : «*Невозможно одновременно использовать `balancerTag` и `outboundTag` . При одновременном использовании будет работать только `outboundTag` .*» В одном правиле задавайте либо тег исходящего, либо тег балансировщика.

Встроенные правила эталонного шаблона

В стандартном `config.json` секция `routing` содержит три правила (в этом порядке):

1. `inboundTag: ["api"] → outboundTag: "api"` — служебное правило для gRPC-API статистики панели.
2. `ip: ["geoip:private"] → outboundTag: "blocked"` — блокировка частных диапазонов.
3. `protocol: ["bittorrent"] → outboundTag: "blocked"` — блокировка BitTorrent.

Правило `api → api` всегда автоматически поднимается на позицию 0 при сохранении (см. [11.11](#)), чтобы запрос статистики не «съел» вышестоящее catch-all-правило.

Пример правила. Отправить весь трафик к российским сайтам и частным сетям напрямую (мимо прокси), а остальное — на балансировщик. Порядок важен: правило-«направить напрямую» должно стоять выше catch-all. В `routing.rules`:

```
{
  "type": "field",
  "domain": ["geosite:category-ru", "domain:example.ru"],
  "ip": ["geoip:ru", "geoip:private"],
  "outboundTag": "direct"
}
```

Чтобы IP-правила (`geoip:ru`) срабатывали и для доменных запросов, на верхнем уровне маршрутизации обычно нужно `routing.domainStrategy: "IPIfNonMatch"` (см. [11.2](#)).

Преднастроенные группы маршрутизации (Базовые соединения)

В режиме «Базовые соединения» панель помогает собрать типовые правила из готовых списков:

Группа	Поля	Подсказка
Блокировка по протоколам/сайтам	—	«Настройте, чтобы клиенты не имели доступа к определенным протоколам»
Блокировка по странам	Заблокированные IP-адреса, Заблокированные домены	«Эти параметры будут блокировать трафик в зависимости от страны назначения.»
Прямые соединения	Прямые IP-адреса, Прямые домены	«Прямое соединение означает, что определенный трафик не будет перенаправлен через другой сервер.»
Правила IPv4	—	«Эти параметры позволят клиентам маршрутизироваться к целевым доменам только через IPv4»
Правила WARP	—	«Эти опции будут направлять трафик в зависимости от конкретного пункта назначения через WARP.»
Маршрутизация NordVPN	—	«Эти опции будут направлять трафик в зависимости от конкретного пункта назначения через NordVPN.»

MTProto-inbound: маршрутизация Telegram-трафика через Xray

У MTProto-inbound есть переключатель «**Route through Xray**» (по умолчанию выключен) и необязательный выбор **Outbound**. При включении панель добавляет в конфиг Xray петлевой SOCKS-мост с тегом самого inbound, и mtg направляет Telegram-трафик через него. После этого исходящим Telegram-трафиком управляет маршрутизатор: его можно сопоставлять обычными правилами на вкладке Routing по тегу inbound либо принудительно направить в выбранный outbound или балансировщик через поле **Outbound**. Оставьте **Outbound** пустым, чтобы решение принимали правила маршрутизации.

11.4. Outbounds (исходящие подключения)

Список `outbounds`. Кнопки: **Создать исходящее подключение**, **Изменить исходящее подключение**.

Подсказка: «Изменение шаблона конфигурации, чтобы определить исходящие подключения для этого сервера».

В эталонном шаблоне два обязательных outbound-a:

- `protocol: "freedom", tag: "direct"` — прямой выход в интернет (с `domainStrategy: "AsIs"` и `finalRules: [{action: "allow"}]`);
- `protocol: "blackhole", tag: "blocked"` — «чёрная дыра» для заблокированного трафика.

Общие поля формы outbound

Поле	Подпись (RU)	Описание
Тег	Тег (подсказка: «Уникальный тег»)	Уникальный идентификатор outbound. Плейсхолдер: «уникальный-тег». Валидация: «Тег обязателен», «Тег уже используется другим исходящим».
Протокол	—	Тип исходящего (см. ниже).
Адрес / Порт	Адрес / Порт	Цель подключения. Адрес и порт обязательны.
Отправить через	Отправить через	Локальный IP-адрес исходящего интерфейса (<code>sendThrough</code>). Плейсхолдер: «локальный IP».
Target Strategy	Target Strategy	Новое в 3.5.0. Как резолвится домен назначения перед подключением. 11 значений: <code>AsIs</code> (по умолчанию, не резолвить), <code>UseIP</code> , <code>UseIPv4</code> , <code>UseIPv6</code> , <code>UseIPv6v4</code> , <code>UseIPv4v6</code> (резолв с fallback), <code>ForceIP</code> , <code>ForceIPv6v4</code> , <code>ForceIPv6</code> , <code>ForceIPv4v6</code> , <code>ForceIPv4</code> (требуют успешного резолва). Пусто = <code>AsIs</code> (в конфиг не пишется); для <code>freedom</code> значение читается с fallback на прежний <code>domainStrategy</code> (легаси-ключ по-прежнему записывается для старых ядер).
Dialer проху (цепочка)	—	Подсказка: «Подключайте это исходящее через другое исходящее (по тегу), чтобы построить цепочку прокси. Оставьте пустым для прямого подключения.» Плейсхолдер: «Выберите исходящее для цепочки». Реализуется через <code>streamSettings.sockopt.dialerProxy</code> .

Выпадающий список **Dialer Proxy** показывает не только локальные outbounds, но и теги outbounds из подписок — так цепочку можно построить и через выход, полученный по подписке. Из списка по-прежнему

исключены blackhole-outbound и сам редактируемый outbound. Оставьте поле пустым для прямого подключения.

Поддерживаемые протоколы outbound

Поддерживаемые формой протоколы:

- **freedom** — прямой выход. Поля `settings.domainStrategy`, `finalRules` (см. ниже), Happy Eyeballs. Тестированию не подлежит («*Outbound has no testable endpoint*»).
- **blackhole** — отбрасывает трафик. Поле **Тип ответа**. Не тестируется.
- **socks**, **http** — список `settings.servers[]` с `address / port`; поле **Пароль авторизации**. Для протокола **http** ниже полей **Username/Password** есть редактор **Headers** (Заголовки) — пары ключ/значение для CONNECT-заголовков, отправляемых вышестоящему HTTP-прокси. Эти заголовки сохраняются при повторном открытии и сохранении outbound (ранее терялись). Учтите: применяются только заголовки уровня настроек (`settings.headers`); заголовки на уровне отдельного сервера xray-core игнорирует.
- **vmess** — `settings.vnext[]` (`address / port`).
- **vless** — `settings.address / settings.port`.
- **trojan**, **shadowsocks** — `settings.servers[]`.
- **wireguard** — `settings.peers[]` с `endpoint`, плюс ключи (см. 11.8).
- **hysteria** — `settings.address / settings.port` (UDP-транспорт).

Для outbound типа **loopback** доступен блок **Sniffing** с теми же параметрами, что и у inbound: включение, **destOverride**, **Metadata Only**, **Route Only** и список **исключённых доменов**.

В маске **UDP** (FinalMask) для **Hysteria2** доступны дополнительные режимы. У маски **Salamander** есть селектор **Mode** со значениями **Salamander** и **Gecko**: режим Gecko добавляет случайное дополнение пакетов с полями **Min/Max** размера (`packetSize`, диапазон 1–2048, по умолчанию 512–1200) — это защищает от фингерпринтинга по длине пакетов. У маски **Realm** (UDP hole-punching) появился необязательный блок **TLS Config** с полями **Server Name** (SNI), **ALPN** (`h3 / h2 / http/1.1`), **Fingerprint** (uTLS) и переключателем **Allow Insecure**.

С 3.5.0 добавлен новый тип **TCP-маски** — **XMC (Minecraft)** (`xmc`): поток маскируется под протокол Minecraft. Поля: **Hostname** (необязательно; «адрес сервера, имитируемый в рукопожатии»), **Usernames** (необязательный список; «имена игроков, предлагаемые проберам; по умолчанию ядро использует Dream») и обязательный **Password** (16 символов, автогенерация с кнопкой 🔄; «пароль обфускации»). Важно: **сочетание TCP-Finalmask с REALITY отклоняется при сохранении** — оно роняло Xray-core при первом подключении (ошибка: «Finalmask is not supported with REALITY security...»; см. Xray-core#6453). UDP-маска с REALITY допустима; сохранённые ранее некорректные комбинации автоматически «лечатся» (маска отбрасывается) при сборке конфига.

Пример: цепочка через вышестоящий SOCKS. Outbound `upstream` ходит на внешний SOCKS5-прокси, а `chained` отправляет свой трафик через него (`dialerProxy`), образуя цепочку. В `outbounds`:

```
[
  {
    "tag": "upstream",
    "protocol": "socks",
    "settings": {
      "servers": [{ "address": "203.0.113.10", "port": 1080 }]
    }
  },
  {
    "tag": "chained",
    "protocol": "freedom",
    "streamSettings": {
      "sockopt": { "dialerProxy": "upstream" }
    }
  }
]
```

Теперь правило маршрутизации с `outboundTag: "chained"` будет выводить трафик в интернет через `upstream`.

Импорт outbound из share-ссылки

Outbound можно импортировать из share-ссылки (`vless://`, `vmess://` и т.п.). При импорте сохраняются и настройки мультимплексора **xmux** (XHTTP), переданные в блоке `extra=` ссылки: после импорта их значения подставляются в подформу **XMUX** созданного outbound.

Поля Mux (мультиплексирование)

Макс. параллелизм, Макс. соединений, Макс. повторных использований, Макс. запросов, Макс. секунд повторного использования, Период keep alive. Эти параметры настраивают mux/XUDP-поведение исходящего.

Sockopts (настройки сокета)

Группа **Sockopts**: Интервал keep alive, Mark (fwmark), Интерфейс, Только IPv6, Принимать проху protocol, Proxy protocol, TCP user timeout (мс), TCP keep-alive idle (с). Здесь же задаётся dialer-проху цепочки.

Freedom finalRules (переопределение блокировки частных IP)

Для freedom-outbound доступна группа **Финальные правила**:

Поле	Подпись	Описание
<code>overrideXrayPrivateIp</code>	Переопределить дефолтный блок частных IP в Xray	Снимает встроенный в Xray запрет на исходящие к приватным IP.
<code>action</code>	Действие	<code>allow</code> (как в эталонном шаблоне: <code>finalRules: [{action: "allow"}]</code>), <code>redirect</code> (Redirect) или прочие.
<code>blockDelay</code>	Задержка блока (мс)	Задержка перед отбрасыванием соединения.

Поле	Подпись	Описание
redirect / fragment	Redirect / Fragment	Действия перенаправления и фрагментации трафика.

Маска fragment: пофрагментные Lengths и Delays

В маске **fragment** (тип fragment в FinalMask, для TCP) одиночные поля Length и Delay заменены списками **Lengths** и **Delays**: для каждого сегмента можно задать отдельный диапазон длины (например 100–200) и задержки в миллисекундах (например 10–20 или 0). Строки списков можно добавлять и удалять; ранее сохранённые одиночные значения переносятся в массив из одного элемента автоматически.

Прочие поля формы

- **UDP over TCP** и **Версия UoT** — для shadowsocks-подобных протоколов.
- **Без gRPC-заголовка**, **Размер chunk Uplink** — параметры gRPC-транспорта.
- TLS/uTLS-поля: **Проверять имя peer**, **Pinned SHA256**, **Short ID**, **Vision testpre**, плейсхолдер «имя сервера».

Тестирование исходящих

Кнопки: **Тест**, **Тестировать все**. Состояния: **Тестирование соединения...**, **Тест успешен**, **Тест не пройден**, **Не удалось протестировать исходящее подключение**. Результат: **Результат теста**, задержка в миллисекундах.

Два режима (подсказка: «TCP: быстрый dial-only probe. HTTP: полный запрос через xray.»):

- **TCP** (mode=tcp) — простой dial до host:port, выполняется параллельно по всем эндпоинтам, ~таймаут 5 с. Проверяет только TCP-достижимость, не валидирует прокси-протокол. Для freedom / blackhole / tera blocked вернёт «Outbound has no testable endpoint».
- **HTTP** (mode=http или пусто) — поднимает временный экземпляр Xray, прогоняет реальный HTTP-запрос (probe URL = серверный outboundTestUrl), измеряет реальную задержку. Авторитетный, но дорогой режим: сериализуется глобальной блокировкой («Another outbound test is already running, please wait»). Тайм-аут одной попытки — 10 с, окно ожидания результата — 15 с (увеличены, чтобы здоровые outbounds на медленных или туннелированных каналах не помечались как «Failed»). При неудаче реальная причина (ошибка DNS, connection refused, истечение дедлайна, ошибка TLS и т. п.) пишется в лог панели/Xray, на который указывают общие сообщения о тайм-ауте.
- **Real delay (новое в 3.5.0)** — третий режим переключателя. Использует тот же временный экземпляр Xray, но сообщает **полное время «холодного» запроса, включая установление туннеля** — цифру, близкую к той, что показывают клиентские приложения. Подсказка переключателя: «TCP: быстрый dial-пробник. HTTP: полный запрос через xray. Real delay: полное время, включая установление соединения». Обычный HTTP-режим с 3.5.0 измеряется по «тёплому» (keep-alive) соединению, поэтому его цифры ниже и ближе к задержке одного запроса.

UDP-протоколы (wireguard, hysteria) и UDP-транспорты (kcp, quic, hysteria) **всегда** тестируются в HTTP-режиме, даже если запрошен TCP — голый UDP-dial не отличает «живой»

эндпоинт от «мёртвого». Для wireguard в тестовой конфигурации принудительно ставится `noKernelTun: true`.

Колонки Egress и Country (с 3.5.0)

После успешного **HTTP** или **Real delay** теста панель через тот же временный маршрут запрашивает метаданные Cloudflare trace и показывает в списке outbound две колонки: **Egress** — IP выхода (IPv4/IPv6, каждый скрыт за «глазом»; пока теста не было — прочерк с подсказкой «Выполните HTTP-тест, чтобы показать IP выхода и страну») и **Country** — флаг и название страны выхода; если выход идёт через Cloudflare WARP, добавляется оранжевая метка WARP. TCP-тест эти колонки не заполняет; на мобильных данные показываются в карточке outbound.

Батчевая проверка и разбивка по этапам

Тест и **Тестировать все** в HTTP-режиме поднимают один общий временный экземпляр Xray на пакет outbounds, для каждого создают петлевой SOCKS-inbound с правилом и параллельно отправляют через него реальный HTTP-запрос; **Тестировать все** проверяет outbounds пачками. **Тестировать все** проверяет и outbounds, полученные из подписок (таблица «из подписок», только для чтения) — их строки также подсвечиваются результатом теста. При этом outbounds `freedom` («direct») и `dns` не тестируются ни в одном режиме (это не прокси): кнопка теста у них недоступна, **Тестировать все** их пропускает, а серверная защита запрещает их HTTP-тест даже при прямом вызове API. Помимо успеха/ошибки попап-результат показывает HTTP-статус ответа и разбивку времени по этапам: **Proxy connect** (подключение к прокси), **TLS via outbound** (TLS через outbound) и **First byte** (время до первого байта) — это помогает понять, на каком шаге возникла задержка или сбой.

Статистика трафика outbounds

Панель ведёт счётчики трафика по тегам (`up / down / total`). Кнопка сброса сбрасывает счётчики для конкретного тега или для всех (`tag = "-alltags-"`). Поля **Информация об учетной записи** и **Статус исходящего подключения** показывают сводку.

11.5. Балансировщики (Balancers)

Список `routing.balancers`. Кнопки: **Создать балансировщик**, **Редактировать балансировщик**.

На вкладке Balancers есть столбцы живого состояния: **Live Target** показывает текущую активную цель балансировщика в работающем Xray, а **Override** позволяет вручную переопределить выбор цели (значение **Auto (strategy)** возвращает выбор по стратегии). Состояние обновляется отдельной кнопкой. Если балансировщик ещё не активен в работающем Xray, панель предложит сначала сохранить изменения или запустить Xray.

Поле	Подпись (RU)	Описание
Тег	Тег (подсказка: «Уникальный тег»)	Уникальный идентификатор. Плейсхолдер: «уникальный тег балансировщика». Валидация: «Тег обязателен», «Тег уже используется другим балансировщиком».

Поле	Подпись (RU)	Описание
Селекторы	Селекторы	Список тегов outbound (по подстроке), среди которых балансировщик выбирает выход. Должен быть выбран хотя бы один: « <i>Выберите хотя бы одно исходящее</i> ».
Fallback	Fallback	Резервный тег, если ни один селектор не подошёл. С 3.5.0 можно выбрать и другой балансировщик (в списке такие варианты помечены синей меткой «Balancer», подсказка «Выберите другой балансировщик как fallback»). Xray нативно этого не умеет, поэтому панель сама строит скрытый loopback-outbound <code>_bl_<цель></code> и правило маршрутизации (путь: балансировщик → loopback → сервер → целевой балансировщик → outbound; предупреждение о лишнем «хопе» показывается в форме). Есть защита от циклов (ошибка «нельзя выбрать — возникла бы циклическая зависимость», вариант в списке блокируется с подсказкой пути цикла) и от удаления используемого балансировщика («Нельзя удалить — используется как fallback у: ...»). Префикс тега <code>_bl_</code> зарезервирован (в тегах запрещён); служебные <code>_bl_</code> -объекты скрыты из списков Outbounds/Routing, а колонки Fallback/Live Target/Override показывают имя целевого балансировщика, а не loopback-тег.
Стратегия	Стратегия	Алгоритм выбора (см. ниже).

Стратегия и параметры наблюдения

Стратегия (`strategy.type`) определяет, как балансировщик выбирает outbound из селекторов. Значения Xray-core: `random` (случайно), `roundRobin` (по кругу), `leastPing` (минимальная задержка по результатам observatory), `leastLoad` (минимальная нагрузка). Для `leastLoad / leastPing` используются параметры из `strategy.settings` :

Поле	Подпись	Описание
<code>expected</code>	Ожидаемое	Плейшолдер: « <i>оптимальное число узлов</i> » — целевое число живых узлов.
<code>maxRtt</code>	Макс. RTT	Верхняя граница допустимого RTT при отборе кандидатов.
<code>tolerance</code>	Допуск	Допуск (tolerance) при сравнении задержек/нагрузки.
<code>baselines</code>	Baselines	Пороговые значения задержки для группировки узлов.
<code>costs</code>	Costs	Весовые коэффициенты (cost) для отдельных тегов.

Примеры стратегий. Блок `strategy` живёт внутри балансировщика (в JSON — рядом с `tag` и `selector`):

```
"strategy": { "type": "random" }      // случайный выбор из селекторов
"strategy": { "type": "roundRobin" }  // по кругу, поочерёдно
"strategy": { "type": "leastPing" }   // минимальная задержка (нужен наблюдатель)
```

Для `leastLoad` параметры задаются в `settings` :

```

"strategy": {
  "type": "leastLoad",
  "settings": {
    "expected": 2,
    "maxRTT": "1s",
    "tolerance": 0.05,
    "baselines": ["500ms", "1s", "2s"],
    "costs": [
      { "regex": false, "match": "proxy-premium", "value": 0.1 },
      { "regex": true, "match": "^proxy-cheap-.+$", "value": 5 }
    ]
  }
}

```

Как это отрабатывает (на примере). Пусть наблюдатель намерил задержки по выходам: **A = 250 мс** , **B = 280 мс** , **C = 700 мс** , **D = 1500 мс** . С настройками выше выбор идёт так:

1. **maxRTT: "1s"** — выходы с задержкой выше 1 с отбрасываются: **D** (1500 мс) выбывает. Остаются **A** , **B** , **C** .
2. **baselines + expected** — выходы группируются по порогам задержки, и берётся **наименьший** порог, в который попадает не менее **expected** выходов. Порог **500ms** уже содержит **A** и **B** — это 2 (= **expected**), поэтому выбирается группа { **A** , **B** }. **C** (700 мс) в выбор не попадает, пока быстрых хватает (он — «горячий резерв»).
3. **tolerance: 0.05** — внутри выбранной группы выходы, чьи задержки отличаются не более чем на 5%, считаются равноценными, и нагрузка делится между ними поровну. **A** (250) и **B** (280) отличаются на ~12% (> 5%), поэтому при прочих равных предпочтение у более быстрого **A** ; будь разница в пределах 5% — трафик шёл бы и через **A** , и через **B** .
4. **costs** — до сравнения корректируют «стоимость» отдельных выходов: меньшее **value** делает выход привлекательнее, большее — наоборот. В примере **proxy-premium** получает **0.1** (становится «дешевле» и выбирается охотнее), а все **proxy-cheap-*** (по регулярному выражению, **regex: true**) — **5** (становятся «дороже» и используются в последнюю очередь). Так можно мягко приоритезировать выходы, не исключая их жёстко.

Итог: трафик пойдёт в основном через **A** (при близких задержках — поровну с **B**), **C** останется резервом, **D** исключён, пока его RTT не опустится ниже **maxRTT** .

Наблюдатель: **observatory** и **burstObservatory** (замеры для **leastPing** / **leastLoad**)

Стратегии **leastPing** и **leastLoad** сами ничего не измеряют — им нужны данные о задержке и доступности каждого outbound. Их собирает **наблюдатель** (**observatory**): он периодически «пингует» каждый отслеживаемый outbound и запоминает время отклика и доступность. Те же данные показываются на вкладке «**Обсерватория**» (статусы **Активен** / **Недоступен**, «**Последняя активность**», «**Последняя попытка**»).

С версии **3.4.2** страница «Балансировщики» разделена на вкладки «**Настройки балансировщика**» (таблица и добавление) и «**Observatory**», а наблюдатель редактируется **структурной формой** (прежний сырой JSON-редактор убран). Панель сама создаёт и удаляет наблюдателей по необходимости: обычный **observatory** — для стратегии **Least Ping**, расширенный **burstObservatory** — для **Least Load**, а также для **Random/**

Round-robin с заданным fallbackTag (этого требует xray-core; при очистке последнего fallbackTag наблюдатель убирается). Создание или удаление наблюдателя требует полного перезапуска Xray.

Доступны два варианта:

- **observatory** — простой: subjectSelector + probeURL + probeInterval.
- **burstObservatory** — расширенный, с тонкой настройкой пинга через pingConfig; удобен для нескольких выходов.

Пример блока burstObservatory:

```
{
  "subjectSelector": ["WS-SE", "WS-FR", "WS-PL"],
  "pingConfig": {
    "destination": "https://www.google.com/generate_204",
    "interval": "1m",
    "connectivity": "http://connectivitycheck.platform.hicloud.com/generate_204",
    "timeout": "5s",
    "sampling": 2
  }
}
```

Назначение полей:

Поле	Что задаёт
subjectSelector	Список префиксов тегов outbound для наблюдения. Xray берёт все outbound, чьи теги начинаются с указанных строк. В примере наблюдаются выходы WS-SE..., WS-FR..., WS-PL... . Эти теги должны совпадать с тем, что выбрано в Селекторах балансировщика.
pingConfig.destination	URL, запрашиваемый через каждый outbound для замера задержки. Берут «лёгкую» страницу с ответом 204 без тела — например https://www.google.com/generate_204 . Время до ответа и есть измеренная задержка.
pingConfig.interval	Как часто пинговать каждый outbound. Строка длительности: "1m" — раз в минуту, также "30s", "5m" и т.п. Чаще — свежее данные, но больше фонового трафика.
pingConfig.connectivity	(необязательно) URL проверки базовой связности самого сервера. Если он недоступен — значит проблема в сети сервера, и наблюдатель не помечает outbound как недоступный (защита от ложных срабатываний при локальном сбое). Обычно тоже endpoint с ответом 204 .
pingConfig.timeout	Сколько ждать ответ на один пинг, прежде чем считать попытку неудачной (например "5s").
pingConfig.sampling	Сколько последних замеров хранить и усреднять по каждому outbound. 2 — учитывать два последних пинга (сглаживает случайные скачки).
pingConfig.httpMethod	Новое в 3.4.2. HTTP-метод проб: HEAD (по умолчанию) или GET .

На вкладке **«Observatory»** те же параметры задаются формой (а не JSON). Для обычного observatory это **Watched Outbounds** (subjectSelector, только чтение), **Probe URL** (probeURL, по умолчанию https://www.google.com/generate_204), **Probe Interval** (probeInterval, 1m) и **Enable Concurrency**

(`enableConcurrency` , по умолчанию включено). Для `burstObservatory` — перечисленные выше поля плюс новое «**HTTP-метод**». Если у балансировщика есть оба наблюдателя, между формами Observatory и Burst переключает сегментный переключатель.

Как всё связать:

1. На вкладке «**Observatory**» настройте параметры наблюдателя (он создаётся автоматически под выбранную стратегию; `subjectSelector` следует за селекторами балансировщика).
2. Создайте балансировщик: **Стратегия** = `leastPing` , в **Селекторах** укажите те же теги outbound (`WS-SE` , `WS-FR` , `WS-PL`).
3. Направьте на него трафик правилом маршрутизации (поле **Тег балансировщика**, см. 11.3).
4. Перезапустите Xray. На вкладке «**Обсерватория**» появятся статусы выходов, а балансировщик начнёт выбирать самый быстрый из живых.

В одном правиле нельзя одновременно задать `balancerTag` и `outboundTag` — работает только `outboundTag` .

Что происходит при удалении outbound или балансировщика (с 3.4.2)

При удалении outbound или балансировщика панель в том же действии **вычищает ссылки на него в маршрутизации** и показывает **предпросмотр последствий** в диалоге подтверждения (заголовок «При удалении также обновится ваша маршрутизация:», далее по каждому правилу — «удалено (не осталось назначения)» или «оставлено (теперь использует ...)», и «Балансировщик ... удалён (не осталось целей)»). Так предотвращается «висячая» ссылка (`balancerTag` / `outboundTag` / `dialerProxy`), из-за которой раньше xray-core мог не запуститься при следующем перезапуске. Удаление балансировщика: правила сохраняют свой `outboundTag` , иначе удаляются. Удаление outbound: его тег убирается из селекторов и `fallbackTag` балансировщиков, очищается `dialerProxy` у других outbound, опустевшие балансировщики удаляются вместе с теперь-беспредметными правилами, наблюдатели пересобираются.

11.6. DNS

Раздел `dns` . Включение: **Включить DNS** (подсказка: «Включить встроенный DNS-сервер»).

С 3.5.0 DNS-серверы на **приватных IP** (например, свой AdGuard/Pi-hole в той же docker-сети) работают «из коробки»: панель автоматически вставляет и поддерживает управляемое allow-правило (`direct`, ограниченное портом DNS-сервера) **перед** блокирующим правилом `geoip:private` и синхронизирует его при изменении `dns.servers` . Раньше собственные DNS-запросы Xray молча блокировались этим правилом, добавляя ~4 с задержки на каждый новый домен; ручные правила больше не нужны (созданные вручную не затрагиваются).

Общие параметры DNS

Поле	Подпись (RU)	JSON	Описание / подсказка
<code>tag</code>	Название тега DNS	<code>dns.tag</code>	«Этот тег будет доступен как входящий тег в правилах маршрутизации.» Позволяет маршрутизировать сами DNS-запросы через <code>inboundTag</code> .
<code>clientIp</code>	IP клиента	<code>dns.clientIp</code>	«Используется для уведомления сервера о указанном местоположении IP во время DNS-запросов» (EDNS Client Subnet).
<code>strategy</code>	Стратегия запроса	<code>dns.queryStrategy</code>	«Общая стратегия разрешения доменных имен». Значения: <code>UseIP</code> , <code>UseIPv4</code> , <code>UseIPv6</code> .
<code>disableCache</code>	Отключить кэш	<code>dns.disableCache</code>	«Отключает кэширование DNS».
<code>disableFallback</code>	Отключить резервный DNS	<code>dns.disableFallback</code>	«Отключает резервные DNS-запросы».
<code>disableFallbackIfMatch</code>	Отключить резервный DNS при совпадении	<code>dns.disableFallbackIfMatch</code>	«Отключает резервные DNS-запросы при совпадении списка доменов DNS-сервера».
<code>enableParallelQuery</code>	Включить параллельные запросы	—	«Включить параллельные DNS-запросы к нескольким серверам для более быстрого разрешения».
<code>useSystemHosts</code>	Использовать системные Hosts	<code>dns.useSystemHosts</code>	«Использовать файл <code>hosts</code> из установленной системы».

Пример блока `dns`. Запросы к доменам Google резолвятся через DoH-сервер Cloudflare, всё остальное — через `1.1.1.1`; на запросы Google ожидаются только не-приватные IP. На верхнем уровне конфига:

```
"dns": {
  "tag": "dns-inbound",
  "queryStrategy": "UseIPv4",
  "servers": [
    {
      "address": "https://cloudflare-dns.com/dns-query",
      "domains": ["geosite:google"],
      "expectIPs": ["geoip:private"]
    },
    "1.1.1.1"
  ]
}
```

Строка-сервер ("1.1.1.1") без полей — это сервер по умолчанию для всех прочих доменов. Тег `dns-inbound` затем можно использовать как `inboundTag` в правилах маршрутизации, чтобы направлять сами DNS-запросы через нужный outbound.

Кэш устаревших записей

Поле	Подпись	Описание
<code>serveStale</code>	Использовать устаревшие	«Возвращать устаревшие результаты из кэша во время обновления в фоне».
<code>serveExpiredTTL</code>	TTL устаревших	«Срок действия (секунды) устаревших записей кэша; 0 = бессрочно».

DNS-серверы (список `dns.servers`)

Кнопки: **Создать DNS**, **Редактировать DNS**, **Удалить все** (подтверждение: «Все DNS-серверы будут удалены из списка. Это действие нельзя отменить.»). Шаблоны: **Использовать шаблон**, окно **Шаблоны DNS**, включая пресет **Семейный**.

При нажатии **Редактировать DNS** на записи DNS-сервера (как и на записи Fake DNS) окно редактирования подставляет сохранённые значения сервера, а не значения по умолчанию.

Поля DNS-сервера:

Поле	Подпись	Описание
<code>address</code>	—	Адрес DNS (IP, DoH-URL, <code>localhost</code> , <code>fakedns</code> и т.п.).
<code>domains</code>	Домены	Список доменов, для которых используется этот сервер.
<code>expectIPs</code>	Ожидаемые IP	Принимать ответ только если IP попадает в список.
<code>unexpectIPs</code>	Неожидаемые IP	Отбрасывать ответы с указанными IP.
<code>skipFallback</code>	Пропустить Fallback	Не использовать этот сервер как fallback.
<code>finalQuery</code>	Финальный запрос	Помечает сервер как финальный в цепочке.
<code>timeoutMs</code>	Тайм-аут (мс)	Таймаут запроса к серверу.

Hosts (статические записи)

Группа **Hosts** (`dns.hosts`). Кнопка **Добавить Host**; пустое состояние **Host не определены**. Поля: домен (плейсхолдер: «Домен (напр. `domain:example.com`)») и значения (плейсхолдер: «IP или домен — введите и нажмите Enter»).

Логи DNS

См. 11.10: флаг **Логи DNS** (`dnsLog`) в секции логирования.

11.7. Fake DNS

Раздел `fakedns`. Кнопки: **Создать Fake DNS**, **Редактировать Fake DNS**.

Поле	Подпись	Описание
<code>ipPool</code>	Подсеть пула IP	CIDR-диапазон, из которого выдаются фиктивные IP (например <code>198.18.0.0/15</code>).
<code>poolSize</code>	Размер пула	Сколько адресов держать в кольцевом пуле.

Fake DNS используется в связке с sniffing на inbound: ядро выдаёт клиенту фиктивный IP, запоминает соответствие домен[?]IP и восстанавливает домен при маршрутизации. Чтобы Fake DNS работал, DNS-сервер с адресом `fakedns` должен быть добавлен в список DNS-серверов.

Пример: связка Fake DNS + DNS-сервер. Сначала задаём пул фиктивных адресов, затем добавляем DNS-сервер `fakedns`, чтобы доменные запросы получали IP из этого пула:

```
"fakedns": [
  { "ipPool": "198.18.0.0/15", "poolSize": 65535 }
],
"dns": {
  "servers": [
    { "address": "fakedns", "domains": ["geosite:geolocation-!cn"] },
    "1.1.1.1"
  ]
}
```

Дополнительно на inbound нужно включить sniffing с `destOverride: ["fakedns"]`, иначе ядру неоткуда взять реальный домен для восстановления.

11.8. WireGuard / WARP / NordVPN

Поля WireGuard (`wireguard`)

Поле	Подпись	Описание
<code>secretKey</code>	Секретный ключ	Приватный ключ локального интерфейса.
<code>publicKey</code>	Публичный ключ	Публичный ключ реер.

Поле	Подпись	Описание
psk	Общий ключ	PreShared Key (опционально).
allowedIPs	Разрешенные IP-адреса	Диапазоны, маршрутизируемые в туннель.
endpoint	Конечная точка	host:port peer.
domainStrategy	Стратегия домена	Стратегия резолва для WireGuard-outbound.

Cloudflare WARP (warp)

Интеграция использует API `https://api.cloudflareclient.com/v0a4005` (client-version `a-6.30-3596`). Действия контроллера (`/xray/warp/:action`): `config`, `reg`, `license`, `data`, `del`.

Пошагово:

1. **Создать аккаунт WARP** → `reg` : панель генерирует/принимает приватный (`privateKey`) и публичный (`publicKey`) ключи, регистрирует устройство в Cloudflare и сохраняет `access_token`, `device_id`, `license_key`, `private_key` (а также `client_id`) в настройке `warp`.
2. **Лицензионный ключ WARP / WARP+** → `license` : установка 26-символьного ключа WARP+ (плейсхолдер: «26-символьный ключ WARP+»). При ошибке: «Не удалось установить лицензию WARP.» Если конфиг ещё не получен: «Сначала получите WARP-конфиг.»
3. **Информация об аккаунте: Имя устройства, Модель устройства, Устройство включено, Тип аккаунта, Роль, WARP+ data, Квота, Использование.**
4. **Добавить исходящий** — создаёт WireGuard-outbound с полученными ключами и эндпоинтом Cloudflare.
5. **Удалить аккаунт** → `del` : очищает сохранённые данные WARP.

NordVPN (nord / nordvpn)

Интеграция использует NordLynx (= WireGuard). Действия контроллера (`/xray/nord/:action`): `countries`, `servers`, `reg`, `setKey`, `data`, `del`.

Пошагово:

1. **Токен доступа** → `reg` : панель запрашивает у `api.nordvpn.com` учётные данные NordLynx и извлекает `nordlynx_private_key`. Сохраняет `private_key` и `token` в настройке `nord`. Альтернатива — `setKey` : ввести **Приватный ключ** напрямую (не может быть пустым).
2. **Страна** → `countries` загружает список стран; **Город** (либо **Все города**).
3. **Сервер** → `servers` загружает серверы выбранной страны (`countryId` валидируется как число — защита от инъекций). Фильтр: показываются только серверы с **Нагрузкой** > 7%. Если серверов нет: «Серверов для выбранной страны не найдено». Если у сервера нет публичного ключа NordLynx: «Выбранный сервер не сообщает публичный ключ NordLynx.»
4. Создание/обновление исходящего: тосты «Исходящий NordVPN добавлен» / «Исходящий NordVPN обновлён».

Приоритет IPv4 и userspace TUN

Генерируемые мастерами WARP и NordVPN WireGuard-outbounds используют `domainStrategy: "ForceIPv4v6"` (приоритет IPv4 с откатом на IPv6 на v6-only хостах) вместо `ForceIP` — это устраняет «зависание» рукопожатия на хостах с наполовину настроенным IPv6, когда выбирается AAAA-запись эндпоинта Cloudflare. Кроме того, для них включён userspace TUN (`noKernelTun: true`) вместо kernel TUN: последний требует прав и fwmark-маршрутизации и тихо падает на многих VPS, тогда как встроенная проверка соединения панели всегда тестирует через userspace TUN — теперь реальный трафик и проверка идут одним путём. Изменение действует только для вновь добавляемых или сброшенных outbounds; уже сохранённые шаблоны сохраняют свои настройки.

11.9. Reverse-прокси и TUN

Reverse (реверс-прокси)

Раздел `reverse` конфигурации Xray. В форме outbound есть переключатель на тип **Реверс-прокси**. Кнопки: **Создать реверс-прокси**, **Редактировать реверс-прокси**.

Поле	Подпись	Описание
Тип	Тип	Bridge или Portal — две роли реверс-прокси Xray.
Домен	Домен	Служебный домен-метка для пары bridge [?] portal.
Тег / Соединение	Тег / Соединение	Теги для связки bridge и portal.
Reverse Tag	Тег реверс-прокси	Подсказка: «Тег исходящего подключения для простого реверс-прокси VLESS. Оставьте пустым для отключения.» Плейсхолдер: «тег исходящего (пусто = отключено)». Реализует упрощённый VLESS reverse.

В форме outbound присутствуют также поля обратного потока: **Обратный sniffing**, **Воркеры**, **Зарезервировано**, **Мин. интервал загрузки (мс)**, **Макс. размер загрузки (байт)**.

TUN (tun)

Поле	Подпись	Описание	По умолчанию
name	—	«Имя интерфейса TUN.»	xray0
mtu	—	«Максимальная единица передачи. Максимальный размер пакетов данных.»	1500
<code>userLevel</code>	Уровень пользователя	«Все соединения, установленные через этот входящий поток, будут использовать этот уровень пользователя.»	0

11.10. Логи и статистика (Stats, metrics)

Лог (log)

Подсказка: «Логи могут замедлять работу сервера. Включайте только нужные вам виды логов при необходимости!» Секция `log` эталонного шаблона: `access: "none", error: "", loglevel: "warning", dnsLog: false, maskAddress: ""`.

Поле	Подпись	JSON	Описание	По умолчанию
<code>logLevel</code>	Уровень логов	<code>loglevel</code>	«Уровень журнала для журналов ошибок...» Значения: <code>debug</code> , <code>info</code> , <code>warning</code> , <code>error</code> , <code>none</code> .	<code>warning</code>
<code>accessLog</code>	Логи доступа	<code>access</code>	«Путь к файлу журнала доступа. Специальное значение «none» отключает логи доступа.»	<code>none</code>
<code>errorLog</code>	Логи ошибок	<code>error</code>	«Путь к файлу логов ошибок. Специальное значение «none» отключает логи ошибок.»	<code>""</code> (по умолчанию)
<code>dnsLog</code>	Логи DNS	<code>dnsLog</code>	«Включить логи запросов DNS»	<code>false</code>
<code>maskAddress</code>	Маскировка адреса	<code>maskAddress</code>	«При активации реальный IP-адрес заменяется на маскировочный в логах.»	<code>""</code> (выкл.)

Статистика (stats / policy)

Группа **Статистика**. Включает счётчики в `policy.system` и `policy.levels`. В эталонном шаблоне: `statsInboundUplink: true, statsInboundDownlink: true, statsOutboundUplink: false, statsOutboundDownlink: false`; для уровня 0 — `statsUserUplink: true, statsUserDownlink: true`.

Поле	Подпись	Описание	По умолчанию
<code>statsInboundUplink</code>	Статистика входящего аплинка	«Включает сбор статистики для исходящего трафика всех входящих прокси.»	<code>true</code>
<code>statsInboundDownlink</code>	Статистика входящего даунлинка	«Включает сбор статистики для входящего трафика всех входящих прокси.»	<code>true</code>
<code>statsOutboundUplink</code>	Статистика исходящего аплинка	«Включает сбор статистики для исходящего трафика всех исходящих прокси.»	<code>false</code>
<code>statsOutboundDownlink</code>	Статистика исходящего даунлинка	«Включает сбор статистики для входящего трафика всех исходящих прокси.»	<code>false</code>

Статистика по клиентам и inbounds (uplink/downlink) — основа отображения трафика в дашборде и у клиентов; отключать её не рекомендуется. Outbound-статистика по умолчанию выключена и нужна, только если вы отслеживаете трафик по тегам исходящих.

Metrics

В эталонном шаблоне присутствует секция `metrics` (`listen: "127.0.0.1:11111"`, `tag: "metrics_out"`) и соответствующий API `metrics_out`. Панель использует этот listener для сбора метрик и снимков observatory: она парсит `metrics.listen` из шаблона, опрашивает `/debug/vars` и агрегирует историю задержек по тегам. Если вы меняете адрес/порт `metrics.listen`, панель будет обращаться к новому адресу; удаление секции `metrics` отключит сбор графиков observatory.

Тестирование outbound в HTTP-режиме поднимает **отдельный временный** экземпляр Xray со своим `metrics-listener` на случайном порту — это не тот же listener, что в основном конфиге.

11.11. Сохранение, перезапуск и автоматические преобразования

Кнопки

Кнопка	Действие
Сохранить	POST <code>/xray/update</code> : валидирует и сохраняет шаблон + <code>outboundTestUrl</code> .
Перезапуск Xray	Перезагружает сервис с сохранённой конфигурацией. Подтверждение: «Перезапустить xray?» / «Перезагружает сервис xray с сохранённой конфигурацией.»

Тосты: успех — «Xray успешно перезапущен», «Xray успешно остановлен»; ошибки — «Произошла ошибка при перезапуске Xray.», «Произошла ошибка при остановке Xray.» Окно **Вывод перезапуска Xray** показывает диагностический вывод ядра.

Горячее применение изменений (без полного перезапуска)

Изменения inbounds, outbounds и правил маршрутизации применяются «вживую»: при нажатии **Сохранить** панель вычисляет разницу между старым и новым конфигом и применяет только изменившиеся части через gRPC-API Xray (HandlerService/RoutingService), не перезапуская процесс. Полный перезапуск выполняется автоматически лишь тогда, когда меняются секции без API горячей перезагрузки (`log`, `dns`, `policy`, `observatory` и т. п.). Поэтому на странице Xray отдельная кнопка «Перезапустить» не нужна — **Сохранить** само применяет изменения. Перезапуск ядра при необходимости по-прежнему выполняется автоматически (см. также авто-перезагрузку при обновлении подписок и ротации WARP).

Восстановление шаблона по умолчанию

Эндпоинт `GET /xray/getDefaultJsonConfig` возвращает эталонный шаблон (`config.json`, встроенный в бинарник). Им можно сбросить конфигурацию к заводской.

Автоматические преобразования при сохранении

При сохранении настроек Xray панель выполняет (в этом порядке):

1. **Снятие обёрток** — снимает обёртки вида `{ "xraySetting": <конфиг>, "inboundTags": ..., "outboundTestUrl": ... }`, если они случайно попали в значение (иначе слои накапливались бы при каждом сохранении). Снимается до 8 слоёв.
2. **Проверка конфигурации** — JSON разбирается в структуру конфигурации Xray; при ошибке — отказ с «*xray template config invalid*».
3. **Гарантия правила статистики** — правило `inboundTag: ["api"] → outboundTag: "api"` принудительно поднимается на позицию 0 в `routing.rules` (или добавляется, если отсутствует). Это гарантирует, что запрос gRPC-статистики панели не будет перехвачен вышестоящим catch-all-правилом (иначе клиенты могут отображаться офлайн с нулевым трафиком при работающем прокси).

Из-за п. 3 не пытайтесь убрать или передвинуть правило `api → api` — панель всё равно вернёт его на место при следующем сохранении. Это служебная инфраструктура статистики, а не пользовательский маршрут.

11.12. Outbound из подписки (с авто-обновлением)

Начиная с версии 3.3.0 панель умеет импортировать `outbound` 'ы напрямую из URL подписки — того же формата, что отдают провайдеры VPN для клиентских приложений. Подписки регулярно пересчитываются в фоне, поэтому набор `outbound` 'ов на сервере поддерживается в актуальном состоянии без ручного редактирования шаблона конфигурации. С 3.5.0 у `ss://` -ссылок (SIP002) с `query`-параметрами (`?plugin=`, `?type=`) или хвостовым `/` порт разбирается корректно (раньше он молча становился `0`); ссылка с действительно некорректным портом пропускается, а не импортируется сломанной.

В русском интерфейсе раздел называется «Подписки исходящих», описание: «Импорт исходящих из удалённых URL подписок (vmess/vless/trojan/ss/...). Теги остаются неизменными для использования в балансировщиках и правилах маршрутизации. Обновление выполняется автоматически.» Раздел расположен на странице Xray, над панелью настройки `outbound` 'ов.

Как это устроено

Подписки хранятся отдельно от шаблона конфигурации Xray. Шаблон **никогда не перезаписывается**: полученные из подписок `outbound` 'ы добавляются в итоговую конфигурацию на лету при каждой генерации конфига Xray.

Добавление подписки

В форме «Добавить подписку» доступны следующие поля:

Поле	Ключ	По умолчанию	Назначение
URL подписки	<code>url</code>	— (обязательно)	Адрес подписки. Плейсхолдер: «https://... (список ссылок в base64)». Принимается только HTTP(S); адрес проверяется на безопасность.
Примечание	<code>remark</code>	пусто	Произвольная метка (плейсхолдер «напр. узлы НК»).
Префикс тега	<code>tagPrefix</code>	<code>subN-</code>	Префикс, с которого начинаются теги импортируемых <code>outbound</code> 'ов. Если оставить пустым, панель сама подберёт наименьший свободный номер вида <code>sub1-</code> , <code>sub2-</code> и т.д.
Интервал обновления	<code>updateInterval</code>	600 секунд (10 минут)	Как часто подписка перечитывается. В UI задаётся в часах/минутах.
Включено	<code>enabled</code>	да (<code>true</code>)	Только включённые подписки попадают в конфиг и обновляются автоматически.
Разрешить приватные адреса	<code>allowPrivate</code>	нет (<code>false</code>)	Разрешает URL на localhost, LAN и приватные IP. По умолчанию выключено в целях защиты от SSRF — включайте только для доверенного локального источника.
Перед ручными исходящими	<code>prepend</code>	нет (<code>false</code>)	Если включено, <code>outbound</code> 'ы этой подписки ставятся перед ручными <code>outbound</code> 'ами шаблона, и один из них может стать <code>outbound</code> 'ом по умолчанию. Иначе они добавляются после .

Кнопка «**Предпросмотр**» (`POST /outbound-subs/parse`) позволяет до сохранения скачать и распарсить URL и увидеть, какие `outbound` 'ы и теги получатся; ничего в базу при этом не пишется. Если по URL ничего не распознано, выводится «По этому URL исходящих не найдено.»

Порядок нескольких подписок в общем списке `outbound` 'ов задаётся приоритетом (`priority`) и меняется стрелками вверх/вниз (`POST /outbound-subs/:id/move`).

Какие форматы подписки принимаются

Тело ответа по URL обрабатывается так:

- Содержимое сначала пробуется как **base64** (стандартный и URL-safe варианты, с автодополнением паддинга и удалением пробелов/переводов строк). Если это base64 — оно декодируется; иначе берётся как есть.
- Затем тело разбивается на строки. Каждая непустая строка, не начинающаяся с `#` , разбирается как ссылка. Нераспознанные строки (комментарии, неподдерживаемые протоколы) молча пропускаются.
- Поддерживаемые схемы ссылок: `vmess://` , `vless://` , `trojan://` , `ss://` (Shadowsocks), `hysteria2://` / `hy2://` , `wireguard://` / `wg://` .

То есть подходит обычная подписка вида «base64-кодированный список ссылок», как у большинства провайдеров.

Стабильные теги

Каждой ссылке вычисляется устойчивая «идентичность» (ядро URI без фрагмента-примечания; для vmess — внутренний JSON без поля `ps`). Соответствие «идентичность → тег» сохраняется, и при следующем обновлении тот же сервер получает тот же тег, даже если изменились примечание или второстепенные параметры. Это специально сделано, чтобы балансировщики и правила маршрутизации продолжали работать после обновлений:

- Точный тег в балансировщике/правиле продолжит указывать на тот же сервер.
- Префиксный/wildcard-селектор (например, `hk-*`) автоматически подхватит новые серверы, которые подписка вернёт позже — это рекомендуемый способ «подписаться на пул».
- Если сервер исчезает из подписки, его тег просто пропадает из итогового массива `outbound` 'ов; при наличии `fallbackTag` у балансировщика Xray использует его.
- Если провайдер сменил UUID/хост/учётные данные сервера, идентичность меняется — это считается новым `outbound` 'ом с новым тегом.

Внутри одной выгрузки теги дедуплицируются суффиксом `-N`. Теги из подписок сохраняют не-ASCII символы (например, кириллицу) и остаются читаемыми: буквы и цифры Unicode сохраняются в slug, а пунктуация заменяется на дефис — теги из кириллических имён больше не сводятся к одним цифрам.

Как работает авто-обновление

- Фоновая задача обновления подписок запускается по расписанию **каждые 5 минут**.
- На каждом запуске она перебирает все включённые подписки и обновляет только те, у которых истёк собственный интервал: подписка обновляется, если ещё ни разу не обновлялась либо если с последнего обновления прошло не меньше её `updateInterval`. Таким образом задача проверяет подписки часто, но каждая конкретная подписка перечитывается не чаще своего `updateInterval` (по умолчанию 10 минут). В UI это отражено соответствующей подсказкой.
- Обновление: URL заново проверяется на безопасность как публичный (приватные адреса блокируются, если у подписки не выставлен `allowPrivate`), запрос идёт через прокси-клиент панели с заголовком `User-Agent: 3x-ui-outbound-sub/1.0`. Цепочка редиректов ограничена 10 переходами, и каждый хоп тоже проверяется на приватность (защита от SSRF). Ожидается HTTP 200; иначе фиксируется ошибка.
- После успешного парсинга результат сохраняется, проставляется время последнего обновления, очищается ошибка. При ошибке её текст виден в UI как «Последняя ошибка», а ранее полученные `outbound` 'ы остаются в силе.
- Если хотя бы одна подписка реально обновилась, задача помечает Xray на перезапуск и шлёт UI-инвалидацию, чтобы интерфейс подтянул новые `outbound` 'ы. Фактическая перезагрузка Xray происходит на ближайшем 30-секундном цикле менеджера.

Ручное обновление одной подписки — кнопка **«Обновить сейчас»** (`POST /outbound-subs/:id/refresh`); она тоже помечает Xray на перезапуск. Добавление, изменение, удаление подписки также взводят флаг перезапуска Xray (при удалении его `outbound` 'ы выпадают из конфига на следующей перезагрузке). UI подсказывает: «После добавления или обновления перезапустите Xray (или дождитесь следующей автоперезагрузки), чтобы исходящие стали активными.»

Как это попадает в конфиг Xray

При каждой генерации конфигурации Xray активные подписочные `outbound` 'ы делятся на две группы — `prepend` (флаг «Перед ручными исходящими») и остальные — и сшиваются с шаблоном: `[prepend подписок] + [outbound'ы шаблона] + [остальные подписки]`. Внутри каждой группы подписки идут по приоритету. Ручные `outbound` 'ы из шаблона при этом не затрагиваются; если массив `outbound` 'ов шаблона почему-то не парсится, подписочные `outbound` 'ы в него не подмешиваются (чтобы не потерять ручные).

Импортированные `outbound` 'ы дополнительно отображаются в самой панели `outbound` 'ов отдельным блоком **«Из подписок исходящих (только для чтения)»** — редактировать их там нельзя, управление только через раздел «Подписки исходящих».

11.13. Ротация IP в WARP

В 3X-UI можно поднять WARP-outbound — исходящее WireGuard-подключение к Cloudflare WARP (тег `warp` в конфиге Xray). Панель сама регистрирует на серверах Cloudflare устройство-аккаунт, получает WireGuard-ключи и адреса и подставляет их в `outbound` с тегом `warp`. Через такой `outbound` трафик выходит в интернет под IP-адресом Cloudflare WARP. Новинка версии 3.3.0 — возможность менять этот исходящий IP вручную или по расписанию, не пересоздавая аккаунт WARP вручную.

Управление находится в разделе **Xray** в карточке WARP (после нажатия «Создать аккаунт WARP» и получения конфига; до этого действия недоступны — панель подскажет «Сначала получите WARP-конфиг»).

Что происходит при смене IP

Кнопка **«Сменить IP»** запускает смену IP. Логика:

1. Генерируется новая пара ключей WireGuard.
2. С новым ключом заново регистрируется устройство WARP на серверах Cloudflare (новый `device_id`, `access_token`, адреса и peer-данные).
3. Новые данные записываются в WARP-outbound конфига Xray: обновляются `secretKey`, `address` (v4 /32 и v6 /128), `reserved` (из `client_id`), а также `publicKey` и `endpoint` у peer.
4. Если ранее был задан лицензионный ключ WARP+ (длиной не менее 26 символов), он автоматически переустанавливается на новый аккаунт. При неудаче это лишь предупреждение в логах — смена IP не отменяется.
5. После успешной смены Xray помечается как требующий перезапуска, чтобы новый `outbound` вступил в силу.

При успехе интерфейс показывает «IP-адрес WARP успешно изменён!».

Автоматическая ротация по расписанию

В карточке WARP есть переключатель **«Автоматическое обновление IP-адреса»** и поле **«Интервал (дни)»**. Подсказка: «0 — отключить. Автоматически меняет IP-адрес.»

Параметр	Значение
Настройка в БД	<code>warpUpdateInterval</code> (целое, ≥ 0)
Значение по умолчанию	<code>0</code> (авто-ротация выключена)
Единица измерения	дни
<code>0</code>	отключает автоматическую смену
<code>> 0</code>	менять IP каждые N дней

Запись интервала сохраняет `warpUpdateInterval`, и при значении больше 0 сбрасывает «время последнего обновления» на текущий момент — иначе планировщик сменил бы IP уже на ближайшем тике.

Расписание исполняет фоновая задача, запускаемая раз в час — то есть панель раз в час проверяет, не пора ли ротировать. Алгоритм проверки:

- если интервал ≤ 0 — ничего не делает;
- если «время последнего обновления» равно 0 (например, интервал выставили прямой правкой БД) — это первый запуск: задача лишь фиксирует базовую отметку времени и НЕ меняет IP сразу;
- если с момента последнего обновления прошло не меньше `интервал × 24 × 3600` секунд — выполняется та же смена IP, обновляется отметка времени и планируется перезапуск Xray.

Важная деталь: ручная смена по кнопке «Сменить IP» тоже сбрасывает отметку времени последнего обновления. Поэтому после ручной ротации отсчёт автоматического интервала начинается заново и плановая смена не сработает сразу следом.

«Через прокси панели»

Изменено в 3.3.1. Отдельная настройка «Сетевой прокси панели» (`panelProxy`) убрана. Исходящий трафик самой панели (в том числе запросы к WARP API) теперь направляется через выбранный **outbound для трафика панели** — Xray-outbound или балансировщик (см. раздел 13). Описание ниже относится к версиям до 3.3.1.

Все запросы к API Cloudflare WARP (регистрация, получение конфига, установка лицензии, смена IP) идут не напрямую, а через HTTP-клиент панели с таймаутом 15 секунд. Этот клиент уважает настройку **«Сетевой прокси панели»** (`panelProxy`) из настроек панели.

Из описания настройки: прокси маршрутизирует собственные исходящие запросы панели (обновления гео-баз, проверки версий Xray/панели, Telegram, а теперь и обращения к WARP) — чтобы обходить серверную фильтрацию. Принимаются адреса вида `socks5://` или `http(s)://`, например локальный SOCKS-входящий самого Xray. Если поле пустое или прокси задан неверно — используется прямое подключение (поведение не ломается).

Польза для WARP: если сервер не может напрямую достучаться до `api.cloudflareclient.com`, регистрация и ротация раньше падали. Теперь, указав в `panelProxy` рабочий прокси (в т.ч. собственный inbound Xray), можно гарантировать доступность WARP API и работоспособность как ручной кнопки, так и плановой ротации.

Когда это полезно

- Регулярная смена исходящего IP для outbound, который ходит через WARP, — снижает риск блокировок и трекинга по одному адресу.
 - «Освежить» IP вручную, если текущий адрес Cloudflare попал в чёрные списки или работает медленно.
 - Серверы, у которых нет прямого доступа к Cloudflare WARP API: маршрутизация запросов через `panelProxy` делает регистрацию и ротацию работоспособными.
-

12. Узлы (мультипанель, master/slave)

Раздел **Узлы** превращает обычную установку 3X-UI в **центральный (мастер-) панель**, которая удалённо наблюдает за другими (дочерними) панелями 3X-UI и управляет ими. Каждый узел — это отдельная установка 3X-UI на своём сервере; мастер обращается к ней по её собственному HTTP API, опрашивает её состояние и синхронизирует на неё inbounds и клиентов, которые им назначены. Это и есть возможность **мультипанели**: вместо того чтобы заходить в каждую панель по отдельности, вы видите все серверы в одном списке и управляете ими централизованно.

Важный принцип: **узел — это не агент, а полноценная панель 3X-UI**. Мастер ничего на ней не «устанавливает» — он лишь подключается к её API по токenu. Удаление узла из списка останавливает только мониторинг; сама удалённая панель при этом не затрагивается (подсказка: «Это остановит мониторинг узла. Сама удалённая панель не будет затронута»).

12.1. Сводка вверху списка

Над таблицей узлов выводятся агрегированные счётчики:

Поле	Описание
Всего узлов	Общее число узлов в списке.
В сети	Сколько узлов имеют статус <code>online</code> .
Не в сети	Сколько узлов имеют статус <code>offline</code> .
Средняя задержка	Усреднённая задержка (ping) до узлов, в миллисекундах.

12.2. Добавление и редактирование узла

Кнопки **Добавить узел** и **Изменить узел** открывают форму с полями узла.

Обязательны (подсказка: «Имя, адрес, порт и токен API обязательны») поля **Имя**, **Адрес**, **Порт** и **API Токен**.

При нажатии «Сохранить» (как при добавлении, так и при изменении) панель **сначала проверяет достижимость узла** с таймаутом 6 секунд. Если узел не отвечает, запись не сохраняется и показывается ошибка. То есть нельзя добавить заведомо недоступный узел.

Поля формы

Поле (RU)	По умолчанию	Допустимые значения	Описание
Имя	— (обязательно)	непустая строка, уникальная	Внутреннее имя узла. На колонку имени наложена уникальность — два узла с одинаковым именем создать нельзя. Подсказка-плейсхолдер: <code>napr.de-frankfurt-1</code> . При сохранении пробелы по краям обрезаются.
Примечание	пусто	любая строка	Необязательная заметка/описание узла. На работу не влияет.

Поле (RU)	По умолчанию	Допустимые значения	Описание
Схема	<code>https</code>	<code>http / https</code>	Протокол подключения к удалённой панели. Если оставить пустым или указать недопустимое значение, нормализация выставит <code>https</code> . Если узел отвечает по обычному HTTP, а схема стоит <code>https</code> , панель вернёт понятную подсказку: «the server speaks HTTP, not HTTPS; set the node scheme to http».
Адрес	— (обязательно)	хост или IP	Адрес удалённой панели. Плейсхолдер: <code>panel.example.com</code> или <code>1.2.3.4</code> . Адрес нормализуется; по умолчанию приватные/локальные адреса запрещены ради защиты от SSRF — см. «Разрешить частный адрес».
Порт	— (обязательно)	целое 1–65535	Порт веб-панели удалённого узла. Значения вне диапазона отклоняются («node port must be 1-65535»).
Базовый путь	<code>/</code>	строка пути	Базовый путь (web base path) удалённой панели, если он задан. Нормализуется: гарантированно начинается и заканчивается на <code>/</code> (пустое значение → <code>/</code>). К нему панель дописывает <code>panel/api/server/status</code> при опросе.
API Токен	— (обязательно)	токен удалённой панели	Bearer-токен для доступа к API узла. Передаётся в заголовке <code>Authorization: Bearer <token></code> . Плейсхолдер: «Токен со страницы Настроек удалённой панели». Подсказка: «Удалённая панель показывает свой токен API в разделе Настройки → Токен API». То есть токен надо создать на самом узле (Настройки → Токен API), а сюда вставить.
Включён	<code>true</code>	да/нет	Включает мониторинг и синхронизацию узла. Выключенные узлы не опрашиваются фоновыми задачами (heartbeat и traffic-sync их пропускают) и не участвуют в массовом обновлении панели.
Разрешить частный адрес	<code>false</code>	да/нет	Снимает SSRF-защиту и разрешает подключаться к узлу по приватному/локальному адресу. Подсказка: «Включить только для узлов в частной сети или VPN». Включайте только когда узел действительно находится в приватной сети или доступен через VPN.

Получение и регенерация токена на стороне узла

Токен берётся на удалённой панели в разделе **Настройки** → **Токен API**. Там же его можно перевыпустить: кнопка **Сгенерировать токен заново** с предупреждением: «Повторная генерация аннулирует текущий токен. Любая центральная панель, использующая его, потеряет доступ до обновления. Продолжить?». После регенерации старый токен в мастер-панели перестанет работать — его нужно обновить в форме узла.

Исходящее подключение (Connection outbound)

Поле **Connection outbound** (Исходящее подключение, `outboundTag`) задаёт, как трафик обращений мастера к API этого узла покидает сервер. Если выбрать в нём тег Xray-outbound, обращения панели к узлу пойдут не напрямую, а через указанный outbound; панель сама добавит в рабочую конфигурацию мост-`inbound` на `loopback` и применит изменение на лету, без перезапуска. Подсказка: «Route this node's panel API traffic through the selected Xray outbound. A loopback bridge inbound is added to the running config automatically and applied live. Leave empty for a direct connection».

Селектор устроен как панельный выбор outbound: теги сгруппированы на **Outbounds** (обычные исходящие) и **Balancers** (балансировщики), blackhole-исходящие из списка скрыты. Пустое значение (плейсхолдер «Direct connection») = прямое подключение к узлу.

Импорт inbound (выбор синхронизируемых inbound-ов)

В форме узла есть настройка **Импорт inbound** (`inboundSyncMode`) с двумя режимами: **Все inbound** (`all` , по умолчанию) и **Выбранные** (`selected`). По умолчанию мастер синхронизирует на узел все inbounds, у которых выбран этот узел; существующие узлы продолжают работать в режиме «Все inbound».

В режиме **Выбранные** под полем появляется мультिवыбор тегов inbound. Нажмите **Загрузить inbound** — мастер по введённым (ещё не сохранённым) параметрам подключения запросит у узла список его inbound-ов (эндпоинт `POST /panel/api/nodes/inbounds`) и покажет их теги; отметьте нужные. Панель будет синхронизировать и разворачивать на узел только отмеченные теги, а остальные inbounds, существующие непосредственно на узле, останутся нетронутыми — мастер их не удаляет и ими не управляет.

Пример: запросить список inbound-ов узла для выборочного импорта. В теле передаются ещё не сохранённые параметры подключения; в ответе — теги доступных на узле inbound-ов:

```
POST /panel/api/nodes/inbounds
Content-Type: application/json

{ "name": "de-fra-1", "scheme": "https", "address": "node1.example.com",
  "port": 2053, "basePath": "/", "apiToken": "abcdef..." }
```

12.3. Проверка TLS (для https-узлов)

Группа полей задаёт, как мастер проверяет HTTPS-сертификат узла. Эти настройки **актуальны только для схемы https** ; для http -узлов они игнорируются.

Проверка TLS — выпадающий список, подсказка: «Как панель проверяет HTTPS-сертификат узла. Закрепление или Пропуск — для самоподписанных сертификатов (только https-узлы)».

Режим (RU)	Значение	По умолчанию	Описание
Проверять (стандартный CA)	<code>verify</code>	да (default)	Обычная проверка цепочки сертификата доверенным CA. Подходит для узлов с публичным/Let's Encrypt сертификатом. Используется также для всех http -узлов.
Закрепить сертификат (SHA-256)	<code>pin</code>	—	Цепочка CA не проверяется, но SHA-256 листового сертификата узла сверяется с сохранённым отпечатком (constant-time сравнение). Сохраняет защиту от MITM для самоподписанных сертификатов. Требуется заполнить поле отпечатка.
Пропустить проверку	<code>skip</code>	—	Проверка сертификата полностью отключается. Предупреждение: «Пропуск проверки убирает защиту от атак «человек посередине» — токен API может быть перехвачен. Лучше закрепить сертификат».

К трём режимам выше в 3.4.0 добавлен четвёртый — **Mutual TLS (client certificate)** (`mtls`), доступный, как и остальные, только для схемы https .

Режим (RU)	Значение	По умолчанию	Описание
Mutual TLS (клиентский сертификат)	mtls	—	Помимо проверки сертификата узла, мастер дополнительно подтверждает себя узлу клиентским сертификатом , выпущенным его собственным CA. Для узла в этом режиме API-токен становится необязательным — узел опознаёт мастер по сертификату. При выборе режима показывается подсказка: «This node authenticates the panel with a client certificate. Copy this panel's CA from the Node mTLS section onto the node, set its Trusted parent CA, then restart it».

Чтобы включить взаимный TLS для узла: на стороне узла задайте режим **Mutual TLS**, скопируйте CA управляющей панели из раздела **Node mTLS** (см. ниже), пропишите его на узле как **доверенный родительский CA** и перезапустите узел.

Если выбрать любое значение, кроме `skip`, `pin` или `mtls`, нормализация принудительно выставит `verify`.

Закрепление сертификата

При выборе **Закрепить сертификат** появляются:

- **SHA-256 закреплённого сертификата** — поле ввода. Принимается отпечаток в **base64** (формат `pinnedPeerCertSha256` из Xray) либо в **hex** с двоеточиями или без (стиль `openssl -fingerprint`). Подсказка: «SHA-256 сертификата узла в base64 или hex. Нажмите «Получить», чтобы считать его с узла сейчас». Плейсхолдер: «SHA-256 в base64 или hex». При выборе `pin` пустой или некорректный отпечаток вызывает ошибку валидации при сохранении.

Пример: один и тот же отпечаток в двух форматах. Поле принимает любой из вариантов — оба означают один сертификат:

```
# base64 (формат pinnedPeerCertSha256 из Xray)
607TNg3l2k0pq8R1sT2uV3wX4yZ5a6B7c8D9e0F1g2=

# hex с двоеточиями (стиль openssl x509 -fingerprint -sha256)
E8:E2:D3:60:DE:5D:9A:4D:29:AB:CF:11:B2:7C:34:...
```

Если отпечаток ещё неизвестен, нажмите **Получить** — мастер сам считает его с узла по HTTPS и подставит в поле.

- Кнопка **Получить** — подключается к узлу по HTTPS без проверки сертификата и считывает SHA-256 текущего листового сертификата (эндпоинт `POST /certFingerprint`), подставляя его в поле. После успеха — «Текущий сертификат узла получен»; при неудаче — «Не удалось получить сертификат». Доступно только для https-узлов.

Node mTLS (взаимная TLS-аутентификация между панелями)

На странице **Узлы** есть отдельный раздел **Node mTLS** — настройка взаимной TLS-аутентификации, добавляющей к API-токену второй фактор (клиентский сертификат) для вызовов «панель → узел». Взаимный TLS включается по желанию; если поля раздела пусты, узлы работают по прежней схеме —

только с API-токеном (подсказка: «Mutual TLS adds a client-certificate factor on top of the API token for node-to-node calls. It is opt-in: leave it empty to keep token-only auth»). В разделе две операции:

- **Скопировать СА этой панели** (`POST /panel/api/nodes/mtls/ca`) — копирует корневой сертификат (CA) этой панели в буфер обмена. Этот СА нужно передать управляемым узлам, чтобы они доверяли клиентскому сертификату панели; на самих узлах после этого выставляется режим проверки TLS **Mutual TLS** (подсказка: «Hand this CA to the nodes this panel manages, then set their TLS verification to Mutual TLS»). После копирования — «CA certificate copied to clipboard».
- **Доверенный родительский СА** (`Trusted parent CA`, `POST /panel/api/nodes/mtls/trustCA`) — поле, которое используется, когда сама эта панель выступает узлом для вышестоящей (управляющей) панели. Вставьте сюда СА управляющей панели, чтобы требовать от неё клиентский сертификат, и нажмите **Save trust CA**. Изменение требует **перезапуска панели** (подсказка: «When this panel is itself a node, paste the managing panel's CA here to require its client certificate. Restart the panel to apply»).

12.4. Что показывается по каждому узлу

Колонки таблицы и поля карточки узла (наблюдаемое состояние, заполняется при каждом heartbeat-опросе):

Поле (RU)	Описание
Статус	<code>online</code> / <code>offline</code> / <code>unknown</code> — см. ниже.
CPU	Загрузка процессора удалённого сервера в процентах.
Память	Использование ОЗУ в процентах (вычисляется как <code>current/total*100</code>).
Аптайм	Время непрерывной работы сервера (в секундах).
Задержка	Время ответа узла на последний опрос (мс).
Последний пинг	Время последнего успешного heartbeat (unix-секунды; <code>0</code> = «никогда»; недавнее значение показывается как «только что»).
Версия Xray	Версия Xray-core, запущенного на узле.
Версия панели	Версия 3X-UI на узле — сравнивается с актуальной для индикатора обновления.
(inbounds)	Сколько inbounds физически размещено на этом узле.
(клиенты)	Число клиентов на inbounds узла.
(онлайн)	Сколько клиентов узла сейчас в сети.
(исчерпано)	Сколько клиентов узла истекли или исчерпали лимит трафика . Отключённые вручную клиенты в этот счётчик не входят.
(скорость)	Текущая (живая) скорость передачи на inbound-ах, размещённых на узле.

Счётчики inbounds/клиентов/онлайн привязываются к узлу по его стабильному GUID (`panelGuid`), а не по локальному id, — чтобы клиент на подузле учитывался именно под подузлом, а не под промежуточным узлом, через который он синхронизируется.

Для inbound-ов, размещённых на узле, страница показывает онлайн-клиентов, счётчики и **текущую скорость передачи**. Привязка по стабильному GUID корректно разводит и «клонированные» узлы с одинаковым `panelGuid`.

Статусы узла

Статус	RU	Когда устанавливается
online	В сети	Узел ответил <code>success=true</code> на опрос <code>panel/api/server/status</code> .
offline	Не в сети	Узел не ответил, вернул HTTP-ошибку, <code>success=false</code> либо нераспознаваемый ответ.
unknown	Неизвестно	Начальное значение, пока узел ещё ни разу не опрошен.

При неуспешном опросе текст ошибки сохраняется и показывается понятной формулировкой, что помогает диагностировать причину «offline».

12.5. Действия над узлом

- **Проверить соединение** (`POST /test`) — в форме узла проверяет связь по введённым (ещё не сохранённым) параметрам с таймаутом 6 с. Результат: «Соединение в порядке ({ms} мс)» или «Не удалось подключиться». Удобно для отладки адреса/порта/токена/TLS до сохранения.
- **Проверить сейчас** (кнопка «Проверить сейчас», `POST /probe/:id`) — внеплановый опрос уже сохранённого узла; немедленно обновляет статус и метрики (CPU/память/аптайм/задержку/версии) и записывает heartbeat. При неудаче — «Проверка не удалась».

Пример: проверить и опросить узел через API мастера. «Проверить соединение» испытывает ещё не сохранённые параметры из формы:

```
POST /panel/api/nodes/test
Content-Type: application/json

{ "scheme": "https", "address": "de-frankfurt-1.example.com", "port": 2053,
  "basePath": "/", "apiToken": "eyJhbGciOi...", "tlsMode": "verify" }
```

Внеплановый опрос уже сохранённого узла с id 7:

```
POST /panel/api/nodes/probe/7
```

- **Обновить панель** (`POST /updatePanel` с телом `{ids:[...]}`) — запускает на узле его штатный самообновлятель: узел скачивает последний релиз 3X-UI и перезапускается на нём. Кнопка **Обновить выбранные ({count})** выполняет это для нескольких отмеченных узлов сразу. Рядом с узлом показывается индикатор: **Доступно обновление** или **Актуально**, исходя из сравнения версии панели узла с последней.

Пример: обновить несколько узлов одним запросом. В теле передаются id отмеченных узлов; обновятся только включённые и `online`, остальные вернутся как пропущенные.


```
POST /panel/api/nodes/updatePanel
Content-Type: application/json

{ "ids": [3, 7, 12] }
```

Ответ вида «Обновление запущено на 2 узлах, 1 не удалось»: узел 12, например, мог быть offline и поэтому пропущен.

- Подтверждение: «Обновить {count} узлов до последней версии? Каждый выбранный узел загрузит последний релиз и перезапустится. Обновляются только включённые узлы в сети».
- **Обновляются только включённые узлы в статусе online**. Выключенный узел в результатах помечается «node is disabled», offline — «node is offline». Итог: «Обновление запущено на {ok} узлах, {failed} не удалось». Если не выбрано ни одного подходящего узла — «Выберите хотя бы один включённый узел в сети».

В диалоге подтверждения обновления (как для одиночного узла, так и для массового) есть флажок **Обновить до канала разработки (последний коммит)**. Если его отметить, выбранные узлы установят rolling-сборку dev-latest (последний коммит ветки main) вместо стабильного релиза; при снятом флажке узел обновляется по своему обычному каналу. При включённом флажке под ним выводится предупреждение: «Сборки разработки отслеживают каждый коммит в main и не являются стабильными релизами — автоматического отката нет». Флаг dev пробрасывается через POST /panel/api/nodes/updatePanel на узел, и тот запускает обновление именно по dev-каналу.

- **Set Cert from Panel** (вспомогательное, GET /webCert/:id) — при создании inbound на узле позволяет подставить пути к **собственному** web-TLS-сертификату узла (а не центральной панели), чтобы файлы существовали именно на узле. Требуется, чтобы узел был включён и доступен.
- **Удалить узел** (POST /del/:id) — подтверждение: «Удалить узел "{name}"? Это остановит мониторинг узла. Сама удалённая панель не будет затронута». Удаляет запись узла и его накопленную статистику трафика; удалённая панель продолжает работать как обычно. **Удалить узел можно только после того, как с него сняты все inbounds**. Если к узлу всё ещё привязан хотя бы один inbound (через node_id), панель отклонит удаление с ошибкой вида «cannot delete node: N inbound(s) still attached to it; detach or delete them first» — сначала открепите или удалите эти inbounds, затем удаляйте узел. Это исключает «осиротевшие» inbounds с висящей ссылкой на удалённый узел.

12.6. История метрик

Кнопка/график истории обращается к GET /history/:id/:metric/:bucket. Доступные метрики: **cpu** и **mem** — они накапливаются при каждом успешном heartbeat. Размер интервала агрегации (bucket, в секундах) ограничен белым списком:

Пример: запрос истории. График загрузки CPU узла 7 с агрегацией по 60-секундным интервалам (возвращается до 60 точек):

```
GET /panel/api/nodes/history/7/cpu/60
```

Для памяти и режима «реального времени» (2 с) — соответственно .../7/mem/60 и .../7/cpu/2. Значения вне белого списка отклоняются («invalid metric» / «invalid bucket»).

Bucket (с)	Назначение
2	Режим реального времени
30	Интервалы по 30 с
60	Интервалы по 1 мин
120	Интервалы по 2 мин
180	Интервалы по 3 мин
300	Интервалы по 5 мин

Возвращается до 60 точек. Недопустимая метрика или bucket отклоняются («invalid metric» / «invalid bucket»).

12.7. Как синхронизируются inbounds и клиенты

Inbound «принадлежит» узлу через поле `node_id` (в редакторе inbound выбирается узел):

Пример: токен в форме узла. Токен берётся на дочерней панели (Настройки → API Токен) и вставляется в поле **API Токен** мастера. При каждом опросе мастер шлёт его в заголовке:

```
GET https://panel.example.com:2053/panel/api/server/status
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.abc123...
```

Если на дочерней панели задан **базовый путь** (web base path), например `/secret/`, мастер сам подставит его перед `panel/api/server/status` → `https://panel.example.com:2053/secret/panel/api/server/status`.

- 1. Развёртывание конфигурации (reconcile).** При любом изменении inbound/клиента, привязанного к узлу, узел помечается «грязным». Фоновая задача для каждого включённого узла **в статусе online** при наличии изменений разворачивает на узел его inbounds (по `node_id`) и затем сбрасывает флаг «грязного» состояния. Узел, который выключен, offline или «грязный», считается «ожидающим» — деплой на него откладывается до восстановления связи.
- 2. Сбор трафика.** Та же задача запрашивает у узла снимок трафика и сливает его в локальную статистику. На основе слитого трафика выполняется проверка исчерпания лимитов/сроков и при необходимости отключение клиентов; счётчик «исчерпано» по узлу отражает именно это. Если узел недоступен, его онлайн-клиенты очищаются.

Для клиента, привязанного сразу к нескольким панелям, master в той же задаче дополнительно рассылает узлам **суммарный по всем панелям** расход трафика этого клиента (отдельной таблицей на узле, ключ — GUID мастера; перезаписывается при каждой отправке, поэтому сброс на стороне мастера тоже распространяется). На узле в трафике клиента отображается большее из двух значений — локальное или присланное, — а при превышении общей квоты клиент отключается **локально на самом узле** (через тот же механизм перезапуска Xray при авто-отключении, что разрывает уже установленные соединения). Это устраняет ситуацию, когда узел видел только свою долю трафика, недосчитывал расход и продолжал обслуживать клиента, уже исчерпавшего общий лимит. При сбросе трафика, авто-продлении или удалении клиента присланные счётчики очищаются.

При **первой** синхронизации inbound, размещённого на узле (добавление нового узла или повторный импорт inbound), мастер инициализирует счётчики трафика клиентов реальными значениями с узла. Раньше в этой ситуации общий счётчик inbound переносился корректно, а индивидуальные счётчики клиентов обнулялись, и мастер занижал расход клиентов на всю историю, накопленную до подключения узла. Теперь, если inbound создаётся в той же синхронизации, новая строка `client_traffics` наследует значение счётчика с узла (baseline выставляется равным ему, поэтому следующая дельта равна нулю и трафик не учитывается дважды). Засев счётчика применяется только для inbound, созданного в этом же проходе: клиент, заново появившийся под уже существующим inbound, по-прежнему стартует с нуля (защита от «фантомного» трафика), а только что удалённый клиент не «воскресает» при пересоздании его inbound. 3. **Heartbeat**. Отдельная фоновая задача периодически опрашивает все **включённые** узлы (с ограничением параллелизма) через `panel/api/server/status`, обновляет статус/метрики/версии и, при наличии веб-клиентов, рассылает обновлённое дерево узлов по WebSocket.

12.8. Цепочки узлов (подузлы / транзитивные узлы)

Топология может быть не плоской: узел сам может быть мастером для своих узлов. Такие нижестоящие панели показываются у вас как **Подузлы** — это **проекции «только для чтения»**, полученные от прямого узла.

- Подсказка: «Только для чтения: подчинённый узел, доступный через {parent}. Управляйте им из собственной панели {parent}». То есть подузел нельзя здесь редактировать, удалять или обновлять — все операции с ним выполняются из панели его прямого родителя.
- Идентичность подузла определяется его GUID; благодаря этому онлайн-клиенты и inbounds учитываются именно под тем физическим узлом, который их хостит, даже в цепочке `Node1 → Node2 → Node3` (мастер «проходит» на один уровень вглубь через каждый прямой узел).
- Если прямой узел становится недоступен, его кэш подузлов очищается, и подузлы пропадают из дерева до восстановления связи.

12.9. Исправления 3.5.0 (стабильность узлов)

Пакет исправлений в 3.5.0, заметных оператору узлов:

- **Сохранение клиента без изменений больше не рушит трафик узла.** Раньше любое сохранение клиента (даже «открыл — сохранил») отправляло на узел полное обновление inbound с пересозданием обработчика Xray — узел числился онлайн, но не пропускал трафик до ручного перезапуска. Теперь по-ор сохранения ничего не делают, а реальные правки уходят одним лёгким обновлением без пересоздания.
- **Host-переопределения узла принимаются в мастер:** при первом принятии inbound узла его строки Hosts (SNI, отпечаток, ALPN и т. д.) импортируются, и подписки несут правильные TLS-параметры.
- **Авто-продление на узле открывает свежее окно квоты** (пакет вида «100 ГБ каждые 30 дней» снова выдаёт полный объём).
- **Удаление клиента на мастере полностью удаляет его на узлах** (запись + трафик), а не только отвязывает; остатки прежних массовых удалений можно убрать действием «Удалить непривязанных клиентов» на узле.
- **Inbound'ы узла не выметаются до первого принятия:** сохранение свежедобавленного узла больше не может удалить его живые inbound'ы.
- **Один некорректный inbound не останавливает синхронизацию трафика узла** (заодно легаси-inbound'ы `socks` на узлах переименовываются в `mixed`).

- **Проверка конфликта портов ограничена своим узлом** — правка адреса прослушивания узлового inbound больше не отклоняется из-за такого же порта на другом узле («port ... already used by inbound ...»).

12.10. Узлы: новое в 3.3.0

В версии 3.3.0 раздел **Узлы** получил три заметных улучшения: корректную атрибуцию трафика и онлайн-клиентов в многоуровневых (multi-hop) топологиях, синхронизацию client-IP между узлами и отдельный индикатор статуса для случая, когда панель узла жива, а ядро Xray на ней упало. Слова inbound/outbound далее не переводятся.

1. Multi-hop: правильная атрибуция трафика по цепочке под-узлов

Раньше счётчики (число inbound, онлайн-клиентов, исчерпанных) считались на уровне «прямого» узла. Если у вас цепочка вида Мастер → Узел1 → Узел2 → Узел3, всё, что физически живёт на Узел2 / Узел3, ошибочно приписывалось Узел1, через который оно дошло до мастера. В 3.3.0 атрибуция идёт по реальному источнику.

Как это устроено:

- **Под-узлы становятся видны как отдельные строки.** Каждая панель публикует список своих прямых узлов; включаются только узлы с известным Guid — стабильная идентичность нужна, чтобы приписать узел на один «прыжок» вверх. Мастер периодически (из heartbeat-задания) подтягивает эти списки и кэширует их, а затем добавляет к прямым узлам «транзитивные» под-узлы.
- **Транзитивные узлы — только для чтения.** В UI они помечаются как **«Подузел»** с подсказкой: *«Только для чтения: подчинённый узел, доступный через {родитель}. Управляйте им из собственной панели {родитель}.»* Кнопок управления у такой строки нет — узлом управляют с панели его непосредственного родителя.
- **Иерархия через GUID.** У прямого узла ParentGuid — это GUID самого мастера; у транзитивного — GUID его родительского узла. Так строится дерево.
- **Источник правды для счётчиков — origin_node_guid на inbound.** Это panelGuid узла, который физически держит этот inbound. Оно проставляется при синхронизации inbound с узла и **сохраняется как есть при дальнейших прыжках**, поэтому глубоко вложенный inbound приписывается реальному узлу, а не промежуточному. По этому GUID пересчитываются счётчики числа inbounds, онлайн-клиентов и исчерпанных клиентов. Логика выбора ключа:

Состояние inbound	Чему приписывается
origin_node_guid задан	этому GUID (реальный узел-источник)
пусто, но задан node_id	синтетический GUID узла (старый билд, ещё не сообщил свой panelGuid)
пусто и node_id пуст	собственному GUID мастера (inbound на локальном Xray)

Онлайн-клиенты так же группируются по GUID, поэтому каждая строка узла показывает только тех, кто реально подключён именно к нему.

Что видит пользователь: в плоской топологии (узлы напрямую под мастером) ничего не меняется — счётчики по GUID и по `id` совпадают. Но как только появляется цепочка узлов, в списке возникают строки-«Подузлы», и числа inbound/онлайн/исчерпанных у каждого узла теперь отражают именно его собственную нагрузку, а не сумму всего, что прошло через него транзитом.

2. Синхронизация client-IP из access.log между узлами

Лимит по IP (`limitIp` у клиента) опирается на адреса, которые Xray пишет в свой `access.log`. Раньше каждый узел видел только подключения к себе, поэтому ограничение «не более N IP на клиента» не работало в кластере: клиент мог подключиться к разным узлам и обойти лимит. В 3.3.0 наблюдаемые IP синхронизируются по всему кластеру.

Как это работает:

- На каждом узле фоновое задание парсит `access.log`, извлекая по каждой строке IP, email клиента и метку времени, и складывает их в локальную таблицу (по одной записи на email, IP хранятся как JSON-массив `{ip, timestamp}`). Локальные адреса `127.0.0.1` и `::1` отбрасываются.
- Синхронизация **раз в 10 секунд** выполняет двусторонний обмен по каждому включённому узлу в сети: тянет IP с узла и сливает их в локальную таблицу, а затем отдаёт узлу сводную картину мастера.
- Слияние объединяет старые и входящие наблюдения **без двойного учёта** одного IP, увиденного на нескольких узлах, и **без воскрешения устаревших** записей: применяется тот же порог давности, что и в локальном задании — **30 минут**. По каждому IP сохраняется самая свежая метка времени. Записи с чужих узлов получают новый локальный `id` (id-пространства узлов независимы); конкурентная вставка одного и того же email защищена от дублей.
- При подсчёте лимита «живым» считается IP, который либо замечен в текущем сканировании локально, либо имеет очень свежую метку из синхронизированной базы (**в пределах 2 минут**). Именно это и заставляет лимит работать в масштабе всего кластера, даже если адрес был замечен на другом узле. При превышении лимита самые старые «живые» IP отправляются в журнал `fail2ban`, а соединения принудительно разрываются (`remove/re-add` клиента через Xray API).

Что видит пользователь: ограничение по числу IP теперь действует на весь кластер, а не на каждый узел по отдельности; в панели по клиенту видны IP, замеченные на любом узле (в пределах окна в 30 минут). Отдельной кнопки/настройки для этого нет — синхронизация идёт автоматически в фоне, при условии что у узла включён и доступен `access.log` (для самого лимита также нужен `Fail2Ban` на узле).

3. Отдельный индикатор статуса: панель узла онлайн, но Xray упал

Раньше статус узла был, по сути, «в сети / не в сети». Если панель узла отвечала, узел считался онлайн — даже когда ядро Xray на нём не работало, и клиенты фактически не могли подключиться. В 3.3.0 здоровье панели и здоровье ядра Xray разделены.

Как это устроено:

- При опросе узла мастер берёт из ответа удалённого `/panel/api/server/status` поля `xray.state` и `xray.errorMsg` и сохраняет их в узле. Эти поля заполняются даже при успешном пинге панели, когда ядро нездорово, — именно чтобы отличить доступность панели от состояния Xray.
- Значения `xray.state`: `"running"` (работает), `"stop"` (остановлен), `"error"` (ошибка).

- Эти значения транслируются в статусы узла. К привычным добавились новые:

Ключ статуса	Русская подпись	Когда показывается
<code>online</code>	«В сети»	панель отвечает, Xray работает (<code>running</code>)
<code>offline</code>	«Не в сети»	панель недоступна / пинг не прошёл
<code>unknown</code>	«Неизвестно»	состояние ещё не определено
<code>xrayError</code>	«Ошибка Xray»	панель онлайн, но ядро Xray в состоянии <code>error</code> (есть <code>errorMsg</code>)
<code>xrayStopped</code>	«Остановлен»	панель онлайн, но Xray остановлен (<code>stop</code>)

- Для подобного состояния в UI используется **отдельный фиолетовый индикатор** (цвет, отличный от зелёного «в сети» и красного «не в сети»). Фиолетовый прямо сигнализирует: до узла дозвониться можно, проблема — в самом ядре Xray, а не в сети или в самой панели.

Что видит пользователь: вместо вводящего в заблуждение «зелёного» при упавшем ядре узел подсвечивается **фиолетовым** со статусом **«Ошибка Xray»** или **«Остановлен»**. Это сразу показывает, что чинить надо Xray на узле (перезапустить ядро, посмотреть `errorMsg`), а не разбираться с доступностью самого узла. Тот же `xrayState` / `xrayError` прокидывается и в транзитивные под-узлы (см. пункт 1), так что некорректное состояние ядра видно по всей цепочке.

13. Настройки панели

Раздел «Настройки» (заголовок страницы — **Настройки**, англ. *Panel Settings*) управляет поведением самой веб-панели 3X-UI: на каком адресе и порту она слушает, как защищена, как взаимодействует с Telegram-ботом и внешними сервисами, в каком часовом поясе выполняет запланированные задачи. Каждый параметр хранится в таблице `settings` базы данных в виде пары «ключ — значение»; если значение в БД отсутствует, применяется значение по умолчанию.

Важно — применение изменений. Любое изменение на этой странице нужно сохранить кнопкой **Сохранить** (*Save*), а затем перезапустить панель, чтобы изменения вступили в силу. Дословная подсказка: «Сохраните изменения и перезапустите панель для их применения.» При сохранении показывается уведомление «Настройки изменены».

13.1. Сохранение и перезапуск панели

Элемент	Назначение
Сохранить (<i>Save</i>)	Записывает все поля формы в БД (<code>POST /panel/setting/update</code>). Перед записью значения проходят валидацию — некорректные адреса, порты или пути будут отклонены, и панель вернёт ошибку.
Перезапустить панель (<i>Restart Panel</i>)	Перезапускает веб-сервер панели (<code>POST /panel/setting/restartPanel</code>). Перезапуск происходит с задержкой 3 секунды. Подсказка: «Вы уверены, что хотите перезапустить панель? Подтвердите, и перезапуск произойдёт через 3 секунды. Если панель будет недоступна, проверьте лог сервера». При успехе — «Панель успешно перезапущена».
Восстановить настройки по умолчанию (<i>Reset to Default</i>)	Удаляет все сохранённые в БД настройки, после чего панель использует значения по умолчанию. Учётные данные администратора не сбрасываются этой операцией.

Перезапуск выполняется отправкой процессу панели сигнала `SIGHUP` (или через зарегистрированный хук перезапуска). На Windows автоматический перезапуск через сигнал не поддерживается. **Изменения параметров прослушивания (IP, порт, путь, домен, сертификаты, часовой пояс) применяются только после перезапуска панели.**

13.2. Общие настройки (вкладка «Панель» / *General*)

Язык интерфейса (*Language*)

Язык веб-интерфейса панели. Доступные языки: `en-US` (английский), `ru-RU` (русский), `zh-CN`, `zh-TW`, `fa-IR`, `ar-EG`, `es-ES`, `id-ID`, `ja-JP`, `pt-BR`, `tr-TR`, `uk-UA`, `vi-VN`. Это настройка отображения и не влияет на работу Xray.

Тип календаря (*Calendar Type*)

- Ключ: `datepicker`

- **Значение по умолчанию:** `gregorian` (григорианский).
- **Назначение:** тип календаря, используемый в выборе дат (например, при задании срока действия клиентов). Подсказка: «Запланированные задачи будут выполняться в соответствии с этим календарём.» Альтернативное значение — персидский (джалали) календарь, что востребовано в иранской аудитории панели.

Размер нумерации страниц (*Pagination Size*)

- **Ключ:** `pageSize`
- **Значение по умолчанию:** `25`
- **Допустимые значения:** целое от `0` до `1000`.
- **Назначение:** число строк на странице в таблицах (списках подключений/inbound). Подсказка: «Определить размер страницы для таблицы подключений. Установите 0, чтобы отключить» — при `0` постраничный вывод отключается, и все записи показываются единым списком.
- **Перезапуск панели не требуется** (настройка отображения).

Перезапускать Xray после авто-отключения (*Restart Xray After Auto Disable*)

- **Ключ:** `restartXrayOnClientDisable`
- **Значение по умолчанию:** `true`
- **Назначение:** при автоматическом отключении клиента (по истечению срока действия или достижению лимита трафика) Xray перезапускается, чтобы разорвать уже установленные соединения такого клиента. Подсказка: «Когда клиент автоматически отключается из-за окончания срока действия или лимита трафика, перезапускать Xray.» Сама функция не изменилась — переключатель просто живёт на вкладке «Панель» (*General*) рядом с прочими общими настройками.

Модель примечания и символ разделения (*Remark Model & Separation Character*)

- **Ключ:** `remarkModel`
- **Значение по умолчанию:** `-ieo`
- **Назначение:** задаёт, как формируется имя (remark) конфигурации в подписке. Строка состоит из **первого символа** — разделителя, и далее **последовательности букв-порядка**:
 - `i` — примечание inbound (*inbound remark*);
 - `e` — email клиента;
 - `o` — дополнительная метка (*extra*).

При значении по умолчанию `-ieo` разделителем служит `-`, а порядок частей: inbound → email → extra (например, `MyInbound-user@mail-extra`). Пустые части пропускаются. Поле «Пример примечания» (*Sample Remark*) в интерфейсе показывает предпросмотр формируемого имени. Включение email в имя дополнительно зависит от параметра «Включать Email в название» в настройках подписки (раздел о подписке).

Пример: как значение `remarkModel` влияет на имя конфигурации. Допустим, inbound называется `VLESS-Reality`, email клиента — `alex@vpn`, а дополнительная метка — `RU`. Тогда:

Значение поля	Итоговое имя (remark)
-ieo (по умолчанию)	VLESS-Reality-alex@vpn-RU
_ie	VLESS-Reality_alex@vpn
-ei	alex@vpn-VLESS-Reality
o (пробел-разделитель, только метка)	RU

Первый символ строки – всегда разделитель; остальные буквы задают, какие части и в каком порядке войдут в имя.

13.3. Доступ к панели: IP, порт, путь, домен, сертификат

Эта группа определяет сетевую точку входа панели. **Все изменения здесь применяются только после перезапуска панели.**

Поле	Ключ	Значение по умолчанию	Описание
IP-адрес для управления панелью (<i>Listen IP</i>)	webListen	"" (пусто)	IP, на котором слушает веб-панель. Пусто = слушать на всех IP. Подсказка: «Оставьте пустым для подключения с любого IP». Если задан, должен быть корректным IP-адресом (иначе сохранение отклоняется).
Домен панели (<i>Listen Domain</i>)	webDomain	"" (пусто)	Доменное имя панели для проверки запроса по домену. Пусто = принимать подключения с любых доменов и IP. Подсказка: «Оставьте пустым для подключения с любых доменов и IP.»
Порт панели (<i>Listen Port</i>)	webPort	2053	Порт, на котором работает панель. Подсказка: «Порт, на котором работает панель». Допустимо 1–65535. Порт должен быть свободным; панель и служба подписки не могут одновременно использовать одинаковую пару IP:порт.
URI-путь (<i>URI Path</i>)	webBasePath	/	Базовый путь URL панели (basePath). Подсказка: «Должен начинаться с '/' и заканчиваться '/'». При сохранении панель автоматически добавляет ведущий и завершающий /, если они отсутствуют. Запрещённые символы в пути отклоняются.

Сертификат панели (TLS / HTTPS)

Поле	Ключ	Значение по умолчанию	Описание
Путь к файлу публичного ключа сертификата панели (<i>Public Key Path</i>)	webCertFile	""	Полный путь к файлу сертификата (цепочки). Подсказка: «Введите полный путь, начинающийся с '/'».
Путь к файлу приватного ключа сертификата панели (<i>Private Key Path</i>)	webKeyFile	""	Полный путь к файлу приватного ключа. Подсказка: «Введите полный путь, начинающийся с '/'».

Если задан **хотя бы один** из путей сертификата/ключа, панель при сохранении пытается загрузить пару «сертификат + ключ»; при ошибке (несуществующий файл, несоответствие ключа и сертификата) сохранение отклоняется. Когда заданы оба корректных пути, панель переходит на HTTPS. Оба поля пустые = панель работает по обычному HTTP.

Предупреждения безопасности (*Security warnings*). Панель показывает блок «Ваша панель может быть открыта:» с предупреждениями, если обнаруживает небезопасную конфигурацию:

- работа по обычному HTTP — «настройте TLS для продакшна»;
- стандартный порт 2053 — «измените его на случайный»;
- базовый путь по умолчанию / — «измените его на случайный»;
- стандартный путь подписки /sub/ и JSON-подписки /json/ — «измените его». Это рекомендации, а не блокировки.

13.4. Сессия, прокси панели и доверенные прокси (вкладка «Прокси и сервер» / *Proxy and Server*)

Продолжительность сессии (*Session Duration*)

- **Ключ:** `sessionMaxAge`
- **Значение по умолчанию:** 360 (минут, т.е. 6 часов).
- **Допустимые значения:** от 1 до 525600 минут (1 год).
- **Назначение:** как долго администратор остаётся авторизованным без повторного входа. Единица — **минута**. Подсказка: «Продолжительность сессии в системе (значение: минута)».

Outbound для трафика панели (*Panel Traffic Outbound*)

- **Ключ:** `panelOutbound`
- **Значение по умолчанию:** "" (пусто = прямое подключение).
- **Назначение:** задаёт Xray-outbound, через который панель отправляет **собственные запросы** — проверки версий и загрузку панели/Xray, обращения к Telegram, обычное обновление geo-файлов — чтобы обойти серверную фильтрацию GitHub/Telegram. Поле представляет собой **выпадающий список**: в нём перечислены outbound'ы из шаблона конфигурации Xray, outbound'ы из подписок на outbound, а также маршрутные **балансировщики** (отдельной группой). Outbound'ы типа `blackhole` из списка исключены — маршрутизировать загрузку в «чёрную дыру» бессмысленно. Дословная подсказка: «Маршрутизирует собственные запросы панели — проверки версий и загрузки панели/Xray, Telegram и обычное обновление geo-файлов — через этот исходящий Xray для обхода серверной фильтрации GitHub/Telegram. Локальный мост-входящий добавляется в работающую конфигурацию автоматически и применяется на лету. Встроенное в Xray автообновление Geodata не затрагивается; у него свой исходящий для загрузки. Оставьте пустым для прямого подключения.»

Как это работает. При выборе outbound'а панель сама добавляет в рабочий конфиг служебный loopback-inbound (SOCKS-мост с тегом `panel-egress`) и правило маршрутизации, которое заворачивает собственный HTTP-трафик панели на выбранный outbound. Если выбран

балансировщик, в правило подставляется `balancerTag`, и трафик панели распределяется между его участниками. Мост и правило применяются **на лету**, без полного перезапуска панели. Оставьте поле пустым для прямого соединения. Встроенное в Xray автообновление гео-данных этой настройкой **не затрагивается** — у него собственный outbound внутри Xray-роутинга.

- **Формат:** `socks5://` (или `socks5h://`) либо `http(s)://`, при необходимости с авторизацией вида `socks5://user:pass@host:port`. Поддерживаемые схемы строго: `socks5`, `socks5h`, `http`, `https` — иные схемы считаются недопустимыми, и панель тогда откатывается на прямое подключение. Типичный пример — локальный SOCKS-входящий самого Xray.
- Дословная подсказка: «Маршрутизирует исходящие запросы самой панели (обновления гео, проверки версий Xray/панели, Telegram) через этот прокси для обхода серверной фильтрации GitHub/Telegram. Принимает `socks5://` или `http(s)://`, напр. локальный SOCKS-входящий Xray. Оставьте пустым для прямого подключения.»
- Невалидный прокси не приводит к ошибке сохранения — панель просто использует прямое соединение и пишет предупреждение в лог.

Пример значений поля. Если на сервере уже поднят локальный SOCKS-входящий Xray на порту `10808`, направьте через него собственные запросы панели:

```
socks5://127.0.0.1:10808
```

Для внешнего HTTP-прокси с авторизацией:

```
http://user:pass@proxy.example.com:8080
```

После сохранения и перезапуска панель будет тянуть обновления гео-баз, проверять версии и обращаться к Telegram уже через указанный прокси.

Доверенные CIDR прокси (*Trusted proxy CIDRs*)

- **Ключ:** `trustedProxyCIDRs`
- **Значение по умолчанию:** `127.0.0.1/32,::1/128` (только локальный хост).
- **Формат:** список IP-адресов или CIDR-подсетей через запятую (например `10.0.0.0/8, 192.168.1.5`). Каждый элемент проверяется как IP или CIDR — некорректное значение отклоняется при сохранении.
- **Назначение:** перечисляет источники, которым разрешено устанавливать заголовки `X-Forwarded-Host`, `X-Forwarded-Proto` и заголовок реального IP клиента. Дословная подсказка: «IP/CIDR через запятую, которым разрешено устанавливать заголовки `forwarded host, proto` и `client IP`». Нужно настраивать, если панель работает за обратным прокси (nginx, Caddy и т.п.), чтобы корректно определять клиентские IP и схему.

Пример: панель за обратным прокси. Если nginx стоит на том же хосте и проксирует запросы на панель, оставьте доверие только локальному хосту (значение по умолчанию):

```
127.0.0.1/32,::1/128
```

Если прокси находится на отдельном сервере во внутренней сети `10.0.0.0/8`, добавьте его подсеть, иначе панель проигнорирует переданные им заголовки и будет видеть IP прокси вместо реального клиента:

```
127.0.0.1/32, ::1/128, 10.0.0.0/8
```

Пример соответствующего блока `nginx`, который передаёт реальный IP и схему:

```
proxy_set_header X-Real-IP      $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
proxy_set_header X-Forwarded-Host $host;
```

13.5. Telegram-бот (вкладка «Telegram-бот» / *Telegram Bot*)

Включить Telegram бота (*Enable Telegram Bot*)

- **Ключ:** `tgBotEnable`
- **Тип/по умолчанию:** логический, `false`.
- **Назначение:** включает работу Telegram-бота. Подсказка: «Доступ к функциям панели через Telegram-бота».

Telegram-токен (*Telegram Token*)

- **Ключ:** `tgBotToken`
- **По умолчанию:** `""`.
- **Назначение:** токен бота. Подсказка: «Необходимо получить токен у менеджера ботов Telegram @botfather».
- **Особенность безопасности:** токен относится к секретным значениям. В ответе панели на чтение настроек он не возвращается (поле очищается, отдаётся только флаг «настроен/не настроен»). Если при сохранении поле оставить пустым, прежний сохранённый токен **сохраняется** (не затирается).

Язык Telegram-бота (*Telegram Bot Language*)

- **Ключ:** `tgLang`
- **По умолчанию:** `en-US`.
- **Назначение:** язык сообщений бота (независимо от языка веб-интерфейса). Список доступных языков совпадает с языками панели.

User ID администратора бота (*Admin Chat ID*)

- **Ключ:** `tgBotChatId`
- **По умолчанию:** `""`.
- **Формат:** один или несколько числовых Telegram User ID **через запятую**.
- **Назначение:** получатели уведомлений и администраторы, которым разрешено управлять панелью через бота. Подсказка: «Один или несколько User ID администратора(-ов) Telegram-бота. Для получения User ID используйте @userinfobot или команду '/id' в боте.»

Частота уведомлений (*Notification Time*)

- **Ключ:** `tgRunTime`
- **По умолчанию:** `@daily` (раз в сутки).
- **Формат:** строка в формате **Crontab** (поддерживаются как стандартные cron-выражения, так и сокращения вида `@daily`, `@hourly`, `@every 1h`). Подсказка: «Укажите интервал уведомлений в формате Crontab». Управляет периодическими отчётами бота.

Примеры значений поля.

Значение	Когда бот шлёт отчёт
<code>@daily</code>	раз в сутки в полночь (по умолчанию)
<code>@hourly</code>	каждый час
<code>@every 6h</code>	каждые 6 часов
<code>0 9 * * *</code>	ежедневно в 09:00
<code>30 8 * * 1</code>	каждый понедельник в 08:30

Время считается в часовом поясе из настройки «Часовой пояс» (п. 13.6).

SOCKS-прокси (*SOCKS Proxy*)

- **Ключ:** `tgBotProxy`
- **По умолчанию:** `""`.
- **Назначение:** SOCKS5-прокси отдельно для подключения бота к Telegram. Подсказка: «Если для подключения к Telegram вам нужен прокси Socks5, настройте его параметры согласно руководству.» Применяется именно к трафику бота (отличается от общего «Сетевого прокси панели» из п. 13.4).

Telegram API Server (*Telegram API Server*)

- **Ключ:** `tgBotAPIServer`
- **По умолчанию:** `""` (использовать стандартный сервер `api.telegram.org`).
- **Формат:** URL `http(s)://...`; при сохранении проходит проверку корректности URL — невалидный адрес отклоняется. Подсказка: «Используемый API-сервер Telegram. Оставьте пустым, чтобы использовать сервер по умолчанию.» Нужен для самостоятельно развёрнутого Telegram Bot API server.

Уведомления бота (группа «Уведомления» / *Notifications*)

Поле	Ключ	По умолчанию	Описание
Резервное копирование базы данных (<i>Database Backup</i>)	<code>tgBotBackup</code>	<code>false</code>	Отправлять в Telegram файл резервной копии БД вместе с отчётом. Подсказка: «Отправлять уведомление с файлом резервной копии базы данных».

Поле	Ключ	По умолчанию	Описание
Уведомление о входе (<i>Login Notification</i>)	<code>tgBotLoginNotify</code>	<code>true</code>	Уведомлять при попытке входа в панель. Подсказка: «Отображает имя пользователя, IP-адрес и время, когда кто-то пытается войти в вашу панель.»
Задержка уведомления об истечении сессии (<i>Expiration Date Notification</i>)	<code>expireDiff</code>	<code>0</code>	За сколько дней до истечения срока действия клиента слать уведомление. <code>0</code> — отключено. Допустимо <code>>= 0</code> . Подсказка: «Получение уведомления об истечении срока действия сессии до достижения порогового значения (значение: день)».
Порог трафика для уведомления (<i>Traffic Cap Notification</i>)	<code>trafficDiff</code>	<code>0</code>	Порог остатка трафика для уведомления. Подсказка: «Получение уведомления об исчерпании трафика до достижения порога (значение: ГБ)». Допустимо <code>0–100</code> .
Порог нагрузки на ЦП (<i>CPU Load Notification</i>)	<code>tgCpu</code>	<code>80</code>	Уведомлять администраторов, если загрузка CPU превышает порог (в %). Допустимо <code>0–100</code> . Подсказка: «Уведомление администраторов в Telegram, если нагрузка на ЦП превышает этот порог (значение: %)».

13.6. Дата и время (вкладка «Дата и время» / *Date and Time*)

Часовой пояс (*Time Zone*)

- **Ключ:** `timeLocation`
- **Значение по умолчанию:** `Local` (системный часовой пояс сервера).
- **Формат:** имя зоны из базы IANA tz (например, `Europe/Moscow`, `UTC`, `Asia/Tehran`).
- **Назначение:** часовой пояс, в котором панель выполняет запланированные задачи (отчёты бота, сброс/проверки трафика, истечение сроков). Подсказка: «Запланированные задачи выполняются в соответствии со временем в этом часовом поясе».
- **Валидация:** при сохранении зона проверяется — несуществующая зона отклоняется. Если позже в БД окажется некорректное значение, панель в рантайме откатится на `Local`, а при недоступности и его — на `UTC`.

13.7. Внешний трафик и поведение Xray (вкладка «Внешний трафик» / *External Traffic*)

Поле	Ключ	По умолчанию	Описание
Информация о внешнем трафике (<i>External Traffic Inform</i>)	<code>externalTrafficInformEnable</code>	<code>false</code>	Уведомлять внешний API при каждом обновлении трафика. Подсказка: «Уведомлять внешний API при каждом обновлении трафика.»

Поле	Ключ	По умолчанию	Описание
URI информации о внешнем трафике (<i>External Traffic Inform URI</i>)	<code>externalTrafficInformURI</code>	<code>""</code>	URL, на который панель отправляет обновления трафика. Проходит проверку корректности URL при сохранении. Подсказка: «Обновления трафика отправляются на этот URI».
Перезапускать Xray после автоотключения (<i>Restart Xray After Auto Disable</i>)	<code>restartXrayOnClientDisable</code>	<code>true</code>	Перезапускать Xray, когда клиент автоматически отключается из-за истечения срока или превышения лимита трафика. Подсказка: «Когда клиент автоматически отключается из-за окончания срока действия или лимита трафика, перезапускать Xray.» Переключатель находится на вкладке «Панель» (General) — см. п. 13.2; здесь приведён для полноты.

13.8. Прочее: шаблон конфигурации Xray и проверочный URL

Шаблон конфигурации Xray (*xrayTemplateConfig*)

- **Ключ:** `xrayTemplateConfig`
- **По умолчанию:** встроенный (embedded) JSON-шаблон, поставляемый со сборкой.
- **Назначение:** базовый JSON-шаблон конфигурации Xray-core, поверх которого панель достраивает inbound/outbound. Это значение **не отдаётся** в обычной выдаче всех настроек и редактируется на отдельной странице конфигурации Xray, а не в общем списке полей настроек панели. Стандартный шаблон по умолчанию доступен через `GET /panel/setting/getDefaultJsonConfig`.

URL проверки исходящих (*xrayOutboundTestUrl*)

- **Ключ:** `xrayOutboundTestUrl`
- **По умолчанию:** `https://www.google.com/generate_204`
- **Назначение:** URL, используемый при проверке работоспособности исходящих (outbound) подключений. При установке проходит санитизацию как HTTP(S)-URL.

13.9. Учётная запись администратора и API-токены

Эти параметры находятся на смежной вкладке («Учётная запись» / *Authentication*) и подробно рассмотрены в разделе о безопасности; здесь — краткая сводка ключей.

- **Смена учётных данных** (поля «Текущий логин», «Текущий пароль», «Новый логин», «Новый пароль») сохраняется отдельным запросом `POST /panel/setting/updateUser`. Требуется верный текущий логин и пароль; новые логин и пароль не должны быть пустыми. Сообщения: «Вы успешно изменили учётные данные администратора.» / «Неверное имя пользователя или пароль».
- **Двухфакторная аутентификация (2FA)** — ключи `twoFactorEnable` (по умолчанию `false`) и секретный `twoFactorToken`. Токен — секрет: при включённом 2FA пустое поле при сохранении не

затирает существующий токен. При **первом** включении 2FA панель инвалидирует текущие сессии (повышается «эпоха входа»).

- **API-токены** управляются отдельными эндпоинтами (`/panel/setting/apiTokens...`): список, создание (`apiTokens/create`), удаление, включение/выключение. Сам токен показывается **только один раз при создании** и не хранится в читаемом виде: «Скопируйте этот токен сейчас. В целях безопасности он не хранится в читаемом виде и больше не будет показан.»

Подробности по 2FA, паролям, LDAP-синхронизации и форматам подписки (JSON/Clash, fragmentation, noises, mux) вынесены в соответствующие отдельные разделы руководства.

13.10. Изменения API в 3.3.0 (важно для интеграций)

В версии 3.3.0 изменилась структура путей серверного API. Если у вас есть внешние интеграции (скрипты, боты, центральные панели, CI-задачи), которые обращаются к панели по HTTP, их **нужно поправить**, иначе они перестанут работать.

⚠ BREAKING CHANGE: эндпоинты `/panel/setting/*` и `/panel/xray/*` переехали под `/panel/api`

Раньше управление настройками панели и конфигурацией Xray жило отдельно, под путями `/panel/setting/*` и `/panel/xray/*`. Теперь оба набора зарегистрированы внутри общей API-группы `/panel/api`. Старые пути **удалены полностью** — запрос по ним вернёт 404.

Зачем это сделано: вся группа `/panel/api` проходит через единую проверку доступа, то есть эти эндпоинты теперь принимают тот же заголовок `Authorization: Bearer <token>`, что и остальной API. API-токен — это полноценный администраторский доступ, и таким образом вся поверхность API стала единообразной.

Что НЕ изменилось: страницы веб-интерфейса (SPA-маршруты) `/panel/settings` и `/panel/xray` остались на месте — речь только о серверных API-эндпоинтах.

Таблица соответствия путей (старый → новый)

Префикс ко всем путям ниже — просто добавилось `api/` после `/panel/`.

Было (≤ 3.2.x)	Стало (3.3.0)	Метод
<code>/panel/setting/all</code>	<code>/panel/api/setting/all</code>	POST
<code>/panel/setting/defaultSettings</code>	<code>/panel/api/setting/defaultSettings</code>	POST
<code>/panel/setting/update</code>	<code>/panel/api/setting/update</code>	POST
<code>/panel/setting/updateUser</code>	<code>/panel/api/setting/updateUser</code>	POST
<code>/panel/setting/restartPanel</code>	<code>/panel/api/setting/restartPanel</code>	POST
<code>/panel/setting/getDefaultJsonConfig</code>	<code>/panel/api/setting/getDefaultJsonConfig</code>	GET
<code>/panel/setting/apiTokens</code>	<code>/panel/api/setting/apiTokens</code>	GET
<code>/panel/setting/apiTokens/create</code>	<code>/panel/api/setting/apiTokens/create</code>	POST

Было (≤ 3.2.x)	Стало (3.3.0)	Метод
/panel/setting/apiTokens/delete/:id	/panel/api/setting/apiTokens/delete/:id	POST
/panel/setting/apiTokens/setEnabled/:id	/panel/api/setting/apiTokens/setEnabled/:id	POST
/panel/xray/	/panel/api/xray/	POST
/panel/xray/update	/panel/api/xray/update	POST
/panel/xray/getDefaultJsonConfig	/panel/api/xray/getDefaultJsonConfig	GET
/panel/xray/getXrayResult	/panel/api/xray/getXrayResult	GET
/panel/xray/getOutboundsTraffic	/panel/api/xray/getOutboundsTraffic	GET
/panel/xray/resetOutboundsTraffic	/panel/api/xray/resetOutboundsTraffic	POST
/panel/xray/testOutbound	/panel/api/xray/testOutbound	POST
/panel/xray/warp/:action	/panel/api/xray/warp/:action	POST
/panel/xray/nord/:action	/panel/api/xray/nord/:action	POST
/panel/xray/outbound-subs (и /outbound-subs/*)	/panel/api/xray/outbound-subs (и /outbound-subs/*)	GET/POST/DELETE

Сами имена под-путей, тела запросов и форматы ответов не менялись — изменился **только префикс**.

Как починить существующие интеграции

1. Найдите в своих скриптах/конфигах все вхождения `/panel/setting/` и `/panel/xray/`.
2. Замените префикс: добавьте `api/` сразу после `/panel/` (например, `/panel/setting/all` → `/panel/api/setting/all`).
3. Тела запросов, параметры и формат ответов править не нужно — меняется только URL.
4. Поскольку настройки и Xray-конфигурация теперь под `/panel/api`, к ним можно (и нужно) обращаться по тому же API-токену `Authorization: Bearer <token>`, что и к `/panel/api/inbounds/*` и прочим эндпоинтам. Не забывайте про CSRF-middleware, которое включено на всю группу `/panel/api`.

Пример: чтение всех настроек через API. Раньше (≤ 3.2.x):

```
curl -sk -X POST "https://panel.example.com:2053/MyPath/panel/setting/all" \
-H "Authorization: Bearer <token>"
```

Теперь (3.3.0) — добавили `api/` после `/panel/`:

```
curl -sk -X POST "https://panel.example.com:2053/MyPath/panel/api/setting/all" \
-H "Authorization: Bearer <token>"
```

Аналогично перезапуск панели: `POST /panel/api/setting/restartPanel`. Старый путь `/panel/setting/restartPanel` теперь вернёт 404.

Типизированный API: схемы и документация (Swagger / OpenAPI)

В 3.3.0 OpenAPI-спецификация стала полноценно типизированной. Раньше типизированные ответы описывались пустым объектом `{}`; теперь компоненты и схемы (`components.schemas`) генерируются прямо из моделей данных. Благодаря этому:

- Swagger UI показывает реальные модели данных, а не безликие заглушки.
- Внешние генераторы (`openapi-generator` и т. п.) могут построить по спецификации готовые клиенты на нужном языке.
- К каждому типизированному ответу прикреплён `$ref` на конкретную модель и приложены примеры ответов.

Где смотреть документацию API:

- **Встроенная страница Swagger.** В меню панели — пункт «Документация API» (SPA-маршрут `/panel/api-docs`). Здесь интерактивно перечислены все эндпоинты с описаниями, телами запросов и примерами ответов.
 - **Сырая OpenAPI 3.0-спецификация** отдаётся по адресу `/panel/api/openapi.json`. Этот URL можно скормить напрямую в Postman, Insomnia или `openapi-generator`. Спецификация встроена в бинарник на этапе сборки; при работе панели под нестандартным `webBasePath` поле `servers` в спецификации автоматически переписывается под текущий базовый путь, чтобы кнопка «Try it out» и внешние генераторы били по правильному префиксу.
-

14. Telegram-бот

Панель 3X-UI содержит встроенного Telegram-бота, через которого можно получать уведомления о состоянии сервера и клиентов, а также управлять отдельными клиентами прямо из мессенджера. Бот работает по технологии long polling (постоянный опрос Telegram), поэтому ему не требуется внешний домен или открытый порт — достаточно исходящего доступа к серверам Telegram.

Бот различает два типа собеседников:

- **Администратор** — пользователь, чей Telegram User ID указан в настройках бота (поле «User ID администратора бота»). Имеет доступ ко всем функциям: статистике сервера, бэкапу, управлению клиентами, перезапуску Xray.
- **Клиент** — любой другой пользователь, чей Telegram User ID привязан к конкретному клиенту входящего подключения (поле `tgId` клиента). Видит только информацию о собственных подписках.

Пример: привязка клиента к Telegram. Чтобы пользователь получал статистику по своей подписке, его числовой Telegram User ID записывается в поле `tgId` клиента. В JSON-настройках клиента это выглядит так:

```
{
  "email": "ivan",
  "id": "6f1e6b1a-0c3d-4f2a-9b7e-1a2b3c4d5e6f",
  "tgId": "123456789",
  "enable": true,
  "limitIp": 2,
  "totalGB": 53687091200,
  "expiryTime": 0
}
```

После этого пользователь с User ID `123456789` сможет запросить у бота `/usage ivan` и увидеть свою статистику. Тот же ID администратор может проставить через кнопку « Установить пользователя Telegram» в карточке клиента — вручную в JSON лезть не обязательно.

14.1. Включение и настройка бота

Все параметры бота задаются в панели в разделе **Настройки** → **Telegram-бот**. После изменения настроек достаточно их сохранить — панель применяет их сразу, перезапуск панели не требуется. Если изменены флаг включения (`tgBotEnable`), токен, User ID администраторов или адрес API-сервера, панель автоматически останавливает и заново запускает бота с новыми параметрами. Прежнее правило о необходимости перезапуска панели после смены токена больше не действует.

Поле (UI)	Ключ настройки	Значение по умолчанию	Описание
Включить Telegram бота	<code>tgBotEnable</code>	<code>false</code>	Главный переключатель. Подсказка: «Доступ к функциям панели через Telegram-бота». Пока выключен, бот не запускается и задания уведомлений не планируются.

Поле (UI)	Ключ настройки	Значение по умолчанию	Описание
Telegram-токен	tgBotToken	(пусто)	Токен бота. Подсказка: «Необходимо получить токен у менеджера ботов Telegram @botfather». Без непустого токена бот не стартует.
SOCKS-прокси	tgBotProxy	(пусто)	Прокси для подключения к Telegram. Подсказка (RU): «Если для подключения к Telegram вам нужен прокси Socks5, настройте его параметры согласно руководству».
Telegram API Server	tgBotAPIServer	(пусто)	Альтернативный API-сервер Telegram. Подсказка: «Используемый API-сервер Telegram. Оставьте пустым, чтобы использовать сервер по умолчанию».
User ID администратора бота	tgBotChatId	(пусто)	Один или несколько Telegram User ID администраторов через запятую. Подсказка: «Для получения User ID используйте @userinfobot или команду /id в боте».
Частота уведомлений для администраторов от бота	tgRunTime	@daily	Расписание периодического отчёта в формате crontab. Подсказка: «Укажите интервал уведомлений в формате Crontab».
Резервное копирование базы данных	tgBotBackup	false	Подсказка: «Отправлять уведомление с файлом резервной копии базы данных». Прикладывает бэкап к периодическому отчёту.
Уведомление о входе	tgBotLoginNotify	true	Подсказка: «Отображает имя пользователя, IP-адрес и время, когда кто-то пытается войти в вашу панель».
Порог нагрузки на ЦП для уведомления	tgCpu	80	Порог загрузки CPU в процентах (валидация 0–100). Подсказка: «Уведомление администраторов в Telegram, если нагрузка на ЦП превышает этот порог (значение: %)». При значении 0 проверка CPU отключена.
Язык Telegram-бота	—	—	Язык, на котором бот формирует все сообщения.

Получение токена через @BotFather

1. Откройте в Telegram диалог с **@BotFather**.
2. Отправьте команду `/newbot` и следуйте инструкциям (имя бота и уникальное `username`, оканчивающееся на `bot`).
3. BotFather выдаст токен вида `123456789:AA...`. Скопируйте его в поле **Telegram-токен**.

Получение User ID администратора

User ID — это числовой идентификатор аккаунта (не username). Узнать его можно двумя способами:

- Написать боту **@userinfobot**.
- Запустить уже настроенного бота и отправить ему команду `/id` — он вернёт ваш ID.

Полученное число впишите в поле **User ID администратора бота**. Чтобы назначить нескольких администраторов, перечислите их ID через запятую (например, 11111111,22222222). Каждый ID проверяется как целое число; некорректное значение приведёт к ошибке запуска бота.

Пример: значение поля «User ID администратора бота». Один администратор — просто число:

123456789

Два администратора через запятую (пробелы можно не ставить):

123456789,987654321

Каждое значение должно быть целым числом. Запись вида @username или 123 456 (с пробелом внутри числа) не подойдёт — бот не запустится.

Прокси

Поддерживаются схемы socks5:// , http:// и https:// . Если поле прокси оставлено пустым, бот пытается использовать общий прокси панели (если он задан и его схема поддерживается). URL с неподдерживаемой схемой или некорректным синтаксисом игнорируется — бот соединяется напрямую. Прокси полезен, когда прямой доступ к API Telegram с сервера заблокирован.

Уведомления по электронной почте (SMTP)

Помимо Telegram, те же события можно получать письмами. Канал настраивается в разделе **Настройки** → **Email** на вкладке **SMTP Settings**:

Поле (UI)	Ключ настройки	Значение по умолчанию	Описание
Enable Email Notifications	smtpEnable	false	Главный переключатель email-уведомлений через SMTP.
SMTP Host	smtpHost	(пусто)	Хост SMTP-сервера (например, smtp.gmail.com).
SMTP Port	smtpPort	587	Порт SMTP-сервера.
SMTP Username	smtpUsername	(пусто)	Имя пользователя для аутентификации SMTP. Используется и как адрес отправителя (From).
SMTP Password	smtpPassword	(пусто)	Пароль для аутентификации SMTP. Хранится скрыто; если пароль уже задан, поле показывает признак «настроен», и его можно оставить пустым, чтобы сохранить текущий.
Recipients	smtpTo	(пусто)	Список получателей через запятую (например, admin@example.com, ops@example.com).
Encryption	smtpEncryptionType	starttls	Тип шифрования соединения: none (без шифрования), starttls (STARTTLS) или tls (неявный TLS).

Кнопка **Send Test Email** отправляет пробное письмо и показывает результат по этапам: **Connection** (подключение), **Authentication** (аутентификация) и **Send** (отправка). Если что-то не так, диагностика указывает, на каком этапе возникла ошибка (например, «Authentication failed — check username and password» или «Server requires STARTTLS — change encryption type»), что упрощает подбор параметров.

На второй вкладке (**Notifications**) выбираются события, о которых будут приходить письма, — теми же карточками-группами, что и для Telegram (см. «Шина событий и выбор уведомлений» в разделе 14.5).

API-сервер Telegram

По умолчанию бот обращается к официальному API Telegram. В поле **Telegram API Server** можно указать адрес собственного Bot API сервера (`telegram-bot-api`). URL проверяется на безопасность; заблокированный или некорректный адрес отбрасывается, и используется сервер по умолчанию.

14.2. Главное меню и кнопки

Меню вызывается командой `/start`. Кнопки — это inline-клавиатура, прикреплённая к сообщению; набор кнопок зависит от того, администратор вы или клиент.

Меню администратора

Кнопка	Действие
 Отсортированный отчёт об использовании трафика	Перечисляет всех клиентов, отсортированных по трафику, с расходом каждого; «лишние» email без данных помечаются «  Нет результатов».
 Состояние сервера	Сводка по серверу (см. раздел 14.5). Кнопка «  Обновить» перерисовывает данные.
Сбросить весь трафик	Сбрасывает счётчики трафика всех клиентов. Запрашивает подтверждение («Вы уверены?  »), затем по каждому клиенту выводит «  Успешно» либо «  Неудача», в конце — «  Сброс трафика завершён для всех клиентов».
 Бэкап БД	Высылает файл базы данных и <code>config.json</code> (см. раздел 14.6).
 Лог банов	Высылает файлы лог-файлов забаненных по IP-лимиту адресов.
 Входящие подключения	Сводка по всем inbound: Remark, порт, трафик, число клиентов, дата окончания.
 Скоро конец	Список inbound и клиентов, у которых скоро исчерпается трафик или срок (см. раздел 14.5).
 Команды	Выводит справку по командам администратора.
 Онлайн	Количество и список клиентов, находящихся онлайн; нажатие на email открывает карточку клиента. Кнопка «  Обновить».
 Все клиенты	Открывает выбор inbound, затем список его клиентов — для просмотра/управления.
 Новый клиент	Запускает мастер добавления клиента (выбор inbound → черновик → подтверждение).
Настройки подписки / индивидуальные ссылки / QR-код	Выбор inbound и клиента для получения ссылки на подписку, индивидуальных ссылок или QR-кодов.

Меню клиента

Клиенту доступен ограниченный набор кнопок:

Кнопка	Действие
Статистика клиента	Показывает данные по всем подпискам, привязанным к Telegram User ID клиента.
Команды	Выводит справку по командам клиента.
Настройки подписки	Выбор своего клиента → ссылка на подписку.
Индивидуальные ссылки	Выбор своего клиента → индивидуальные ссылки.
QR-код	Выбор своего клиента → QR-коды.

Если у пользователя нет ни одного клиента с его Telegram User ID, бот отвечает: « Ваша конфигурация не найдена!  Пожалуйста, попросите администратора использовать ваш Telegram User ID в конфигурации. Ваш User ID: ...». Этот ID нужно передать администратору, чтобы тот вписал его в поле клиента.

14.3. Команды бота

С версии 3.5.0 в меню «/» Telegram зарегистрированы **восемь** команд:

Команда	Описание (из меню)	Доступ	Что делает
/start	Показать главное меню	все	Приветствие; администратору дополнительно показывает «Добро пожаловать в бота управления!» и главное меню.
/help	Справка по боту	все	Выводит общее приветствие и предложение выбрать пункт меню.
/status	Проверить статус бота	все	Отвечает «  Бот функционирует нормально».
/id	Показать ваш Telegram ID	все	Возвращает «  Ваш User ID: ...». Удобно для получения собственного User ID.
/usage	Показать расход клиента: /usage email	все (см. ниже)	Добавлена в меню в 3.5.0.
/inbound	Поиск inbound: /inbound remark (admin)	админ	Добавлена в меню в 3.5.0.
/restart	Перезапустить ядро Xray (admin)	админ	Добавлена в меню в 3.5.0.
/clearall	Сбросить трафик всех клиентов (admin)	админ	Новая в 3.5.0: запрашивает подтверждение (кнопки «Отмена» / «Подтвердить сброс трафика») и обнуляет трафик всех клиентов.

Детали команд-аргументов:

- **/usage [Email]** — поиск клиента по email.
 - Для **администратора** показывает полную карточку клиента (с кнопками управления).

- Для **клиента** показывает только его собственную подписку с указанным email (по привязке Telegram User ID). Без аргумента бот просит указать email: « Пожалуйста, укажите email для поиска».
- **/inbound [имя подключения]** — только для администратора. Ищет inbound по Remark и выводит его параметры со статистикой всех клиентов. Без аргумента (или для клиента) — « Неизвестная команда».
- **/restart** — только для администратора. Перезапускает Xray Core. Возможные ответы: « Ядро Xray успешно перезапущено», « Xray Core не запущен» (если ядро не работает), « Ошибка при перезапуске Xray-core. <Ошибка>». Любые аргументы после **/restart** приводят к сообщению о неизвестной команде с подсказкой **/restart**.

В групповых чатах команда вида **/команда@botusername** обрабатывается, только если username совпадает с именем текущего бота.

Ещё три изменения 3.5.0: в списке **онлайн-клиентов** кнопки подписаны email – примечание inbound (одинаковые имена в разных inbound больше не путаются); сообщения **бэкапа** и **бан-лога** начинаются со строки **Hostname==...** — удобно, когда один бот обслуживает несколько панелей; поиск карточки клиента по **Telegram ID** работает независимо от форматирования JSON настроек (раньше «компактные» записи, пришедшие с узлов или из импорта, не находились).

Справка администратора (кнопка «Команды»):

 Для перезапуска Xray Core: **/restart**
 Для поиска клиента по email: **/usage [Email]**
 Для поиска входящих подключений (со статистикой клиентов): **/inbound [имя подключения]**
 Ваш Telegram User ID: **/id**

Справка клиента:

\$
 Для просмотра информации о вашей подписке: **/usage [Email]**
 Ваш Telegram User ID: **/id**

14.4. Управление клиентом (только администратор)

Открыв карточку клиента (через «Все клиенты», «Онлайн», «Скоро конец» или **/usage**), администратор видит сведения о клиенте (email, привязанные inbound, статус «Активен», статус соединения, дата окончания, расход трафика) и inline-кнопки управления:

Кнопка	Назначение
 Обновить	Перечитать карточку клиента.
 Сбросить трафик	Обнулить счётчик трафика клиента. Требуется подтверждения «  Подтвердить сброс трафика?».
 Лимит трафика	Задать лимит трафика. Готовые значения: ☹ Безлимит (0), 1/5/10/20/30/40/50/60/80/100/150/200 GB или «  Своё» — ввод числа на  встроенной цифровой клавиатуре (кнопки 0–9, «  » — сброс к 0, «  » — стереть последнюю цифру, «  Подтвердить: N»). Значение задаётся в гигабайтах.

Кнопка	Назначение
 Изменить дату окончания	Готовые варианты: ☹ Безлимит, « Своеё», добавить 7/10/14/20 дней, 1/3/6/12 месяцев. Положительное число продлевает срок (прибавляет дни к текущей дате окончания либо к «сейчас», если срок уже истёк); 0 снимает ограничение по сроку.
 Лог IP	Показывает зафиксированные IP-адреса клиента (с отметками времени, если есть). Из лога доступны « Обновить » и « Очистить IP » (с подтверждением « Подтвердить очистку IP?»).
 Лимит IP	Ограничение одновременных IP. Варианты: ☹ Безлимит (0), 1–10 или « Своеё » (цифровая клавиатура).
 Установить пользователя Telegram	Показывает текущий привязанный Telegram User ID клиента; позволяет очистить привязку (« Удалить пользователя Telegram » с подтверждением). Привязка нового пользователя выполняется через системный выбор контакта Telegram.
 Вкл./Выкл.	Включает или отключает клиента. Требуется подтверждения « Подтвердить вкл/выкл пользователя?».

Все операции, изменяющие конфигурацию (лимит трафика/IP, дата окончания, привязка/отвязка Telegram-пользователя, вкл/выкл), при необходимости помечают Xray на перезапуск, чтобы изменения вступили в силу. После успешной операции бот выводит подтверждение вида «  : ... » и заново показывает карточку.

Любой ввод цифр в мастерах ограничен значением < 999999.

14.5. Уведомления и отчёты

Уведомления отправляются всем администраторам (всем User ID из `tgBotChatId`).

Шина событий и выбор уведомлений

Уведомления построены на единой шине событий, а каналов доставки два — **Telegram** и **электронная почта (SMTP)**. Для каждого канала отдельно выбирается, о каких событиях оповещать. В **Настройки** → **Telegram** это делается на вкладке **Notifications**, в **Настройки** → **Email** — на одноимённой вкладке.

События сгруппированы карточками; у каждой группы есть мастер-переключатель со счётчиком включённых событий (n/всего) и промежуточным состоянием, когда выбрана только часть. Доступны группы:

- **Outbound** — «Down» (`outbound.down`) и «Up» (`outbound.up`): падение и восстановление outbound.
- **Xray Core** — «Crash» (`xray.crash`): аварийное завершение ядра Xray.
- **Nodes** — «Down» (`node.down`) и «Up» (`node.up`): узел стал недоступен или восстановился.
- **System** — «CPU high (%)» (`cpu.high`) и «Memory high (%)» (`memory.high`): высокая загрузка процессора и оперативной памяти. У обоих событий рядом находится инлайн-поле порога в процентах.
- **Security** — «Login attempt» (`login.attempt`): попытка входа в панель.

Набор включённых событий хранится отдельно: для Telegram — в `tgEnabledEvents` , для Email — в `smtpEnabledEvents` . По умолчанию в обоих каналах включены «Login attempt» и «CPU high» (значение `login.attempt,cpu.high`).

Уведомление о входе в панель

Управляется галочкой **Уведомление о входе** (`tgBotLoginNotify` , по умолчанию включено). При каждой попытке входа в веб-панель администраторам приходит сообщение:

- При успехе: «  Успешный вход в панель.» + хост, имя пользователя, IP, время.
- При неудаче: «  Ошибка входа в панель.» + хост, **причина** (например, «Ошибка 2FA» при неверном втором факторе), имя пользователя, IP, время.

Превышение нагрузки на CPU и память

Раз в минуту панель проверяет загрузку процессора и оперативной памяти. Если порог `tgCpu` > 0 и средняя загрузка CPU за минуту его превышает, администраторам уходит: «  Загрузка процессора составляет N%, что превышает пороговое значение M%». Аналогично проверяется загрузка RAM против порога `tgMemory` (по умолчанию 80%) — событие «Memory high (%)».

Оба порога задаются инлайн-полями рядом с событиями «CPU high (%)» и «Memory high (%)» в группе **System** на вкладке Notifications (см. «Шина событий и выбор уведомлений» ниже). Для канала Email действуют отдельные ключи `smtpCpu` и `smtpMemory` . При значении порога 0 соответствующая проверка отключена.

Периодический отчёт (по расписанию)

Планируется по cron-выражению из поля **Частота уведомлений** (`tgRunTime` , по умолчанию `@daily`). Если значение пустое или некорректное, используется `@daily` . Отчёт включает:

Конструктор расписания

Поле **Частота уведомлений для администраторов от бота** задаётся не вводом строки вручную, а через конструктор расписания. Сначала в выпадающем списке выбирается режим:

- **@every** — **повторять с интервалом** — появляются поле числа и выбор единицы (**Секунды / Минуты / Часы**); итог собирается в выражение вида `@every 6h` .
- **@hourly** — **каждый час**, **@daily** — **каждый день в 00:00**, **@weekly** — **каждую неделю**, **@monthly** — **каждый месяц** — готовые пресеты, которые сохраняются как соответствующий макрос (`@hourly` , `@daily` , `@weekly` , `@monthly`).
- **Произвольный (crontab)** — поле для собственного crontab-выражения. Планировщик панели работает с включёнными секундами, поэтому произвольное выражение состоит из **6 полей**: секунда, минута, час, день месяца, месяц, день недели (например, `0 30 8 * * *` — каждый день в 08:30:00). При переключении на этот режим поле заполняется crontab-эквивалентом текущего выбора, чтобы было от чего отталкиваться.

Пример: значения поля «Частота уведомлений» (`tgRunTime`). Поддерживаются как готовые сокращения, так и полный crontab-формат:

Значение	Когда срабатывает
<code>@daily</code>	раз в сутки в полночь (значение по умолчанию)

Значение	Когда срабатывает
@hourly	каждый час
@every 6h	каждые 6 часов
0 9 * * *	каждый день в 09:00
0 9 * * 1	каждый понедельник в 09:00
0 */12 * * *	каждые 12 часов (в 00:00 и 12:00)

Порядок полей в crontab: минута, час, день месяца, месяц, день недели.

1. Строку « Запланированные отчёты: <расписание>» и текущие дату/время.
2. **Состояние сервера** (см. ниже).
3. Блок «Скоро конец» по inbound и клиентам.
4. Персональные уведомления клиентам с привязанным Telegram User ID — каждому клиенту-неадмину уходит список его подписок, у которых скоро исчерпание трафика/срока (с учётом отключённых).
5. Если включено **Резервное копирование базы данных** (`tgBotBackup`) — бэкап БД администраторам.

Состояние сервера содержит: хост, версию 3X-UI и Xray, IPv4/IPv6, время работы (в днях), среднюю нагрузку (Load1/2/3), RAM (текущая/всего), число онлайн-клиентов, счётчики TCP/UDP-соединений, суммарный сетевой трафик (↑/↓) и статус Xray.

«Скоро конец» показывает:

- по inbound: число отключённых и число «скоро исчерпывающихся», затем перечисление таких inbound (Remark, порт, трафик, дата окончания);
- по клиентам: то же, плюс карточки клиентов и кнопки с их email (нажатие открывает карточку клиента).

Пороги «скоро исчерпания» берутся из общих настроек панели: запас по трафику (в ГБ) и запас по сроку (в днях). «Исчерпывающимся» считается inbound/клиент, у которого до лимита трафика осталось меньше порога ИЛИ до даты окончания осталось меньше порога.

14.6. Бэкап и логи

- **Бэкап БД** (кнопка « Бэкап БД» или галочка в периодическом отчёте): бот присылает время бэкапа, файл базы данных (`x-ui.db` , либо `x-ui.dump` для PostgreSQL) и файл конфигурации Xray `config.json` .

Имя файла бэкапа, который присылает бот, формируется по адресу сервера: используется значение **webDomain**, а если оно не задано — публичный IP сервера. Это помогает понять, с какого сервера пришёл файл, когда бэкапы собираются с нескольких панелей. Если адрес определить не удалось, применяется обобщённое имя.

- **Лог банов** (кнопка « Лог банов»): присылает текущий и предыдущий файлы лога IP-адресов, забаненных по превышению лимита IP. Пустые файлы не отправляются.

14.7. Особенности работы

- **Длинные сообщения** разбиваются на части (порог ~2000 символов), inline-клавиатура прикрепляется к последней части.
 - **Параллельность**: команды и нажатия кнопок обрабатываются конкурентно (пул до 10 одновременных обработчиков).
 - **Надёжность отправки**: при ошибках соединения сообщения отправляются повторно с экспоненциальной задержкой (1с/2с/4с, до 3 попыток).
 - **Кэширование**: данные «Состояние сервера» кэшируются, чтобы частые нажатия «Обновить» не нагружали систему.
 - **Перезапуск бота**: при сохранении настроек, затрагивающих бота (флаг включения, токен, User ID администраторов или адрес API-сервера), панель сама останавливает предыдущий цикл опроса и запускает новый с актуальными параметрами – перезагрузка панели для этого не нужна. Одновременно работает только один экземпляр приёма обновлений.
-

15. Гео-базы (geoip / geosite и кастомные)

Гео-базы — это бинарные файлы `.dat`, которые Xray-core использует для маршрутизации и фильтрации трафика по принадлежности к стране (IP-диапазоны) или по категории доменов. Панель умеет загружать и обновлять как стандартный набор гео-файлов, так и произвольные пользовательские источники, указанные по URL. Все файлы хранятся в каталоге `bin` рядом с бинарником Xray (путь по умолчанию `bin`, переопределяется переменной окружения `XUI_BIN_FOLDER`).

15.1. Что такое geoip.dat и geosite.dat

- **geoip.dat** — база соответствий «IP-адрес → код страны/региона». Применяется в правилах маршрутизации в виде `geoip:<код>`, например `geoip:ru`, `geoip:cn`, а также для специальных меток `geoip:private` (приватные/локальные сети). По смыслу это ответ на вопрос «в какой стране находится этот IP».
- **geosite.dat** — база соответствий «домен → категория/список». Применяется в виде `geosite:<категория>`, например `geosite:category-ads-all` (рекламные домены), `geosite:google`, `geosite:ru`. По смыслу это сгруппированные списки доменов.

Эти файлы нужны, чтобы строить правила вида «весь трафик на российские IP/домены — напрямую, остальное — через outbound» и подобные. Сами правила задаются в разделе маршрутизации Xray; гео-базы лишь поставляют для них данные. Без актуальных гео-файлов правила, ссылающиеся на `geoip: / geosite:`, не сработают или будут опираться на устаревшие списки.

Пример: правило «российские домены и IP — напрямую». Такое правило в разделе маршрутизации направляет весь трафик к российским ресурсам в outbound с тегом `direct`:

```
{
  "type": "field",
  "domain": ["geosite:category-ru"],
  "ip": ["geoip:ru"],
  "outboundTag": "direct"
}
```

15.2. Стандартные гео-файлы и их обновление

Панель содержит фиксированный «белый список» (allowlist) из шести стандартных файлов с жёстко прописанными источниками загрузки. Обновление выполняется через `POST /panel/api/server/updateGeofile/:fileName` (или без имени файла — для обновления всех сразу).

Пример: обновление одного файла и всех сразу через API. Обновить только `geoip_RU.dat`:

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/updateGeofile/
geoip_RU.dat' \
-H 'Cookie: 3x-ui=<session-cookie>'
```

Обновить все шесть стандартных файлов одним запросом (имя файла не указано):

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/updateGeofile' \
-H 'Cookie: 3x-ui=<session-cookie>'
```

Успешный ответ:

```
{ "success": true, "msg": "Geofile updated successfully", "obj": null }
```

Имя файла	Источник (репозиторий релизов)
geoip.dat	github.com/Loyalsoldier/v2ray-rules-dat (geoip.dat)
geosite.dat	github.com/Loyalsoldier/v2ray-rules-dat (geosite.dat)
geoip_IR.dat	github.com/chocolate4u/Iran-v2ray-rules (geoip.dat)
geosite_IR.dat	github.com/chocolate4u/Iran-v2ray-rules (geosite.dat)
geoip_RU.dat	github.com/runetfreedom/russia-v2ray-rules-dat (geoip.dat)
geosite_RU.dat	github.com/runetfreedom/russia-v2ray-rules-dat (geosite.dat)

Особенности обновления стандартных файлов:

- **Кнопка обновления одного файла.** Перед загрузкой выводится подтверждение: «Вы действительно хотите обновить геофайл?» с пояснением «Это обновит файл #filename#.» (англ. *Do you really want to update the geofile? This will update the #filename# file.*). При успехе всплывает уведомление «Геофайлы успешно обновлены» (англ. *Geofile updated successfully*).
- **Кнопка «Обновить все»** (англ. *Update all*) загружает все шесть файлов. Подтверждение: «Это обновит все геофайлы.» (англ. *This will update all geofiles.*).
- **Условная загрузка.** Если локальный файл уже есть, в запрос подставляется заголовок `If-Modified-Since` с временем модификации файла. Ответ сервера `304 Not Modified` означает, что файл не менялся — повторно он не скачивается, обновляется только метка времени файла.
- **Безопасность имени файла.** Принимаются только имена из allowlist; имя проверяется на отсутствие `..`, разделителей пути `/` и `\`, абсолютных путей и обязано соответствовать шаблону `^[a-zA-Z0-9._-]+\.dat$`. Любое имя вне списка отклоняется с ошибкой «Invalid geofile name».
- **Перезапуск Xray.** После загрузки гео-файлов Xray-core перезапускается, чтобы он пересчитал обновлённые базы. Если перезапуск не удался, в сообщение об ошибке добавляется соответствующая строка.

Обновление гео-баз из командной строки (x-ui)

Гео-базы можно обновлять и без панели — через интерактивное меню `x-ui` (пункт обновления гео-файлов) либо неинтерактивной командой `x-ui update-all-geofiles`. Для каждого файла набора (geoip/geosite, включая наборы IR и RU) выводится отдельный статус: «обновлён», «уже актуален» или «ошибка загрузки». При неудачной загрузке ложное сообщение об успехе не печатается. Перезапуск Xray (а значит и разрыв активных соединений) происходит только если хотя бы один файл действительно был обновлён; если ни один файл не изменился (все вернули `304 Not Modified`), панель и Xray не перезапускаются.

15.3. Авто-обновление гео-данных средствами Xray (Geodata Auto-Update)

Дополнительные `.dat` -источники по произвольному URL добавляются не средствами панели, а через нативную секцию `geodata` Xray-core. Соответствующий раздел вынесен в модальное окно обновлений Xray (Дашборд → обновления Xray, `xrayUpdates`) — это вкладка «Автообновление Geodata» (англ. *Geodata Auto-Update*). Панель здесь лишь редактирует ключ `geodata` в шаблоне конфигурации Xray; скачиванием, проверкой и горячей перезагрузкой файлов занимается само ядро Xray.

В верхней части раздела показана подсказка: «Xray скачивает эти файлы по расписанию и перезагружает их без перезапуска. URL должны быть HTTPS. Файл должен уже существовать в папке bin, прежде чем Xray сможет его обновлять.» (англ. *Xray downloads these files on schedule and hot-reloads them without a restart. URLs must be HTTPS. Each file must already exist in the bin folder once before Xray can update it.*).

Поля раздела

- **Расписание (cron)** (англ. *Schedule (cron)*) — строка cron из 5 полей; значение по умолчанию `0 4 * * *` (ежедневно в 04:00). При сохранении проверяется, что строка содержит ровно 5 полей, иначе выводится ошибка «Cron должен содержать 5 полей, напр. 0 4 * * *».
- **Скачивать через outbound (необязательно)** (англ. *Download through outbound (optional)*) — раскрывающийся список с тегами доступных outbound (плюс outbound подписок), через который Xray будет загружать файлы; outbound с протоколом `blackhole` в список не попадают. Поле можно оставить пустым — тогда используется прямое подключение. Этот выбор не зависит от outbound для собственных запросов панели (см. §11): у авто-обновления geodata свой отдельный outbound для загрузки.
- **Список файлов** — каждая строка задаёт пару «URL + Имя файла» (англ. *File name*). URL должен начинаться с `https://` (иначе «Для каждого файла нужен HTTPS URL.»). Имя файла указывается простым, без путей и разделителей — только символы `^[A-Za-z0-9._-]+$` (иначе «Имя файла должно быть простым, например geosite_custom.dat (без путей).»). При вводе URL панель пытается подставить имя файла автоматически из последнего сегмента пути. Кнопка «Добавить файл» (англ. *Add file*) добавляет строку, кнопка-корзина удаляет её.

Если список пуст, показывается подсказка: «Файлы не настроены. В правилах маршрутизации файлы указываются как `ext:geosite_custom.dat:category`.» (англ. *No files configured. Reference files in routing rules as ext:geosite_custom.dat:category.*).

Сохранение

Кнопка «Сохранить и перезапустить Xray» (англ. *Save & Restart Xray*) выводит подтверждение «Сохранить настройки geodata?» с пояснением «Шаблон конфигурации Xray будет обновлён, а Xray перезапущен.» (англ. *Save geodata settings? This updates the Xray config template and restarts Xray.*). После сохранения ключ `geodata` записывается в шаблон конфигурации (`POST /panel/api/xray/update`) и Xray перезапускается (`POST /panel/api/server/restartXrayService`). Если список файлов пуст, ключ `geodata` из шаблона удаляется.

Важные особенности:

- **Файл должен уже существовать в bin**. Xray обновляет только те `.dat`-файлы, которые на момент запуска уже присутствуют в папке `bin`. Поэтому новый кастомный файл сначала кладут в `bin` вручную (или хотя бы создают там пустую/устаревшую версию под нужным именем), и лишь затем Xray начинает поддерживать его в актуальном состоянии по расписанию.
- **Горячая перезагрузка**. После планового скачивания Xray перечитывает обновлённые базы без полного перезапуска процесса.
- **Совместимость**. Ранее скачанные гео-файлы (как стандартные, так и кастомные) продолжают работать в правилах маршрутизации с синтаксисом `ext:` без изменений.

Если список пуст, отображается подсказка: «Пользовательских источников гео пока нет — нажмите «Добавить», чтобы создать» (англ. *No custom geo sources yet — click Add to create one*).

Колонки таблицы и поля источника

Поле (UI)	JSON	Значение по умолчанию	Описание
Тип (<i>Type</i>)	<code>type</code>	— (обязательно)	Тип ресурса: только <code>geosite</code> или <code>geoip</code> . Определяет имя итогового файла.
Псевдоним (<i>Alias</i>)	<code>alias</code>	— (обязательно)	Короткий идентификатор источника. Из него и типа строится имя файла.
URL (<i>URL</i>)	<code>url</code>	— (обязательно)	Прямая ссылка на <code>.dat</code> -файл (http/https).
Включено (<i>Enabled</i>)	—	—	Признак активности источника в списке.
Обновлено (<i>Last updated</i>)	<code>lastUpdatedAt</code>	<code>0</code>	Время последнего успешного обновления (Unix-время; <code>0</code> — ещё не обновлялся).
Маршрутизация (<code>ext:...</code>) (<i>Routing (ext:...)</i>)	—	—	Готовая строка для правил маршрутизации: <code>ext:<файл.dat>:tag</code> .
Действия (<i>Actions</i>)	—	—	Кнопки «Изменить», «Удалить», «Обновить сейчас».

Дополнительно в БД хранятся служебные поля: `localPath` (фактический путь к файлу в каталоге `bin`), `lastModified` (значение заголовка `Last-Modified` от сервера, используется для условной загрузки), `createdAt` и `updatedAt`.

Именование файлов

Имя итогового файла формируется автоматически из типа и псевдонима:

- тип `geoip` → `geoip_<alias>.dat`;
- тип `geosite` → `geosite_<alias>.dat`.

Например, источник с типом `geosite` и псевдонимом `myads` создаст файл `geosite_myads.dat`.

Пример: добавление источника через API. Добавить свой список рекламных доменов как `geosite` -ресурс с псевдонимом `myads` :

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/customGeo/add' \
-H 'Cookie: 3x-ui=<session-cookie>' \
-H 'Content-Type: application/json' \
-d '{
  "type": "geosite",
  "alias": "myads",
  "url": "https://example.com/lists/myads.dat"
}'
```

Панель скачает файл в каталог `bin` как `geosite_myads.dat` , сохранит запись и перезапустит Xray.

Кнопки и действия

- **Добавить** (англ. *Add*) — открывает форму «Добавить источник» (англ. *Add custom geo*). Кнопка сохранения — «Сохранить» (англ. *Save*). API: `POST /add` .
- **Изменить** (англ. *Edit*) — форма «Изменить источник» (англ. *Edit custom geo*). API: `POST /update/:id` . При смене типа или псевдонима старый файл удаляется, новый скачивается заново.
- **Удалить** (англ. *Delete*) — подтверждение «Удалить этот пользовательский источник?» (англ. *Delete this custom geo source?*). Удаляет запись из БД и сам `.dat` -файл. API: `POST /delete/:id` . При успехе: «Пользовательский гео-файл «<имя>» удалён».
- **Обновить сейчас** (англ. *Update now*) — перекачивает конкретный источник и обновляет метку времени. API: `POST /download/:id` . При успехе: «Geofile «<имя>» обновлён».
- **Обновить все** — обновляет все пользовательские источники сразу. API: `POST /update-all` . При полном успехе: «Все пользовательские источники обновлены» (англ. *All custom geo sources updated*). Если хотя бы один источник не обновился, операция считается частично неуспешной с сообщением «Не удалось обновить один или несколько пользовательских источников» (англ. *One or more custom geo sources failed to update*), а в ответе перечисляются успешные и неуспешные источники.

После любого из действий (добавление, изменение, удаление, обновление, обновление всех при наличии успехов) Xray-core перезапускается.

Пошагово: добавление источника

1. Нажмите «Добавить».
2. В поле «Тип» выберите `geosite` или `geoip` .
3. В поле «Псевдоним» введите идентификатор (только строчные латинские буквы, цифры, `-` и `_` ; подсказка-плейсхолдер: `a-z 0-9 _ -`).
4. В поле «URL» укажите прямую ссылку на `.dat` -файл (должна начинаться с `http://` или `https://`).
5. Нажмите «Сохранить». Панель сразу скачает файл в каталог `bin` , сохранит запись и перезапустит Xray.

15.4. Валидация и ограничения

При создании и изменении источника выполняются строгие проверки. Сообщения об ошибках:

Условие	Сообщение (RU)	Сообщение (EN)
Тип не <code>geosite / geoip</code>	Тип должен быть <code>geosite</code> или <code>geoip</code>	<i>Type must be geosite or geoip</i>
Пустой псевдоним	Укажите псевдоним	<i>Alias is required</i>
Недопустимые символы в псевдониме (не <code>^[a-z0-9_-]+\$</code>)	Псевдоним содержит недопустимые символы	<i>Alias must match allowed characters</i>
Псевдоним зарезервирован	Этот псевдоним зарезервирован	<i>This alias is reserved</i>
Пустой URL	Укажите URL	<i>URL is required</i>
URL не парсится	Некорректный URL	<i>URL is invalid</i>
Схема не <code>http/https</code>	URL должен использовать <code>http</code> или <code>https</code>	<i>URL must use http or https</i>
Пустой/некорректный хост, либо заблокирован SSRF-защитой	Некорректный хост URL	<i>URL host is invalid</i>
Дубликат «тип + псевдоним»	Такой псевдоним уже используется для этого типа	<i>This alias is already used for this type</i>
Источник не найден	Источник не найден	<i>Custom geo source not found</i>
Ошибка скачивания	Ошибка загрузки	<i>Download failed</i>

Подсказки в форме (валидация на клиенте): «Псевдоним: только a-z, цифры, - и _» (*Alias may only contain lowercase letters, digits, - and _*) и «URL должен начинаться с `http://` или `https://`» (*URL must start with http:// or https://*).

Дополнительные технические ограничения:

- **Зарезервированные псевдонимы.** Нельзя использовать псевдонимы, конфликтующие со стандартными файлами. Зарезервированы (сравнение регистронезависимое, дефис приравнивается к подчёркиванию): `geoip`, `geosite`, `geoip_ir`, `geosite_ir`, `geoip_ru`, `geosite_ru`. Например, `geosite-ru` будет отклонён как `geosite_ru`.
- **SSRF-защита.** Хост URL резолвится в IP, и если он указывает на приватный/внутренний адрес, загрузка блокируется (пользователь видит «Некорректный хост URL»). Это не даёт использовать панель для обращений к внутренним сервисам.
- **Защита от path traversal.** Конечный путь файла обязан находиться внутри каталога `bin` (с разрешением симлинков); попытка выйти за его пределы отклоняется.
- **Минимальный размер файла.** Скачанный файл считается валидным только если он не меньше 64 байт; слишком маленький файл отвергается с ошибкой загрузки.
- **Прокси и условная загрузка.** Если в настройках панели задан прокси, скачивание идёт через него; в остальных случаях используется прямое соединение с SSRF-безопасным транспортом. Как и для стандартных файлов, применяется `If-Modified-Since / 304 Not Modified` (повторно неизменённый файл не качается). Таймаут загрузки — 10 минут, проба доступности URL (HEAD, при необходимости — частичный GET) — 12 секунд.

15.5. Автопроверка при старте панели

При запуске панель проходит по всем пользовательским источникам и для каждого проверяет наличие и целостность локального файла (файл отсутствует, является каталогом или меньше 64 байт). Если файл отсутствует или повреждён, выполняется проба источника и попытка повторной загрузки. Это гарантирует, что после переустановки или потери каталога `bin` пользовательские гео-файлы будут восстановлены автоматически.

15.6. Использование гео-баз в правилах маршрутизации

В правилах маршрутизации Xray гео-базы используются в полях вроде `domain / ip` через префиксы:

- **geoip**: для IP-баз — `geoip:<код>`. Примеры: `geoip:ru`, `geoip:cn`, `geoip:private`. Берётся из `geoip.dat` (или `geoip_RU.dat` и т.п., если правило указывает на конкретный файл).
- **geosite**: для доменных баз — `geosite:<категория>`. Примеры: `geosite:category-ads-all`, `geosite:google`, `geosite:ru`. Берётся из `geosite.dat`.

Пример: блокировка рекламы через geosite. Правило, отправляющее все рекламные домены в «чёрную дыру» (предполагается outbound с тегом `blocked` и протоколом `blackhole`):

```
{
  "type": "field",
  "domain": ["geosite:category-ads-all"],
  "outboundTag": "blocked"
}
```

Для **пользовательских** файлов используется синтаксис внешнего файла `ext:`. Подсказка в UI: «В правилах маршрутизации используйте значение как `ext:файл.dat:тег` (замените `тег`).» (англ. *In routing rules use the value column as `ext:file.dat:tag` (replace tag).*). Формат:

```
ext:<имя_файла.dat>:<тег>
```

где `<имя_файла.dat>` — это `geoip_<alias>.dat` или `geosite_<alias>.dat`, а `<тег>` — конкретный список/категория внутри файла. Панель в колонке «Маршрутизация (ext:...)» подсказывает готовый шаблон вида `ext:geosite_myads.dat:tag` — остаётся заменить `tag` на нужный тег. Имя такого файла задаётся в разделе «Автообновление Geodata» (см. §15.3) в поле «Имя файла» — например `geosite_custom.dat`; на него ссылаются в правилах как `ext:geosite_custom.dat:category`.

Пример: правило на основе пользовательского файла. Если добавлен источник типа `geosite` с псевдонимом `myads`, а внутри `.dat`-файла список помечен тегом `ads`, правило маршрутизации выглядит так:

```
{
  "type": "field",
  "domain": ["ext:geosite_myads.dat:ads"],
  "outboundTag": "blocked"
}
```

Для IP-источника (тип `geoip`, псевдоним `mycorp`, тег `office`) поле будет `"ip":`
`["ext:geoip_mycorp.dat:office"]`.

16. Эксплуатация: бэкапы, логи, обновление, CLI

Этот раздел охватывает повседневное обслуживание панели: создание и восстановление резервных копий базы данных, просмотр журналов (логов) панели и Xray, перезапуск и остановку сервисов, обновление Xray и самой панели, периодические задачи (cron) и удаление панели. Часть операций выполняется из веб-интерфейса (вкладки на странице «Дашборд» и «Настройки панели»), часть — из консольного меню `x-ui` на сервере.

16.1. Резервное копирование и восстановление базы данных

Все данные панели (входящие/inbound, клиенты, группы, узлы, настройки) хранятся в одной базе. Управление бэкапами доступно на странице «Дашборд» во вкладке «Резервная копия», заголовок блока — «Бэкап и восстановление».

Панель поддерживает два движка БД, и поведение бэкапа от этого зависит:

- **SQLite** (вариант по умолчанию) — данные лежат в файле `x-ui.db`.
- **PostgreSQL** — если панель настроена на PostgreSQL, в блоке отображается подсказка:

«Эта панель работает на PostgreSQL. „Резервная копия“ скачивает архив `pg_dump (.dump)`, а „Восстановление“ загружает его обратно через `pg_restore`. Восстановление также принимает базу SQLite (.db) или миграционный дамп SQLite и импортирует их данные в PostgreSQL. На сервере должны быть установлены клиентские инструменты PostgreSQL (`pg_dump` и `pg_restore`).»

Экспорт (создание копии)

Кнопка «Экспорт базы данных» (англ. `Back Up`) скачивает файл резервной копии на ваше устройство.

Движок БД	Имя файла	Что происходит на сервере
SQLite	<code>{host}_ГГГГ-ММ-ДД_ЧЧММСС.db</code>	Сначала выполняется checkpoint WAL, чтобы файл содержал последние записи, затем файл целиком читается и отдаётся на скачивание
PostgreSQL	<code>{host}_ГГГГ-ММ-ДД_ЧЧММСС.dump</code>	Запускается <code>pg_dump</code> , архив отдаётся на скачивание

С версии 3.4.2 имя скачиваемого файла резервной копии содержит адрес, по которому вы открыли панель, и **дату-время**: `{host}_ГГГГ-ММ-ДД_ЧЧММСС` с расширением `.db` (SQLite) или `.dump` (PostgreSQL) — например `panel.example.com_2026-06-27_000000.db`. Так файлы группируются по серверу и сортируются по времени, а несколько копий за один день не перезаписывают друг друга. То же имя используется в бэкапах, которые присылает бот Telegram (`host` берётся из домена/IP панели, запасной вариант — `x-ui`).

Подсказки в интерфейсе:

- SQLite: «Нажмите, чтобы скачать файл .db, содержащий резервную копию вашей текущей базы данных на ваше устройство.»
- PostgreSQL: «Нажмите, чтобы скачать дамп PostgreSQL (.dump) текущей базы данных на ваше устройство.»

Технически экспорт — это запрос `GET /panel/api/server/getDb`. Имя вложения формируется сервером (Content-Disposition) в зависимости от движка.

Имя файла резервной копии формируется по адресу сервера, а не фиксированным `x-ui.db` / `x-ui.dump`. При скачивании из браузера оно берётся из адреса панели в адресной строке (хост запроса), иначе — из настроенного веб-домена, а при его отсутствии — из публичного IP сервера (сначала IPv4, затем IPv6), с откатом на `x-ui`. Так бэкапы с разных серверов легко различать. Расширение остаётся `.db` для SQLite и `.dump` для PostgreSQL; бэкапы через Telegram именуются по тому же домену/IP.

Пример: скачивание бэкапа через API. Тот же экспорт можно получить запросом из консоли — например, для скрипта автоматического резервного копирования. Нужна авторизованная сессия (cookie входа):

```
# 1) Логинимся и сохраняем cookie сессии
curl -s -c cookies.txt \
  -d 'username=admin&password=admin' \
  https://panel.example.com:2053/panel/login

# 2) Скачиваем файл базы (имя задаёт сервер: x-ui.db или x-ui.dump)
curl -s -b cookies.txt -OJ \
  https://panel.example.com:2053/panel/api/server/getDb
```

Если панель открыта по базовому пути (Web Base Path), его нужно добавить в URL: `...:2053/<base_path>/panel/api/server/getDb`.

Импорт (восстановление)

Кнопка **«Импорт базы данных»** (англ. `Restore`) открывает выбор файла и загружает его на сервер для восстановления (POST `/panel/api/server/importDB`, поле формы `db`).

Подсказки в интерфейсе:

- SQLite: «Нажмите, чтобы выбрать и загрузить файл .db или миграционный дамп (.dump) с вашего устройства для восстановления базы данных.»
- PostgreSQL: «Нажмите, чтобы выбрать и загрузить бэкап PostgreSQL (.dump), базу SQLite (.db) или миграционный дамп SQLite для восстановления базы данных. Это заменит все текущие данные.»

С 3.5.0 тип файла определяется по содержимому (PGDMP — архив `pg_dump`; заголовок «SQLite format 3» — база `.db`; ведущие `PRAGMA / BEGIN TRANSACTION` — текстовый SQL-дамп), а не по расширению; диалог выбора принимает `.dump`, `.db` на обеих СУБД. SQLite-файлы, загруженные на PostgreSQL-панель, импортируются в PostgreSQL одной транзакцией (см. 3.14); перед импортом файл проверяется как настоящая панельная БД, старые схемы автоматически домигрируются. Перед восстановлением `pg_dump`-архива панель заранее (до остановки Xray) проверяет его читаемость: если дамп создан более новым PostgreSQL, выводится ошибка с точной командой — например, «...run 'x-ui pgclient 17'...».

Процесс импорта для SQLite (важно понимать, что он атомарный и с откатом):

1. Загруженный файл проверяется на формат — это должна быть валидная база SQLite; иначе возвращается ошибка «Invalid db file format».
2. Файл сохраняется во временный `x-ui.db.temp` и проходит проверку целостности.
3. **Xray останавливается** перед заменой БД.
4. Текущая база переименовывается в резервную `x-ui.db.backup` (fallback).
5. Временный файл перемещается на место рабочей БД, выполняется инициализация и миграции схемы, затем миграция inbound.
6. **Если любой шаг завершился ошибкой** — выполняется откат: восстанавливается прежняя база из `x-ui.db.backup`, и Xray перезапускается на старых данных.
7. При успехе fallback-файл удаляется, и **Xray автоматически перезапускается** уже на восстановленных данных.

Сообщения интерфейса по итогу:

Результат	Текст
Успех	«База данных успешно импортирована»
Ошибка импорта	«Произошла ошибка при импорте базы данных»
Ошибка чтения файла	«Произошла ошибка при чтении базы данных»

Восстановление полностью заменяет текущие данные. Поскольку Xray в процессе кратковременно останавливается, существующие подключения клиентов на время импорта прерываются.

Файл миграции между движками (SQLite ⇌ PostgreSQL)

Отдельно от обычного бэкапа есть функция **«Скачать файл миграции»** (Download Migration, запрос `GET /panel/api/server/getMigration`). Она формирует переносимый файл для перехода на другой движок БД:

С 3.5.0 кнопка показывается **только на PostgreSQL-панелях** (на SQLite обычный бэкап `.db` теперь восстанавливается прямо в PostgreSQL — отдельный экспорт не нужен):

Текущий движок	Что скачивается	Имя файла	Назначение
PostgreSQL	База SQLite, собранная из данных PostgreSQL	<code>x-ui.db</code>	Перевести панель обратно на SQLite

Подсказка: «Нажмите, чтобы скачать базу данных SQLite (.db), собранную из ваших данных PostgreSQL и готовую для запуска панели на SQLite.»

Преобразование `.db ⇌ .dump` для SQLite можно выполнить и из CLI командой `x-ui migratedb [file]` (см. раздел 16.7). Также с 3.5.0 миграция между движками стала **транзакционной и без потерь** (переносятся в том числе группы клиентов и глобальные счётчики трафика; неудачный импорт не меняет целевую БД), а для обновления клиентских инструментов PostgreSQL появилась команда `x-ui pgclient [версия]` и пункт **10. Install/Upgrade client tools (pg_dump/pg_restore)** в меню PostgreSQL (при необходимости подключается официальный репозиторий PostgreSQL).

Бэкап через Telegram-бот

Если настроен Telegram-бот (см. раздел про уведомления), он умеет присылать резервную копию прямо в чат администратора. Бэкап через Telegram включает **два файла**: саму базу данных (`x-ui.db` , либо `x-ui.dump` на PostgreSQL) и конфигурацию Xray `config.json` . Сообщению предшествует строка « Время резервного копирования: ...».

Есть два способа получить бэкап в Telegram:

1. **По запросу.** Кнопка «  **Бэкап БД** » в меню бота — бот немедленно отправляет файлы в текущий чат.
2. **Автоматически с отчётом.** В настройках бота есть переключатель «**Резервное копирование базы данных**» (Database Backup) с описанием «Отправлять уведомление с файлом резервной копии базы данных». Когда он включён, при каждой периодической рассылке отчёта бот после отчёта присылает резервную копию всем администраторам. Период отправки отчёта задаётся расписанием cron бота (см. раздел 16.6). Между файлами и между администраторами бот делает паузы, чтобы не превышать лимиты Telegram.

Бэкап через бота отправляется только если бот запущен; на PostgreSQL он также требует наличия `pg_dump` на сервере.

16.2. Просмотр логов

В панели два независимых средства просмотра логов, оба открываются с вкладки «**Логи**» на «Дашборде». Каждое окно умеет обновляться (иконка «обновить» в заголовке) и скачивать показанное в файл `x-ui.log` (кнопка с иконкой загрузки).

Логи панели (приложения / syslog)

Окно логов панели (`POST /panel/api/server/logs/{count}`). Элементы управления:

Элемент	Значение по умолчанию	Описание
Количество строк	20	Выпадающий список: 20 / 50 / 100 / 500 / 1000
Уровень	Info	Минимальный уровень: Debug / Info / Notice / Warning / Error
SysLog (галочка)	выключено	Откуда брать логи: из буфера приложения или из системного журнала
Автообновление (галочка)	выключено	Пересчитывать лог каждые 5 секунд (см. ниже)

Поведение зависит от галочки **SysLog**:

- **Выключено (по умолчанию):** логи берутся из внутреннего кольцевого буфера панели, отфильтрованные по выбранному уровню. Записи отображаются с уровнем (DEBUG / INFO / NOTICE / WARNING / ERROR) и источником: **X-UI:** — сообщения самой панели, **XRAY:** — проброшенные сообщения Xray.

Простые уведомления без штампа времени и уровня (например, системное сообщение «Syslog is not supported» на Windows) теперь показываются целиком, как есть. Распознаётся строго формат YYYY/MM/DD LEVEL – тело ; всё прочее выводится без разбора, поэтому такие строки больше не обрезаются (раньше первые три слова ошибочно трактовались как дата/время/уровень).

- **Включено:** панель выполняет на сервере `journalctl -u x-ui --no-pager -n <count> -p <level>`, то есть показывает системный журнал службы `x-ui`. Допустимое количество строк – от 1 до 10000; уровень принимает значения syslog (`emerg/0`, `alert/1`, `crit/2`, `err/3`, `warning/4`, `notice/5`, `info/6`, `debug/7`). На Windows режим SysLog не поддерживается – будет показано предупреждение, что нужно снять галочку и использовать логи приложения. Если `systemd` /служба недоступны, появится сообщение об ошибке запуска `journalctl`.

Пример: тот же журнал из консоли сервера. Когда панель недоступна (например, не стартует), системный журнал можно прочитать напрямую – это ровно та команда, которую панель выполняет в режиме SysLog:

```
# последние 100 строк уровня warning и выше
journalctl -u x-ui --no-pager -n 100 -p warning

# следить за журналом в реальном времени
journalctl -u x-ui -f
```

Уровень в этом окне фильтрует **вывод**. Какой минимальный уровень вообще пишется в консоль/ syslog, определяется уровнем логирования панели (переменная окружения, по умолчанию `Info` ; в файл панель всегда пишет на уровне `DEBUG`).

Логи доступа Xray (журнал доступа)

Отдельное окно для access-лога Xray (`POST /panel/api/server/xraylogs/{count}`). Оно разбирает строки журнала доступа Xray и показывает их таблицей: **Date, From, To, Inbound, Outbound, Email**.

Начиная с 3.4.1 это окно и кнопка его вызова на карточке статуса Xray подписаны **«Логи доступа»** (Access Logs) – раньше они назывались просто «Логи». Переименование сделано, чтобы не путать просмотрщик access-лога Xray с просмотрщиком логов самой панели, у которого прежде было такое же название.

Элемент	Значение по умолчанию	Описание
Количество строк	20	20 / 50 / 100 / 500 / 1000
Фильтр	пусто	Текстовый поиск по подстроке (применяется по нажатию Enter)
Автообновление (галочка)	выключено	Перечитывать лог каждые 5 секунд (см. ниже)
Direct (галочка)	включено	Показывать прямые соединения (трафик через freedom-outbound)
Blocked (галочка)	включено	Показывать заблокированные соединения (трафик в blackhole-outbound)

Элемент	Значение по умолчанию	Описание
Proxy (галочка)	включено	Показывать проксируемый трафик

Тип события определяется автоматически по тегу исходящего соединения в строке лога: соответствие тегам `freedom` → «DIRECT» (зелёный), `blackhole` → «BLOCKED» (красный), всё остальное → «PROXY» (синий).
Строки `api` → `api` и пустые строки пропускаются.

Автообновление. В обоих окнах логов (и «Логи», и «Логи доступа») есть флажок **«Автообновление»** (`Auto Update`). Если его включить, содержимое лога автоматически пересчитывается каждые 5 секунд с сохранением всех текущих настроек окна — выбранного числа строк, уровня/фильтра и галочек `Direct` / `Blocked` / `Proxy`. Опрос прекращается, как только окно закрыто или флажок снят.

Чтобы это окно показывало записи, у Xray должен быть включён **журнал доступа** с путём к файлу (не `none`) — см. ниже. Если access-лог отключён или файл недоступен, окно будет пустым («No Record...»).

16.3. Уровень и настройка логирования Xray

Параметры журналирования самого Xray задаются на странице **«Конфигурации Xray»** в блоке **«Лог»** (`Log`) с предупреждением:

«Логи могут замедлять работу сервера. Включайте только нужные вам виды логов при необходимости!»

Поле	Перевод	Значение по умолчанию	Описание
Уровень логов (<code>logLevel</code>)	Log Level	<code>warning</code>	Уровень детализации лога ошибок Xray. Допустимые значения: <code>debug</code> , <code>info</code> , <code>notice</code> , <code>warning</code> , <code>error</code> . Подсказка: «Уровень журнала для журналов ошибок, указывающий информацию, которую необходимо записать.»
Логи доступа (<code>accessLog</code>)	Access Log	<code>none</code>	Путь к файлу журнала доступа. Спецзначение <code>none</code> отключает логи доступа. Подсказка: «Путь к файлу журнала доступа. Специальное значение „none“ отключает логи доступа.»
Логи ошибок (<code>errorLog</code>)	Error Log	пусто (путь по умолчанию)	Путь к файлу логов ошибок; <code>none</code> отключает их. Подсказка: «Путь к файлу логов ошибок. Специальное значение „none“ отключает логи ошибок.»
Логи DNS (<code>dnsLog</code>)	DNS Log	<code>false</code> (выкл.)	Включить логирование DNS-запросов. Подсказка: «Включить логи запросов DNS».
Маскировка адреса (<code>maskAddress</code>)	Mask Address	пусто (выкл.)	При активации реальный IP-адрес автоматически заменяется на маскировочный в логах. Подсказка: «При активации реальный IP-адрес заменяется на маскировочный в логах.»

Так как по умолчанию **«Логи доступа» = none**, окно «Логи Xray» (раздел 16.2) изначально пусто. Чтобы оно заработало, задайте здесь путь к access-логу и перезапустите Xray.

Обратите внимание: пустой access-лог влияет только на это окно. Список онлайн-клиентов на «Дашборде» и лимит количества IP в форме клиента **не зависят** от access-лога — панель определяет онлайн-клиентов и считает их IP-адреса через online-stats API ядра Xray (статистику по соединениям). На старых версиях ядра, где этого API нет, панель автоматически возвращается к прежнему способу (чтение access-лога), и тогда для лимита IP путь к access-логу здесь по-прежнему нужен.

Лимит по количеству IP и fail2ban. Само ограничение по количеству IP у клиента (поле «IP Limit» в форме клиента и при массовом добавлении) применяется на сервере только если установлен **fail2ban** — именно он банит превышающие лимит адреса. Панель проверяет наличие fail2ban (GET /panel/api/server/fail2banStatus); если его нет, поле «IP Limit» становится недоступным с поясняющей подсказкой (на Windows — отдельным сообщением), а ранее заданные лимиты на таких серверах автоматически обнуляются, поскольку всё равно не действовали. Блокировка fail2ban распространяется и на TCP, и на UDP. С 3.5.0 «мёртвое» соединение (клиент пропал без корректного закрытия TCP) фиксируется **один раз**, а не при каждом 10-секундном сканировании: строка [LIMIT_IP] и цикл отключения повторяются только при реальном переподключении, поэтому счётчик fail2ban не накручивается и завышать maxretry больше не нужно (формат лога и failregex не менялись). На обычных серверах fail2ban теперь ставится автоматически при установке и обновлении панели (см. раздел 16.5).

Пример: блок `log`, при котором окно «Логи Xray» начнёт показывать записи. В JSON-конфигурации Xray это выглядит так:

```
{
  "log": {
    "loglevel": "warning",
    "access": "./access.log",
    "error": "",
    "dnsLog": false,
    "maskAddress": ""
  }
}
```

Главное — заменить `"access": "none"` на путь к файлу (например, `"./access.log"`). После сохранения перезапустите Xray, и таблица в окне «Логи Xray» наполнится строками.

16.4. Управление Xray: остановка и перезапуск

Состоянием Xray управляют с карточки Xray на «Дашборде». Текущее состояние отображается одним из значений: **Запущен** (Running), **Остановлен** (Stopped), **Неизвестно** (Unknown), **Ошибка** (Error). При ошибке доступна всплывающая подсказка «Ошибка при запуске Xray».

Кнопка	Перевод	Эндпоинт	Действие
Стоп	Stop	POST /panel/api/server/stopXrayService	Останавливает процесс Xray. При успехе — уведомление-предупреждение «Xray service has been stopped».
Перезапуск	Restart	POST /panel/api/server/restartXrayService	Перезапускает (или запускает) Xray с применением текущей конфигурации. При успехе — уведомление «Xray service has been restarted successfully».

После любой из операций панель рассылает новое состояние по WebSocket, поэтому статус на «Дашборде» обновляется без перезагрузки страницы. Если операция завершилась ошибкой, состояние Xray становится «Ошибка», и текст ошибки попадает в уведомление.

Помимо ручного перезапуска, панель сама проверяет, не нужно ли перезапустить Xray (фоновая задача каждые 30 с) и не упал ли процесс (проверка каждую секунду) — см. раздел 16.6.

Монитор здоровья туннеля (автоперезапуск Xray)

В 3.4.1 появился необязательный **монитор здоровья туннеля**. Если он включён, панель периодически проверяет доступность заданного URL и после нескольких неудачных проверок подряд автоматически перезапускает ядро Xray — это помогает восстановить туннель, переставший пропускать трафик. По умолчанию монитор **отключён** и настраивается **только переменными окружения службы** (в веб-интерфейсе его настроек нет — так задумано авторами).

Включает монитор переменная `XUI_TUNNEL_HEALTH_MONITOR=true`. Переменную `XUI_TUNNEL_HEALTH_PROXY` следует направить на локальный xray-inbound (например `socks5://127.0.0.1:1080`) — тогда проба идёт через сам Xray и проверяет именно туннель; без неё проверяется лишь связность хоста, и перезапуск не починит проблему сетевого подключения сервера. Остальные переменные задают параметры проверки:

Переменная	Назначение	По умолчанию
<code>XUI_TUNNEL_HEALTH_MONITOR</code>	Включить монитор (вкл/выкл)	<code>false</code>
<code>XUI_TUNNEL_HEALTH_PROXY</code>	Прокси, через который идёт проба (укажите локальный xray-inbound)	пусто
<code>XUI_TUNNEL_HEALTH_URL</code>	URL, который проверяется	<code>https://www.cloudflare.com/cdn-cgi/trace</code>
<code>XUI_TUNNEL_HEALTH_INTERVAL</code>	Интервал между проверками	<code>30s</code>
<code>XUI_TUNNEL_HEALTH_TIMEOUT</code>	Таймаут одной проверки	<code>10s</code>
<code>XUI_TUNNEL_HEALTH_FAILURES</code>	Число неудач подряд до перезапуска	<code>3</code>
<code>XUI_TUNNEL_HEALTH_COOLDOWN</code>	Минимальная пауза между перезапусками	<code>5m</code>

Перезапуск Xray разрывает соединения всех подключённых клиентов, поэтому имеет смысл оставлять интервал и порог числа неудач достаточно большими, чтобы случайный сбой одной пробы не приводил к лишним перезапускам.

16.5. Перезапуск и обновление панели

Перезапуск панели

На странице **«Настройки панели»** есть действие **«Перезапустить панель»** (`Restart Panel` , `POST /panel/api/setting/restartPanel`). При подтверждении панель перезапускается **через 3 секунды**.

Сообщения:

- Подтверждение: «Вы уверены, что хотите перезапустить панель? Подтвердите, и перезапуск произойдёт через 3 секунды. Если панель будет недоступна, проверьте лог сервера.»
- Успех: «Панель успешно перезапущена».

Технически на Linux перезапуск выполняется отправкой сигнала `SIGHUP` процессу панели (либо через зарегистрированный хук). На Windows отправка `SIGHUP` не поддерживается.

Самообновление панели (Update Panel)

На «Дашборде» доступна функция **«Обновить панель»** (`Update Panel`) — обновление 3X-UI до последнего релиза прямо из веб-интерфейса.

Перед обновлением панель сверяет версии (`GET /panel/api/server/getPanelUpdateInfo`), запрашивая последний релиз 3x-ui с GitHub:

Поле	Перевод
Текущая версия панели	Current panel version
Последняя версия панели	Latest panel version
Панель обновлена / «Обновлено»	Panel is up to date / Up to date — показывается, если новой версии нет

Запуск обновления — `POST /panel/api/server/updatePanel` . Диалог подтверждения:

- «Вы действительно хотите обновить панель?»
- «Это обновит 3X-UI до версии #version# и перезапустит сервис панели.»

После запуска — всплывающее сообщение «Обновление панели началось» (`Panel update started`); при неудачной проверке версий — «Проверка обновления панели не удалась» (`Panel update check failed`).

Что происходит на сервере: самообновление поддерживается **только на Linux** (на других ОС вернётся ошибка «panel web update is supported only on Linux installations»). Панель скачивает официальный скрипт `update.sh` с GitHub (`raw.githubusercontent.com/MHSanaei/3x-ui/main/update.sh`) и запускает его в отдельном процессе: предпочтительно через `systemd-run` в отдельном юните (`x-ui-web-update-`

`<timestamp>`), а при отсутствии `systemd` — как отдельный отсоединённый процесс. По завершении скрипт обновляет компоненты и перезапускает службу панели. Для запуска требуется `bash` .

Если в ходе обновления скрипт сгенерировал новый случайный базовый путь веб-панели (Web Base Path), служба `x-ui` перезапускается автоматически, чтобы новый путь сразу заработал. (Без перезапуска сервер продолжал бы отдавать старый путь, а интерфейс показывал бы новый, и новый адрес был бы недоступен до ручного перезапуска.)

Dev-канал (rolling), с 3.4.2. Кнопка версии панели на «Дашборде» теперь всегда открывает окно обновления (раньше на актуальной стабильной сборке она вела на страницу релизов GitHub), а переключатель **«Dev-канал»** (`devChannelEnable`) показывается всегда — даже на стабильной сборке. Включив его, при следующем обновлении можно перейти на «катящийся» dev-канал (сборки по каждому коммиту в `main`). При включении выводится предупреждение: «Dev-сборки отслеживают каждый коммит в `main` и не являются стабильными релизами — автоматического отката нет». Без явного действия пользователя ничего не обновляется.

Канал обновления Dev (rolling-сборки по коммитам)

Помимо обычного обновления до стабильного релиза, есть необязательный **«Канал разработки»** (`Dev`). Переключатель появляется в окне обновления панели **только на dev-сборках** (CI-сборки, собранные по отдельному коммиту); на стабильных релизах его не видно. При включении панель будет обновляться до rolling-сборки `dev-latest` , которая отслеживает каждый коммит ветки `main` и не является стабильным релизом — показывается предупреждение, что dev-сборки нестабильны и автоматического отката нет. В режиме dev окно показывает «Текущий коммит» / «Последний коммит» вместо номеров версий. Функция доступна только на Linux с `systemd` .

На dev-сборках панель показывает свою версию как `dev+<короткий-коммит>` вместо вводящего в заблуждение стабильного номера — в бейдже боковой панели, на карточке «Дашборда», в окне обновления, в отчёте Telegram-бота о статусе и в выводе команды `x-ui -v` . На стабильных релизах вид версии не меняется.

На узлах (нодах) панель этого же 3x-ui обновляется централизованно через `POST /panel/api/nodes/updatePanel` — см. раздел про узлы.

Автоматическая установка fail2ban

Чтобы лимит по количеству IP у клиентов (раздел 16.3) работал из коробки, при установке и обновлении панели на обычном сервере `fail2ban` теперь устанавливается и настраивается автоматически (прежде это происходило только в Docker-образе). Поведением управляет переменная окружения `XUI_ENABLE_FAIL2BAN` : настройка выполняется, если переменная не задана или равна `true` . Ручной запуск доступен командой `x-ui setup-fail2ban` . Сбой настройки `fail2ban` не прерывает установку или обновление панели.

Установка и обновление на IPv6-only хостах

Скрипты `install.sh` и `update.sh` теперь корректно работают на серверах только с IPv6: загрузка релиза, скрипта `x-ui.sh` и сервис-файлов больше не использует принудительно IPv4 (`curl -4`), а берёт доступный протокол. Поэтому панель можно установить и обновить и на хосте без IPv4-адреса.

Исправления скриптов в 3.5.0

- **RHEL-семейство** (Rocky/Alma/RHEL/Oracle): локальная установка PostgreSQL из меню теперь работает (аутентификация по паролю вместо `ident`), а `fail2ban` для лимита IP ставится из **EPEL**.
- **Arch/Manjaro**: установка и обновление больше не запускают полный апгрейд системы `pacman -Syu` — только обновление базы пакетов.
- **32-битный ARM**: скачанный бинарь Xray именуется `xray-linux-arm32` — именно так, как его запускает панель (раньше обновление клало файл под другим именем, и панель продолжала гонять старое ядро).
- **IP-сертификат**: автоопределённый публичный IPv4 показывается для подтверждения (Enter — принять) перед выпуском; при отказе — проверенный ручной ввод.
- **Выбор АСМЕ-порта**: Enter (принять порт 80 по умолчанию) больше не выводит ложное «Your input is invalid».

Переопределение порта панели переменной XUI_PORT

Порт прослушивания веб-панели можно переопределить переменной окружения `XUI_PORT` — она действует только на время работы текущего процесса и **не изменяет** сохранённое значение `webPort` в базе. Допустимы значения от 1 до 65535; пустое, некорректное или выходящее за диапазон значение игнорируется (используется `webPort`) с предупреждением в логе. Это удобно при развёртывании, в первую очередь в Docker: при использовании bridge-сети публикуемый порт контейнера должен совпадать с `XUI_PORT` — например, `XUI_PORT=8080` и `ports: "8080:8080"`.

Обновление и переключение версии Xray-core

Здесь же на «Дашборде» можно управлять версией Xray-core отдельно от панели.

- **Обновления Xray** (Xray Updates) / **Выбор версии** (Version) — выпадающий список доступных версий. Подсказки: «Выберите нужную версию» и предупреждение «Важно: старые версии могут не поддерживать текущие настройки».
- Установка/смена версии — `POST /panel/api/server/installXray/{version}`. Диалог: «Переключить версию Xray» / «Вы точно хотите сменить версию Xray?». При успехе — «Xray успешно обновлён».

Пример: смена версии Xray-core запросом к API. Версия указывается тегом релиза из XTLS/Xray-core (с префиксом `v`). Например, переключиться на `v1.8.24`:

```
curl -s -b cookies.txt -X POST \
  https://panel.example.com:2053/panel/api/server/installXray/v1.8.24
```

(`cookies.txt` — файл с cookie из примера в разделе 16.1.) После установки Xray перезапустится автоматически с выбранной версией.

На сервере при смене версии Xray сначала останавливается, скачивается архив нужной версии с GitHub (XTLS/Xray-core), бинарник распаковывается и заменяется, после чего Xray перезапускается с проверкой контрольных размеров архива/бинарника.

16.6. Периодические задачи (cron)

Панель регистрирует ряд фоновых задач при запуске. Их расписания фиксированы (не настраиваются в UI, за исключением расписания Telegram-отчёта и LDAP-синхронизации). Ниже — задачи, относящиеся к эксплуатации.

Задача	Расписание	Назначение
Проверка работы Xray	каждую 1 с	Контроль, что процесс Xray запущен
Проверка необходимости перезапуска Xray	каждые 30 с	Перезапуск, если конфигурация помечена как изменённая
Сбор трафика Xray	каждые 5 с (старт через 5 с после запуска)	Учёт трафика inbound/клиентов
Проверка IP клиентов	каждые 10 с	Контроль лимита IP по логу
Heartbeat и синхронизация трафика узлов	каждые 5 с	Обмен с узлами (нодами)
Очистка логов	ежедневно (@daily)	Очищает IP-limit логи и persistent access-log, ротируя текущий лог в *.prev.log. С 3.5.0 ежедневная очистка затрагивает и error-лог Xray (раньше — только access)
Ограничение роста логов Xray	каждые 10 минут (@every 10m)	Новое в 3.5.0. Обрезает access- и error-лог Xray, когда любой из них превышает 64 МБ ; отключённые логи (none /пусто) не трогаются
Сброс трафика по периоду	@hourly , @daily , @weekly , @monthly	Сбрасывает счётчики трафика тех inbound (и их клиентов), у которых задан соответствующий период автосброса
Telegram-отчёт	задаётся в настройках бота (по умолчанию @daily)	Рассылка отчёта администраторам; при включённой опции — с приложенной резервной копией БД (раздел 16.1)
Сброс hash-хранилища Telegram	каждые 2 м	Только при включённом боте
Контроль нагрузки CPU для Telegram	каждые 10 с	Только если задан порог CPU > 0

Дополнительно:

- **Периодический сброс трафика** срабатывает только для тех inbound, у которых выбран соответствующий режим автосброса (ежечасно/ежедневно/еженедельно/ежемесячно). Задача сбрасывает трафик самого inbound и всех его клиентов.
- **Проверка истечения и исчерпания.** Отключение клиентов по истечении срока и при исчерпании лимита трафика выполняется в рамках учёта трафика: клиенты с истёкшим expiry_time либо

исчерпанным объёмом помечаются и отключаются, при необходимости рассчитывается следующий срок (для циклических лимитов и режима «отсчёт с первого использования»). На «Дашборде» и в списках это отражается статусами «Истекло»/«Исчерпано»/«Скоро закончится».

- **Автоматический бэкап в Telegram** – это побочный эффект задачи отчёта, отдельного cron-расписания только для бэкапа нет. Поэтому частота автобэкапа равна частоте отчёта бота.

16.7. Консольное меню и CLI (`x-ui`)

На сервере панель управляется командой `x-ui`. Без аргументов открывается интерактивное меню «3X-UI Panel Management Script»; с аргументом выполняется конкретная подкоманда. Пункты меню, относящиеся к эксплуатации:

№ в меню	Пункт	Действие
1	Install	Установка панели (скачивает и запускает <code>install.sh</code>)
2	Update	Обновление всех компонентов <code>x-ui</code> до последней версии без потери данных; после – автоматический перезапуск
3	Update to Dev Channel (latest commit)	Обновление до rolling-сборки <code>dev-latest</code> (последний коммит ветки <code>main</code>) с подтверждением (см. 16.5)
4	Update Menu	Обновление только самого скрипта меню <code>x-ui</code>
5	Legacy Version	Установка указанной (старой) версии панели по введённому номеру (например, <code>2.4.0</code>)
6	Uninstall	Полное удаление панели и Xray (см. 16.8)
7	Reset Username & Password	Сброс логина/пароля администратора
8	Reset Web Base Path	Сброс базового пути веб-панели
9	Reset Settings	Сброс настроек к значениям по умолчанию
10	Change Port	Смена порта панели
11	View Current Settings	Просмотр текущих настроек
12–14	Start / Stop / Restart	Запуск, остановка, перезапуск службы панели
15	Restart Xray	Перезапуск только Xray
16	Check Status	Текущий статус службы
17	Logs Management	Просмотр и очистка логов (см. ниже)
18–19	Enable / Disable Autostart	Включить/выключить автозапуск службы при старте ОС
27	Update Geo Files	Обновление гео-файлов (GeoIP/GeoSite)
25	PostgreSQL Management	Управление PostgreSQL

Нумерация пунктов меню изменилась в 3.4.1: из-за добавления пункта 3 «Update to Dev Channel» все последующие пункты сдвинулись на единицу. Всего пунктов стало 28, выбор вводится в диапазоне [0–28] .

Управление логами в CLI (пункт 16)

Подменю «Logs Management» теперь открывается пунктом **17** (раньше – 16):

- **Debug Log** – потоковый просмотр журнала службы: `journalctl -u x-ui -e --no-pager -f -p debug` (на Alpine – `grep` по `/var/log/messages`).
- **Clear All logs** – очистка системного журнала: `journalctl --rotate + journalctl --vacuum-time=1s` , после чего служба перезапускается. (Недоступно на Alpine.)

Прямые подкоманды `x-ui`

Все доступные подкоманды:

Команда	Описание
<code>x-ui</code>	Открыть административное меню
<code>x-ui start</code>	Запустить панель
<code>x-ui stop</code>	Остановить панель
<code>x-ui restart</code>	Перезапустить панель
<code>x-ui restart-xray</code>	Перезапустить Xray
<code>x-ui status</code>	Текущий статус
<code>x-ui settings</code>	Показать текущие настройки
<code>x-ui enable</code>	Включить автозапуск при старте ОС
<code>x-ui disable</code>	Отключить автозапуск
<code>x-ui log</code>	Просмотр логов
<code>x-ui banlog</code>	Просмотр логов банов Fail2ban
<code>x-ui setup-fail2ban</code>	Установить и настроить fail2ban для лимита IP (см. 16.5)
<code>x-ui update</code>	Обновить панель

| `x-ui update-dev` | Обновить панель до канала разработки (rolling-сборка `dev-latest`) ||
`x-ui update-all-geofiles` | Обновить все гео-файлы (с последующим перезапуском) || `x-ui migrateDB`
[`file`] | Преобразование базы `.db` в `.dump` (SQLite) || `x-ui legacy` | Установить устаревшую версию ||
`x-ui install` | Установить панель || `x-ui uninstall` | Удалить панель |

Команда `x-ui update` скачивает и запускает официальный `update.sh` (тот же, что и веб-обновление из раздела 16.5), запрашивая подтверждение: «This function will update all x-ui

components to the latest version, and the data will not be lost.» По завершении панель автоматически перезапускается.

Флаги `-webCert` / `-webCertKey` в подкоманде `setting`. Пути к сертификату и приватному ключу веб-панели можно задать прямо в подкоманде `x-ui setting -webCert <путь> -webCertKey <путь>` — указание любого из этих флагов сохраняет соответствующий путь (как и отдельная подкоманда `cert`), и панель сразу переходит на HTTPS.

Получение API-токена через CLI

Команда получения API-токена через CLI (пункт меню/команда `x-ui`) не показывает ранее выданный токен. API-токены хранятся только в виде хешей, поэтому существующий токен нельзя получить в открытом виде. Если токены уже настроены, команда сообщает их количество, советует управлять токенами в панели (**Settings** → **API Tokens**, см. раздел про API-токены) и сразу генерирует **новый запасной токен** с именем вида `cli-fallback-<timestamp>` и выводит его, чтобы CLI оставался полезным без захода в интерфейс.

16.8. Удаление панели

Удаление выполняется из CLI — пункт меню **5 (Uninstall)** или команда `x-ui uninstall`. Перед удалением запрашивается подтверждение (по умолчанию «нет»): «Are you sure you want to uninstall the panel? xray will also be uninstalled!».

При подтверждении скрипт:

1. Останавливает службу и отключает её автозапуск (`systemctl stop/disable x-ui`, либо на Alpine — `rc-service / rc-update`), удаляет unit-файл службы и перезагружает конфигурацию `systemd`.
2. Удаляет каталоги данных и приложения (`/etc/x-ui/`, каталог установки) и env-файл службы (`/etc/default/x-ui`, `/etc/conf.d/x-ui` или `/etc/sysconfig/x-ui` — в зависимости от дистрибутива).
3. Удаляет сам скрипт `x-ui` и выводит сообщение «Uninstalled Successfully», а также команду для повторной установки.

Если панель использовала PostgreSQL (в env-файле `XUI_DB_TYPE=postgres`), после удаления файлов панели скрипт дополнительно спрашивает, нужно ли удалить и сам сервер PostgreSQL вместе со всеми его базами: «Also purge PostgreSQL and delete all of its data?». Запрос требует явного подтверждения (по умолчанию — отказ) и сопровождается предупреждением: удаление затронет **ВСЕ** базы PostgreSQL на машине, в том числе принадлежащие другим приложениям, и необратимо. При отказе PostgreSQL и его данные остаются нетронутыми.

Удаление необратимо: вместе с панелью удаляется Xray и все данные (включая базу). Если данные могут понадобиться, заранее сделайте экспорт базы (раздел 16.1).

16.9. Команда `x-ui migrateDB`

Начиная с версии 3.3.0 управляющий скрипт `x-ui.sh` получил подкоманду `migrateDB` — обёртку вокруг встроенного бинарника `x-ui` (`x-ui migrate-db`) для конвертации базы данных панели SQLite между двумя форматами: бинарным `.db` и переносимым текстовым дампом `.dump` (обычный SQL-текст).

Что делает команда

Команда работает в двух направлениях, причём направление определяется **автоматически** по входному файлу:

Направление	Как называется	Что происходит
<code>.db → .dump</code>	dump (выгрузка)	бинарная база SQLite выгружается в текстовый SQL-файл
<code>.dump → .db</code>	restore (восстановление)	из текстового SQL-файла заново собирается бинарная база SQLite

Под капотом скрипт вызывает бинарник панели:

- выгрузка: `x-ui migrate-db --src <вход> --dump <выход>`
- восстановление: `x-ui migrate-db --restore <вход> --out <выход>`

Синтаксис вызова

```
x-ui migrateDB [file.db|file.dump] [output]
```

- **[file.db|file.dump]** — входной файл (первый аргумент). Если его не указать, берётся установленная база панели по умолчанию: `/etc/x-ui/x-ui.db`.
- **[output]** — путь к выходному файлу (второй аргумент). Необязателен: при отсутствии имя выбирается автоматически рядом со входным файлом (см. ниже).

Примеры:

```
x-ui migrateDB                                # выгрузить /etc/x-ui/x-ui.db -> /etc/x-ui/
x-ui.dump
x-ui migrateDB /etc/x-ui/x-ui.db backup.dump
x-ui migrateDB backup.dump restored.db        # собрать .db из дампа
```

Как определяется направление

Скрипт смотрит на расширение входного файла:

- `*.db`, `*.sqlite`, `*.sqlite3` → режим **dump** (выгрузка в текст);
- `*.dump`, `*.sql` → режим **restore** (сборка базы).

Если расширение не распознано, скрипт читает первые 16 байт файла: сигнатура SQLite format 3 означает бинарную базу (режим dump), иначе файл считается дампом (режим restore).

Имя выходного файла, если второй аргумент не задан:

- при выгрузке — то же имя, что у входа, с расширением `.dump` ;
- при восстановлении — то же имя с расширением `.db` .

Защитные проверки и поведение

- **Наличие бинарника.** Если бинарник `x-ui` не найден или не исполняемый — выводится ошибка «x-ui binary not found ... Is the panel installed?».
- **Поддержка функции в сборке.** Скрипт проверяет, что бинарник умеет `migrate-db --dump/--restore` (через `x-ui migrate-db -h`). Если нет — предлагается сначала обновить панель командой `x-ui update` .
- **Существование входного файла.** При отсутствии входного файла печатается ошибка и строка с синтаксисом вызова.
- **Перезапись выхода.** Если выходной файл уже существует, запрашивается подтверждение (по умолчанию «нет»); без подтверждения операция отменяется. При восстановлении старый выходной файл предварительно удаляется.
- **Защита «живой» базы.** При восстановлении в базу по умолчанию `/etc/x-ui/x-ui.db` , когда панель запущена, операция отклоняется с требованием сначала остановить панель (`x-ui stop`) либо выбрать другой выходной путь. Это предотвращает перезапись рабочей базы у работающего сервиса.
- При неуспехе сборки базы недописанный выходной файл удаляется.

Зачем это нужно

- **Бэкап.** Текстовый `.dump` — человекочитаемый, удобен для хранения в системах контроля версий и для дифференциального просмотра содержимого базы.
- **Перенос.** Дамп переносим между машинами и устойчив к различиям версий формата файла SQLite — на новом сервере из него собирается рабочая `.db` .
- **Диагностика.** Из `.dump` можно глазами посмотреть структуру и данные панели, не имея под рукой инструментов SQLite.

Интерактивный режим

Помимо прямого вызова, конвертация доступна из интерактивного меню. В подменю PostgreSQL (`x-ui` → раздел работы с PostgreSQL) есть пункт **9. Convert SQLite .db <-> .dump** : она спрашивает путь к входному файлу (по умолчанию `/etc/x-ui/x-ui.db`) и к выходному (можно оставить пустым для авто-именования), а направление, как и в CLI-режиме, определяется автоматически.

Документ подготовлен по исходному коду 3X-UI. Если какой-то пункт интерфейса в вашей версии отличается — приоритет у поведения панели и подсказок в самом UI.